

Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

КОНСПЕКТ ЛЕКЦИЙ

по дисциплине

**«Архитектура автоматизированных систем
обработки информации и управления»**

**Профиль: «Архитектура программных
систем»**

ЧАСТЬ 1

РАЗРАБОТКА ПРИЛОЖЕНИЙ НА ЯЗЫКЕ C#

Москва – 2026

Оглавление

1	КРАТКАЯ ХАРАКТЕРИСТИКА ЯЗЫКА ПРОГРАММИРОВАНИЯ C#	8
2	СОЗДАНИЕ КОНСОЛЬНОГО ПРИЛОЖЕНИЯ НА ЯЗЫКЕ C#	8
3	БАЗОВЫЕ ВОЗМОЖНОСТИ ЯЗЫКА C#, НЕОБХОДИМЫЕ ДЛЯ СОЗДАНИЯ КОНСОЛЬНЫХ ПРИЛОЖЕНИЙ.....	14
3.1	Консольный ввод-вывод	14
3.2	Строчковая интерполяция	16
3.3	Преобразования типов	17
3.4	Простые массивы, условные операторы и циклы	19
3.5	Задание для закрепления материала	21
4	ОСНОВНЫЕ ТИПЫ ДАННЫХ ЯЗЫКА C#	25
4.1	Организация типов данных в языке C#	25
4.2	Базовые типы данных	26
5	ОСНОВНЫЕ КОНСТРУКЦИИ ПРОГРАММИРОВАНИЯ ЯЗЫКА C#	29
5.1	Пространства имен и сборки	29
5.2	Основная программа и параметры командной строки.....	32
5.3	XML-комментарии.....	36
5.4	Вызов методов, передача параметров и возврат значений	38
5.5	Условные операторы и операторы сопоставления с образцом	41
5.6	Операторы цикла	47
5.7	Использование массивов	54
5.8	Работа с NULL.....	58
6	ОСНОВЫ РАБОТЫ С ФАЙЛАМИ	63
6.1	Получение данных о файлах и каталогах.....	63
6.1.1	Использование класса <i>Directory</i> для работы с каталогами.....	63
6.1.2	Использование класса <i>File</i> для работы с файлами	67
6.1.3	Использование класса <i>Path</i> для формирования путей	68
6.2	Чтение и запись файлов	72
6.2.1	Запись и чтение файла целиком	73
6.2.2	Добавление данных в файл	74
6.2.3	Чтение и запись массива строк	75
6.2.4	Ленивое чтение строк.....	77
6.2.5	Разбор текстового файла на подстроки	78
6.2.6	Бинарные файлы.....	78
6.3	Оператор USING и автоматическое освобождение ресурсов	84
6.4	Класс STRINGBUILDER для безопасного формирования строк.....	87
6.5	Стандартные потоки, перенаправление и конвейеры командной строки	91

7	РАБОТА СО СТАНДАРТНЫМИ КОЛЛЕКЦИЯМИ	106
7.1	ОБОБЩЕННЫЙ СПИСОК	107
7.2	НЕОБОБЩЕННЫЙ СПИСОК	110
7.3	ОБОБЩЕННЫЕ СТЕК И ОЧЕРЕДЬ	111
7.4	ОБОБЩЕННЫЙ СЛОВАРЬ	115
7.5	КОРТЕЖ	121
7.6	НОВЫЙ СИНТАКСИС КОРТЕЖЕЙ	122
7.7	ЗАПИСИ (RECORDS)	129
8	ОБЪЯВЛЕНИЕ И НАСЛЕДОВАНИЕ КЛАССОВ В C#	131
8.1	ОБЪЯВЛЕНИЕ КЛАССА И ЕГО ЭЛЕМЕНТОВ	132
8.2	ОБЪЯВЛЕНИЕ КОНСТРУКТОРА	135
8.3	ОБЪЯВЛЕНИЕ МЕТОДОВ	139
8.3.1	<i>Модификаторы видимости</i>	<i>139</i>
8.3.2	<i>Параметры по умолчанию и именованные аргументы</i>	<i>140</i>
8.3.3	<i>Локальные функции</i>	<i>141</i>
8.4	ОБЪЯВЛЕНИЕ СВОЙСТВ	144
8.4.1	<i>Области видимости аксессоров</i>	<i>146</i>
8.4.2	<i>Автоопределяемые свойства</i>	<i>147</i>
8.4.3	<i>Вычисляемые свойства</i>	<i>148</i>
8.4.4	<i>Роль свойств в языке C#</i>	<i>148</i>
8.4.5	<i>Возможности свойств в C# 6 и C# 7</i>	<i>149</i>
8.4.6	<i>Init-аксессоры</i>	<i>150</i>
8.4.7	<i>Required-свойства</i>	<i>152</i>
8.5	ФИНАЛИЗАТОРЫ	155
8.6	ОБЪЯВЛЕНИЕ СТАТИЧЕСКИХ ЭЛЕМЕНТОВ КЛАССА	157
8.7	СТАТИЧЕСКИЕ КЛАССЫ	159
8.8	КОНСТАНТЫ И СТАТИЧЕСКИЕ ПОЛЯ ТОЛЬКО ДЛЯ ЧТЕНИЯ	161
8.9	НАСЛЕДОВАНИЕ КЛАССОВ	163
8.9.1	<i>Вызов конструкторов при наследовании</i>	<i>165</i>
8.9.2	<i>Вызов других конструкторов текущего класса</i>	<i>166</i>
8.9.3	<i>Ключевое слово base для доступа к элементам базового класса</i>	<i>167</i>
8.9.4	<i>Проверка и приведение типов</i>	<i>167</i>
8.10	ВИРТУАЛЬНЫЕ И АБСТРАКТНЫЕ МЕТОДЫ	170
8.10.1	<i>Виртуальные методы и их переопределение</i>	<i>171</i>
8.10.2	<i>Полиморфизм</i>	<i>172</i>
8.10.3	<i>Переопределение и сокрытие методов (override и new)</i>	<i>174</i>
8.10.4	<i>Абстрактные классы и абстрактные методы</i>	<i>176</i>
8.10.5	<i>Запечатывание переопределённых методов</i>	<i>178</i>
8.10.6	<i>Виртуальные свойства</i>	<i>179</i>
8.11	МЕТОДЫ РАСШИРЕНИЯ	183

8.12	ЧАСТИЧНЫЕ КЛАССЫ	187
8.12.1	Частичные методы	191
8.13	ПАТТЕРН «ТЕКУЧИЙ ИНТЕРФЕЙС» (FLUENT INTERFACE) И ЦЕПОЧКИ ВЫЗОВОВ МЕТОДОВ 194	
8.14	СОЗДАНИЕ ДИАГРАММЫ КЛАССОВ В VISUAL STUDIO	197
9	СИТУАЦИОННОЕ МЕТАГРАФОВОЕ ОПИСАНИЕ ООП В C#	202
9.1	КЛАСС КАК СИТУАЦИЯ.....	202
9.2	НАСЛЕДОВАНИЕ КАК ПЕРЕХОД МЕЖДУ СИТУАЦИЯМИ.....	203
9.3	ЦЕПОЧКА ВИРТУАЛИЗАЦИИ КАК ПРОЦЕСС С ВЕТВЛЕНИЕМ И/ИЛИ.....	204
9.4	ОСНОВЫ МЕТАГРАФОВОГО ПОДХОДА ДЛЯ ОПИСАНИЯ СИТУАЦИЙ	205
10	РАСШИРЕННЫЕ ВОЗМОЖНОСТИ ОБЪЕКТНО-ОРИЕНТИРОВАННОГО ПРОГРАММИРОВАНИЯ В ЯЗЫКЕ C#	209
10.1	ОБРАБОТКА ИСКЛЮЧЕНИЙ	209
10.1.1	Блоки <i>try</i> , <i>catch</i> и <i>finally</i>	209
10.1.2	Генерация исключений и оператор <i>throw</i>	212
10.1.3	Часто используемые стандартные исключения	213
10.1.4	Создание собственных исключений.....	214
10.1.5	Фильтры исключений.....	217
10.1.6	Выражение <i>throw</i>	218
10.1.7	Рекомендации по обработке исключений	220
10.2	ИНТЕРФЕЙСЫ.....	223
10.2.1	Основы работы с интерфейсами.....	223
10.2.2	Возможности интерфейсов в современном C#.....	225
10.2.3	Наследование классов от интерфейсов.....	226
10.2.4	Автоматическая генерация заглушек	227
10.2.5	Неявная и явная реализация интерфейса.....	228
10.3	СТРУКТУРЫ (STRUCT)	234
10.3.1	Структура как тип-значение	234
10.3.2	Неизменяемые структуры (<i>readonly struct</i>)	235
10.3.3	Производительность: массив структур и массив классов	236
10.4	ПЕРЕЧИСЛЕНИЯ	236
10.5	ПЕРЕГРУЗКА ОПЕРАТОРОВ	243
10.6	ОБОБЩЕНИЯ	250
10.7	ДЕЛЕГАТЫ	259
10.8	Лямбда-выражения	262
10.9	ОБОБЩЕННЫЕ ДЕЛЕГАТЫ FUNC И ACTION И (УСТАРЕВШИЙ) PREDICATE	266
10.10	ГРУППОВЫЕ ДЕЛЕГАТЫ	270
10.11	СОБЫТИЯ	272
11	РЕФЛЕКСИЯ	278

11.1	РАБОТА СО СБОРКАМИ.....	278
11.2	РАБОТА С ТИПАМИ ДАННЫХ.....	279
11.3	ДИНАМИЧЕСКИЕ ДЕЙСТВИЯ С ОБЪЕКТАМИ КЛАССОВ.....	283
11.4	РАБОТА С АТТРИБУТАМИ.....	284
11.5	ИСПОЛЬЗОВАНИЕ РЕФЛЕКСИИ НА УРОВНЕ ОТКОМПИЛИРОВАННЫХ ИНСТРУКЦИЙ	288
11.6	САМООТБРАЖАЕМОСТЬ.....	291
12	ТЕХНОЛОГИЯ LINQ	293
12.1	МОТИВАЦИЯ: ЗАЧЕМ НУЖЕН LINQ.....	293
12.2	КРАТКАЯ ИСТОРИЯ LINQ И ЕГО МЕСТО В .NET.....	297
12.3	ПРЕДМЕТНАЯ ОБЛАСТЬ: СОТРУДНИКИ, ОТДЕЛЫ, ПРОЕКТЫ	300
12.3.1	Определение классов	301
12.3.2	Заполнение тестовых данных.....	302
12.3.3	Структура данных	303
12.4	LINQ TO OBJECTS: ОСНОВНЫЕ ОПЕРАЦИИ.....	304
12.4.1	Два синтаксиса: <i>query syntax</i> и <i>method syntax</i>	305
12.4.2	Выборка и проекция (<i>Select</i>)	306
12.4.3	Фильтрация (<i>Where</i>)	307
12.4.4	Сортировка (<i>OrderBy, ThenBy</i>).....	308
12.4.5	Соединения (<i>Join</i>)	309
12.4.6	Связь «много-ко-многим»	311
12.4.7	Группировка и агрегатные функции.....	313
12.4.8	Операции над множествами	315
12.4.9	Разделение данных на страницы (пагинация) – <i>Skip, Take</i>	316
12.4.10	Декартово произведение.....	317
12.5	LINQ TO OBJECTS: РАСШИРЕННЫЕ ВОЗМОЖНОСТИ	318
12.5.1	12.5.1 Отложенное и немедленное выполнение	318
12.5.2	Материализация результатов.....	319
12.5.3	Получение отдельных элементов	321
12.5.4	Конструкция <i>let</i> для промежуточных вычислений	323
12.5.5	Генерация последовательностей.....	324
12.5.6	Агрегатная функция <i>Aggregate</i>	325
12.6	КАК УСТРОЕН LINQ: ПРОВАЙДЕРЫ И ДЕРЕВЬЯ ВЫРАЖЕНИЙ.....	326
12.6.1	Два интерфейса: <i>IEnumerable</i> и <i>IQueryable</i>	326
12.6.2	Делегаты и деревья выражений	327
12.6.3	12.6.3 Схема работы LINQ-провайдера.....	329
12.6.4	Существующие LINQ-провайдеры	330
12.6.5	Практическое следствие: ограничения <i>IQueryable</i>	331
12.7	LINQ ДЛЯ РАЗЛИЧНЫХ ФОРМАТОВ ДАННЫХ	332
12.7.1	LINQ to XML	333
12.7.2	LINQ to JSON	334

12.7.3	<i>LINQ to YAML</i>	336
12.7.4	<i>Принцип обработки различных источников данных</i>	337
12.7.5	<i>12.7.5 Другие LINQ-провайдеры и библиотеки</i>	338
12.8	ORM и ENTITY FRAMEWORK CORE	339
12.8.1	<i>От ручного SQL к ORM: постановка проблемы</i>	339
12.8.2	<i>Понятие ORM</i>	340
12.8.3	<i>Установка Entity Framework Core для SQLite</i>	341
12.8.4	<i>Классы модели данных</i>	342
12.8.5	<i>Контекст базы данных (DbContext)</i>	344
12.8.6	<i>Создание базы данных и заполнение данными</i>	345
12.8.7	<i>LINQ-запросы через Entity Framework Core</i>	347
12.8.8	<i>Сравнение подходов</i>	350
12.9	ТИПИЧНЫЕ ОШИБКИ И РЕКОМЕНДАЦИИ	351
12.9.1	<i>Многократное перечисление IEnumerable</i>	351
12.9.2	<i>Загрузка всей таблицы вместо фильтрации на сервере</i>	351
12.9.3	<i>Забывший Include в EF Core</i>	352
12.9.4	<i>Выбор между query syntax и method syntax</i>	352
12.10	ИТОГИ	353
12.11	ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ МАТЕРИАЛА (LINQ TO OBJECTS)	353
13	РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКИХ ИНТЕРФЕЙСОВ	355
13.1	ПАТТЕРНЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА: MVC, MVP, MVVM	355
13.1.1	<i>Паттерн MVC (Model — View — Controller)</i>	355
13.1.2	<i>Паттерн MVP (Model — View — Presenter)</i>	356
13.1.3	<i>Паттерн MVVM (Model — View — ViewModel)</i>	356
13.2	ОСНОВЫ РАЗРАБОТКИ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИИ WINDOWS FORMS	357
13.2.1	<i>Создание проекта</i>	358
13.2.2	<i>Пример работы с кнопкой и текстовым полем</i>	360
13.2.3	<i>Пример работы с таймером</i>	366
13.2.4	<i>Пример открытия дочерних окон</i>	369
13.2.5	<i>Архитектурный аспект: «Super Form» и паттерн MVP применительно к Windows Forms</i>	376
13.3	WPF И ПАТТЕРН MVVM	376
13.3.1	<i>Привязка данных (Data Binding)</i>	380
13.3.2	<i>Ключевые выводы по WPF + MVVM</i>	380
13.4	РАЗДЕЛ 15.5. ASP.NET CORE MVC	381
14	ПРИМЕР МНОГОПОТОЧНОГО ПОИСКА В ТЕКСТОВОМ ФАЙЛЕ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИИ WINDOWS FORMS	381
14.1	ЧТЕНИЕ ИНФОРМАЦИИ ИЗ ТЕКСТОВОГО ФАЙЛА	383
14.2	ЧЕТКИЙ ПОИСК В ТЕКСТОВОМ ФАЙЛЕ	385

14.3	НЕЧЕТКИЙ ПОИСК В ТЕКСТОВОМ ФАЙЛЕ	388
14.4	ФОРМИРОВАНИЕ ОТЧЕТА	394

1 Краткая характеристика языка программирования C#

Современный объектно-ориентированный язык программирования общего назначения C# разработан компанией Microsoft для платформы .NET.

С его помощью можно разрабатывать консольные приложения, оконные приложения, веб-приложения, серверные API. Для обработки данных предназначены технологии LINQ и Entity Framework.

Первая версия C# появилась в начале 2000 годов и с тех пор активно развивается. Создателем языка C# является Андерс Хейлсберг, который до работы над языком C# был разработчиком компилятора с языка Паскаль и среды разработки Delphi. Это безусловно сказалось на C#, который, не смотря на синтаксис, унаследованный от C++, впитал в себя лучшие структурные черты Паскаля.

2 Создание консольного приложения на языке C#

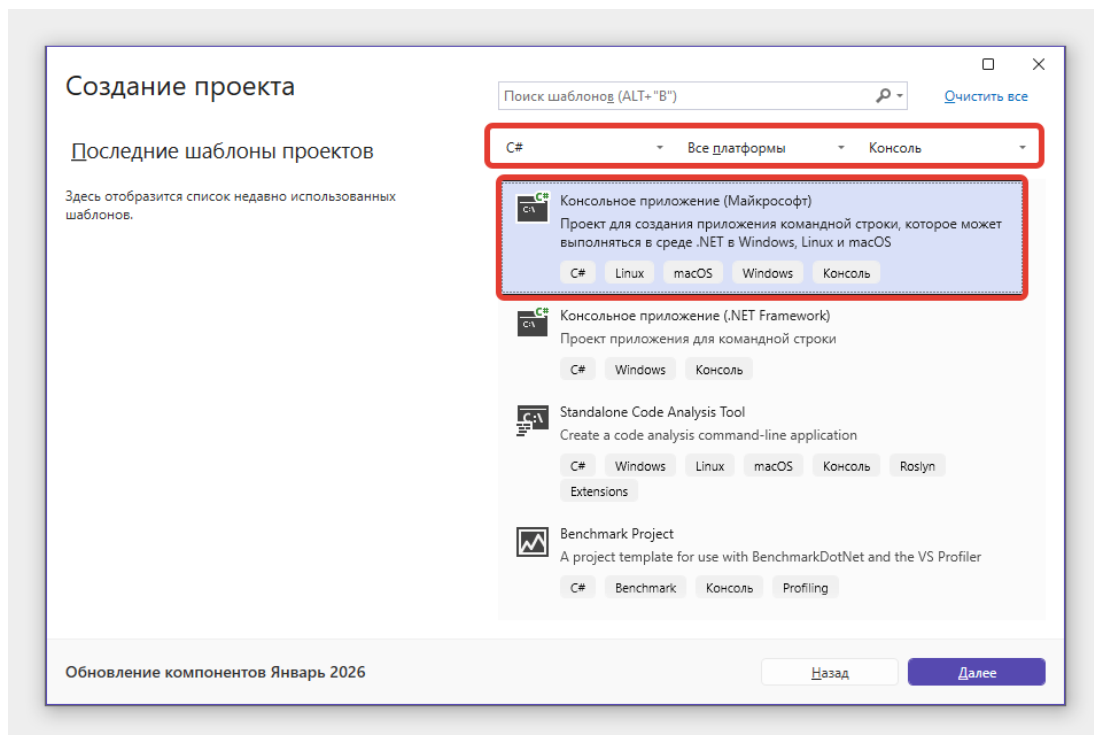


Рис. 1. Окно создания проекта.

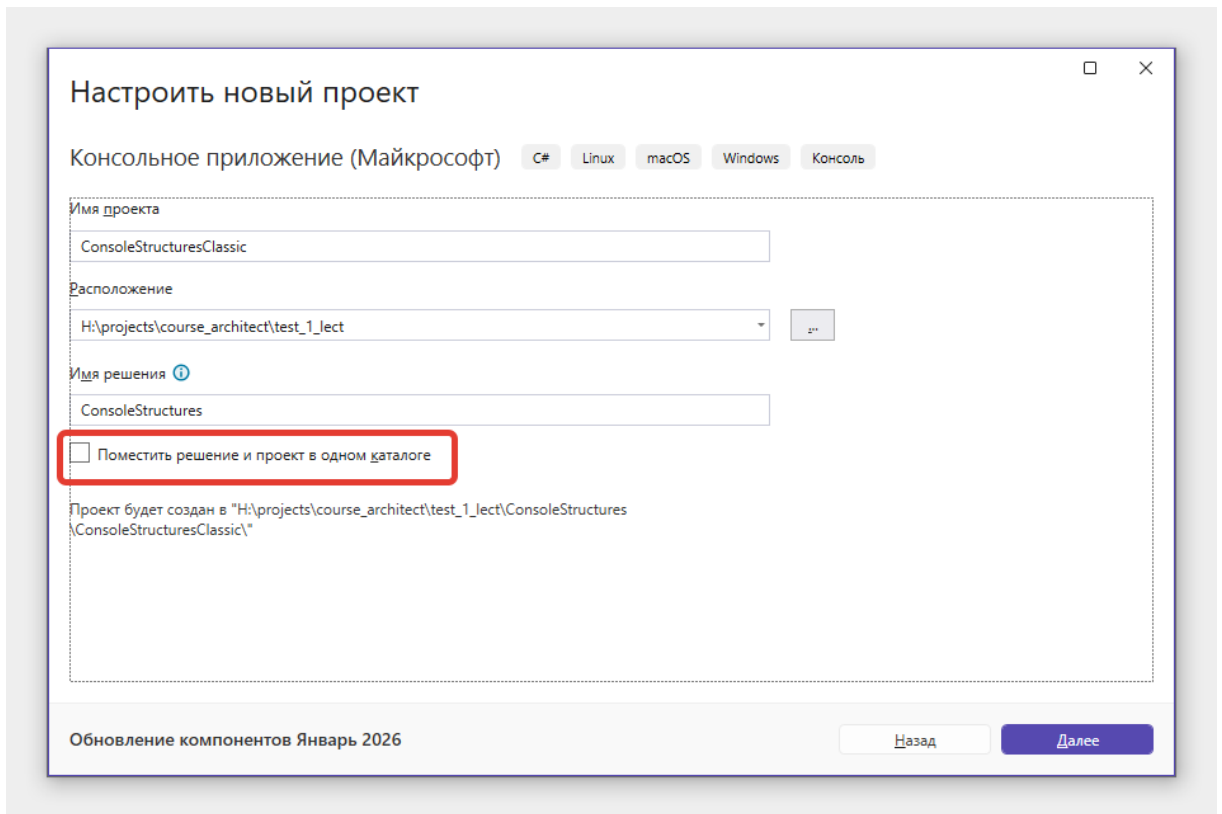


Рис. 2. Окно настройки проекта.

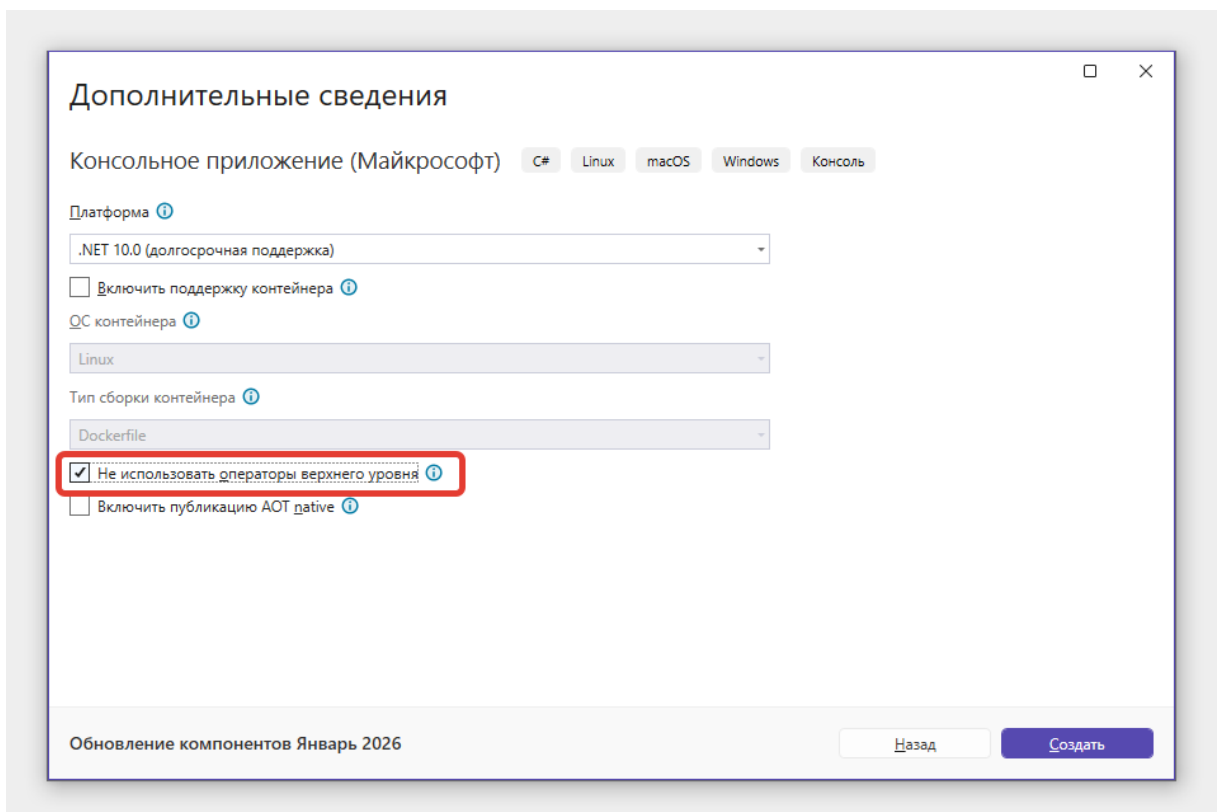


Рис. 3. Окно настройки проекта – дополнительные сведения.

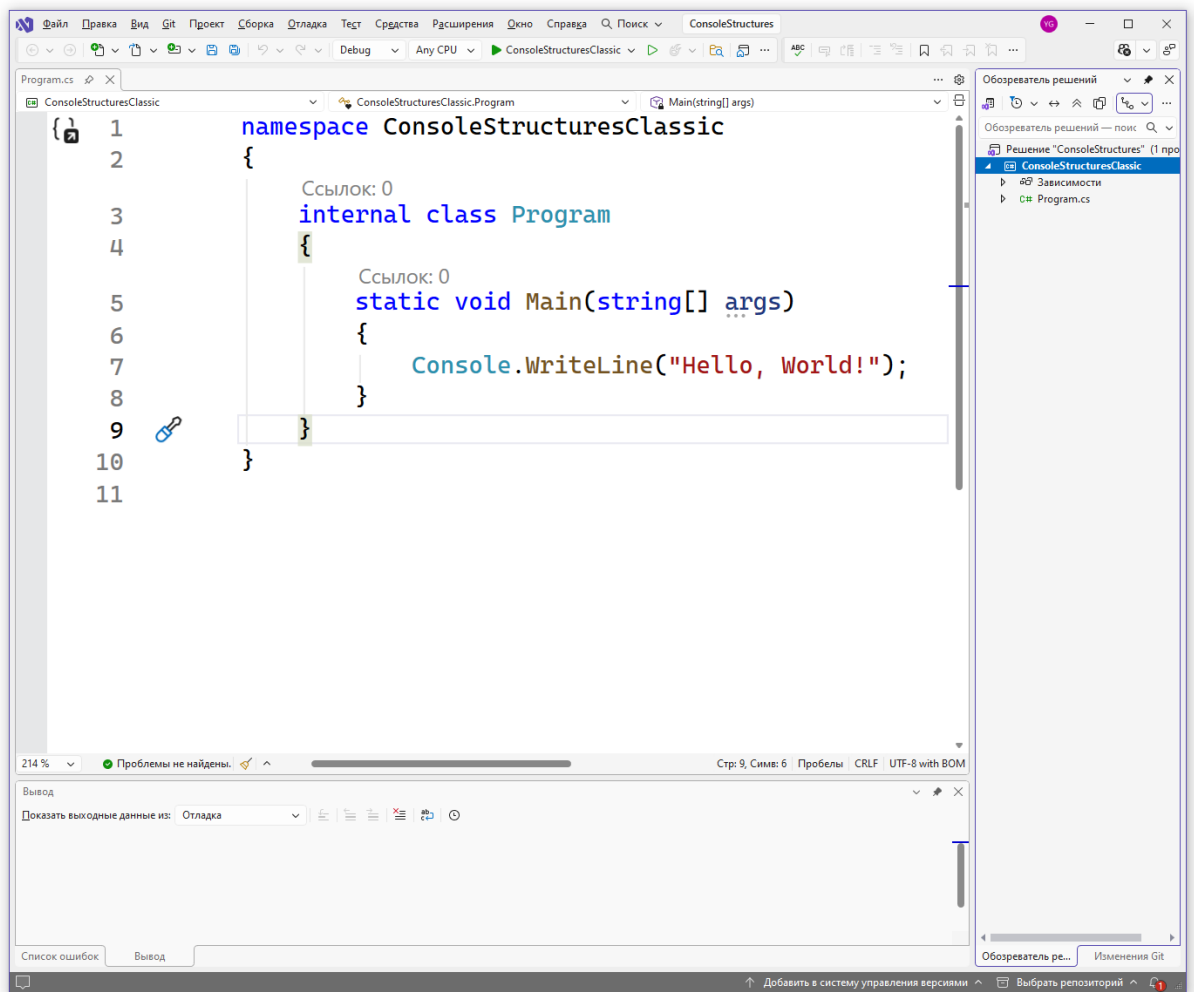


Рис. 4. Внешний вид с кодом «классического» проекта.

Запустим проект горячей клавишей F5.

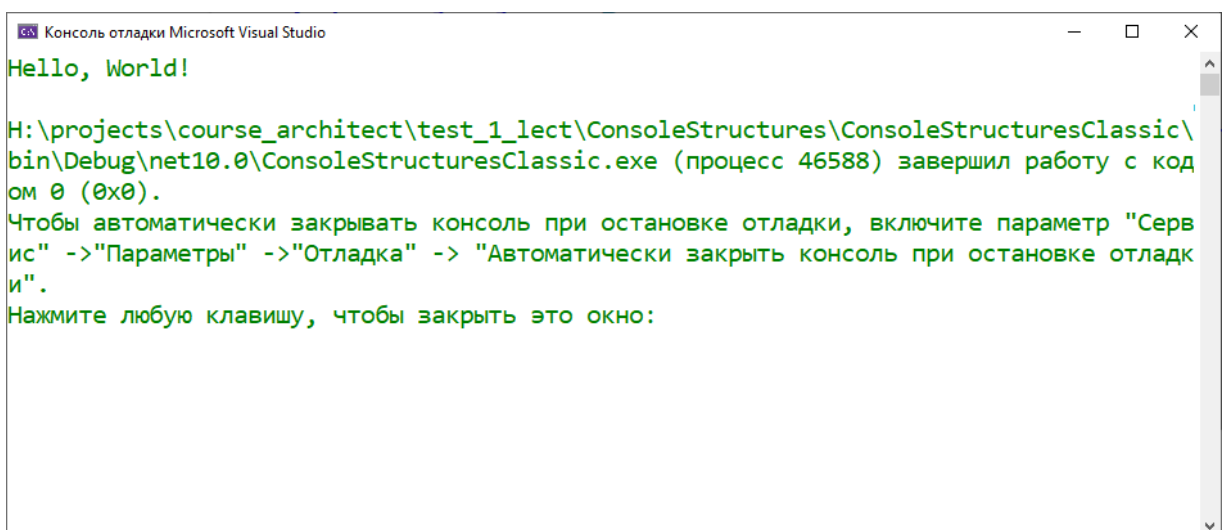


Рис. 5. Консоль с результатом запуска проекта.

Нажмем правую кнопку на решении, в контекстном меню выберем пункты «Добавить», «Создать в проект».

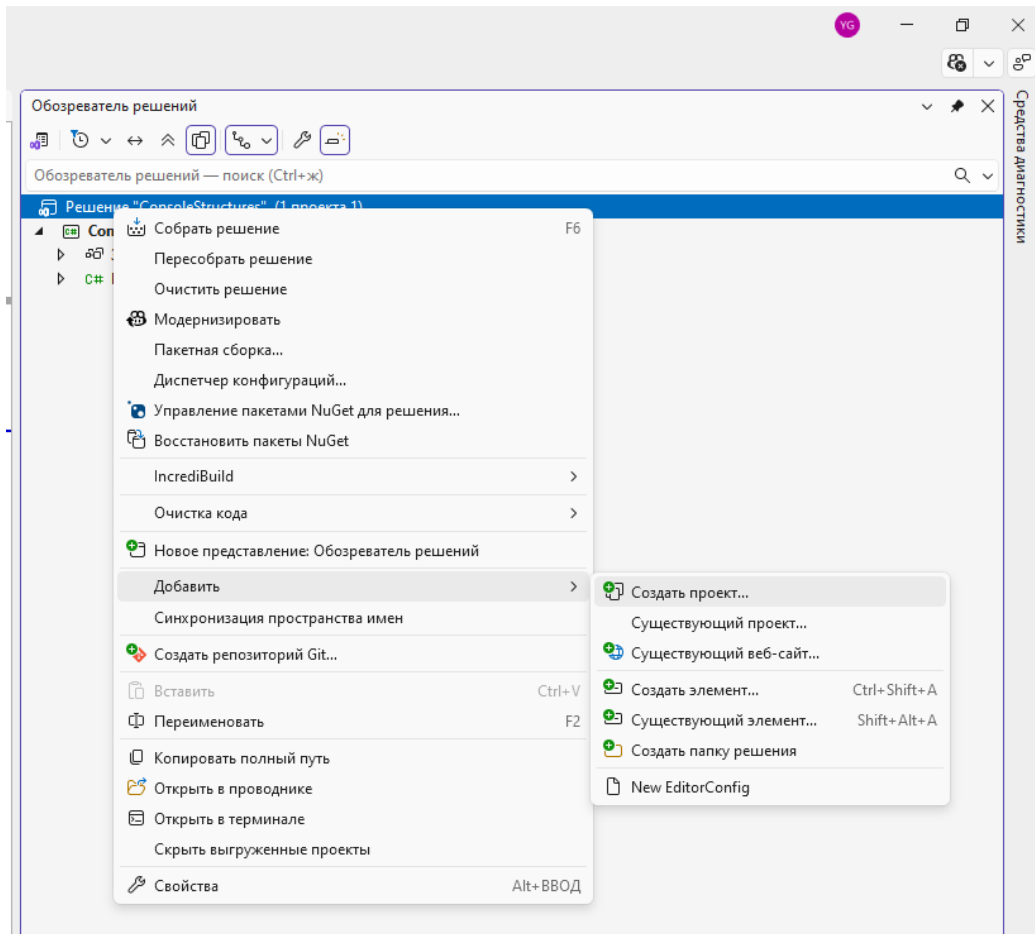


Рис. 6. Добавление проекта в решение.

Открывается окно выполнения проекта аналогично рис. 1, снова будем создавать консольный проект.

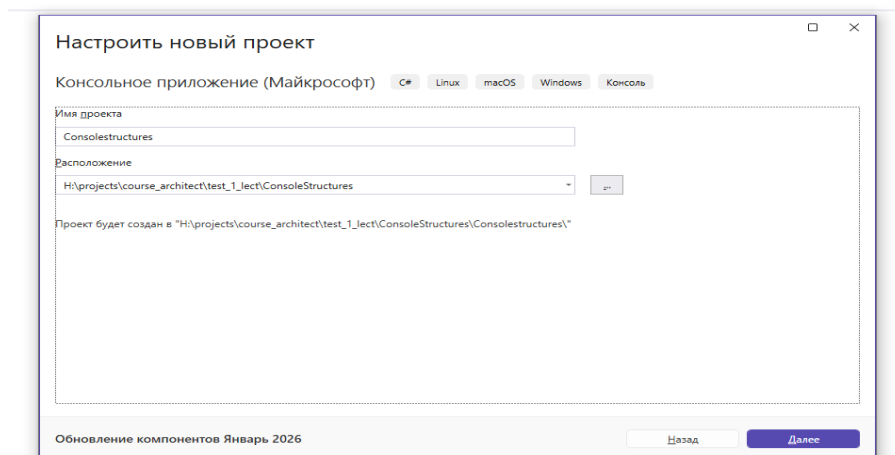


Рис. 7. Добавление проекта в решение – настройка проекта.

Обратите внимание, что в появившемся окне нет пункта про решение, так как оно уже создано.

В создаваемом проекте будем использовать «операторы верхнего уровня», флаг «Не использовать операторы верхнего уровня» должен быть выключен.

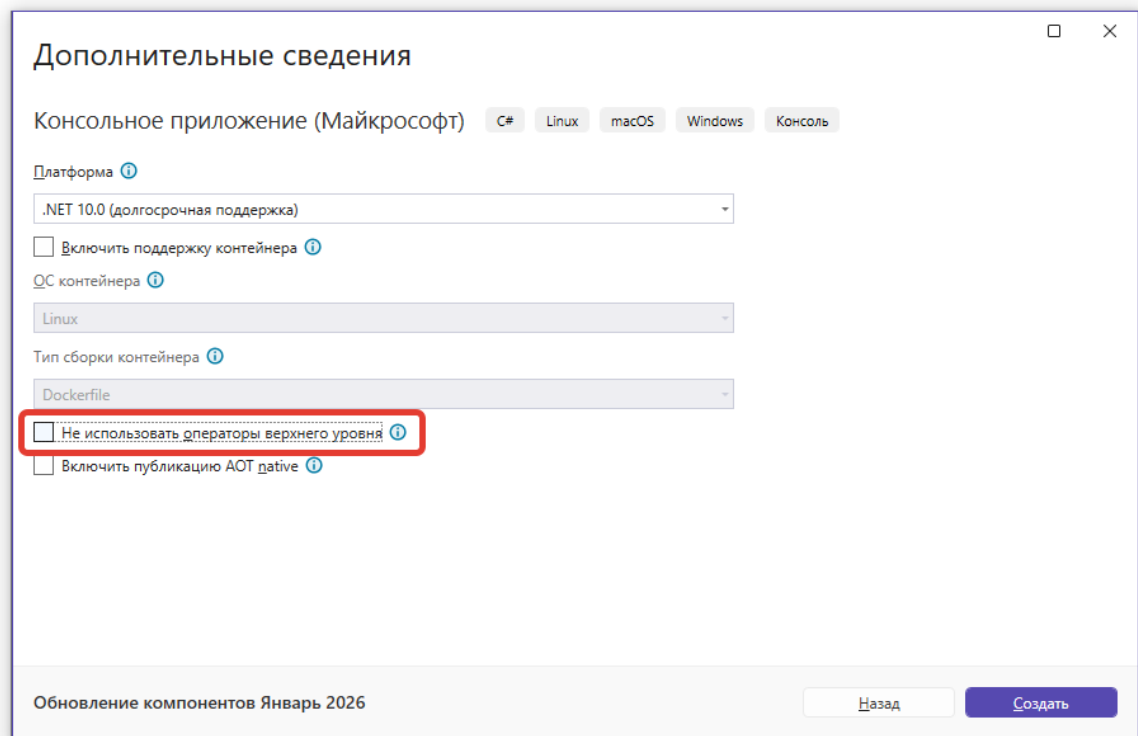


Рис. 8. Добавление проекта в решение – настройка проекта, доп. сведения.

Начиная с .NET 6 в консольном проекте по умолчанию используются операторы верхнего уровня (инструкции верхнего уровня) которые позволяют обходиться без описания класса и функции Main.

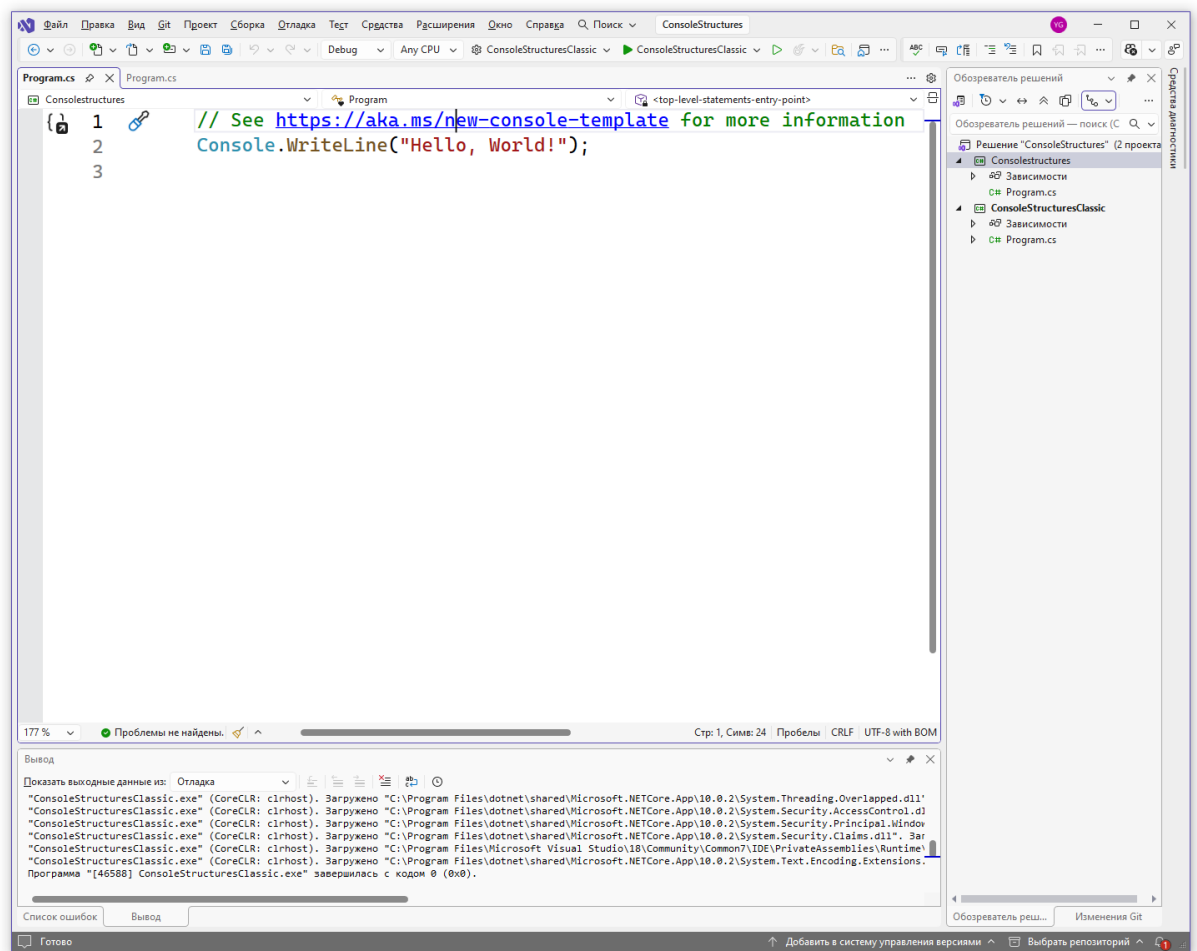


Рис. 9. Внешний вид проекта с инструкциями верхнего уровня.

Результат запуска проекта совпадает с рисунком 5.

Сделаем текущий проект активным запускаемым проектом. Для этого необходимо нажать правую кнопку на новом проекте, в контекстном меню выбрать пункт «Назначить в качестве запускаемого проекта». После этого новый проект будет отображаться жирным шрифтом в списке проектов.

В решении Visual Studio только один проект может быть назначен как запускаемый проект. Это текущий активный проект, который запускается при нажатии горячей клавиши F5.

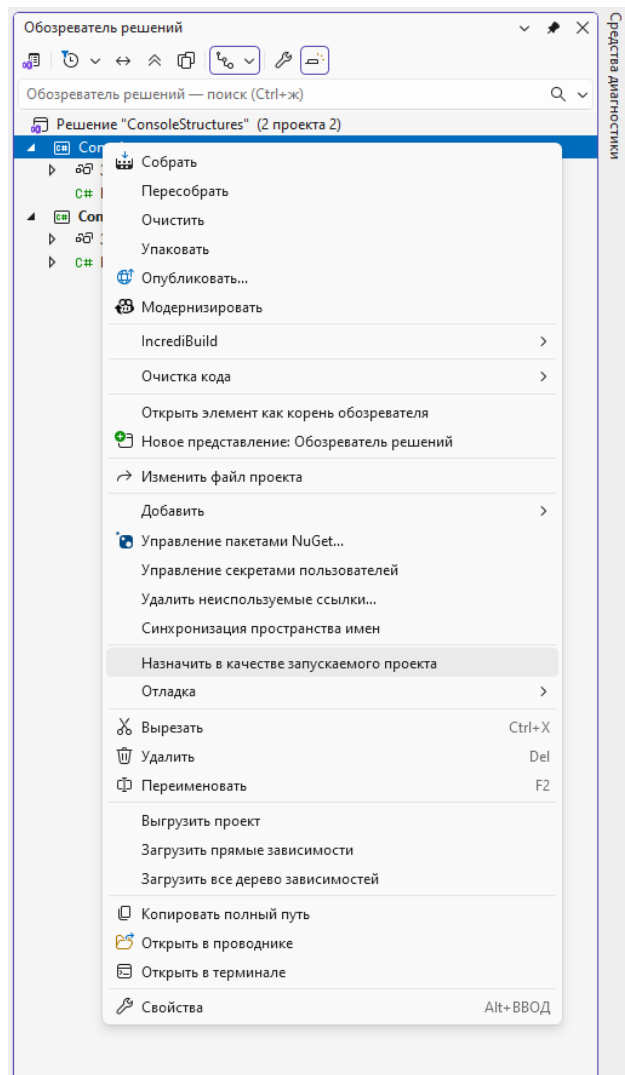


Рис. 10. Назначение запускаемого проекта (текущего активного проекта).

3 Базовые возможности языка С#, необходимые для создания консольных приложений

3.1 Консольный ввод-вывод

Средства консольного ввода-вывода в С# напоминают аналогичные средства в Java и очень уступают по возможностям консольному вводу-выводу в C++.

Ввод-вывод в консоль осуществляется посредством класса `Console`, у которого предусмотрены статические методы ввода-вывода данных:

- `Console.WriteLine` – вывод данных с переводом строки;
- `Console.Write` – вывод данных без перевода строки;
- `Console.Read` – чтение текущего символа;
- `Console.ReadKey` – чтение текущего символа или функциональной клавиши;
- `Console.ReadLine` – чтение строки до нажатия ввода.

У класса `Console` также есть методы для чтения и изменения цвета текста, размеров окна и другие.

Для ввода данных в консоли существует только метод `Console.ReadLine`, позволяющий ввести только строковый тип данных, но не позволяющий вводить числовые типы данных. Здесь язык `C#` уступает по возможностям консольной библиотеке `C++`. Поэтому в языке `C#` при разработке консольных приложений актуальным является преобразование строкового типа данных в другие типы.

Методы `Console.WriteLine` и `Console.Write` имеют большое количество перегрузок для вывода данных различных типов.

Также существует перегрузка для форматированного вывода, когда первым параметром указывается строка формата, а далее передается произвольное количество параметров, которые подставляются в строку формата.

Пример использования метода `WriteLine` со строкой формата:

```
int p1 = 2;
int p2 = 4;
Console.WriteLine("{0} умножить на {1} = {2}", p1, p2, p1*p2);
```

В консоль будет выведено:

```
2 умножить на 4 = 8
```

Фигурные скобки в строке формата означают ссылку на соответствующий параметр, причем параметры нумеруются с нуля. Таким образом, вместо `{0}` будет подставлена переменная `p1`, вместо `{1}` – переменная `p2`, вместо `{2}` – выражение `p1*p2`.

Пример, содержащий форматирование параметра:

```
double d3 = 1.12345678;
Console.WriteLine("{0} округленное до 3 разрядов = {0:F3}", d3);
```

В консоль будет выведено:

1,12345678 округленное до 3 разрядов = 1,123

Выражение {0:F3} означает, что нулевой параметр нужно вывести в виде числа с плавающей точкой, округлив до 3 знаков после разделителя разрядов.

ЗАДАНИЕ

Введите название страны и столицы страны и выведите в консоль, например «Москва – столица России!».

3.2 Строковая интерполяция

Строковая интерполяция (string interpolation) – это возможность упрощения синтаксиса, которая появилась в версии C# 6.

Строковая интерполяция позволяет указывать в строке шаблон для ее форматирования на основе переменных. Необходимо отметить, что похожие возможности есть в других языках программирования, в частности в Python и PHP.

Пример, использующий строковую интерполяцию:

```
string City = "Moscow";
string Country = "Russia";

//Старый способ вывода с помощью конкатенации строк
Console.WriteLine(City + ", " + Country);
//Старый способ вывода с помощью string.Format
Console.WriteLine(string.Format("{0}, {1}", City, Country));
//Новый способ вывода с помощью строковой интерполяции
Console.WriteLine($"{City}, {Country}");
```

Признаком использования строковой интерполяции является то, что при объявлении строки перед кавычками ставится символ доллара. Если в

такой строке указано выражение в фигурных скобках, то оно автоматически вычисляется.

Необходимо отметить, что строковая интерполяция в настоящее время является основным способом соединения строк при консольном выводе.

Результаты вывода в консоль:

Moscow, Russia

Moscow, Russia

Moscow, Russia

ЗАДАНИЕ

Введите название страны и столицы страны и выведите в консоль с использованием строковой интерполяции, например «Москва – столица России!».

3.3 Преобразования типов

Преобразование типов выполняется аналогично тому, как это происходит в C++ и Java с использованием оператора приведения типов – круглые скобки.

Пример преобразования переменной типа double в тип int:

```
double d1 = 123.45;
int i1 = (int)d1;
```

Очень часто, особенно при консольном вводе, необходимо преобразовать строковое представление числа (введенное с клавиатуры) в числовой формат. Это можно сделать тремя способами.

Первый состоит в том, что у каждого класса-типа существует статический метод Parse, преобразующий строку в числовой тип данных.

Пример преобразования типов с использованием метода Parse:

```
int i2 = int.Parse("123");
double d2 = double.Parse("123,45");
```

Метод Parse использует настройки локализации операционной системы, в частности для переменной типа double в качестве разделителя целой и дробной части используется запятая. Если строка не содержит корректного представления числа, метод Parse генерирует исключение FormatException (исключения подробнее рассмотрены ниже).

При втором способе вместо метода Parse используется TryParse. В случае неудачного преобразования данный метод не генерирует исключение, но возвращает логическое значение false. Возвращаемое число представляется выходным параметром (out-параметр).

Пример использования метода TryParse:

```
int i3;
bool result = int.TryParse("123", out i3);
```

В современных версиях C# можно объявить переменную при вызове функции:

```
bool result = int.TryParse("123", out int i3);
```

Третий способ заключается в применении класса Convert, который содержит большое количество статических методов и может преобразовать значения большинства базовых между собой.

Пример использования класса Convert:

```
int i4 = Convert.ToInt32("123");
```

Если же строка не содержит корректное представление числа, класс Convert, как и метод Parse, генерирует исключение FormatException.

ЗАДАНИЕ

Введите с клавиатуры целое число (оно будет введено как строка). Преобразуйте введенную строку к целому числу. С использованием строковых интерполяций выведите, например «Число 333 введено успешно» или «Введенная строка qwerty не может быть преобразована в число».

Если конвертация в целое число произведена успешно (используйте условный оператор из примера в следующем разделе), то с помощью метода Parse и класса Convert преобразуйте целое число в действительное число. Выведите полученные действительные числа с помощью строковой интерполяции.

3.4 Простые массивы, условные операторы и циклы

Для объявления массивов в C# используется синтаксис:

```
int[] array;
```

В языке C++ квадратные скобки должны ставить не после имени типа, а после имени переменной: `int array[]`. В Java допустимы оба варианта объявления.

Если необходимо присвоить начальные значения элементам массива, то форма объявления следующая:

```
int[] array = new int[3] {1, 2, 3};
```

Аналогично для строкового типа:

```
string[] strs =  
    new string[3] { "строка1", "строка2", "строка3" };
```

Пример условного оператора if:

```
if (str == "строка1")  
{  
    Console.WriteLine("if: str == \"строка1\"");  
}  
else  
{  
    Console.WriteLine("if: str != \"строка1\"");  
}
```

В качестве условия проверяется значение логического типа, который отсутствует в стандартном C++, но есть в Паскале, Java, C#.

Счетный цикл `for` такой же как в C++ и Java. В круглых скобках после `for` указываются три оператора – начальное присвоение значения переменной цикла; условие выхода из цикла; оператор, который выполняется при переходе к следующему шагу цикла. Операторы разделяются точкой с запятой.

Пример цикла `for`:

```
for (int i = 0; i < 3; i++)  
    Console.WriteLine(i);
```

В данном примере оператор вывода переменной `i` не вложен в фигурные скобки, использование скобок не является обязательным, так как это единственный оператор цикла. Если бы в цикле выполнялось несколько действий, то их необходимо было бы вложить в фигурные скобки.

ЗАДАНИЕ

Объявите в программе массив со значениями: 1,2,3,4,5,6,7,8 (ввод с клавиатуры не используется. Выведите на экран только четные элементы массива.

3.5 Задание для закрепления материала

Необходимо написать на C# программу в виде консольного приложения, которая решает биквадратное уравнение и сортирует корни. Программа выполняет следующие действия:

1. Позволяет ввести с клавиатуры три коэффициента – a, b, c.
2. Не нужно делать проверок на нулевые значения, некорректный ввод, предполагается, что пользователь вводит корректные значения для случая имеющего точно четыре корня.
3. Вычисляет корни по формуле:

$$x_{1,2,3,4} = \pm \sqrt{\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}}$$

4. Не нужно делать дополнительных проверок, предполагается, что дискриминант всегда больше нуля, и корней точно четыре.
5. Полученные корни необходимо поместить в массив или изначально сохранять корни как элементы массива.
6. Полученный массив корней необходимо отсортировать по возрастанию любым алгоритмом. Пожалуйста укажите в комментариях какой алгоритм сортировки Вы используете.
7. Отсортированный массив необходимо вывести на экран.

Решение:

```
using System;

// ===== ОСНОВНОЙ КОД ПРОГРАММЫ =====

double a, b, c;
double[] roots = new double[4];
double[] roots_bubble = new double[4];
double[] roots_selection = new double[4];
int n;
string[] input;

Console.WriteLine("Решение биквадратного уравнения:  $ax^4 + bx^2 + c = 0$ ");
Console.WriteLine();

// Ввод коэффициентов
Console.Write("Введите коэффициенты a, b, c через пробел: ");
input = Console.ReadLine().Split(' ');
a = double.Parse(input[0]);
b = double.Parse(input[1]);
c = double.Parse(input[2]);

Console.WriteLine();
Console.WriteLine("-----");
Console.WriteLine("Уравнение: {0:F2}x^4 + {1:F2}x^2 + {2:F2} = 0", a, b, c);
Console.WriteLine("-----");

// Решение уравнения
n = SolveBiquadratic(a, b, c, roots);

Console.WriteLine();
Console.WriteLine("КОРНИ ДО СОРТИРОВКИ:");
PrintArray(roots, n);

// Копируем массив для двух разных сортировок
CopyArray(roots, roots_bubble, n);
CopyArray(roots, roots_selection, n);

// Применяем обе сортировки
BubbleSort(roots_bubble, n);
SelectionSort(roots_selection, n);

// Выводим результаты
Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine("РЕЗУЛЬТАТЫ СОРТИРОВКИ:");
Console.WriteLine("=====");

Console.WriteLine();
Console.WriteLine("1. ПУЗЫРЬКОВАЯ СОРТИРОВКА (Bubble Sort):");
PrintArray(roots_bubble, n);

Console.WriteLine();
Console.WriteLine("2. СОРТИРОВКА ВЫБОРОМ (Selection Sort):");
PrintArray(roots_selection, n);

// Проверка идентичности результатов
Console.WriteLine();
Console.WriteLine("-----");
if (ArraysEqual(roots_bubble, roots_selection, n))
{
    Console.WriteLine("Оба метода дают ОДИНАКОВЫЙ результат");
}
}
```

```

else
{
    Console.WriteLine("ВНИМАНИЕ: Результаты РАЗЛИЧАЮТСЯ!");
}

Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine("ТЕСТОВЫЕ ПРИМЕРЫ:");
Console.WriteLine("=====");
Console.WriteLine("1) a=1, b=-5, c=4 → корни: -2, -1, 1, 2");
Console.WriteLine("2) a=1, b=-10, c=9 → корни: -3, -1, 1, 3");
Console.WriteLine("3) a=1, b=-13, c=36 → корни: -3, -2, 2, 3");
Console.WriteLine("4) a=1, b=-17, c=16 → корни: -4, -1, 1, 4");
Console.WriteLine("=====");

Console.WriteLine();
Console.WriteLine("Нажмите любую клавишу для выхода...");
Console.ReadKey();

// ===== ФУНКЦИИ =====

/* Решение биквадратного уравнения  $ax^4 + bx^2 + c = 0$ 
   Возвращает количество корней */
static int SolveBiquadratic(double a, double b, double c, double[] roots)
{
    // Замена  $y = x^2$ , получаем  $ay^2 + by + c = 0$ 
    double D = b * b - 4 * a * c;
    double y1 = (-b + Math.Sqrt(D)) / (2 * a);
    double y2 = (-b - Math.Sqrt(D)) / (2 * a);

    // Находим  $x = \pm\sqrt{y}$  для каждого y
    roots[0] = Math.Sqrt(y1);
    roots[1] = -Math.Sqrt(y1);
    roots[2] = Math.Sqrt(y2);
    roots[3] = -Math.Sqrt(y2);

    return 4;
}

/* Пузырьковая сортировка массива по возрастанию */
static void BubbleSort(double[] arr, int n)
{
    int i, j;
    double temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

/* Сортировка выбором – находим минимум и ставим на своё место */
static void SelectionSort(double[] arr, int n)
{
    int i, j, min_idx;

```

```

double temp;

// Для каждой позиции находим минимум в оставшейся части
for (i = 0; i < n - 1; i++)
{
    min_idx = i; // предполагаем, что минимум на позиции i

    // Ищем минимум среди оставшихся элементов
    for (j = i + 1; j < n; j++)
    {
        if (arr[j] < arr[min_idx])
        {
            min_idx = j;
        }
    }

    // Меняем местами найденный минимум с элементом на позиции i
    temp = arr[i];
    arr[i] = arr[min_idx];
    arr[min_idx] = temp;
}

/* Вывод массива */
static void PrintArray(double[] arr, int n)
{
    for (int i = 0; i < n; i++)
    {
        Console.WriteLine(" x{0} = {1,7:F3}", i + 1, arr[i]);
    }
}

/* Копирование массива */
static void CopyArray(double[] src, double[] dst, int n)
{
    for (int i = 0; i < n; i++)
    {
        dst[i] = src[i];
    }
}

/* Проверка идентичности массивов */
static bool ArraysEqual(double[] arr1, double[] arr2, int n)
{
    for (int i = 0; i < n; i++)
    {
        if (Math.Abs(arr1[i] - arr2[i]) > 0.0001)
        {
            return false;
        }
    }
    return true;
}

```

4 Основные типы данных языка C#

4.1 Организация типов данных в языке C#

Особенность типов данных языка C# заключается в том, что для всей среды исполнения .NET используется единая общая система типов – CTS (Common Type System), причем как в C#, так и в Visual Basic .NET и других языках .NET-платформы.

Все типы данных разделяются на типы-значения (value type) и ссылочные типы (reference type).

Типы-значения хранятся в области памяти, доступ к которой организован в виде стека. К ним относятся:

- базовые типы данных;
- перечисления (enum);
- структуры (struct).

Если переменная типа-значения передается как параметр в метод, то в стеке делается копия значения и передается в метод, а исходное значение не изменяется. В частности, такой порядок копирования значений удобен для примитивных типов.

Ссылочные типы хранятся в динамически распределяемой области памяти, которую принято называть кучей (heap). Для работы с ней используются ссылки, фактически являющиеся указателями. Ссылочными типами являются:

- классы;
- интерфейсы;
- массивы;
- делегаты.

Если переменная ссылочного типа передается в метод как параметр, то фактически передается указатель на эту переменную, при этом не делается копия значения (как в случае типа-значения). Поэтому если

значение переменной, переданное по ссылке, изменяется в методе, то исходное значение вследствие этого также меняется. Такой порядок копирования значений удобен, в частности, для передачи объектов классов.

В отличие от C++, в языке C# использование типов-значений и ссылочных типов практически ничем не различается, для обозначения ссылок не применяют специальные символы «&». Использовать указатели (которые как и в C++ обозначаются символом «*») можно в так называемом небезопасном (unsafe) режиме. Этот режим подходит для взаимодействия с аппаратными средствами, оптимизации производительности, в обычных приложениях применять unsafe-режим категорически не рекомендуется. В данном издании unsafe-режим не рассматривается.

Структура – это механизм языка, который представляет собой аналог класса, но реализован на основе стека. Синтаксически структуры и классы в программе на C# мало различаются. Однако в некоторых случаях использование структур вместо классов позволяет повысить производительность приложения, иногда очень значительно. Например, массив структур хранится в стеке последовательно, обработка такого фрагмента данных не требует дополнительных переходов. Массив объектов классов хранится в виде массива указателей, при их обработке необходимо выполнить дополнительные переходы по указателям, что может существенно увеличить время обработки данных. В современном варианте языка C# структуры в основном являются механизмом оптимизации производительности приложений.

4.2 Базовые типы данных

Базовые типы данных являются типами-значениями и хранятся в стеке.

У каждого типа есть полное наименование, принятое в CTS, а также краткое, используемое в C#. Допустимы оба варианта наименований, но для удобства разработчики обычно предпочитают краткие наименования.

Целочисленные типы данных представлены в таблице 1.

Таблица 1

Целочисленные типы данных

Наименование в C#	Наименование в CTS	Описание
sbyte	System.SByte	Целое 8-битное число со знаком
short	System.Int16	Целое 16-битное число со знаком
int	System.Int32	Целое 32-битное число со знаком
long	System.Int64	Целое 64-битное число со знаком
byte	System.Byte	Целое 8-битное число без знака
ushort	System.UInt16	Целое 16-битное число без знака
uint	System.UInt32	Целое 32-битное число без знака
ulong	System.UInt64	Целое 64-битное число без знака

Типы данных с плавающей точкой представлены в таблице 2.

Таблица 2

Типы данных с плавающей точкой

Наименование в C#	Наименование в CTS	Описание
float	System.Single	32-битное число с плавающей точкой одинарной точности, около 7 значащих цифр после запятой
double	System.Double	64-битное число с плавающей точкой двойной точности, около 15 значащих цифр после запятой
decimal	System.Decimal	128-битное число с плавающей

		точкой повышенной точности, около 28 значащих цифр после запятой
--	--	--

Внутреннее представление типа decimal отличается от внутреннего представления типов float и double.

Описание логического типа представлено в таблице 3.

Таблица 3

Логический тип данных

Наименование в C#	Наименование в CTS	Описание
bool	System.Boolean	Может принимать значения true или false.

В условных операторах C# и Java, как и в Паскале используется логический тип. В классических C и C++ он отсутствует, его роль выполняет целочисленный тип, но в некоторых современных диалектах C++ логический тип применяется.

Описание символьного и строкового типов данных представлено в таблице 4.

Таблица 4

Символьный и строковый типы данных

Наименование в C#	Наименование в CTS	Описание
char	System.Char	Символ Unicode
string	System.String	Строка символов Unicode

Тип string является не типом-значением, а ссылочным типом.

В языке C# ограничителем для символа служит одинарный апостроф, а для строки двойная кавычка. Пример объявления символа и строки:

```
char c = '1';
```



```
string str = "строка";
```

Корневой тип в CTS – тип `System.Object` (в C# – `object`). От него наследуются все ссылочные типы и типы-значения. Сам тип `object` является ссылочным.

Большинство базовых типов являются типами-значениями, но типы `string` и `object` стали исключением из этого правила.

5 Основные конструкции программирования языка C#

5.1 Пространства имен и сборки

Программа на языке C# начинается с операторов `using`, каждый из которых указывает пространство имен для библиотечных классов. Любой класс в языке C# должен быть объявлен в каком-либо пространстве имен с использованием оператора `namespace`.

Пространства имен представляют собой древовидную структуру. В каждой ее ветви содержатся вложенные классы и вложенные пространства имен. Если с помощью оператора `using` подключаются классы какого-либо пространства имен, например

```
using System;
```

то классы вложенных пространств имен при этом автоматически не подключаются. Поэтому их необходимо подключать отдельными директивами:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

Пространства имен представляют собой логическую структуру для систематизации классов. Выясним, как эта структура соотносится с откомпилированными классами.

Откомпилированный бинарный код для платформы .NET хранится в файлах сборок (assembly). Файл может иметь расширение .dll или .exe по аналогии с библиотеками и исполняемыми файлами ОС Windows. Но данные файлы содержат бинарный код, выполняющийся только на платформе .NET. Если же она не установлена, то ОС Windows не запустит такой исполняемый файл.

Файлы сборок следует подключать в разделе References проекта (рис. 11). В русской версии Visual Studio раздел References называется Ссылки. При нажатии правой кнопкой мыши на пункт References открывается диалог по добавлению новых сборок.

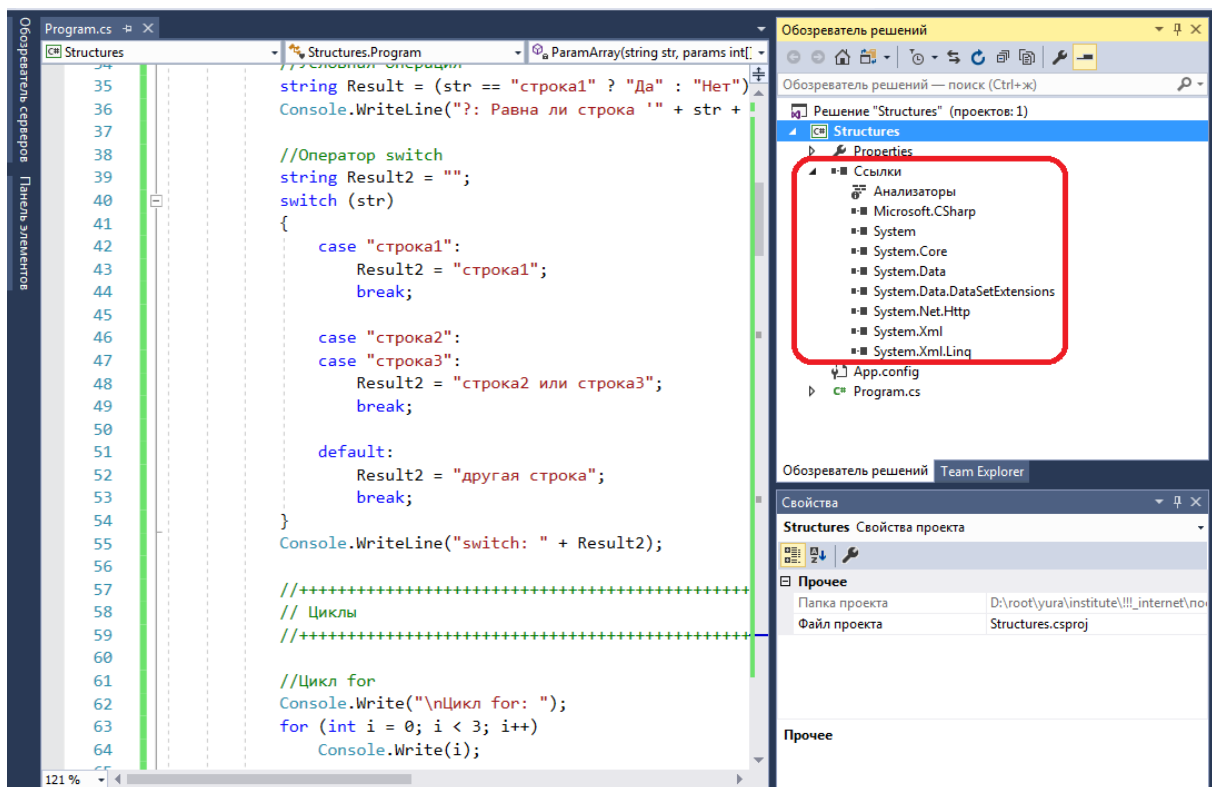


Рис. 11. Файлы сборок.

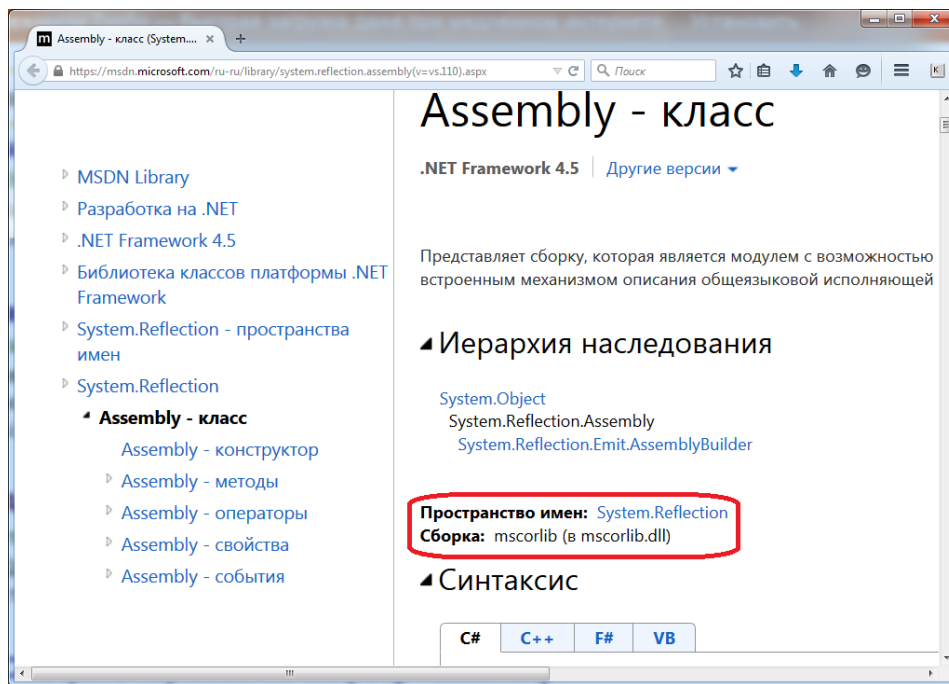


Рис. 12. Указание пространства имен и сборки в справочной системе.

Из рисунка 11 видно, что раздел References обычно содержит те же значения что и операторы using. Однако это принципиально разная информация. В разделе References находятся имена физических файлов сборок – файлов, которые содержат бинарный код, присоединяемый к проекту. Секция using ссылается на логическое название в дереве пространства имен.

При этом возможна ситуация, когда классы из одного и того же пространства имен находятся в разных сборках, и наоборот, одна сборка содержит классы из разных пространств имен. Поэтому в справочной системе Microsoft для каждого класса указано как имя сборки, так и пространство имен (рис 12).

Стоит отметить, что концепция пространства имен – специфика Microsoft.

В классическом языке C++ пространства имен отсутствуют, вместо них применяется подключение заголовочных файлов. Однако в версии языка C++, предлагаемой Microsoft, используется такой же механизм пространств имен, как и в C#.

В Java существует концепция похожая на пространства имен – пакеты (package). По аналогии с пространством имен каждый класс может быть включен в пакет. Но в данной концепции существуют ограничения – пакет является каталогом файловой системы, в который вложены файлы классов. Если имя пакета составное (содержит точку), то это предполагает вложенность соответствующих каталогов. Один файл в Java не может содержать более одного класса, следовательно, файл-класс однозначно соответствует пакету-каталогу. Пространства имен в языке C# – более гибкий механизм, однако сторонники Java полагают, что подобная жесткость позволяет избежать ошибок и несоответствий, встречающихся в пространствах имен.

5.2 Основная программа и параметры командной строки

В тексте программы (если мы не используем инструкции верхнего уровня) приведены конструкции:

```
namespace Structures
{
    internal class Program
    {
        ...
    }
}
```

Команда namespace объявляет пространство имен, а class – класс. Модификатор internal указывает что класс виден только в рамках текущей сборки. Команда namespace может содержать несколько классов. Чтобы сослаться на класс Program следует применить директиву:

```
using Structures;
```

Основной исполняемый метод консольного приложения – метод Main:

```
static void Main(string[] args)
{
    ...
}
```

Строковый массив параметров метода `args` содержит параметры (аргументы) командной строки, которые могут быть заданы при вызове консольного приложения. В Visual Studio в целях отладки данные параметры можно указать в меню «Отладка», «Свойства отладки для проекта ...».

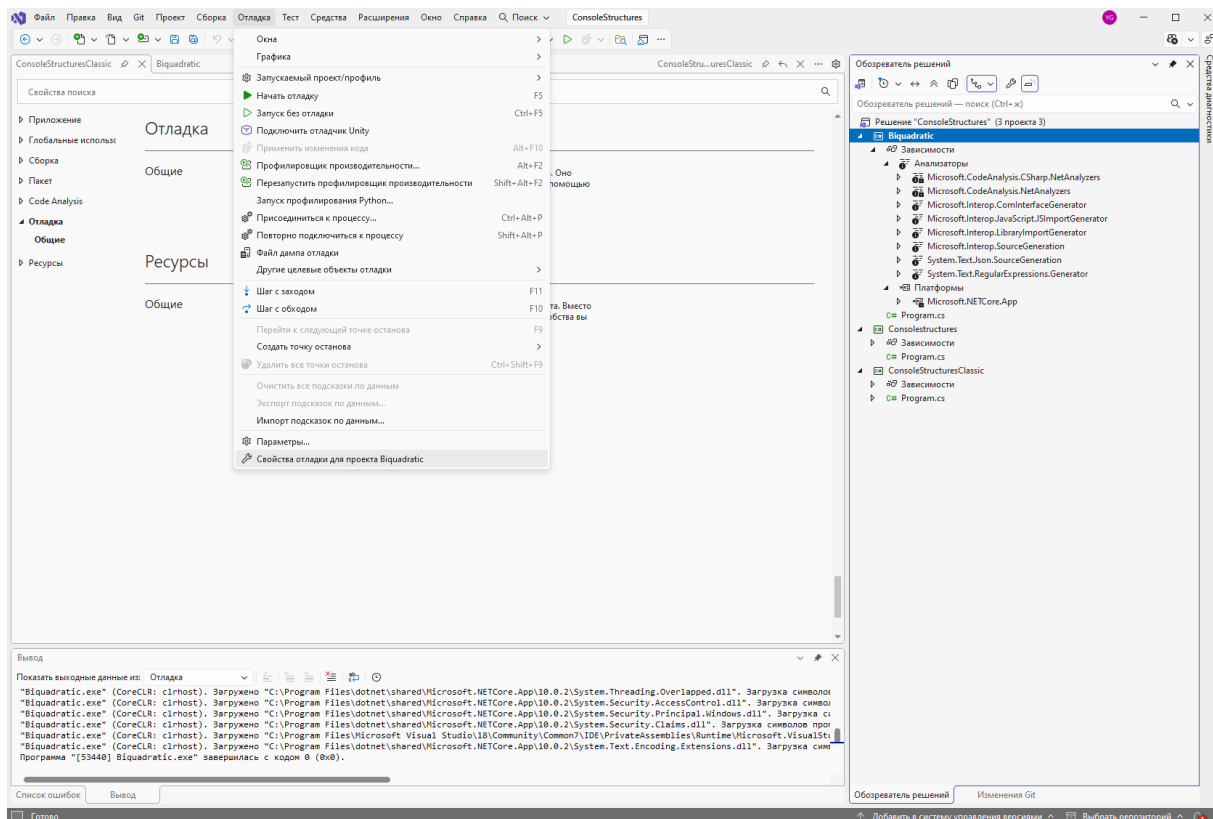


Рис. 13. Меню свойств отладки.

При выборе пункта меню открывается окно «Профили запуска», в котором есть поле для аргументов командной строки, их можно указать через пробел.

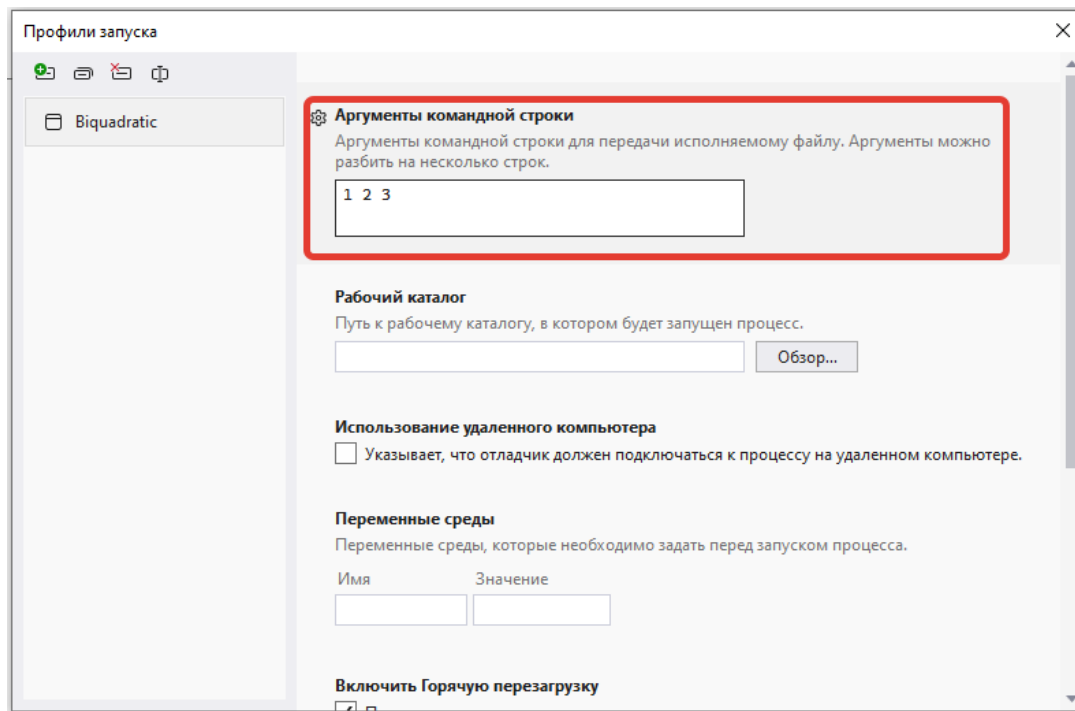


Рис. 14. Окно «Профили запуска».

Следующий фрагмент консольного приложения выводит параметры командной строки:

```
namespace ConsoleStructuresClassic
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Command line args:");
            foreach(string arg in args)
            {
                Console.WriteLine(arg);
            }
        }
    }
}
```

Результаты вывода в консоль:

Command line args:

1
2
3

Развернутый шаблон консольного приложения в настоящее время используется редко. В современных проектах C# используется более компактный код, можно выделить два варианта:

1. Начиная с C# 10 (.NET 6) используется подход file-scoped namespaces (пространства имен с областью действия на файл).

Пример до C# 10:

```
using System;
namespace Structures
{
    internal class FileName
    {
        // код
    }
}
```

Пример начиная с C# 10:

```
using System;

namespace Structures;

internal class FileName
{
    // код
}
```

2. Начиная с C# 9.0 (.NET 5) можно писать код основной консольной программы без объявления класса и пространства имен. Такой шаблон используется по умолчанию в современных версиях Visual Studio.

Пример до C# 9:

```
using System;

namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

```
}
}
```

Пример начиная с C# 9:

```
using System;

Console.WriteLine("Hello, World!");
```

5.3 XML-комментарии

При объявлении классов, методов и других структур возможно задать комментарии к ним в виде XML-тэгов.

Такие комментарии сохраняются в откомпилированной сборке и используются Visual Studio для работы технологии дополнения кода IntelliSense.

Пример метода с XML-комментариями:

```
/// <summary>
/// Метод сложения двух целых чисел
/// </summary>
/// <param name="p1">Первое число</param>
/// <param name="p2">Второе число</param>
/// <returns>Результат сложения</returns>
static int Sum(int p1, int p2)
{
    return p1 + p2;
}
```

Перед XML-комментарием ставится три прямых слеша, далее указывается соответствующий тэг XML. Существует довольно большое количество XML-тэгов комментариев, но наиболее часто используются три из них:

- `summary` – краткое описание;
- `param` – описание входного параметра;
- `returns` – описание возвращаемого значения.

При работе в Visual Studio не нужно вводить полную структуру XML-комментариев. Чтобы добавить блок XML-комментариев, необходимо установить курсор на строку кода перед объявлением функции (класса и т.д.) и три раза набрать прямой слеш «/». После этого автоматически

добавляется заголовок XML-комментария. Visual Studio автоматически определяет, для какой структуры добавляется XML-комментарий, и генерирует набор тэгов, подходящих для данного случая.

Пример автоматически сгенерированных XML-комментариев для метода:

```

/// <summary>
///
/// </summary>
/// <param name="p1"></param>
/// <param name="p2"></param>
/// <returns></returns>
static int Sum(int p1, int p2)
{
    return p1 + p2;
}

```

Информация о наборе параметров при генерации комментария формируется автоматически, однако в случае изменения параметров после добавления XML-комментария невозможна повторная генерация комментария, информация о новых параметрах должна быть добавлена в XML-комментарий вручную.

<pre> /// <summary> /// Функция сложения двух целых чисел /// </summary> /// <param name="p1">Первое число</param> /// <param name="p2">Второе число</param> /// <returns>Сумма двух параметров</returns> static int Sum(int p1, int p2) { return p1 + p2; } static void Main(string[] args) { Sum() } </pre>	<pre> /// <summary> /// Функция сложения двух целых чисел /// </summary> /// <param name="p1">Первое число</param> /// <param name="p2">Второе число</param> /// <returns>Сумма двух параметров</returns> static int Sum(int p1, int p2) { return p1 + p2; } static void Main(string[] args) { Sum(1,) } </pre>
--	---

Рис. 15. Использование XML-комментариев при вызове функции Sum.

При вызове функции Sum на основе XML-комментариев будет автоматически сгенерирована подсказка, которая выводится с помощью IntelliSense (рис. 4).

Использование механизма XML-комментариев позволяет разработчику создавать хорошо документируемый код, при этом документация автоматически используется механизмом IntelliSense при

вызове кода. Поэтому использование XML-комментариев чрезвычайно желательно при разработке проектов.

5.4 Вызов методов, передача параметров и возврат значений

Вызов методов происходит также как и в языках C++ и Java. Метод вызывается по имени, параметры передаются в круглых скобках после его имени. При объявлении метода указываются формальные параметры, при вызове в них выполняется подстановка фактических параметров, с которыми метод осуществляет работу при данном вызове.

В языке C# есть особенность, связанная с передачей параметров – ключевые слова `ref` и `out`.

Если перед параметром указывается ключевое слово `ref`, то значение передается по ссылке, то есть передается ссылка на параметр. Если параметр изменяется в методе, то эти изменения сохраняются в вызывающем методе.

При этом необходимо учитывать, что все значения ссылочных типов (объекты классов) автоматически передаются по ссылке даже без указания ключевого слова `ref`, их изменения в методе автоматически сохраняются.

Таким образом, ключевое слово `ref` актуально прежде всего для типов-значений. Оно является аналогом передачи параметра по указателю в языке C++, хотя в современных вариантах C++ также используется понятие ссылки.

Если перед параметром указывается ключевое слово `out`, то значение параметра обязательно должно быть инициализировано в вызываемом методе, что проверяется на этапе компиляции. До передачи в вызываемый метод значение переменной может быть не инициализировано. Инициализированное значение параметра становится доступно в вызывающем методе.

Ключевые слова `ref` и `out` должны быть указаны и при объявлении параметров в методе, и при вызове метода. Их указание при вызове метода, очевидно, излишне для компилятора, которому достаточно информации при объявлении метода. Однако это позволяет программисту избежать ошибок, связанных с незапланированным изменением значений параметров в методе.

Пример вызова методов:

```
string RefTest = «Значение до вызова функций»;

ParamByVal(RefTest);
Console.WriteLine(«\nВызов функции ParamByVal. Значение переменной:
« + RefTest);

ParamByRef(ref RefTest);
Console.WriteLine(«Вызов функции ParamByRef. Значение переменной: «
+ RefTest);

int x = 2, x2, x3;
ParamOut(x, out x2, out x3);
Console.WriteLine(«Вызов функции ParamOut. X={0}, x^2={1}, x^3={2}»,
x, x2, x3);
```

Пример объявления методов:

```
/// <summary>
/// Передача параметра по значению
/// </summary>
static void ParamByVal(string param)
{
    param = «Это значение НЕ будет передано в вызывающую функцию»;
}

/// <summary>
/// Передача параметра по ссылке
/// </summary>
static void ParamByRef(ref string param)
{
    param = «Это значение будет передано в вызывающую функцию»;
}

/// <summary>
/// Выходные параметры объявляются с помощью out
/// </summary>
static void ParamOut(int x, out int x2, out int x3)
{
    x2 = x * x;
```

```

    x3 = x * x * x;
}

```

В версии языка C# 7 реализовано синтаксическое усовершенствование, которое позволяет объявлять out-параметры непосредственно при вызове функции. Ранее для передачи out-параметров использовался следующий синтаксис:

```

int i;
OutFunction(out i);

```

Теперь можно использовать упрощенную конструкцию:

```

OutFunction(out int i);

```

Использование упрощенной конструкции позволяет существенно сократить код при большом количестве out-параметров.

Если какой-то out-параметр не используется, то его можно заменить на конструкцию discard (_):

```

OutFunction(out int i, out string j, out float j);
// результаты параметров j и k не нужны при вызове
OutFunction(out int i, out string _, out float _);

```

В метод может быть передано переменное количество параметров, для этого при объявлении параметра метода используется ключевое слово params.

Пример вызова метода с переменным количеством параметров:

```

ParamArray("Вывод параметров: ", 1, 2, 333);

```

Пример объявления метода с переменным количеством параметров:

```

/// <summary>
/// Переменное количество параметров задается с помощью params
/// </summary>
static void ParamArray(string str, params int[] ArrayParams)
{
    Console.Write(str);
    foreach (int i in ArrayParams)
        Console.Write(" {0} ", i);
}

```

Параметр с ключевым словом params должен быть объявлен последним в списке параметров.

Для возврата значений из методов, так же как и в языках C++ и Java, используется ключевое слово return.

5.5 Условные операторы и операторы сопоставления с образцом

Пример условного оператора if:

```
if (str == "строка1")
{
    Console.WriteLine("if: str == \"строка1\"");
}
else
{
    Console.WriteLine("if: str != \"строка1\"");
}
```

В качестве условия проверяется значение логического типа, который отсутствует в стандартном C++, но есть в Паскале, Java, C#.

Условный оператор вопрос-двоеточие выполняется аналогично тому, что в C++ и Java. До знака вопроса указывается логическое выражение. Если оно истинно, то выполняется код от вопроса до двоеточия, если ложно – код после двоеточия.

Пример оператора вопрос-двоеточие:

```
string Result = (str == "строка1" ? "Да" : "Нет");
```

Оператор switch выполняется аналогично тем, что в C++ и Java.

Пример оператора switch:

```
switch (str)
{
    case "строка1":
        Result2 = "строка1";
        break;

    case "строка2":
    case "строка3":
        Result2 = "строка2 или строка3";
        break;

    default:
        Result2 = "другая строка";
        break;
}
```

В круглых скобках после switch указывается проверяемое выражение, а после case – возможные варианты проверяемых значений.

Если значения после `switch` и `case` совпадают, то выполняются операторы, указанные в `case` до первого оператора `break`. Если оператор `break` не указан, то выполняются операторы следующей по порядку секции `case`, поэтому обычно каждую секцию `case` завершает оператор `break`. Если ни один оператор `case` не удовлетворяет условию, то выполняются операторы секции `default`.

Операторы сопоставления с образцом (`pattern matching`) – это классическая особенность языков программирования, использующих функциональный подход, таких как F#, Scala, Erlang, Haskell.

В языке C# данная возможность появилась, начиная с версии 7. Сопоставление с образцом похоже на условные операторы и поэтому построено на их основе.

Пусть дан массив объектов, который содержит строки и целые числа:

```
object[] array1 = { 1, "строка 1", 2, "строка 2", 3 };
```

Для того чтобы расширить условный оператор до оператора сопоставления с образцом в язык C# была добавлена конструкция `is`.

Пример сопоставления с образцом на основе условного оператора `if`:

```
foreach(object obj in array1)
{
    if(obj is int i1)
    {
        Console.WriteLine("Число -> " + i1.ToString());
    }
    else if (obj is string s1)
    {
        Console.WriteLine("Строка -> " + s1);
    }
}
```

В данном примере в цикле `foreach` по очереди перебираются элементы массива.

Если элемент массива соответствует целому числу «`obj is int i1`», то он автоматически приводится к целому типу и помещается в переменную `i1`. Далее с ним можно выполнять необходимые действия, в примере это вывод в консоль.

Если элемент массива соответствует строке «obj is string s1», то он автоматически приводится к строке и помещается в переменную s1.

Результаты вывода в консоль:

Число -> 1

Строка -> строка 1

Число -> 2

Строка -> строка 2

Число -> 3

Вторым вариантом сопоставления с образцом в C# является использование оператора switch.

Пример сопоставления с образцом на основе оператора switch:

```
foreach (object obj in array1)
{
    switch(obj)
    {
        case string s1:
            Console.WriteLine("Строка -> " + s1);
            break;
        case int i1 when i1 > 2:
            Console.WriteLine("Число большее 2 -> " +
i1.ToString());
            break;
        case int i1:
            Console.WriteLine("Число -> " + i1.ToString());
            break;
    }
}
```

В этом случае в каждом операторе case указывается проверяемый тип и переменная, в которую сохраняется результат приведения типа.

С использованием ключевого слова when можно задавать так называемые «охранные выражения» (guards), которые накладывают дополнительные ограничения на проверку. В данном примере возможность приведения к целому типу проверяется дважды, случай $i1 > 2$ рассматривается отдельно и возникновение такого случая проверяется с помощью охранного выражения.

Результаты вывода в консоль:

Число -> 1

Строка -> строка 1

Число -> 2

Строка -> строка 2

Число больше 2 -> 3

Современный стиль сопоставления с образцом это switch expression (конструкция появилась в C# 8). Данная конструкция очень напоминает классические конструкции pattern matching из функциональных языков программирования.

```
// Современный стиль (switch expression)
// появился в C# 8
string result21 = str switch
{
    "строка1" => "строка1",
    "строка2" or "строка3" => "строка2 или строка3", // логический
or
    _ => "другая строка" // _ = default
};

// Более сложный пример с типами
int? obj = 3;
string description = obj switch
{
    int i => $"Целое число: {i}",
    null => "Null",
};

// С условиями (guards)
var number = 5;
string category = number switch
{
    < 0 => "Отрицательное",
    0 => "Ноль",
    > 0 and <= 10 => "Малое положительное",
    > 10 => "Большое положительное"
};
Console.WriteLine($"{result21} {obj} {number}");
```

Результаты вывода в консоль:

строка1 3 5

ЗАДАНИЕ

Дан массив:

```
object[] data = { 15, "Hello", -5, null,  
                  "World", 42, 0, "C#", -10 };
```

Напишите программу на C#, которая анализирует массив объектов различных типов. Требуется реализовать 3 метода:

- **Метод AnalyzeWithIf** - используя условный оператор `if` и конструкцию `is`:
 1. Для положительных чисел вывести: "Положительное число: {значение}"
 2. Для нуля вывести: "Ноль: {значение}"
 3. Для отрицательных чисел вывести: "Отрицательное число: {значение}"
 4. Для строк вывести: "Строка: {значение}"
 5. Для `null` вывести: "Пустое значение"
- **Метод AnalyzeWithSwitch** - используя традиционный оператор `switch` с `pattern matching` и `guard` условиями (`when`)
- **Метод GetCategory** - используя современный `switch expression`, который возвращает строку с категорией элемента

Ожидаемый вывод программы должен показывать результаты работы всех трёх методов.

РЕШЕНИЕ:

```

static void example_conditions()
{
    object[] data = { 15, "Hello", -5, null, "World", 42, 0, "C#", -10 };

    Console.WriteLine("=== Анализ с помощью if ===");
    AnalyzeWithIf(data);

    Console.WriteLine("\n=== Анализ с помощью switch ===");
    AnalyzeWithSwitch(data);

    Console.WriteLine("\n=== Категории (switch expression) ===");
    foreach (var item in data)
    {
        string category = GetCategory(item);
        Console.WriteLine($"{item} ?? "null" -> {category}");
    }

    // В решении используются вложенные методы (функции)
    // Метод 1: Использование if с pattern matching
    static void AnalyzeWithIf(object[] array)
    {
        foreach (object obj in array)
        {
            if (obj is null)
            {
                Console.WriteLine("Пустое значение");
            }
            else if (obj is int number && number > 0)
            {
                Console.WriteLine($"Положительное число: {number}");
            }
            else if (obj is int num && num == 0)
            {
                Console.WriteLine($"Ноль: {num}");
            }
            else if (obj is int negative && negative < 0)
            {
                Console.WriteLine($"Отрицательное число: {negative}");
            }
            else if (obj is string str)
            {
                Console.WriteLine($"Строка: {str}");
            }
        }
    }

    // Метод 2: Использование switch с pattern matching и guards
    static void AnalyzeWithSwitch(object[] array)
    {
        foreach (object obj in array)
        {
            switch (obj)
            {
                case null:
                    Console.WriteLine("Пустое значение");
                    break;

                case int i when i > 0:
                    Console.WriteLine($"Положительное число: {i}");
                    break;

                case int i when i == 0:

```

```

        Console.WriteLine($"Ноль: {i}");
        break;

    case int i when i < 0:
        Console.WriteLine($"Отрицательное число: {i}");
        break;

    case string s:
        Console.WriteLine($"Строка: {s}");
        break;
    }
}

// Метод 3: Использование switch expression
static string GetCategory(object obj)
{
    return obj switch
    {
        null => "NULL",
        int i when i > 0 => "ПОЛОЖИТЕЛЬНОЕ",
        int i when i == 0 => "НОЛЬ",
        int i when i < 0 => "ОТРИЦАТЕЛЬНОЕ",
        string s => $"СТРОКА_ДЛИНЫ_{s.Length}",
        _ => "НЕИЗВЕСТНЫЙ_ТИП"
    };
}
}

```

5.6 Операторы цикла

Счетный цикл `for` такой же как в C++ и Java. В круглых скобках после `for` указываются три оператора – начальное присвоение значения переменной цикла; условие выхода из цикла; оператор, который выполняется при переходе к следующему шагу цикла. Операторы разделяются точкой с запятой.

Пример цикла for:

```

for (int i = 0; i < 3; i++)
    Console.Write(i);

```

В данном примере оператор вывода переменной `i` не вложен в фигурные скобки, использование скобок не является обязательным, так как это единственный оператор цикла. Если бы в цикле выполнялось несколько действий, то их необходимо было бы вложить в фигурные скобки.

В языке C# также существует форма счетного цикла, предназначенная для перебора коллекции – `foreach`. В классическом языке C++ такой цикл

отсутствует (однако он появился в новых версиях). В языке Java этот цикл имеет синтаксис `for(переменная : коллекция)`.

Пример цикла foreach:

```
int[] array1 = { 1, 2, 3 };
foreach(int i2 in array1)
    Console.WriteLine(i2);
```

В данном примере `int i2` является объявлением переменной цикла, `in` отделяет переменную цикла от имени коллекции.

Следует отметить, что цикл `foreach` является одним из наиболее часто используемых видов цикла в C#. Это обусловлено тем, что в языке C# существует развитый набор коллекций, а реализация многих алгоритмов связана с перебором коллекций.

Циклы с предусловием и постусловием также очень похожи на те, что в C++ и Java.

Пример цикла с предусловием:

```
int i3 = 0;
while (i3 < 3)
{
    Console.WriteLine(i3);
    i3++;
}
```

Пример цикла с постусловием:

```
int i4 = 0;
do
{
    Console.WriteLine(i4);
    i4++;
} while (i4 < 3);
```

Операторы `break` и `continue` используются аналогично языкам C и C++.

ЗАДАНИЕ

Дан массив:

```
int[] numbers = { 5, -3, 8, 12, -7,  
                  0, 15, -2, 4, 20 };
```

Напишите программу на C#, которая работает с массивом целых чисел и выполняет различные операции, используя разные типы циклов. Требуется реализовать 5 методов:

- 1. Метод PrintEvenIndices** - используя цикл `for`, вывести элементы массива, находящиеся на четных индексах (0, 2, 4, ...)
- 2. Метод CalculateSum** - используя цикл `foreach`, вычислить и вернуть сумму всех элементов массива
- 3. Метод FindFirstGreaterThan** - используя цикл `while` и оператор `break`, найти и вернуть первое число больше заданного значения (параметр метода). Если такого числа нет, вернуть -1
- 4. Метод PrintUntilSumExceeds** - используя цикл `do-while`, выводить элементы массива и их накопленную сумму, пока сумма не превысит заданное значение (параметр метода)
- 5. Метод PrintPositiveOnly** - используя цикл `foreach` и оператор `continue`, вывести только положительные числа из массива (пропуская отрицательные и ноль)

Программа должна вызвать все методы и продемонстрировать их работу.

РЕШЕНИЕ с использованием локальных функций (появились в C# 7):

```
static void example_loops()
{
    int[] numbers = { 5, -3, 8, 12, -7, 0, 15, -2, 4, 20 };

    Console.WriteLine("Исходный массив:");
    Console.WriteLine(string.Join(", ", numbers));
    Console.WriteLine();

    // 1. Вывод элементов на четных индексах (for)
    Console.WriteLine("=== 1. Элементы на четных индексах (цикл for)");
    PrintEvenIndices(numbers);
    Console.WriteLine();

    // 2. Сумма всех элементов (foreach)
    Console.WriteLine("=== 2. Сумма всех элементов (цикл foreach)");
    int sum = CalculateSum(numbers);
    Console.WriteLine($"Сумма: {sum}");
    Console.WriteLine();

    // 3. Первое число больше заданного (while с break)
    Console.WriteLine("=== 3. Первое число > 10 (цикл while с break)");
    int result = FindFirstGreaterThan(numbers, 10);
    if (result != -1)
        Console.WriteLine($"Найдено: {result}");
    else
        Console.WriteLine("Не найдено");
    Console.WriteLine();

    // 4. Вывод пока сумма не превысит значение (do-while)
    Console.WriteLine("=== 4. Вывод пока сумма <= 15 (цикл do-while)");
    PrintUntilSumExceeds(numbers, 15);
    Console.WriteLine();

    // 5. Только положительные числа (foreach с continue)
    Console.WriteLine("=== 5. Только положительные (foreach с continue)");
    PrintPositiveOnly(numbers);

    // Метод 1: Цикл for - элементы на четных индексах
    static void PrintEvenIndices(int[] array)
    {
        Console.Write("Элементы на индексах 0, 2, 4, ....: ");
        for (int i = 0; i < array.Length; i += 2)
        {
```

```

        Console.WriteLine(array[i] + " ");
    }
    Console.WriteLine();
}

// Метод 2: Цикл foreach - сумма элементов
static int CalculateSum(int[] array)
{
    int sum = 0;
    foreach (int num in array)
    {
        sum += num;
    }
    return sum;
}

// Метод 3: Цикл while с break - поиск первого элемента
static int FindFirstGreaterThan(int[] array, int threshold)
{
    int index = 0;
    while (index < array.Length)
    {
        if (array[index] > threshold)
        {
            return array[index]; // или можно использовать
break
        }
        index++;
    }
    return -1; // не найдено
}

// Метод 4: Цикл do-while - вывод до превышения суммы
static void PrintUntilSumExceeds(int[] array, int limit)
{
    int index = 0;
    int currentSum = 0;

    do
    {
        currentSum += array[index];
        Console.WriteLine($"array[{index}] = {array[index]},
накопленная сумма = {currentSum}");
        index++;

        if (currentSum > limit)
        {
            Console.WriteLine($"Сумма превысила {limit}, выход
из цикла");
            break;
        }
    }
    while (currentSum <= limit);
}

```



```

    }

    } while (index < array.Length);
}

// Метод 5: Цикл foreach с continue - только положительные
static void PrintPositiveOnly(int[] array)
{
    Console.Write("Положительные числа: ");
    foreach (int num in array)
    {
        if (num <= 0)
        {
            continue; // пропускаем отрицательные и ноль
        }
        Console.Write(num + " ");
    }
    Console.WriteLine();
}
}

```

Результаты вывода в консоль:

Исходный массив:

5, -3, 8, 12, -7, 0, 15, -2, 4, 20

=== 1. Элементы на четных индексах (цикл for) ===

Элементы на индексах 0, 2, 4, ...: 5 8 -7 15 4

=== 2. Сумма всех элементов (цикл foreach) ===

Сумма: 52

=== 3. Первое число > 10 (цикл while с break) ===

Найдено: 12

=== 4. Вывод пока сумма <= 15 (цикл do-while) ===

array[0] = 5, накопленная сумма = 5

array[1] = -3, накопленная сумма = 2

array[2] = 8, накопленная сумма = 10

array[3] = 12, накопленная сумма = 22

Сумма превысила 15, выход из цикла

=== 5. Только положительные (foreach с continue) ===
 Положительные числа: 5 8 12 15 4 20

5.7 Использование массивов

Для объявления массивов в C# используется синтаксис:

```
int[] array;
```

В языке C++ квадратные скобки должны ставить не после имени типа, а после имени переменной: `int array[]`. В Java допустимы оба варианта объявления.

Если необходимо присвоить начальные значения элементам массива, то форма объявления следующая:

```
int[] array = new int[3] {1, 2, 3};
```

Аналогично для строкового типа:

```
string[] strs =  
    new string[3] { "строка1", "строка2", "строка3" };
```

Многомерные массивы в языке C# бывают двух видов — прямоугольные и зубчатые (jagged).

Объявление прямоугольных массивов очень похоже на их объявление в Паскале.

Пример объявления прямоугольного массива:

```
int[,] matrix = new int[2, 2] { { 1, 2 }, { 3, 4 } };
```

или:

```
int[,] matrix = { {1,2}, {3,4} };
```

или:

```
int[,] matrix = new int[2, 2];  
matrix[0, 0] = 1;  
matrix[0, 1] = 2;  
matrix[1, 0] = 3;  
matrix[1, 1] = 4;
```

Объявление зубчатых массивов похоже на их объявление в языках C++ и Java.

Пример объявления зубчатого массива:

```
int[][] jagged = new int[3][];
jagged[0] = new int[3] { 1, 2, 3 };
jagged[1] = new int[2] { 4, 5 };
jagged[2] = new int[4] { 6, 7, 8, 9 };
```

В случае зубчатого массива каждый подмассив может иметь собственную размерность.

ЗАДАНИЕ

Для квадратной матрицы напишите функции проверки ее симметричности относительно главной диагонали.

Матрица называется симметричной относительно главной диагонали, если элемент на позиции [i, j] равен элементу на позиции [j, i] для всех i и j.

Например, матрица:

1 2 3

2 5 6

3 6 9

является симметричной, так как:

- **matrix[0,1] = 2 равно matrix[1,0] = 2**
- **matrix[0,2] = 3 равно matrix[2,0] = 3**
- **matrix[1,2] = 6 равно matrix[2,1] = 6**

Требуется реализовать:

- **Метод IsSymmetric** - принимает прямоугольный двумерный массив и возвращает true, если матрица симметрична, и false в противном

случае. Если матрица не квадратная, метод должен вернуть false.

- **Метод PrintMatrix - выводит матрицу в форматированном виде**

В основном методе создать и проверить:

- **Симметричную матрицу 3x3**
- **Несимметричную матрицу 3x3**
- **Неквадратную матрицу 2x3**

Программа должна вывести каждую матрицу и результат проверки её симметричности.

РЕШЕНИЕ:

```

static void example_matrices()
{
    // 1. Симметричная матрица
    Console.WriteLine("=== Матрица 1: Симметричная ===");
    int[,] matrix1 = {
        { 1, 2, 3 },
        { 2, 5, 6 },
        { 3, 6, 9 }
    };
    PrintMatrix(matrix1);
    bool result1 = IsSymmetric(matrix1);
    Console.WriteLine($"Симметрична: {(result1 ? "Да" : "Нет")}");
    Console.WriteLine();

    // 2. Несимметричная матрица
    Console.WriteLine("=== Матрица 2: Несимметричная ===");
    int[,] matrix2 = {
        { 1, 2, 3 },
        { 4, 5, 6 },
        { 7, 8, 9 }
    };
    PrintMatrix(matrix2);
    bool result2 = IsSymmetric(matrix2);
    Console.WriteLine($"Симметрична: {(result2 ? "Да" : "Нет")}");
    Console.WriteLine();

    // 3. Неквадратная матрица
    Console.WriteLine("=== Матрица 3: Неквадратная (2x3) ===");
    int[,] matrix3 = {
        { 1, 2, 3 },
        { 4, 5, 6 }
    };
    PrintMatrix(matrix3);
    bool result3 = IsSymmetric(matrix3);
    Console.WriteLine($"Симметрична: {(result3 ? "Да" : "Нет")}");
    Console.WriteLine();

    // 4. Единичная матрица
    Console.WriteLine("=== Матрица 4: Единичная матрица 4x4 ===");
    int[,] matrix4 = {
        { 1, 0, 0, 0 },
        { 0, 1, 0, 0 },
        { 0, 0, 1, 0 },
        { 0, 0, 0, 1 }
    };
    PrintMatrix(matrix4);
    bool result4 = IsSymmetric(matrix4);
    Console.WriteLine($"Симметрична: {(result4 ? "Да" : "Нет")}");
}

```

```

// Метод проверки симметричности матрицы
static bool IsSymmetric(int[,] matrix)
{
    int rows = matrix.GetLength(0); // количество строк
    int cols = matrix.GetLength(1); // количество столбцов

    // Проверка: матрица должна быть квадратной
    if (rows != cols)
    {
        return false;
    }

    // Проверка симметричности
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            // Проверяем, что элемент [i,j] равен элементу [j,i]
            if (matrix[i, j] != matrix[j, i])
            {
                return false;
            }
        }
    }

    return true;
}

// Метод вывода матрицы
static void PrintMatrix(int[,] matrix)
{
    int rows = matrix.GetLength(0);
    int cols = matrix.GetLength(1);

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            Console.Write($"{matrix[i, j],4}");
        }
        Console.WriteLine();
    }
}
}

```

5.8 Работа с null

```

//Ошибка, типу int нельзя присвоить null
int n1 = null;

```

Nullable value types используют структуру `Nullable<T>`. Тип `int?` означает, что переменной можно присвоить или значение типа `int` или `null`.

```
int? nl_number = 42;
int? nl_nullableNumber = null;

if (nl_number.HasValue)
{
    //присваивается значение int (42)
    int nl_value = nl_number.Value;
    Console.WriteLine(nl_value);
}
```

Null-coalescing оператор `??` появился в C# 2. Оператор `??` присваивает правое значение если левое `null`.

```
int nl_result1 = nl_number ?? -1; // 42
int nl_result2 = nl_nullableNumber ?? -1; // -1
Console.WriteLine($"{nl_number} {nl_nullableNumber} {nl_result1} {nl_result2}");
```

Nullable reference types — это ссылочные типы (например, `string`), которые явно помечены как допускающие `null`.

```
string? nl_str1 = "Строка1";
string? nl_str2 = null;
if(nl_str2 == null) Console.WriteLine(nl_str1);
```

Null-conditional оператор `?.` появился в C# 6. Если `nl_str2 == null`, то возвращается `null`, ошибка не возникает.

```
int ? nl_length1 = nl_str1?.Length;
int? nl_length2 = nl_str2?.Length;
Console.WriteLine($"{nl_length1} {nl_length2}");
```

Null-coalescing assignment (присваивание) `??=` появилось начиная с C#8. Присваивание работает, только если `nl_str2 == null`.

```
nl_str2 ??= "!!!";
Console.WriteLine(nl_str2);
```

Для проверки на `null` можно использовать выражения

```
// старый стиль
if (obj == null) { }
// современный стиль начиная с C# 7
if (obj is null) { }
if (obj is not null) { }
```

ЗАДАНИЕ

Напишите программу на C#, которая демонстрирует различные способы работы с null-значениями на примере массива nullable целых чисел.

```
int?[] numbers = { 10, null, 25, null,  
                  30, 15, null };
```

- 1. Вывести исходный массив (для null выводить слово "null")**
- 2. Подсчитать количество null-значений, используя проверку is null**
- 3. Подсчитать количество не-null значений, используя проверку is not null**
- 4. Вычислить сумму всех ненулевых чисел, используя свойство HasValue и Value**
- 5. Создать новый массив, где все null заменены на -1, используя оператор ??**
- 6. Вывести длину строкового представления каждого числа (используя оператор ?. для вызова метода ToString())**
- 7. Заменить все null-значения в исходном массиве на 0, используя оператор ??=**
- 8. Вывести итоговый массив после замены null-значений**

РЕШЕНИЕ:

```

static void example_null()
{
    // Исходные данные
    int?[] numbers = { 10, null, 25, null, 30, 15, null };

    // 1. Вывод исходного массива
    Console.WriteLine("=== 1. Исходный массив ===");
    for (int i = 0; i < numbers.Length; i++)
    {
        // Используем ?. для безопасного вызова ToString()
        string display = numbers[i]?.ToString() ?? "null";
        Console.WriteLine($"numbers[{i}] = {display}");
    }
    Console.WriteLine();

    // 2. Подсчет null-значений (оператор is null)
    Console.WriteLine("=== 2. Подсчет null-значений (is null) ===");
    int nullCount = 0;
    for (int i = 0; i < numbers.Length; i++)
    {
        if (numbers[i] is null)
        {
            nullCount++;
        }
    }
    Console.WriteLine($"Количество null-значений: {nullCount}");
    Console.WriteLine();

    // 3. Подсчет не-null значений (оператор is not null)
    Console.WriteLine("=== 3. Подсчет не-null значений (is not null)");
    int notNullCount = 0;
    for (int i = 0; i < numbers.Length; i++)
    {
        if (numbers[i] is not null)
        {
            notNullCount++;
        }
    }
    Console.WriteLine($"Количество не-null значений: {notNullCount}");
    Console.WriteLine();

    // 4. Сумма ненулевых чисел (HasValue и Value)
    Console.WriteLine("=== 4. Сумма ненулевых чисел (HasValue и Value)");
    int sum = 0;
    for (int i = 0; i < numbers.Length; i++)
    {
        if (numbers[i].HasValue)
        {
            sum += numbers[i].Value;
            Console.WriteLine($"Добавляем numbers[{i}] = {numbers[i].Value}, текущая сумма = {sum}");
        }
    }
}

```

```

    }
}
Console.WriteLine($"Итоговая сумма: {sum}");
Console.WriteLine();

// 5. Замена null на -1 (оператор ??)
Console.WriteLine("=== 5. Создание массива с заменой null на -1
(оператор ??) ===");
int[] numbersWithDefault = new int[numbers.Length];
for (int i = 0; i < numbers.Length; i++)
{
    numbersWithDefault[i] = numbers[i] ?? -1;
    Console.WriteLine($"numbersWithDefault[{i}] =
{numbersWithDefault[i]}");
}
Console.WriteLine();

// 6. Длина строкового представления (оператор ?.)
Console.WriteLine("=== 6. Длина строкового представления (оператор ?.)
===");
for (int i = 0; i < numbers.Length; i++)
{
    // numbers[i]?.ToString() вернет null если numbers[i] == null
    // затем ?.Length вернет null если ToString() вернул null
    // затем ?? 0 заменит null на 0
    int? length = numbers[i]?.ToString()?.Length ?? 0;
    Console.WriteLine($"Длина представления numbers[{i}] = {length}");
}
Console.WriteLine();

// 7. Замена null на 0 в исходном массиве (оператор ??=)
Console.WriteLine("=== 7. Замена null на 0 в исходном массиве
(оператор ??=) ===");
Console.WriteLine("До замены: null-значений = " + nullCount);
for (int i = 0; i < numbers.Length; i++)
{
    // Присваивание произойдет только если numbers[i] == null
    numbers[i] ??= 0;
}

// Проверка что null-значений больше нет
int nullCountAfter = 0;
for (int i = 0; i < numbers.Length; i++)
{
    if (numbers[i] is null)
    {
        nullCountAfter++;
    }
}
Console.WriteLine("После замены: null-значений = " + nullCountAfter);
Console.WriteLine();

// 8. Вывод итогового массива
Console.WriteLine("=== 8. Итоговый массив после замены ===");
for (int i = 0; i < numbers.Length; i++)

```

```

{
    Console.WriteLine($"numbers[{i}] = {numbers[i]}");
}
Console.WriteLine();

// Дополнительная демонстрация
Console.WriteLine("=== ДОПОЛНИТЕЛЬНО: Сравнение способов проверки
===");
int? testValue = null;

// Старый стиль
if (testValue == null)
{
    Console.WriteLine("testValue == null (старый стиль): true");
}

// Современный стиль
if (testValue is null)
{
    Console.WriteLine("testValue is null (современный стиль): true");
}

testValue = 42;
if (testValue is not null)
{
    Console.WriteLine($"testValue is not null: true, значение =
{testValue}");
}
}

```

6 Основы работы с файлами

6.1 Получение данных о файлах и каталогах

Классы для работы с файловой системой объявлены в пространстве имен System.IO. Прежде чем читать или записывать файлы, необходимо уметь находить нужные пути и проверять существование файлов и каталогов. Для этих задач в .NET предусмотрены три основных класса: Directory, File и Path.

6.1.1 Использование класса Directory для работы с каталогами

Наиболее простой способ получения информации о файловой системе — использование статического класса Directory. Если класс является

статическим, то предполагается, что все его свойства и методы являются статическими.

Класс `Directory`, объявленный в пространстве имен `System.IO`, позволяет осуществлять работу с каталогами: создание (метод `CreateDirectory`), удаление (метод `Delete`), перемещение (метод `Move`) каталогов, получение списка файлов и подкаталогов.

Перед обращением к каталогу рекомендуется проверить его существование методом `Directory.Exists`.

Пример:

```
string catalogName = @"c:\";

if (Directory.Exists(catalogName))
{
    Console.WriteLine("Каталог существует: " + catalogName);
}
else
{
    Console.WriteLine("Каталог не найден: " + catalogName);
}
```

Обратите внимание на объявление переменной `catalogName`. Как и в языке C++ в C# символ «\» внутри строки используется для создания специальных символов, например «\n» – перевод строки. В соответствии с правилами языка C# имя каталога должно быть объявлено как «c:\», при этом символ «\» как и в C++ удваивается. Но особенностью языка C# является то, что если перед строковым значением стоит символ «@», то специальные символы игнорируются и удваивать символ «\» не требуется.

Пример создания каталога при его отсутствии:

```
string outputDir = @"c:\output";

if (!Directory.Exists(outputDir))
{
    Directory.CreateDirectory(outputDir);
    Console.WriteLine("Каталог создан: " + outputDir);
}
```

Метод `Directory.GetFiles` получает список файлов, а метод `Directory.GetDirectories` – список подкаталогов указанного каталога.

Пример вывода списка файлов и каталогов в корне диска «C»:

```

Console.WriteLine("\nСписок файлов каталога " + catalogName);
string[] files = Directory.GetFiles(catalogName);
foreach (string file in files)
{
    Console.WriteLine(file);
}

Console.WriteLine("\nСписок подкаталогов каталога " + catalogName);
string[] dirs = Directory.GetDirectories(catalogName);
foreach (string dir in dirs)
{
    Console.WriteLine(dir);
}

```

Результаты вывода в консоль:

Список файлов каталога c:\

c:\appverifUI.dll

c:\bootmgr

c:\BOOTNXT

...

Список подкаталогов каталога c:\

c:\\$GetCurrent

c:\\$Recycle.Bin

c:\\$WinREAgent

...

Для фильтрации файлов по маске используется метод `GetFiles` со вторым параметром — маской поиска, которая позволяет получать только файлы нужного типа, например `*.txt`:

Пример фильтрации файлов по маске:

```

// Только текстовые файлы
string[] txtFiles = Directory.GetFiles(catalogName, "*.txt");

// Файлы, имя которых начинается на "report"
string[] reportFiles = Directory.GetFiles(catalogName, "report*.");

Console.WriteLine($"Найдено txt-файлов: {txtFiles.Length}");
foreach (string file in txtFiles)
{
    Console.WriteLine(file);
}

```

Результаты вывода в консоль:

```
Найдено txt-файлов: 9
c:\eula.1028.txt
c:\eula.1031.txt
c:\eula.1033.txt
```

По умолчанию `GetFiles` просматривает только указанный каталог. Чтобы выполнить поиск во всех вложенных подкаталогах (то есть рекурсивный поиск по подкаталогам), используется перечисление `SearchOption`

Пример рекурсивного поиска по подкаталогам:

```
// Только в указанном каталоге
// Параметр "SearchOption.TopDirectoryOnly" можно не указывать, он
установлен по умолчанию
string catalogName2 = @"c:\python";
string[] files1 = Directory.GetFiles(catalogName2, "*.txt",
SearchOption.TopDirectoryOnly);

// Во всех вложенных подкаталогах
string[] files2 = Directory.GetFiles(catalogName2, "*.txt",
SearchOption.AllDirectories);

Console.WriteLine($"В каталоге: {files1.Length} файлов");
Console.WriteLine($"В каталоге и подкаталогах: {files2.Length} файлов");
```

Метод `GetFiles` загружает весь список файлов в массив сразу. Метод `EnumerateFiles` возвращает тип `IEnumerable<string>` и перечисляет файлы по одному — без загрузки всего списка в память. Для каталогов с большим количеством файлов `EnumerateFiles` работает эффективнее. Аналогичная пара методов существует для каталогов: `GetDirectories` и `EnumerateDirectories`.

Обобщенный тип `IEnumerable<T>` используется для организации «ленивых вычислений» в .NET. Обычные (не ленивые) вычисления производятся в тот момент, когда они были объявлены. «Ленивыми» называются вычисления (или обработка данных) в том случае если они выполняются только тогда, когда какой-либо программный модуль требует результат этих вычислений.

Пример ленивого чтения файлов:

```
// GetFiles – загружает все пути сразу в массив string[]
string[] allFiles = Directory.GetFiles(catalogName);
Console.WriteLine(allFiles.GetType().FullName, allFiles.Length);
Console.WriteLine(allFiles.Length);

// EnumerateFiles – ленивое перечисление, IEnumerable<string>
var lazyFiles = Directory.EnumerateFiles(catalogName, "*.txt");
Console.WriteLine(lazyFiles.GetType().FullName);
int i = 0;
foreach (string file in lazyFiles)
{
    // Файлы читаются по одному – не нужно ждать полной загрузки списка
    Console.WriteLine(file);
    i++;
    if (i >= 3) break;
}
```

Результаты вывода в консоль:

```
System.String[]
36
```

```
System.IO.Enumeration.FileSystemEnumerable`1[[System.String,
System.Private.CoreLib, Version=10.0.0.0, Culture=neutral,
PublicKeyToken=7cec85d7bea7798e]]
c:\eula.1028.txt
c:\eula.1031.txt
c:\eula.1033.txt
```

6.1.2 Использование класса File для работы с файлами

Класс File, как и Directory, является статическим. Помимо операций чтения и записи (рассмотренных далее), он предоставляет методы для проверки существования файла и получения его метаданных.

Метод File.Exists необходимо вызывать перед чтением файла — это позволяет избежать исключения при обращении к несуществующему файлу.

Пример проверки существования файла:

```
string filePath = @"c:\output\report.txt";

if (File.Exists(filePath))
```

```

{
    Console.WriteLine("Файл найден: " + filePath);

    // Метаданные файла
    DateTime created = File.GetCreationTime(filePath);
    DateTime modified = File.GetLastWriteTime(filePath);
    DateTime accessed = File.GetLastAccessTime(filePath);

    Console.WriteLine($"Создан:      {created:dd.MM.yyyy HH:mm}");
    Console.WriteLine($"Изменён:    {modified:dd.MM.yyyy HH:mm}");
    Console.WriteLine($"Открывался: {accessed:dd.MM.yyyy HH:mm}");
}
else
{
    Console.WriteLine("Файл не найден: " + filePath);
}

```

Результаты вывода в консоль:

Файл найден: c:\output\report.txt

Создан: 05.03.2026 05:50

Изменён: 05.03.2026 05:50

Открывался: 05.03.2026 05:50

6.1.3 Использование класса Path для формирования путей

Класс Path предназначен для работы с именами файлов и каталогов. Он не выполняет операций с файловой системой — только разбирает и формирует строки путей. Ключевое назначение класса — создание путей корректным способом, независимо от операционной системы (в частности, корректная работа с разделителями \ в Windows и / в Linux).

При этом важно понимать, что класс Path не проверяет существование реальных путей в операционной системе, он выполняет преобразования на основе информации об устройстве файловой системы.

Основные методы класса Path:

```

string fullPath = @"c:\output\report.txt";

// Фрагменты путей
Console.WriteLine(Path.GetFileName(fullPath));
// report.txt
Console.WriteLine(Path.GetFileNameWithoutExtension(fullPath));
// report
Console.WriteLine(Path.GetExtension(fullPath));

```



```
// .txt
Console.WriteLine(Path.GetDirectoryName(fullPath));
// C:\projects\myapp\data

// Изменить расширение
string csvPath = Path.ChangeExtension(fullPath, ".csv");
Console.WriteLine(csvPath);
// C:\projects\myapp\data\report.csv

// Получить абсолютный путь из относительного
string absPath = Path.GetFullPath(@"data\report.txt");
Console.WriteLine(absPath);
// C:\текущий_каталог\data\report.txt

// Path.Combine – объединение фрагментов путей
string path3 = Path.Combine(@"c:\docs", "report.txt"); //
c:\docs\report.txt
Console.WriteLine(path3);
string path4 = Path.Combine(@"c:\docs", "2024", "report.txt"); // три
части
Console.WriteLine(path4);

// Получение рабочего каталога приложения (в котором находится исполняемый
файл)
string appDir = AppContext.BaseDirectory;
Console.WriteLine(appDir);
```

ЗАДАНИЕ

Напишите консольное приложение на C#, которое:

- 1. Запрашивает у пользователя путь к каталогу**
- 2. Проверяет существование каталога (Directory.Exists)**
- 3. Если каталог не существует — выводит сообщение об ошибке**
- 4. Если существует — выводит полный список файлов с расширением .txt в каталоге и всех подкаталогах (SearchOption.AllDirectories)**
- 5. Для каждого файла: имя файла (Path.GetFileName), дату последнего изменения (File.GetLastWriteTime),**

путь к содержащему его каталогу (Path.GetDirectoryName)

6. Выводит итоговое количество найденных файлов

РЕШЕНИЕ:

```
using System;
using System.IO;

// ===== ПОИСК ТЕКСТОВЫХ ФАЙЛОВ =====

Console.WriteLine("=====");
Console.WriteLine("    ПОИСК ТЕКСТОВЫХ ФАЙЛОВ (.txt)");
Console.WriteLine("=====");
Console.WriteLine();

Console.Write("Введите путь к каталогу: ");
string inputPath = Console.ReadLine()?.Trim() ?? "";

Console.WriteLine();

// Проверка: пользователь не ввёл пустую строку
if (String.IsNullOrEmpty(inputPath))
{
    Console.WriteLine("Ошибка: путь к каталогу не введён.");
}
// Проверка существования каталога
else if (!Directory.Exists(inputPath))
{
    Console.WriteLine($"Ошибка: каталог не существует: {inputPath}");
}
else
{
    // Каталог существует – выполняем поиск
    SearchTextFiles(inputPath);
}

// ===== ФУНКЦИИ =====

/// <summary>
/// Выполняет поиск всех файлов .txt в указанном каталоге
/// и всех его подкаталогах, выводит информацию о каждом файле
/// </summary>
/// <param name="rootPath">Путь к корневому каталогу поиска</param>
static void SearchTextFiles(string rootPath)
{
    Console.WriteLine($"Поиск в каталоге: {rootPath}");
    Console.WriteLine();

    // Получение списка всех .txt файлов в каталоге и подкаталогах
    // SearchOption.AllDirectories – рекурсивный поиск по подкаталогам
    string[] files = Directory.GetFiles(rootPath, "*.txt",
```

```

        SearchOption.AllDirectories);

    if (files.Length == 0)
    {
        Console.WriteLine("Текстовые файлы (.txt) не найдены.");
        return;
    }

    // Заголовок таблицы результатов
    PrintTableHeader();

    // Вывод информации о каждом найденном файле
    foreach (string filePath in files)
    {
        PrintFileInfo(filePath, rootPath);
    }

    // Итоговое количество найденных файлов
    Console.WriteLine(new string('-', 72));
    Console.WriteLine($"Итого найдено файлов: {files.Length}");
}

/// <summary>
/// Выводит заголовок таблицы результатов
/// </summary>
static void PrintTableHeader()
{
    Console.WriteLine(new string('-', 72));
    Console.WriteLine($"{"Имя файла",-28} {"Дата изменения",-18}
Каталог");
    Console.WriteLine(new string('-', 72));
}

/// <summary>
/// Выводит информацию об одном файле:
/// имя файла, дату последнего изменения и путь к каталогу
/// </summary>
/// <param name="filePath">Полный путь к файлу</param>
/// <param name="rootPath">Корневой каталог поиска для сокращения
вывода</param>
static void PrintFileInfo(string filePath, string rootPath)
{
    // Имя файла без пути
    string fileName = Path.GetFileName(filePath);

    // Дата последнего изменения файла
    DateTime lastModified = File.GetLastWriteTime(filePath);
    string lastModifiedStr = lastModified.ToString("dd.MM.yyyy HH:mm");

    // Путь к каталогу, содержащему файл
    string fileDir = Path.GetDirectoryName(filePath);

    // Для удобства чтения выводим путь к каталогу
    // относительно корневого каталога поиска
    string relativePath = GetRelativePath(rootPath, fileDir);

```

```

        Console.WriteLine($"{fileName,-28} {lastModifiedStr,-18}
{relativePath}");
    }

    /// <summary>
    /// Возвращает путь к каталогу относительно корневого каталога.
    /// Если каталог совпадает с корневым – возвращает точку «.»
    /// </summary>
    /// <param name="rootPath">Корневой каталог</param>
    /// <param name="fullPath">Полный путь к каталогу</param>
    /// <returns>Относительный путь или полный путь если не удалось
сократить</returns>
    static string GetRelativePath(string rootPath, string fullPath)
    {
        // Нормализация путей для корректного сравнения
        string normalRoot = rootPath.TrimEnd(Path.DirectorySeparatorChar);
        string normalFull = fullPath.TrimEnd(Path.DirectorySeparatorChar);

        // Если каталог файла совпадает с корневым – возвращаем «.»
        if (string.Equals(normalRoot, normalFull,
            StringComparison.OrdinalIgnoreCase))
        {
            return ".";
        }

        // Если путь начинается с корневого – убираем его из начала
        if (normalFull.StartsWith(normalRoot,
            StringComparison.OrdinalIgnoreCase))
        {
            // +1 для удаления символа разделителя каталогов
            return normalFull.Substring(normalRoot.Length + 1);
        }

        // В остальных случаях возвращаем полный путь
        return fullPath;
    }
}

```

6.2 Чтение и запись файлов

В предыдущем разделе были рассмотрены способы получения путей к файлам и проверки их существования. В данном разделе рассматриваются методы чтения и записи файлов.

Наиболее простой способ работы с текстовыми файлами — использование статического класса `File`, объявленного в пространстве имён `System.IO`. Помимо методов чтения и записи, у класса `File` есть методы для копирования (`Copy`), перемещения (`Move`) и удаления (`Delete`) файлов.

Для более низкоуровневой работы с файлами в .NET предусмотрен класс `FileStream`. Он позволяет выполнять с файлами большое количество операций, в том числе произвольный доступ к позиции внутри файла. Однако данный подход является достаточно низкоуровневым и детально не рассматривается. Следует отметить, что этот подход пришёл в C# из стандартных библиотек языка Java.

6.2.1 Запись и чтение файла целиком

Метод `File.WriteAllText` записывает строку в текстовый файл, создавая файл при его отсутствии или полностью заменяя содержимое существующего. Метод `File.ReadAllText` возвращает всё содержимое текстового файла в виде переменной типа `string`.

Пример ввода строки и сохранения в файл:

```
//Пример ввода строки и сохранения в файл
string currentPath = AppContext.BaseDirectory;

Console.Write("Введите текст для записи в файл: ");
string file1Contents = Console.ReadLine();

// Формирование имени файла
string file1Name = Path.Combine(currentPath, "file1.txt");

// Запись строки в файл
File.WriteAllText(file1Name, file1Contents);

// Чтение строки из файла
string file1ContentsRead = File.ReadAllText(file1Name);
Console.WriteLine("Чтение строки из файла: " + file1ContentsRead);
```

Результаты ввода-вывода в консоль:

Введите текст для записи в файл: пример текста

Чтение строки из файла: пример текста

В данном примере содержимое строки вводится с клавиатуры, затем с помощью метода `File.WriteAllText` записывается в текстовый файл. Метод `Path.Combine` корректно соединяет строку пути к каталогу приложения и

имя создаваемого файла. Если указанный файл уже существует, `File.WriteAllText` полностью заменяет его предыдущее содержимое.

Метод `File.ReadAllText` считывает файл целиком в одну строку. Файл может содержать произвольное количество строк и символов перевода строки.

При этом в каталоге, содержащем исполняемый файл, создаётся текстовый файл `file1.txt` с указанным содержимым.

Перед чтением файла всегда рекомендуется проверять его существование методом `File.Exists`, чтобы избежать исключения при обращении к отсутствующему файлу:

6.2.2 Добавление данных в файл

Методы `File.AppendAllText` и `File.AppendAllLines` добавляют данные в конец существующего файла, не удаляя предыдущее содержимое. Если файл не существует, он создаётся автоматически. Это поведение отличает их от `WriteAllText` и `WriteAllLines`, которые всегда перезаписывают файл целиком.

Пример ведения текстового журнала событий:

```
//Пример ведения текстового журнала событий
string logFile = Path.Combine(AppContext.BaseDirectory, "log.txt");

// WriteAllText – создаёт файл или перезаписывает существующий
File.WriteAllText(logFile, $"[{DateTime.Now:dd.MM.yyyy HH:mm}] Запуск программы\n");

// AppendAllText – добавляет текст в конец файла
File.AppendAllText(logFile, $"[{DateTime.Now:dd.MM.yyyy HH:mm}] Шаг 1 выполнен\n");
File.AppendAllText(logFile, $"[{DateTime.Now:dd.MM.yyyy HH:mm}] Шаг 2 выполнен\n");

// AppendAllLines – добавляет массив строк в конец файла
// каждая строка завершается переводом строки автоматически
string[] newEvents = {
    $"[{DateTime.Now:dd.MM.yyyy HH:mm}] Шаг 3 выполнен",
    $"[{DateTime.Now:dd.MM.yyyy HH:mm}] Завершение программы"
};
File.AppendAllLines(logFile, newEvents);
```

```
Console.WriteLine("Содержимое журнала:");
Console.WriteLine(File.ReadAllText(logFile));
```

Результаты вывода в консоль:

Содержимое журнала:

```
[05.03.2026 07:22] Запуск программы
[05.03.2026 07:22] Шаг 1 выполнен
[05.03.2026 07:22] Шаг 2 выполнен
[05.03.2026 07:22] Шаг 3 выполнен
[05.03.2026 07:22] Завершение программы
```

6.2.3 Чтение и запись массива строк

В классе File есть методы, способные записать (WriteAllLines) или прочитать (ReadAllLines) массив строк. При записи каждый элемент массива помещается в отдельную строку файла. При чтении каждая строка файла становится отдельным элементом возвращаемого массива.

Пример ввода строк с клавиатуры и сохранения в файл:

```
//Пример ввода строк с клавиатуры и сохранения в файл
Console.WriteLine("Введите строки для записи в файл " +
    "(пустая строка – окончание ввода):");

string tempStrTrim = "";

// Список строк
List<string> list = new List<string>();

do
{
    // Временная переменная для хранения строки
    string tempStr = Console.ReadLine();
    // Удаление пробелов в начале и конце введённой строки
    tempStrTrim = tempStr.Trim();

    if (tempStrTrim != "")
    {
        // Непустая строка сохраняется в список
        list.Add(tempStrTrim);
    }
}
while (tempStrTrim != "");

// Формирование имени файла
```

```

string file2Name = Path.Combine(AppContext.BaseDirectory, "file2.txt");

// Запись в файл массива строк
File.WriteAllLines(file2Name, list.ToArray());

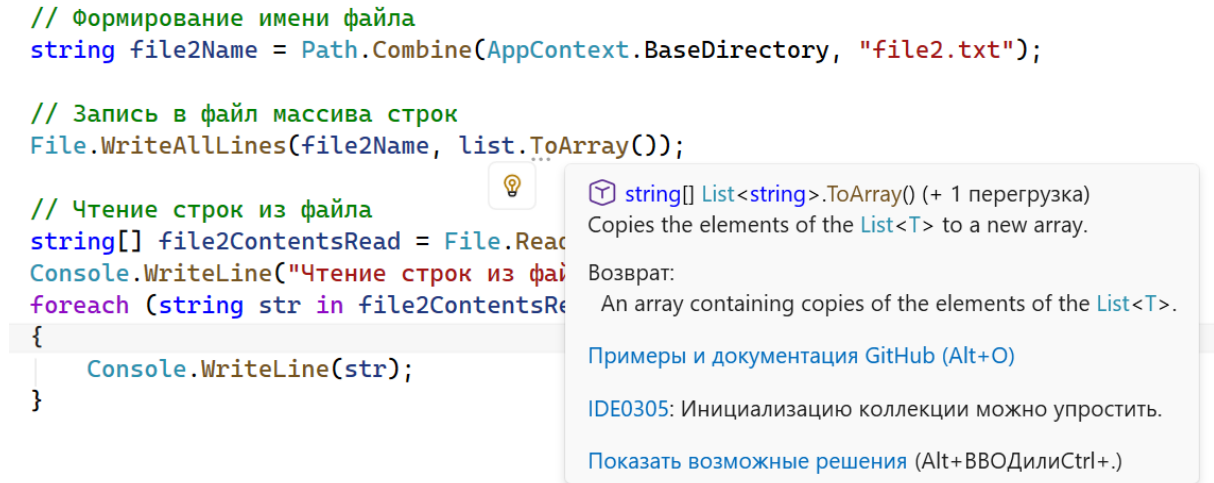
// Чтение строк из файла
string[] file2ContentsRead = File.ReadAllLines(file2Name);
Console.WriteLine("Чтение строк из файла:");
foreach (string str in file2ContentsRead)
{
    Console.WriteLine(str);
}

```

Для хранения строк при вводе используется обобщённый список `List<string>` (обобщенные коллекции мы планируем детально рассмотреть в следующих разделах). Ввод осуществляется с помощью цикла с постусловием — в условии проверяется ввод пустой строки. Метод `Trim` удаляет пробелы в начале и конце строки. Если строка после удаления пробелов не является пустой, она добавляется в список.

В данном примере для проверки строки на пустоту выполняется сравнение с пустой строкой (`tempStrTrim != ""`). Это работает корректно, так как пробелы предварительно удалены методом `Trim`. Для той же задачи в `.NET` предусмотрен статический метод `String.IsNullOrWhiteSpace`, возвращающий `true`, если строка пуста, равна `null` или содержит только пробельные символы.

Для записи в файл применяется метод `File.WriteAllLines`. Список строк преобразуется в массив методом `list.ToArray`. Начиная с `.NET 6` метод `WriteAllLines` принимает `IEnumerable<string>`, поэтому передавать `List<string>` без преобразования в массив также допустимо.

Рис. 16. Подсказка для метода `ToArray()`.

6.2.4 Ленивое чтение строк

Метод `File.ReadAllLines` загружает все строки файла в память сразу и возвращает массив `string[]`. Для файлов большого размера это может потребовать значительного объёма памяти. В качестве альтернативы класс `File` предоставляет метод `ReadLines`, который возвращает `IEnumerable<string>` и читает строки по одной — без загрузки всего файла в память.

Пример сравнения двух подходов:

```
string fileName = Path.Combine(AppContext.BaseDirectory, "file2.txt");

// ReadAllLines – загружает все строки в массив сразу
string[] allLines = File.ReadAllLines(fileName);
Console.WriteLine($"ReadAllLines: загружено {allLines.Length} строк");

// ReadLines – читает строки по одной, работает с foreach
Console.WriteLine("ReadLines:");
foreach (string line in File.ReadLines(fileName))
{
    Console.WriteLine(line);
}
```

6.2.5 Разбор текстового файла на подстроки

На практике для обработки текстового файла удобно считать его целиком методом `File.ReadAllText`, а затем разбить полученную строку на подстроки с помощью метода `string.Split`. Метод `Split` принимает разделитель и возвращает массив подстрок.

Пример разбора текстового файла на слова:

```
string fileName = Path.Combine(AppContext.BaseDirectory, "poem.txt");
string fileContent = File.ReadAllText(fileName);

// Разбивка на строки по символу перевода строки
string[] lines = fileContent.Split('\n');
Console.WriteLine($"Строк в файле: {lines.Length}");

// Разбивка на слова по пробелу
string[] words = fileContent.Split(' ');
Console.WriteLine($"Слов в файле: {words.Length}");

// Разбивка одновременно по нескольким разделителям
char[] separators = { ' ', '\n', '\r', '\t' };
string[] tokens = fileContent.Split(separators,
    StringSplitOptions.RemoveEmptyEntries);
Console.WriteLine($"Токенов в файле: {tokens.Length}");

// Вывод файла по строкам, пробелы и концы строк заменены на символ |
foreach(string line in lines)
{
    string[] lineWords = line.Trim().Split(' ');
    foreach (var item in lineWords)
    {
        Console.Write(item.Trim());
        Console.Write('|');
    }
    Console.WriteLine();
}
```

6.2.6 Бинарные файлы

Для работы с бинарными файлами у класса `File` предусмотрены упрощённые методы — `ReadAllBytes` и `WriteAllBytes`. Они позволяют считать или записать файл как последовательность байтов/

Пример работы с бинарными файлами:

```
string sourceFile = Path.Combine(AppContext.BaseDirectory, "image.jpg");
string destFile = Path.Combine(AppContext.BaseDirectory,
    "image_copy.jpg");
```

```
// Чтение файла как последовательности байтов
byte[] fileBytes = File.ReadAllBytes(sourceFile);
Console.WriteLine($"Размер файла: {fileBytes.Length} байт");

// Запись байтов в новый файл
File.WriteAllBytes(destFile, fileBytes);
Console.WriteLine("Файл скопирован: " + destFile);
```

ЗАДАНИЕ

Напишите консольное приложение на C#, которое реализует простой файловый менеджер заметок. Программа должна выполнять следующие действия:

- При запуске проверить существование файла `notes.txt` в каталоге приложения (`File.Exists`). Если файл не существует — вывести сообщение «Файл заметок не найден, будет создан новый».
- Реализовать меню с вариантами:
- 1 — Добавить заметку: запросить у пользователя текст заметки и дописать её в файл с помощью `File.AppendAllText`. Перед каждой заметкой автоматически добавлять текущую дату и время в формате [дд.ММ.гггг ЧЧ:мм].
- 2 — Показать все заметки: прочитать файл с помощью `File.ReadLines` и вывести строки с нумерацией.
- 3 — Найти заметки по слову: запросить у пользователя слово для поиска, прочитать файл построчно и вывести только строки, содержащие введённое слово (использовать метод `string.Contains`).
- 4 — Очистить все заметки: перезаписать файл пустой строкой с помощью `File.WriteAllText`, вывести подтверждение.
- 0 — Выход: завершить программу.

РЕШЕНИЕ:

```

using System;
using System.IO;

// ===== МЕНЕДЖЕР ЗАМЕТОК =====

// Путь к файлу заметок в каталоге приложения
string notesFile = Path.Combine(AppContext.BaseDirectory, "notes.txt");

// Проверка существования файла при запуске
if (!File.Exists(notesFile))
{
    Console.WriteLine("Файл заметок не найден, будет создан новый.");
}
else
{
    Console.WriteLine($"Файл заметок найден: {notesFile}");
}

Console.WriteLine();

// Главный цикл меню – выполняется до выбора пункта 0
string choice = "";
do
{
    PrintMenu();
    choice = Console.ReadLine()?.Trim() ?? "";
    Console.WriteLine();

    switch (choice)
    {
        case "1":
            AddNote(notesFile);
            break;
        case "2":
            ShowAllNotes(notesFile);
            break;
        case "3":
            SearchNotes(notesFile);
            break;
        case "4":
            ClearNotes(notesFile);
            break;
        case "0":
            Console.WriteLine("До свидания!");
            break;
        default:
            Console.WriteLine("Неверный пункт меню. Попробуйте снова.");
            break;
    }

    Console.WriteLine();
}
while (choice != "0");

// ===== ФУНКЦИИ =====

```

```

/// <summary>
/// Выводит главное меню программы
/// </summary>
static void PrintMenu()
{
    Console.WriteLine("=====");
    Console.WriteLine("          МЕНЕДЖЕР ЗАМЕТОК          ");
    Console.WriteLine("=====");
    Console.WriteLine("  1 – Добавить заметку");
    Console.WriteLine("  2 – Показать все заметки");
    Console.WriteLine("  3 – Найти заметки по слову");
    Console.WriteLine("  4 – Очистить все заметки");
    Console.WriteLine("  0 – Выход");
    Console.WriteLine("=====");
    Console.Write("Ваш выбор: ");
}

/// <summary>
/// Добавляет новую заметку в конец файла с отметкой даты и времени
/// </summary>
/// <param name="filePath">Путь к файлу заметок</param>
static void AddNote(string filePath)
{
    Console.Write("Введите текст заметки: ");
    string noteText = Console.ReadLine() ?? "";

    // Проверка: заметка не должна быть пустой
    if (String.IsNullOrEmpty(noteText))
    {
        Console.WriteLine("Заметка не добавлена: текст пустой.");
        return;
    }

    // Формирование строки заметки с датой и временем
    // DateTime.Now возвращает текущую дату и время
    string timestamp = DateTime.Now.ToString("[dd.MM.yyyy HH:mm]");
    string noteLine = $"{timestamp} {noteText}{Environment.NewLine}";

    // File.AppendAllText – дозапись в конец файла
    // Если файл не существует, он будет создан автоматически
    File.AppendAllText(filePath, noteLine);

    Console.WriteLine("Заметка успешно добавлена.");
}

/// <summary>
/// Выводит все заметки из файла с нумерацией строк
/// </summary>
/// <param name="filePath">Путь к файлу заметок</param>
static void ShowAllNotes(string filePath)
{
    if (!File.Exists(filePath))
    {
        Console.WriteLine("Файл заметок не найден.");
    }
}

```

```

        return;
    }

    Console.WriteLine("----- Все заметки -----");

    int lineNumber = 1;

    // File.ReadLines – читает строки по одной без загрузки всего файла
    foreach (string line in File.ReadLines(filePath))
    {
        // Пропускаем пустые строки
        if (!String.IsNullOrEmpty(line))
        {
            // Выравнивание номера строки по правому краю в поле шириной 3
            Console.WriteLine($"{lineNumber,3}. {line}");
            lineNumber++;
        }
    }

    if (lineNumber == 1)
    {
        Console.WriteLine("Заметок пока нет.");
    }
    else
    {
        Console.WriteLine("-----");
        Console.WriteLine($"Итого заметок: {lineNumber - 1}");
    }
}

/// <summary>
/// Выводит только те заметки, которые содержат заданное слово
/// </summary>
/// <param name="filePath">Путь к файлу заметок</param>
static void SearchNotes(string filePath)
{
    if (!File.Exists(filePath))
    {
        Console.WriteLine("Файл заметок не найден.");
        return;
    }

    Console.Write("Введите слово для поиска: ");
    string searchWord = Console.ReadLine() ?? "";

    if (String.IsNullOrEmpty(searchWord))
    {
        Console.WriteLine("Слово для поиска не введено.");
        return;
    }

    Console.WriteLine($"---- Поиск по слову \"{searchWord}\" ----");

    int foundCount = 0;
    int lineNumber = 1;

```

```

foreach (string line in File.ReadLines(filePath))
{
    if (!String.IsNullOrEmpty(line))
    {
        // string.Contains – проверка вхождения подстроки
        // StringComparison.OrdinalIgnoreCase – поиск без учёта
регистра
        if (line.Contains(searchWord,
StringComparison.OrdinalIgnoreCase))
        {
            Console.WriteLine($"{lineNumber,3}. {line}");
            foundCount++;
        }
        lineNumber++;
    }
}

Console.WriteLine("-----");

if (foundCount == 0)
{
    Console.WriteLine($"Заметок со словом \"{searchWord}\" не
найденно.");
}
else
{
    Console.WriteLine($"Найдено заметок: {foundCount}");
}
}

/// <summary>
/// Удаляет все заметки после подтверждения пользователем
/// </summary>
/// <param name="filePath">Путь к файлу заметок</param>
static void ClearNotes(string filePath)
{
    if (!File.Exists(filePath))
    {
        Console.WriteLine("Файл заметок не найден.");
        return;
    }

    Console.Write("Вы уверены, что хотите удалить все заметки? (да/нет):
");
    string confirm = Console.ReadLine()?.Trim().ToLower() ?? "";

    if (confirm == "да")
    {
        // File.WriteAllText с пустой строкой перезаписывает файл целиком,
        // удаляя всё предыдущее содержимое
        File.WriteAllText(filePath, "");
        Console.WriteLine("Все заметки удалены.");
    }
    else

```

```

    {
        Console.WriteLine("Очистка отменена.");
    }
}

```

6.3 Оператор *using* и автоматическое освобождение ресурсов

Оператор `using` предназначен для работы с ресурсами, требующими освобождения. Например, это открытие файла, которое требует обязательного закрытия.

Оператор `using` используется в коде методов и его не следует путать с инструкцией `using`, которая используется в начале файла для подключения пространств имен.

```

// Фрагмент кода:
using (var file = File.OpenRead("data.txt"))
{
    Process(file);
}

```

```

// Компилятор превращает в:
var file = File.OpenRead("data.txt");
try
{
    Process(file);
}
finally
{
    if (file != null)
        ((IDisposable)file).Dispose();
}

```

Если файл не закрыть явным образом, то возникает ошибка.

Пример без использования using:

```

FileStream file = null;
try
{
    file = File.OpenRead("data.txt");
    // работа с файлом
}
finally
{
    file?.Dispose(); // освобождаем ресурс
}

```


Пример с использованием using:

```
using (FileStream file = File.OpenRead("data.txt"))
{
    // работа с файлом
} // Dispose() вызывается автоматически здесь!
```

В C# 8 появилась конструкция using declaration. В этом случае using ставится перед присваиванием внутри метода.

```
void ModernUsing()
{
    using FileStream file = File.OpenRead("data.txt");

    // работа с файлом
    // ...

} // Dispose() вызовется в конце области видимости
```

Детальный пример с использованием using:

```
using System;
using System.IO;

// ===== ЧТЕНИЕ ФАЙЛА РОЕМ.TXT =====

string poemFile = Path.Combine(AppContext.BaseDirectory, "поем.txt");

// Проверка существования файла (раздел 7.1)
if (!File.Exists(poemFile))
{
    Console.WriteLine($"Файл не найден: {poemFile}");
    return;
}

Console.WriteLine("=====");
Console.WriteLine("    Способ 1: try/finally без using    ");
Console.WriteLine("=====");
ReadWithTryFinally(poemFile);

Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine("    Способ 2: классический using { }    ");
Console.WriteLine("=====");
ReadWithUsing(poemFile);

Console.WriteLine();
Console.WriteLine("=====");
Console.WriteLine("    Способ 3: using declaration (C#8)    ");
Console.WriteLine("=====");
ReadWithUsingDeclaration(poemFile);

// ===== ФУНКЦИИ =====
```

```

/// <summary>
/// Чтение файла с помощью try/finally – явное освобождение ресурса.
/// Демонстрирует то, во что компилятор разворачивает оператор using.
/// </summary>
/// <param name="filePath">Путь к текстовому файлу</param>
static void ReadWithTryFinally(string filePath)
{
    StreamReader reader = null;

    try
    {
        // Открываем файл для чтения
        reader = new StreamReader(filePath);

        Console.WriteLine($"Файл открыт: {Path.GetFileName(filePath)}");
        Console.WriteLine(new string('-', 36));

        int lineNumber = 1;
        string line;

        // ReadLine возвращает null когда файл прочитан до конца
        while ((line = reader.ReadLine()) != null)
        {
            Console.WriteLine($"{lineNumber,3}: {line}");
            lineNumber++;
        }

        Console.WriteLine(new string('-', 36));
        Console.WriteLine($"Прочитано строк: {lineNumber - 1}");
    }
    finally
    {
        // Dispose() вызывается явно в блоке finally.
        // Оператор ?. защищает от ошибки если reader == null
        // (например, если файл не удалось открыть)
        reader?.Dispose();
        Console.WriteLine("StreamReader закрыт в блоке finally.");
    }
}

/// <summary>
/// Чтение файла с помощью классического оператора using.
/// Dispose() вызывается автоматически при выходе из блока using,
/// в том числе при возникновении исключения.
/// </summary>
/// <param name="filePath">Путь к текстовому файлу</param>
static void ReadWithUsing(string filePath)
{
    // Компилятор автоматически разворачивает этот блок
    // в конструкцию try/finally как в примере выше
    using (StreamReader reader = new StreamReader(filePath))
    {
        Console.WriteLine($"Файл открыт: {Path.GetFileName(filePath)}");
        Console.WriteLine(new string('-', 36));
    }
}

```

```

    int lineNumber = 1;
    string line;

    while ((line = reader.ReadLine()) != null)
    {
        Console.WriteLine($"{lineNumber,3}: {line}");
        lineNumber++;
    }

    Console.WriteLine(new string('-', 36));
    Console.WriteLine($"Прочитано строк: {lineNumber - 1}");

} // Dispose() вызывается автоматически здесь
Console.WriteLine("StreamReader закрыт автоматически.");
}

/// <summary>
/// Чтение файла с помощью using declaration – синтаксис C# 8.
/// Dispose() вызывается в конце области видимости метода.
/// Код короче: не нужны фигурные скобки блока using.
/// </summary>
/// <param name="filePath">Путь к текстовому файлу</param>
static void ReadWithUsingDeclaration(string filePath)
{
    // using перед объявлением переменной – конструкция C# 8
    // StreamReader будет закрыт в конце метода
    using StreamReader reader = new StreamReader(filePath);

    Console.WriteLine($"Файл открыт: {Path.GetFileName(filePath)}");
    Console.WriteLine(new string('-', 36));

    int lineNumber = 1;
    string line;

    while ((line = reader.ReadLine()) != null)
    {
        Console.WriteLine($"{lineNumber,3}: {line}");
        lineNumber++;
    }

    Console.WriteLine(new string('-', 36));
    Console.WriteLine($"Прочитано строк: {lineNumber - 1}");

} // Dispose() вызывается автоматически здесь – в конце метода

```

6.4 Класс *StringBuilder* для безопасного формирования строк

Класс `String` в C# содержит неизменяемые строки. Это означает, что при каждом применении оператора `+` для конкатенации строк в памяти

создаётся новый объект `String`, а старый становится недоступным и должен быть удалён сборщиком мусора.

Для небольших фрагментов кода такой подход не вызывает проблем. Однако при формировании больших текстовых отчётов в цикле создаётся большое количество ненужных строковых объектов, что может существенно снизить производительность приложения.

Для решения этой задачи платформа .NET предоставляет класс `StringBuilder` из пространства имён `System.Text`. Класс `StringBuilder` хранит внутренний изменяемый буфер и дописывает данные в его конец без создания промежуточных объектов.

Основные методы и свойства класса `StringBuilder`

- `Append(value)` — Дописывает значение в конец буфера
- `AppendLine(value)` — Дописывает значение и символ перевода строки
- `AppendFormat(format, args)` — Дописывает форматированную строку
- `Insert(index, value)` — Вставляет значение в указанную позицию
- `Remove(start, length)` — Удаляет фрагмент из буфера
- `Replace(old, new)` — Заменяет все вхождения подстроки
- `Clear()` — Очищает буфер
- `ToString()` — Возвращает содержимое буфера как `string`
- `Length` — Текущая длина содержимого буфера

Типичное применение `StringBuilder` — формирование текстового отчёта из нескольких источников данных с последующей записью в файл.

В следующем примере отчёт строится из массива данных, а затем сохраняется методом `File.WriteAllText`:

```
using System;
using System.IO;
using System.Text;

// ===== ФОРМИРОВАНИЕ ОТЧЁТА =====
```

```

// Исходные данные для отчёта
string[] subjects = { "Математика", "Физика", "Информатика",
                      "История",    "Химия",  "Английский" };

int[] scores = { 85, 92, 98, 76, 81, 88 };

string reportFile = Path.Combine(AppContext.BaseDirectory,
    "report.txt");

// Формирование отчёта и запись в файл
BuildAndSaveReport(subjects, scores, reportFile);

// Чтение файла и вывод содержимого (раздел 7.2)
Console.WriteLine("Содержимое сохранённого файла:");
Console.WriteLine(new string('=', 40));
foreach (string line in File.ReadLines(reportFile))
{
    Console.WriteLine(line);
}

// ===== ФУНКЦИИ =====

/// <summary>
/// Формирует текстовый отчёт с помощью StringBuilder
/// и сохраняет его в файл
/// </summary>
/// <param name="subjects">Массив названий предметов</param>
/// <param name="scores">Массив оценок</param>
/// <param name="filePath">Путь к файлу для сохранения
отчёта</param>
static void BuildAndSaveReport(string[] subjects,
                                int[] scores,
                                string filePath)
{
    StringBuilder sb = new StringBuilder();

    // Заголовок отчёта
    sb.AppendLine("=====");
    sb.AppendLine("                ОТЧЁТ ОБ УСПЕВАЕМОСТИ                ");
    sb.AppendLine("=====");
    sb.AppendFormat("Дата формирования: {0:dd.ММ.yyyy HH:mm}\n",
        DateTime.Now);
    sb.AppendLine("-----");

    // Формирование строк таблицы в цикле.
    // Если бы здесь использовался оператор «+»,
    // при каждой итерации создавался бы новый объект String
    int totalScore = 0;

```

```

for (int i = 0; i < subjects.Length; i++)
{
    string grade = GetGrade(scores[i]);

    // AppendFormat – форматированная строка с выравниванием
    sb.AppendFormat("{0,-15} {1,3} балла {2}\n",
        subjects[i], scores[i], grade);

    totalScore += scores[i];
}

// Итоговая строка
double average = (double)totalScore / scores.Length;

sb.AppendLine("-----");
sb.AppendFormat("Средний балл: {0:F1}\n", average);
sb.AppendFormat("Всего предметов: {0}\n", subjects.Length);
sb.AppendLine("=====");

// ToString() – получение итоговой строки из буфера
string reportText = sb.ToString();

// Запись в файл методом File.WriteAllText (раздел 7.2)
File.WriteAllText(filePath, reportText);

Console.WriteLine($"Отчёт сохранён: {filePath}");
Console.WriteLine($"Размер отчёта: {reportText.Length}
символов");
Console.WriteLine();
}

/// <summary>
/// Возвращает текстовую оценку по числовому баллу
/// </summary>
/// <param name="score">Числовой балл</param>
/// <returns>Текстовая оценка</returns>
static string GetGrade(int score)
{
    return score switch
    {
        >= 90 => "Отлично",
        >= 75 => "Хорошо",
        >= 60 => "Удовлетворительно",
        _ => "Неудовлетворительно"
    };
}

```

Содержимое созданного файла:

=====

ОТЧЁТ ОБ УСПЕВАЕМОСТИ

```

=====
Дата формирования: 06.03.2026 11:20
-----
Математика      85 балла  Хорошо
Физика          92 балла  Отлично
Информатика     98 балла  Отлично
История         76 балла  Хорошо
Химия           81 балла  Хорошо
Английский      88 балла  Хорошо
-----
Средний балл:   86,7
Всего предметов: 6
=====

```

ЗАДАНИЕ

Напишите консольное приложение на C#, которое запрашивает у пользователя несколько строк (до ввода пустой строки), формирует с помощью `StringBuilder` нумерованный список введенных строк и сохраняет результат в текстовый файл `list.txt`. Перед каждым пунктом списка добавьте порядковый номер и текущую дату. Используйте `File.ReadLines` для проверки записанного файла.

6.5 Стандартные потоки, перенаправление и конвейеры командной строки

Каждая консольная программа в операционной системе имеет три стандартных потока: поток ввода (`stdin`), поток вывода (`stdout`) и поток ошибок (`stderr`). Это справедливо и для Windows и для Linux и для macOS.

По умолчанию поток ввода связан с клавиатурой, а потоки вывода и ошибок — с окном консоли. Однако операционная система позволяет перенаправлять эти потоки с помощью специальных операторов командной строки. Программа при этом не изменяется — она по-прежнему вызывает `Console.ReadLine` и `Console.WriteLine`, но операционная система подменяет источник или приёмник данных.

Метод `Console.WriteLine` записывает строку в стандартный поток вывода (`stdout`). Метод `Console.Error.WriteLine` записывает строку в поток ошибок (`stderr`). Метод `Console.ReadLine` читает одну строку из стандартного потока ввода (`stdin`); когда поток ввода завершается, метод возвращает `null`.

Свойства `Console.IsInputRedirected` и `Console.IsOutputRedirected` возвращают `true`, если соответствующий поток перенаправлен (данные поступают не с клавиатуры или идут не на экран). Эти свойства позволяют программе адаптировать своё поведение — например, не выводить интерактивные подсказки, когда ввод поступает из файла.

Рассмотрим программу, которая объединяет в себе два режима — генерацию данных (`produce`) и их обработку (`consume`). Режим переключается аргументом командной строки. Программа работает исключительно со стандартными потоками, а перенаправление в файлы выполняется средствами операционной системы:

```
using System.Text;

Console.OutputEncoding = Encoding.UTF8;
Console.InputEncoding = Encoding.UTF8;

if (args.Length == 0)
{
    Console.Error.WriteLine("Использование:");
    Console.Error.WriteLine("  produce_consume produce  

— вывод данных");
    Console.Error.WriteLine("  produce_consume consume  

— обработка данных");
    Console.Error.WriteLine("  produce_consume produce > data.csv  

— вывод в файл");
}
```



```

    Console.Error.WriteLine("  produce_consume consume < data.csv
– ввод из файла");
    Console.Error.WriteLine("  produce_consume produce | tool consume
– конвейер");
    return;
}

switch (args[0].ToLower())
{
    case "produce":
        Produce();
        break;
    case "consume":
        Consume();
        break;
    default:
        Console.Error.WriteLine($"Неизвестная команда: {args[0]}");
        break;
}

void Produce()
{
    string[] cities = { "Москва", "Санкт-Петербург", "Новосибирск",
                        "Екатеринбург", "Казань" };
    Random rng = new(42);

    for (int i = 0; i < 10; i++)
    {
        string city = cities[rng.Next(cities.Length)];
        int temperature = rng.Next(-25, 36);
        // Вывод в stdout – может быть перенаправлен в файл или конвейер
        Console.WriteLine($"{city};{temperature}");
    }

    // Диагностика в stderr – всегда отображается на экране
    Console.Error.WriteLine("[produce] Сгенерировано 10 измерений");
}

void Consume()
{
    Dictionary<string, List<int>> data = new();
    int lineCount = 0;

    string? line;
    // ReadLine() вернёт null при окончании файла, потока или по
    Ctrl+Z/Ctrl+D
    while ((line = Console.ReadLine()) is not null)
    {
        lineCount++;
        string[] parts = line.Split(';');
        if (parts.Length != 2 || !int.TryParse(parts[1], out int temp))
        {
            Console.Error.WriteLine($"[consume] Пропущена строка:
{line}");
            continue;

```

```

    }

    string city = parts[0];
    if (!data.ContainsKey(city))
        data[city] = new List<int>();
    data[city].Add(temp);
}

Console.WriteLine($"Обработано строк: {lineCount}");
Console.WriteLine(new string('-', 40));
foreach (var pair in data.OrderBy(p => p.Key))
{
    double avg = pair.Value.Average();
    int min = pair.Value.Min();
    int max = pair.Value.Max();
    Console.WriteLine($"{pair.Key}: среднее={avg:F1}, мин={min},
макс={max}");
}

Console.Error.WriteLine($"[consume] Обработано {lineCount} строк");
}

```

Обратите внимание, что программа не содержит кода для работы с файлами — она пишет в Console.WriteLine и читает из Console.ReadLine. Вся гибкость достигается средствами операционной системы

Эту программу необходимо запускать из командной строки. Откроем командную строку (терминал), в Windows для этого используется “cmd”.

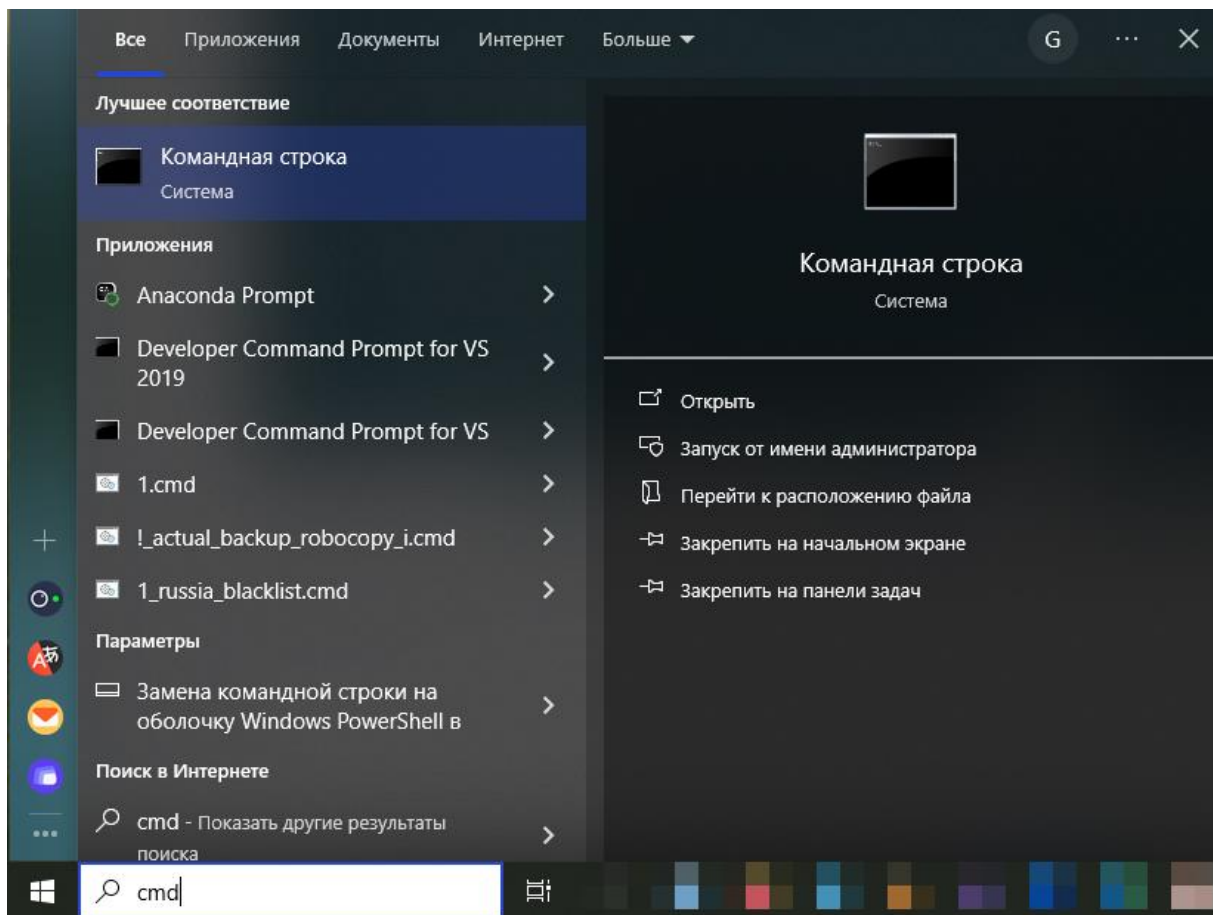


Рис. 17. Открытие cmd в Windows.

Соберем или пересоберем проект в Visual Studio.

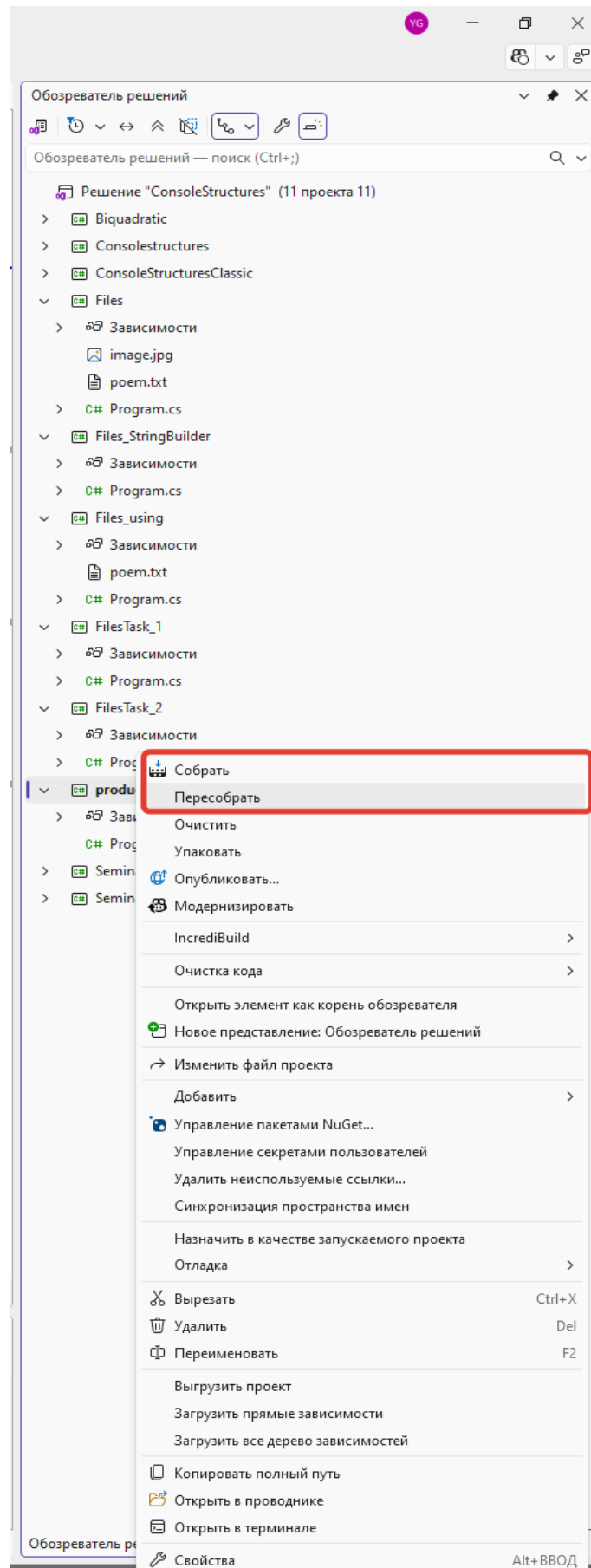


Рис. 18. Сборка или пересборка проекта.

Скопируем путь к решению и перейдем к нему в командной строке.

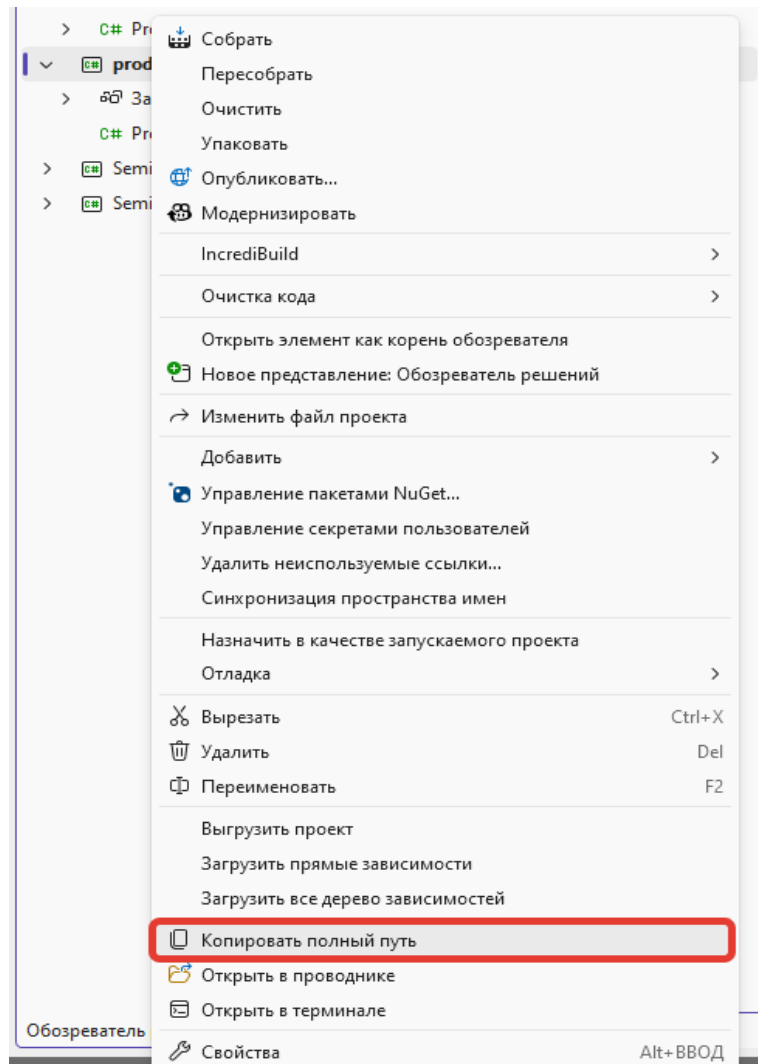


Рис. 19. Копирование пути.

Переключимся в открытую командную строку (терминал) и перейдем в каталог проекта. В Windows будем использовать команду “cd” для смены каталога, “dir” – для просмотра содержимого каталога, букву диска с двоеточием для перехода к другому логическому диску (если это Ваш проект расположен на другом диске), “cls” для очистки экрана, “type” для вывода на экран содержимого файла.

Из скопированного пути нужно удалить название файла проекта с расширением .csproj, после этого последовательно перейти в подкаталоги “bin”, “Debug”, “net...”. Экранная форма с результатами выполнения команд показана на следующем рисунке:

```

Командная строка

C:\Users>cd H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\produce_consume.csproj
Неверно задано имя папки.

C:\Users>cd H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\

C:\Users>h:

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume>dir
Том в устройстве H имеет метку SSD_STORE_2
Серийный номер тома: C8CC-6D2D

Содержимое папки H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume

11.03.2026  03:29  <DIR>      .
11.03.2026  03:29  <DIR>      ..
11.03.2026  03:27  <DIR>      bin
11.03.2026  05:58  <DIR>      obj
11.03.2026  03:27                253 produce_consume.csproj
11.03.2026  03:28                2 969 Program.cs
                2 файлов              3 222 байт
                4 папок   64 162 205 696 байт свободно

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume>cd bin

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin>dir
Том в устройстве H имеет метку SSD_STORE_2
Серийный номер тома: C8CC-6D2D

Содержимое папки H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin

11.03.2026  03:27  <DIR>      .
11.03.2026  03:27  <DIR>      ..
11.03.2026  03:27  <DIR>      Debug
                0 файлов              0 байт
                3 папок   64 162 205 696 байт свободно

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin>cd Debug

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug>dir
Том в устройстве H имеет метку SSD_STORE_2
Серийный номер тома: C8CC-6D2D

Содержимое папки H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug

11.03.2026  03:27  <DIR>      .
11.03.2026  03:27  <DIR>      ..
11.03.2026  05:58  <DIR>      net10.0
                0 файлов              0 байт
                3 папок   64 162 205 696 байт свободно

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug>cd net10.0

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>dir
Том в устройстве H имеет метку SSD_STORE_2
Серийный номер тома: C8CC-6D2D

Содержимое папки H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0

11.03.2026  05:58  <DIR>      .
11.03.2026  05:58  <DIR>      ..
11.03.2026  05:58                439 produce_consume.deps.json
11.03.2026  05:58                8 192 produce_consume.dll
11.03.2026  05:58               162 304 produce_consume.exe
11.03.2026  05:58               11 720 produce_consume.pdb
11.03.2026  05:58                270 produce_consume.runtimeconfig.json
                5 файлов             182 925 байт
                2 папок   64 162 205 696 байт свободно

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>

```

Рис. 20. Результаты выполнения команд в командной строке.

Далее можно выполнять команды конвейера и перенаправления в файл (из файла). Примеры показаны на экранной форме:

```

Командная строка

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>dir
Том в устройстве H имеет метку SSD_STORE_2
Серийный номер тома: C8CC-6D2D

Содержимое папки H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0

11.03.2026  05:58    <DIR>          .
11.03.2026  05:58    <DIR>          ..
11.03.2026  05:58                439 produce_consume.deps.json
11.03.2026  05:58                8 192 produce_consume.dll
11.03.2026  05:58            162 304 produce_consume.exe
11.03.2026  05:58            11 720 produce_consume.pdb
11.03.2026  05:58            270 produce_consume.runtimeconfig.json
                    5 файлов            182 925 байт
                    2 папок             64 162 205 696 байт свободно

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>produce_consume.exe produce > 1.txt
[produce] Сгенерировано 10 измерений

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>type 1.txt
Екатеринбург;-17
Москва;6
Москва;-9
Екатеринбург;6
Москва;21
Санкт-Петербург;-10
Новосибирск;-6
Санкт-Петербург;-10
Новосибирск;-23
Казань;10

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>produce_consume.exe consume < 1.txt
Обработано строк: 10
-----
Екатеринбург: среднее=-5,5, мин=-17, макс=6
Казань: среднее=10,0, мин=10, макс=10
Москва: среднее=6,0, мин=-9, макс=21
Новосибирск: среднее=-14,5, мин=-23, макс=-6
Санкт-Петербург: среднее=-10,0, мин=-10, макс=-10
[consume] Обработано 10 строк

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>produce_consume.exe produce | produce_consume.exe consume
[produce] Сгенерировано 10 измерений
Обработано строк: 10
-----
Екатеринбург: среднее=-5,5, мин=-17, макс=6
Казань: среднее=10,0, мин=10, макс=10
Москва: среднее=6,0, мин=-9, макс=21
Новосибирск: среднее=-14,5, мин=-23, макс=-6
Санкт-Петербург: среднее=-10,0, мин=-10, макс=-10
[consume] Обработано 10 строк

H:\projects\course_architect\test_1_lect\ConsoleStructures\produce_consume\bin\Debug\net10.0>

```

Рис. 21. Результаты вызова программы в командной строке.

Оператор `>` перенаправляет стандартный поток вывода программы в файл. Если файл существует, он будет перезаписан:

```
produce_consume.exe produce > data.csv
```

В этом примере всё, что программа выводит через `Console.WriteLine`, будет записано в файл `data.csv`, а не на экран. При этом сообщения, выводимые через `Console.Error.WriteLine`, по-прежнему отобразятся в консоли — оператор `>` действует только на `stdout`.

Оператор `>>` также перенаправляет `stdout` в файл, но не перезаписывает его, а дописывает данные в конец:

```
produce_consume.exe produce >> data.csv
produce_consume.exe produce >> data.csv
```

После выполнения обеих команд файл `data.csv` будет содержать результаты двух запусков программы.

Оператор `<` подаёт содержимое файла на стандартный поток ввода программы. Программа читает данные через `Console.ReadLine` так же, как если бы они вводились с клавиатуры, но источником является файл:

```
produce_consume.exe consume < data.csv
```

Когда файл прочитан до конца, `Console.ReadLine` возвращает `null` — точно так же, как при завершении потока в конвейере. Именно поэтому стандартный шаблон чтения `while ((line = Console.ReadLine()) is not null)` одинаково корректно работает и с клавиатурой (завершение по `Ctrl+Z` в Windows, `Ctrl+D` в Linux/macOS), и с перенаправлением из файла, и с конвейером.

Поток ошибок имеет числовой дескриптор 2. Операторы `2>` и `2>>` перенаправляют `stderr` в файл:

```
produce_consume.exe consume < data.csv 2> errors.log
```

В этом примере результаты обработки отображаются на экране (`stdout` не перенаправлен), а все диагностические сообщения записываются в файл `errors.log`.

Можно перенаправить и `stdout`, и `stderr` одновременно, каждый в свой файл:

```
produce_consume.exe consume < data.csv > report.txt 2> errors.log
```

Запись `2>&1` перенаправляет поток ошибок туда же, куда направлен поток вывода. Это полезно, когда нужно сохранить весь вывод программы в один файл:

```
produce_consume.exe consume < data.csv > all_output.txt 2>&1
```


Оператор `|` (канал, `pipe`) перенаправляет стандартный поток вывода одной программы на стандартный поток ввода другой:

```
produce_consume.exe produce | produce_consume.exe consume
```

Всё, что первая программа записывает через `Console.WriteLine`, вторая программа получает через `Console.ReadLine`. Конвейер может включать произвольное количество программ:

```
programA | programB | programC
```

В этом случае `stdout` программы `A` подаётся на `stdin` программы `B`, а `stdout` программы `B` — на `stdin` программы `C`. Поток ошибок каждой программы не затрагивается конвейером и выводится на экран — именно поэтому диагностические сообщения следует всегда направлять в `Console.Error`.

Операторы перенаправления и конвейеры можно комбинировать. Это позволяет строить гибкие цепочки обработки данных.

Чтение из файла, обработка в конвейере, вывод на экран:

```
produce_consume.exe produce < input.csv | produce_consume.exe consume
```

Генерация в конвейере, запись конечного результата в файл:

```
produce_consume.exe produce | produce_consume.exe consume > report.txt
```

Полная цепочка: чтение из файла, обработка, запись результата в файл, ошибки — в отдельный файл:

```
produce_consume.exe produce | produce_consume.exe filter 4 |  
produce_consume.exe consume > report.txt 2> errors.log
```

ЗАДАНИЕ

Напишите консольное приложение на C#, которое работает в двух режимах, переключаемых аргументом командной строки. Программа должна работать только со стандартными потоками ввода-вывода, без явного открытия файлов в коде.

Режим `generate` — генерация данных. Программа выводит в `stdout` 20 строк вида `Фамилия;Предмет;Оценка`. Фамилии (не менее 5 различных), предметы (не менее 3 различных) и оценки (от 2 до 5) выбираются случайным образом.

Режим `filter` — фильтрация данных. Программа читает строки из `stdin` и выводит в `stdout` только те строки, в которых оценка не ниже значения, переданного вторым аргументом командной строки. Например, при запуске с аргументами `filter 4` на выход передаются только строки с оценками 4 и 5.

Все диагностические и справочные сообщения должны выводиться через `Console.Error`.

Продемонстрируйте работу программы следующими командами:

**`program.exe generate` — просмотр данных на экране
`program.exe generate > grades.csv` — сохранение данных в файл**

program.exe generate | program.exe filter 4 — фильтр из двух программ

program.exe generate | program.exe filter 4 > good.csv — фильтрация с сохранением в файл

program.exe generate | program.exe filter 4 > report.txt 2> diag.log — результат в файл, диагностика в отдельный файл

РЕШЕНИЕ:

```
// — Данные —————
string[] surnames =
{
    "Иванов", "Петров", "Сидоров", "Кузнецов", "Смирнов",
    "Попов", "Волков", "Новиков"
};

string[] subjects =
{
    "Математика", "Физика", "Информатика", "История", "Химия"
};

// — Разбор аргументов —————

if (args.Length == 0)
{
    PrintUsage();
    return;
}

string mode = args[0].ToLower();

switch (mode)
{
    case "generate":
        Generate();
        break;

    case "filter":
        if (args.Length < 2)
        {
```

```

        Console.Error.WriteLine("[ОШИБКА] Режим filter требует второй
аргумент – минимальную оценку.");
        Console.Error.WriteLine("Пример: program.exe filter 4");
        return;
    }
    if (!int.TryParse(args[1], out int minGrade) || minGrade < 2 ||
minGrade > 5)
    {
        Console.Error.WriteLine($"[ОШИБКА] Некорректная оценка:
\"{args[1]}\". Допустимые значения: 2..5.");
        return;
    }
    Filter(minGrade);
    break;

    default:
        Console.Error.WriteLine($"[ОШИБКА] Неизвестный режим:
\"{mode}\".");
        PrintUsage();
        break;
}

// — Локальные функции —————

void PrintUsage()
{
    Console.Error.WriteLine("=== Программа обработки оценок ===");
    Console.Error.WriteLine("Использование:");
    Console.Error.WriteLine("  program.exe generate          –
сгенерировать 20 строк с оценками");
    Console.Error.WriteLine("  program.exe filter <мин_оценка> –
отфильтровать строки из stdin");
    Console.Error.WriteLine();
    Console.Error.WriteLine("Примеры:");
    Console.Error.WriteLine("  program.exe generate > grades.csv");
    Console.Error.WriteLine("  program.exe generate | program.exe filter
4");
    Console.Error.WriteLine("  program.exe generate | program.exe filter 4
> good.csv");
}

void Generate()
{
    Console.Error.WriteLine("[generate] Генерация 20 записей...");

    var rnd = new Random();

    for (int i = 0; i < 20; i++)
    {
        string surname = surnames[rnd.Next(surnames.Length)];
        string subject = subjects[rnd.Next(subjects.Length)];
        int grade = rnd.Next(2, 6); // от 2 до 5 включительно

        Console.WriteLine($"{surname};{subject};{grade}");
    }
}

```

```

    Console.Error.WriteLine("[generate] Готово. Выведено 20 строк в
stdout.");
}

void Filter(int minGrade)
{
    Console.Error.WriteLine($"[filter] Фильтрация: оценка >= {minGrade}");

    int totalRead = 0;
    int totalPassed = 0;
    string? line;

    while ((line = Console.ReadLine()) != null)
    {
        totalRead++;

        string[] parts = line.Split(';');

        if (parts.Length < 3)
        {
            Console.Error.WriteLine($"[filter] Пропуск некорректной строки
#{totalRead}: \"{line}\"");
            continue;
        }

        if (!int.TryParse(parts[2].Trim(), out int grade))
        {
            Console.Error.WriteLine($"[filter] Не удалось разобрать оценку
в строке #{totalRead}: \"{line}\"");
            continue;
        }

        if (grade >= minGrade)
        {
            Console.WriteLine(line);
            totalPassed++;
        }
    }

    Console.Error.WriteLine($"[filter] Готово. Прочитано: {totalRead},
прошло фильтр: {totalPassed}, отброшено: {totalRead - totalPassed}.");
}

```

7 Работа со стандартными коллекциями

Коллекции – важная часть любого языка программирования. В языке C# существует развитая система коллекций. В данном разделе сначала рассмотрены примеры использования стандартных коллекций, а затем примеры создания нестандартных коллекций.

Подходы к работе с коллекциями в языках C++, Java и C# в целом совпадают. Все коллекции можно разделить на две большие категории – обобщенные (шаблонизированные в C++) и необобщенные.

Использование обобщенной коллекции предполагает, что в нее помещаются элементы обобщенного типа, следовательно все элементы коллекции должны быть одного и того же типа, который указывается при создании обобщенной коллекции. В соответствии с принципами ООП, вместо указанного обобщенного типа могут применяться производные типы. Классы обобщенных коллекций находятся в пространстве имен `System.Collections.Generic`.

Использование необобщенной коллекции предполагает, что в нее помещаются элементы типа `object` – самого базового типа в иерархии типов, а значит, элементы различных типов. Основные классы необобщенных коллекций находятся в пространстве имен `System.Collections`. Список основных классов коллекций приведен в таблице 5.

Таблица 5

Список основных классов коллекций

Коллекция	Обобщенный класс	Необобщенный класс
Список	<code>List<T></code>	<code>ArrayList</code>
Стек	<code>Stack<T></code>	<code>Stack</code>
Очередь	<code>Queue<T></code>	<code>Queue</code>
Множество	<code>HashSet<T></code>	–

Ассоциативный массив, хранящий пары ключ-значение. Такую структуру называют также словарем или хэш-таблицей	Dictionary<TKey, TValue>	Hashtable
---	--------------------------	-----------

Обобщенные коллекции на практике используются чаще, поэтому в шаблоне консольного приложения в Visual Studio по умолчанию подключено пространство имен обобщенных коллекций System.Collections.Generic, а пространство имен необобщенных коллекций System.Collections не подключено. В случае необходимости его следует добавить вручную с помощью ключевого слова using.

В таблице 5 приведены только основные коллекции. В .NET существуют десятки классов коллекций, в том числе различные специализированные, предназначенные для решения конкретных задач (например, битовые коллекции), коллекции для безопасного многопоточного доступа и т.д.

7.1 Обобщенный список

Рассмотрим пример работы с обобщенным списком. Создание объекта списка:

```
List<int> li = new List<int>();
```

Добавление элементов в список:

```
li.Add(1);
li.Add(2);
li.Add(3);
```

В версии языка C# 3.0 появился более удобный синтаксис для инициализации списков:

```
List<int> li1 = new List<int>
{
```

```

        1,2,3
    };

    List<string> li1_str = new List<string>
    {
        "строка 1",
        "строка 2",
        "строка 3"
    };

```

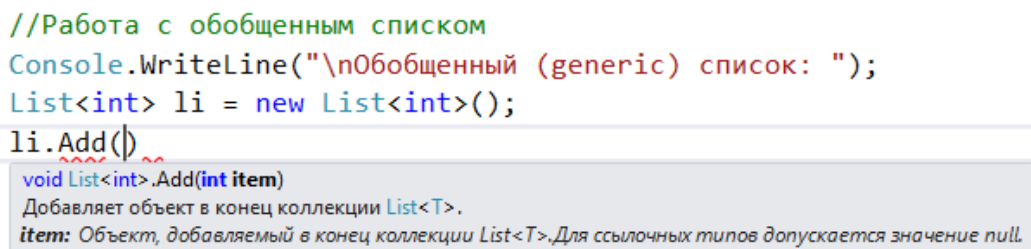
Вывод элементов списка:

```

foreach (int i in li)
{
    Console.WriteLine(i);
}

```

В данном примере в качестве класса-обобщения используется `int`, поэтому создается список целых чисел. Преимущества применения обобщенной коллекции состоят в том, что тип элемента известен как при добавлении, так и при выводе. При добавлении Visual Studio подсказывает тип добавляемого элемента с помощью механизма IntelliSense (рис. 22). Тип элемента уже известен, так как он был ранее указан при создании списка.



```

//Работа с обобщенным списком
Console.WriteLine("\nОбобщенный (generic) список: ");
List<int> li = new List<int>();
li.Add()

```

void List<int>.Add(int item)
 Добавляет объект в конец коллекции List<T>.
 item: Объект, добавляемый в конец коллекции List<T>. Для ссылочных типов допускается значение null.

Рис. 22. Работа IntelliSense для добавления в обобщенный список.

При выводе с использованием цикла `foreach` тип переменной `i` также заранее известен.

Для обобщенного списка перегружен индексатор, который позволяет получить элемент по его номеру в коллекции, элементы нумеруются с нуля. Пример:

```
int item2 = li[1];
```

Свойство `Count` возвращает количество элементов в коллекции:

```
int count = li.Count;
```


Метод `Contains` позволяет проверить существование элемента в коллекции. В данном примере проверяется существование в коллекции числа «3»:

```
bool is3 = li.Contains(3);
```

Метод `Remove` позволяет удалить элемент из коллекции, здесь из списка удаляется число «2»:

```
li.Remove(2);
```

Основное преимущество стандартных коллекций языка C# состоит в том, что для их обработки может быть использован интегрированный язык запросов LINQ. Это чрезвычайно облегчает обработку сложных данных в C#.

Начиная с C# 9 при создании объекта коллекции можно опустить имя типа в правой части выражения, если тип уже указан в левой части (target-typed new):

```
// Старый синтаксис
List<int> li1 = new List<int>();

// Новый синтаксис начиная с C# 9
List<int> li2 = new();

// С инициализацией
List<int> li13 = new() { 1, 2, 3 };
Dictionary<int, string> d1 = new() { [1] = "один", [2] = "два" };
```

Такой синтаксис значительно сокращает объём кода при работе с коллекциями.

В C# 12 появился ещё более компактный синтаксис — выражения коллекций (collection expressions). Квадратные скобки [...] позволяют единообразно инициализировать массивы, списки, множества и другие типы, поддерживающие данный синтаксис:

```
// Массив
int[] arr = [1, 2, 3];

// Список
List<int> li3 = [1, 2, 3];
```

```
// Оператор spread (..) – объединение коллекций
List<int> first = [1, 2, 3];
List<int> second = [4, 5, 6];
List<int> merged = [.. first, .. second]; // [1, 2, 3, 4, 5, 6]
```

Оператор `spread ..` позволяет встраивать содержимое одной коллекции в другую прямо при инициализации. Тип целевой коллекции компилятор определяет из контекста (левой части выражения), поэтому один и тот же литерал `[1, 2, 3]` может создать массив, список или другой подходящий тип.

Ограничение рассмотренного примера заключается в том, что в такой список можно поместить только целые числа и нельзя — элементы других типов.

Если в список необходимо сохранять элементы различных типов, то можно использовать необобщенный список.

7.2 Необобщенный список

Рассмотрим работу с необобщенным списком. Создание объекта списка:

```
ArrayList al = new ArrayList();
```

Добавление элементов различных типов в список:

```
al.Add(333);
al.Add(123.123);
al.Add(«строка»);
```

Вывод элементов списка:

```
foreach (object o in al)
{
    //Для определения типа используется механизм рефлексии
    string type = o.GetType().Name;
    if (type == «Int32»)
    {
        Console.WriteLine(«Целое число: « + o.ToString());
    }
    else if (type == «String»)
    {
        Console.WriteLine(«Строка: « + o.ToString());
    }
}
```

```

    }
    else
    {
        Console.WriteLine(«Другой тип: « + o.ToString());
    }
}

```

Преимущество необобщенного списка – возможность добавления элементов различных типов.

Однако при чтении элементов возникает проблема, связанная с тем, что точно неизвестен тип элемента, который получен из списка, потому что список возвращает элемент базового типа `object`. Решить эту проблему можно с использованием механизма рефлексии.

Механизм рефлексии подробнее будет рассмотрен ниже, здесь же используется возможность получения информации о реальном типе элемента, получаемого из списка. Вызов метода `o.GetType()` для объекта возвращает информацию о его типе данных, а свойство `Name` позволяет получить строковое имя типа. Далее осуществляется ветвление по строковому имени типа и выполняются различные действия в зависимости от различных значений типа данных.

7.3 Обобщенные стек и очередь

Стек и очередь отличаются от списка в первую очередь тем, что здесь производится не перебор элементов коллекции, а доступ к первому или последнему элементу коллекции. Принцип работы стека и очереди показан на рис 23.

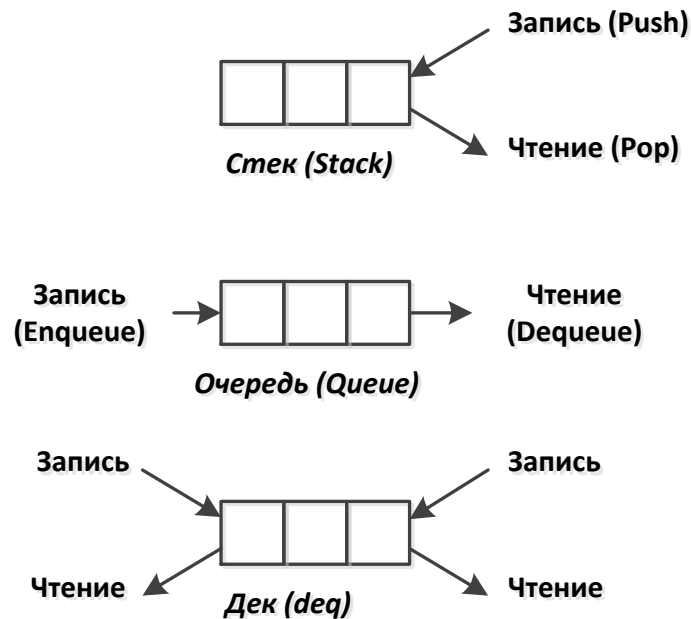


Рис. 23. Стек, очередь и дек.

В случае стека (класс `Stack`) работа происходит с так называемой вершиной стека, которую условно можно считать первым или последним элементом списка данных. Запись (`Push`) производится в вершину стека, чтение (`Pop`) также производится из вершины стека. При чтении элемент автоматически удаляется из стека. Стек реализует принцип работы LIFO (last in, first out – последний вошел, первый вышел).

В случае очереди (класс `Queue`) запись (`Enqueue`) выполняется в конец очереди, а чтение (`Dequeue`) – из начала очереди. При чтении элемент автоматически удаляется из очереди. Очередь реализует принцип работы FIFO (first in, first out – первый вошел, первый вышел).

Стек и очередь являются частным случаем структуры, которую принято называть дек (`deq` – double ended queue, двунаправленная очередь). В этом случае чтение и запись элементов могут проводиться с обоих концов списка данных. В .NET не существует стандартного библиотечного класса, реализующего дек.

Рассмотрим работу со стеком.

Создание стека:

```
Stack<int> st = new Stack<int>();
```

Добавление элементов в стек:

```
st.Push(1);
st.Push(2);
st.Push(3);
```

В реализации стека для .NET возможен перебор элементов с помощью foreach, хотя в классическом стеке такое действие предусматривать не обязательно:

```
foreach(int i in st)
{
    Console.WriteLine(i);
}
```

С использованием метода Peek возможно чтения элемента из вершины стека без удаления (прочитанный элемент не удаляется из вершины стека):

```
int i3 = q.Peek();
```

С применением метода Pop выполняется чтение элемента из вершины стека и его удаление:

```
while (st.Count > 0)
{
    int i = st.Pop();
    Console.WriteLine(i);
}
```

Рассмотрим работу с очередью.

Создание очереди:

```
Queue<int> q = new Queue<int>();
```

Добавление элементов в очередь:

```
q.Enqueue(11);
q.Enqueue(22);
q.Enqueue(33);
```

В реализации очереди для .NET возможен перебор элементов с помощью foreach:

```
foreach (int i in q)
{
    Console.WriteLine(i);
}
```

С использованием метода Peek возможно чтения элемента из начала очереди без удаления:

```
int i3 = q.Peek();
```

С применением метода Dequeue осуществляется чтение элемента из начала очереди и его удаление:

```
while (q.Count > 0)
{
    int i = q.Dequeue();
    Console.WriteLine(i);
}
```

Результаты вывода в консоль для стека и очереди:

Стек:

Перебор элементов стека с помощью foreach:

3

2

1

Получение верхнего элемента без удаления:3

Чтение с удалением элементов из стека:

3

2

1

Очередь:

Перебор элементов очереди с помощью foreach:

11

22

33

Получение первого элемента без удаления:11

Чтение с удалением элементов из очереди:

11

22

33

Необходимо отметить, что чтение данных из стека выполняется в обратном порядке (в соответствии с принципом LIFO), а чтение данных из очереди – в прямом (в соответствии с принципом FIFO).

7.4 Обобщенный словарь

Работа со словарем отличается от работы со списком в основном только тем, что в словарь добавляются пары ключ-значение.

В других языках программирования словарь могут также называть ассоциативным (или ассоциированным) массивом или хэш-таблицей.

Создание объекта словаря:

```
Dictionary<int, string> d = new Dictionary<int, string>();
```

При создании словаря указывается два обобщенных типа – тип ключа и тип значения.

Добавление элементов:

```
d.Add(1, "строка 1");
d.Add(2, "строка 2");
d.Add(3, "строка 3");
```

В версии языка C# 6 появился более удобный синтаксис для инициализации словарей:

```
Dictionary<int, string> d1 = new Dictionary<int, string>
{
    [1] = "строка 1",
    [2] = "строка 2",
    [3] = "строка 3"
};
```

Вывод элементов:

```
foreach (KeyValuePair<int, string> v in d)
{
    Console.WriteLine(v.Key.ToString() + "-" + v.Value);
}
```

При выводе элементов возникает проблема, связанная с тем, что в цикле `foreach` необходимо объявить тип данных, который содержит пару ключ-значение, читаемые из словаря. В качестве такого типа в .NET используется тип `KeyValuePair<TKey, TValue>`. Объект такого типа содержит свойства `Key` и `Value`, возвращающие ключ и значение пары соответственно.

У объекта класса словаря существуют коллекции ключей и значений: `Keys` и `Values`.

Пример вывода ключей и значений:

```
Console.Write("\nКлючи: ");
foreach (int i in d.Keys)
{
    Console.Write(i.ToString() + " ");
}

Console.Write("\nЗначения: ");
foreach (string str in d.Values)
{
    Console.Write(str + " ");
}
```

Для получения значения по ключу можно использовать перегруженный индексатор, что аналогично применению индексатора для списка.

Пример получения значения по ключу:

```
int key = 3;
string val = d[key];
Console.WriteLine("\nДля ключа '" + key.ToString() +
    "' значение '" + val + "'");
```

Однако если указать ключ, отсутствующий в словаре, то это приведет к возникновению исключения `KeyNotFoundException`.

Для решения данной задачи можно также использовать метод `TryGetValue`. Он не генерирует исключение, а возвращает логическое значение: `true` если удалось прочитать значение по ключу и `false` если не удалось.

Пример использования метода `TryGetValue`:

```
string val2 = "";
bool res = d.TryGetValue(key, out val2);
if (res)
{
    Console.WriteLine("\nДля ключа '" + key.ToString() +
        "' значение '" + val2 + "'");
}
```


В отличие от метода `Add`, который генерирует исключение `ArgumentException` при попытке добавить уже существующий ключ, метод `TryAdd` не генерирует исключение, а возвращает `false`:

```
Dictionary<int, string> d2 = new() { [1] = "один" };
bool added1 = d2.TryAdd(2, "два");    // true  – ключ добавлен
bool added2 = d2.TryAdd(1, "один-2"); // false – ключ уже существует
```

Пару ключ-значение при переборе можно деконструировать напрямую в кортеж, не объявляя переменную типа `KeyValuePair<>`:

```
// Старый стиль
foreach (KeyValuePair<int, string> v in d2)
    Console.WriteLine($"{v.Key} - {v.Value}");

// Современный стиль – деконструкция (C# 7+)
foreach (var (k, v) in d2)
    Console.WriteLine($"{k} - {v}");
```

Оба варианта эквивалентны, но второй удобнее.

Обобщенные словари, как и обобщенные списки, часто используются в программах на C#.

ЗАДАНИЕ

Напишите консольное приложение на языке C#, реализующее упрощённую систему учёта студентов с использованием основных обобщённых коллекций.

Программа должна выполнять следующие действия:

Создать `List<string>` с именами пяти студентов, используя синтаксис инициализации. Вывести список. Добавить шестого студента методом `Add`. Проверить наличие студента с заданным именем через `Contains`.

Удалить одного студента через Remove. Вывести итоговый список с нумерацией.

Создать Stack<string> — стек для моделирования истории ответов на экзамене (последний ответивший стоит «на вершине»). Добавить в стек имена трёх студентов. Вывести вершину стека без удаления (Peek). Извлечь всех студентов из стека с выводом (Pop), продемонстрировав порядок LIFO.

Создать Queue<string> — очередь студентов на сдачу лабораторной работы. Добавить в очередь четырёх студентов. Вывести первого в очереди без удаления (Peek). По одному «принять лабораторную» — извлечь всех из очереди с выводом (Dequeue), продемонстрировав порядок FIFO.

Создать Dictionary<string, int> — словарь «имя студента → оценка». Заполнить пятью записями, используя синтаксис инициализатора. Вывести все пары ключ-значение с использованием деконструкции. Найти оценку конкретного студента через TryGetValue. Вычислить и вывести среднюю оценку по группе.

РЕШЕНИЕ:

```
// — 1. List<string>
```

```

        Console.WriteLine("=====");
Console.WriteLine("1. Список студентов (List<string>);");
Console.WriteLine("=====");

// C# 3.0 — синтаксис инициализации коллекции
List<string> students = new List<string>
{
    "Иванов",
    "Петров",
    "Сидоров",
    "Козлов",
    "Новиков"
};

        Console.WriteLine("Исходный список:");
foreach (string s in students)
    Console.WriteLine("  " + s);

// Добавление нового студента
students.Add("Морозов");
Console.WriteLine("\nПосле добавления Морозова:");
foreach (string s in students)
    Console.WriteLine("  " + s);

// Проверка наличия
string searchName = "Петров";
bool found = students.Contains(searchName);
Console.WriteLine($"Студент '{searchName}' в списке: {found}");

// Удаление
students.Remove("Сидоров");
Console.WriteLine("\nПосле удаления Сидорова (с нумерацией):");
for (int i = 0; i < students.Count; i++)
    Console.WriteLine($"  {i + 1}. {students[i]}");

// — 2. Stack<string>


---



Console.WriteLine("\n=====");
Console.WriteLine("2. История ответов на экзамене (Stack<string>);");
Console.WriteLine("=====");

Stack<string> examHistory = new Stack<string>();
    examHistory.Push("Иванов");
examHistory.Push("Козлов");
examHistory.Push("Новиков");

// Peek — без удаления
Console.WriteLine($"Последний ответивший (Peek): {examHistory.Peek()}");

Console.WriteLine("Извлечение из стека (порядок LIFO):");
while (examHistory.Count > 0)
{
    Console.WriteLine($"  Ответил: {examHistory.Pop()}");
}

```

```
// — 3. Queue<string>
```

```
Console.WriteLine("\n=====");
Console.WriteLine("3. Очередь на сдачу лабораторной (Queue<string>)");
Console.WriteLine("=====");
```

```
Queue<string> labQueue = new Queue<string>();
labQueue.Enqueue("Иванов");
labQueue.Enqueue("Петров");
labQueue.Enqueue("Козлов");
labQueue.Enqueue("Морозов");
```

```
// Peek — без удаления
```

```
Console.WriteLine($"Первый в очереди (Peek): {labQueue.Peek()}");
```

```
Console.WriteLine("Приём лабораторных (порядок FIFO):");
```

```
while (labQueue.Count > 0)
```

```
{
    Console.WriteLine($" Принята работа у: {labQueue.Dequeue()}");
}
```

```
// — 4. Dictionary<string, int>
```

```
Console.WriteLine("\n=====");
Console.WriteLine("4. Оценки студентов (Dictionary<string, int>)");
Console.WriteLine("=====");
```

```
// C# 6 — синтаксис инициализатора индексов
```

```
Dictionary<string, int> grades = new Dictionary<string, int>
```

```
{
    ["Иванов"] = 5,
    ["Петров"] = 4,
    ["Козлов"] = 3,
    ["Новиков"] = 5,
    ["Морозов"] = 4
};
```

```
// Вывод с деконструкцией KeyValuePair (C# 7+)
```

```
Console.WriteLine("Все оценки:");
```

```
foreach (var (name, grade) in grades)
    Console.WriteLine($" {name}: {grade}");
```

```
// TryGetValue
```

```
string target = "Козлов";
```

```
if (grades.TryGetValue(target, out int foundGrade))
```

```
    Console.WriteLine($" \nОценка {target}: {foundGrade}");
```

```
else
```

```
    Console.WriteLine($" \nСтудент '{target}' не найден");
```

```
// Средняя оценка
```

```
double avg = 0;
```

```
foreach (var (_, grade) in grades) // _ — discard, имя не нужно
```

```

    avg += grade;
    avg /= grades.Count;
    Console.WriteLine($"Средняя оценка по группе: {avg:F2}");

```

7.5 Кортеж

Кортеж – это сравнительно новый вид коллекции, который появился в C# 4.0. Кортеж является не совсем классической коллекцией. Если, например, список можно представить в виде таблицы из одного столбца и множества строк, а словарь в виде таблицы из двух столбцов и множества строк, то кортеж можно представить в виде таблицы из одной строки и множества столбцов. Если список и словарь имеют фиксированную ширину и могут расти только «по вертикали», то кортеж, наоборот, имеет фиксированную высоту и может расти только «по горизонтали».

Столбцы определяются динамически с помощью обобщенных типов. Пример объявления кортежа:

```

Tuple<int, string, string> group =
    new Tuple<int, string, string>(1, "ИУ", "ИУ-5");

```

Если лямбда-выражения позволяют на лету объявлять методы, то кортежи – структуры, подобные объектам классов.

Когда программист хочет объявить какую-либо вспомогательную структуру данных и не желает объявлять класс, он может воспользоваться кортежем. Однако, недостаток кортежей состоит в том, что поля кортежа в IntelliSense представляются не в виде имен (как в классе) а в виде номеров полей (Item1 ... ItemN). Пример представлен на рис. 21.

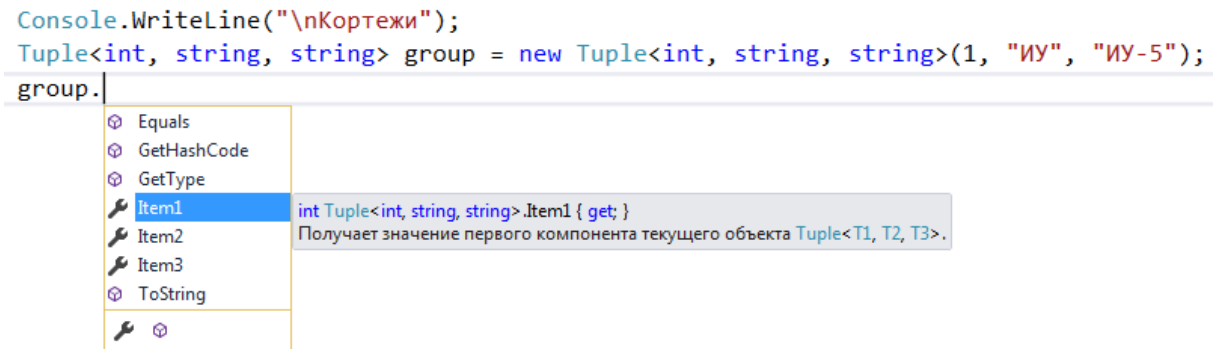


Рис. 24. Работа IntelliSense для кортежа.

Кортеж не может содержать более восьми обобщенных типов, однако обобщенный тип может быть кортежем. Значит, если требуется использовать более восьми полей, то можно создавать вложенные кортежи.

Пример объявления вложенных кортежей:

```
Tuple<int, int, int, int, int, int, int, Tuple<string, string, string>> tuple =
new Tuple<int, int, int, int, int, int, int, Tuple<string, string, string>>
(1, 2, 3, 4, 5, 6, 7, new Tuple<string, string, string>("str1", "str2", "str3"));
```

Кортеж может выступать в качестве обобщенного типа для других коллекций.

Пример объявления списка, элементом которого является кортеж:

```
List<Tuple<int, int, int>> tupleList =
    new List<Tuple<int, int, int>>();
tupleList.Add(new Tuple<int, int, int>(1, 1, 1));
tupleList.Add(new Tuple<int, int, int>(2, 2, 2));
tupleList.Add(new Tuple<int, int, int>(3, 3, 3));
```

Вместо кортежа в данном примере можно было создать отдельный класс, содержащий три целочисленных поля, и помещать в список объекты данного класса.

7.6 Новый синтаксис кортежей

В предыдущем разделе мы рассмотрели классический для языка C# синтаксис кортежей. В последнее время язык C# часто подвергался критике за то, что в нем не поддерживается упрощенный синтаксис для работы с кортежами который используется в языке Python и многих

языках, поддерживающих функциональный подход, таких как Haskell и F#. В версии C# 7.0 этот подход был наконец реализован (он также носит название `ValueTuple`), и мы рассматриваем его в данном разделе.

Теперь кортеж может быть объявлен следующим образом с именованными параметрами:

```
(string strParam, int intParam) tuple1 = ("строка", 333);
Console.WriteLine(tuple1.strParam);
Console.WriteLine(tuple1.intParam);
```

Результаты вывода в консоль:

```
строка
333
```

Таким образом, тип-кортеж создается на лету и инициализируется значениями. В отличие от старого синтаксиса кортежей, где у полей были имена `Item1 ... ItemN`, в новом синтаксисе все поля имеют собственные имена.

Имена полей можно указывать в правой части, в этом случае компилятор автоматически выводит типы полей. Следующий пример аналогичен предыдущему:

```
var tuple2 = (strParam: "строка", intParam: 333);
Console.WriteLine(tuple2.strParam);
Console.WriteLine(tuple2.intParam);
```

Кортеж является изменяемым типом данных, полям можно присваивать новые значения:

```
tuple2.strParam = "новая строка";
```

Кортежи можно использовать в качестве входных параметров функций. Например, пусть объявлена следующая функция:

```
public static void InputTuple((string strP, int intP) tupleParam) {}
```

Ее можно вызвать следующим образом:

```
InputTuple(tuple2);
```

Обратите внимание, что имена параметров кортежей `tuple2` и `tupleParam` не совпадают. Но ошибки не возникает, потому что совпадают

сигнатуры этих кортежей (у обоих два параметра, первый типа `string`, а второй типа `int`).

Кортежи также можно использовать в качестве выходных параметров функций. Например, пусть объявлена следующая функция, которая возвращает кортеж:

```
public static (string strParam, int intParam) OutputTuple()
{
    return ("строка", 333);
}
```

Ее можно вызвать следующим образом, в этом случае переменная `tuple3` будет кортежем:

```
var tuple3 = OutputTuple();
```

Можно при вызове произвести разыменование кортежа, в этом случае переменные `str1` и `int1` будут заполнены значениями из соответствующих полей кортежа:

```
(string str1, int int1) = OutputTuple();
```

Обратите внимание, что имена параметров кортежей (`string strParam`, `int intParam`) и (`string str1`, `int int1`) не совпадают. Но ошибки не возникает, так как совпадают сигнатуры кортежей.

Новый синтаксис кортежей упрощает создание типов, их передачу в функции и возвращение из функций.

Если при деконструкции кортежа некоторые поля не нужны, их можно отбросить с помощью оператора `discard` — символа `_`:

```
// Нужны только имя и возраст, город не нужен
(string name, int age, _) = GetPerson();
Console.WriteLine($"{name}, {age} лет");

public static (string name, int age, string city) GetPerson() =>
("Иван", 18, "Москва");
```

Оператор `_` можно применять к нескольким полям одновременно. Это позволяет явно показать, что часть данных намеренно игнорируется.

Начиная с C# 8 кортежи можно использовать в switch expression для реализации логики, зависящей от комбинации нескольких значений:

```
static string GetDirection(int dx, int dy) =>
    (dx, dy) switch
    {
        (0, 1) => "Север",
        (0, -1) => "Юг",
        (1, 0) => "Восток",
        (-1, 0) => "Запад",
        (0, 0) => "Стоп",
        _ => "Диагональ"
    };

Console.WriteLine(GetDirection(0, 1)); // Север
Console.WriteLine(GetDirection(1, 0)); // Восток
Console.WriteLine(GetDirection(1, 1)); // Диагональ
```

Данная конструкция позволяет лаконично описывать логику, зависящую от нескольких факторов одновременно, без вложенных операторов if.

ЗАДАНИЕ

Напишите консольное приложение на C#, демонстрирующее работу с кортежами.

Программа должна выполнять следующие действия:

Объявить кортеж старого синтаксиса `Tuple<string, string, int>` с данными о книге (название, автор, год издания). Вывести поля через `Item1`, `Item2`, `Item3`.

Объявить аналогичный кортеж новым синтаксисом `ValueTuple` с именованными полями `title`, `author`, `year`. Убедиться, что поля доступны по именам.

Написать функцию `GetBookStats`, которая принимает `List<(string Title, string Author, int Year)>` и возвращает кортеж `(string Oldest, string Newest, double AvgYear)` с названием самой старой книги, самой новой книги и средним годом издания. Вызвать функцию и вывести результат. Продемонстрировать деконструкцию результата. Продемонстрировать применение оператора `discard` для игнорирования одного из возвращаемых полей.

Написать функцию `ClassifyBook`, использующую `switch expression` с кортежом из двух значений (жанр, год) для определения описания книги. Вызвать её для нескольких вариантов входных данных.

РЕШЕНИЕ:

```
// — 1. Старый синтаксис Tuple<>


---


Console.WriteLine("=====");
Console.WriteLine("1. Кортеж старого синтаксиса (Tuple<>)");
Console.WriteLine("=====");

Tuple<string, string, int> oldBook =
    new Tuple<string, string, int>("Мастер и Маргарита", "Булгаков",
1967);

// Поля доступны только как Item1, Item2, Item3
Console.WriteLine($"Название : {oldBook.Item1}");
Console.WriteLine($"Автор      : {oldBook.Item2}");
Console.WriteLine($"Год        : {oldBook.Item3}");

// — 2. Новый синтаксис ValueTuple с именованными полями


---


Console.WriteLine("\n=====");
Console.WriteLine("2. Кортеж нового синтаксиса (ValueTuple)");
```

```

Console.WriteLine("=====");

var newBook = (title: "Преступление и наказание",
               author: "Достоевский",
               year: 1866);

    // Поля доступны по именам – гораздо читаемее
    Console.WriteLine($"Название : {newBook.title}");
Console.WriteLine($"Автор      : {newBook.author}");
Console.WriteLine($"Год        : {newBook.year}");

// Кортеж изменяем
newBook.year = 2024; // переиздание
Console.WriteLine($"Год переиздания: {newBook.year}");

// — 3. Функция, возвращающая кортеж


---



Console.WriteLine("\n=====");
Console.WriteLine("3. Функция с кортежем в качестве возвращаемого
значения");
Console.WriteLine("=====");

List<(string Title, string Author, int Year)> library = new()
{
    ("Война и мир",           "Толстой",      1869),
    ("Преступление и наказание", "Достоевский", 1866),
    ("Мастер и Маргарита",    "Булгаков",  1967),
    ("1984",                  "Оруэлл",    1949),
    ("Гарри Поттер",          "Роулинг",   1997)
};

    // Вызов и сохранение в переменную кортежа
    var stats = GetBookStats(library);
    Console.WriteLine($"Самая старая книга : {stats.Oldest}");
Console.WriteLine($"Самая новая книга  : {stats.Newest}");
Console.WriteLine($"Средний год          : {stats.AvgYear:F1}");

// Деконструкция кортежа
(string oldest, string newest, double avgYear) = GetBookStats(library);
    Console.WriteLine($"Через деконструкцию:");
Console.WriteLine($"Старая: {oldest}, Новая: {newest}, Средний год:
{avgYear:F1}");

// Discard – средний год нам не нужен
(string oldest2, string newest2, _) = GetBookStats(library);
    Console.WriteLine($"C discard (средний год отброшен):");
Console.WriteLine($"Старая: {oldest2}, Новая: {newest2}");

// — 4. switch expression с кортежом из двух значений


---



Console.WriteLine("\n=====");
Console.WriteLine("4. switch expression с кортежом (C# 8)");
Console.WriteLine("=====");

```

```

// Проверяем несколько комбинаций
string[] genres = { "Роман", "Фантастика", "Роман", "Детектив" };
int[] years = { 1869, 1997, 2020, 1950 };

for (int i = 0; i < genres.Length; i++)
{
    string desc = ClassifyBook(genres[i], years[i]);
    Console.WriteLine($" ({genres[i]}, {years[i]}) -> {desc}");
}

//
=====
// Локальные функции
//
=====

// Функция принимает список кортежей и возвращает кортеж со статистикой
static (string Oldest, string Newest, double AvgYear)
    GetBookStats(List<(string Title, string Author, int Year)> books)
{
    if (books.Count == 0)
        return ("-", "-", 0);

    string oldest = books[0].Title;
    string newest = books[0].Title;
    int minYear = books[0].Year;
    int maxYear = books[0].Year;
    double sum = 0;

    foreach (var (title, _, year) in books) // author не нужен – discard
    {
        sum += year;
        if (year < minYear) { minYear = year; oldest = title; }
        if (year > maxYear) { maxYear = year; newest = title; }
    }

    return (oldest, newest, sum / books.Count);
}

// Функция использует switch expression с кортежом из двух значений (C# 8)
static string ClassifyBook(string genre, int year) =>
    (genre, year) switch
    {
        ("Роман", < 1900) => "Классический роман XIX века",
        ("Роман", >= 1900 and < 2000) => "Роман XX века",
        ("Роман", >= 2000) => "Современный роман",
        ("Фантастика", >= 1990) => "Современная фантастика",
        ("Фантастика", _) => "Классическая фантастика",
        ("Детектив", _) => "Детектив",
        _ => $"Книга жанра '{genre}'"
    };

```

7.7 Записи (records)

В C# 9 появился механизм records (записи) как альтернатива кортежам. Записи решают ту же задачу, что и кортежи — быстрое объявление небольших структур данных — однако обладают рядом дополнительных возможностей: именованные поля, встроенное сравнение по значению, автоматическое приведение к строковому типу с помощью метода ToString. Пример:

```
// Объявление записи (одна строка)
record Book(string Title, string Author, int Year);

// Использование
Book b1 = new Book("Евгений Онегин", "Пушкин", 1833);
Book b2 = new Book("Евгений Онегин", "Пушкин", 1833);

Console.WriteLine(b1);           // Book { Title = Евгений Онегин,
... }
Console.WriteLine(b1 == b2);      // True — сравнение по значению

// Создание копии с изменёнными полями (with-expression)
Book b3 = b1 with { Year = 1837 };
Console.WriteLine(b3.Year);      // 1837
```

Данные созданной записи по умолчанию изменить нельзя:

```
public record Person(string Name, int Age);

// Компилятор генерирует:
// public string Name { get; init; }
// public int Age { get; init; }

var p = new Person("Alice", 30);
p.Name = "Bob"; // ❌ Ошибка компиляции — init-only
```

Но можно явно сделать свойства изменяемыми:

```
public record Person(string Name, int Age)
```

```
{
    public string Name { get; set; } = Name; //  теперь можно менять
}
```

Тип `record` является ссылочным типом, то есть хранится в динамически распределяемой памяти. В C# 10 появился `record struct` — запись на основе структуры (типа-значения), что аналогично по смыслу `ValueTuple`, но с именованными полями и более читаемым синтаксисом:

```
record struct Point(double X, double Y);
```

```
Point p = new Point(1.0, 2.5);
```

```
p.X = 2.0;
```

```
Console.WriteLine(p.X); // 1
```

Поля `record struct` по умолчанию можно изменять.

ЗАДАНИЕ

Напишите консольное приложение на C#:

1. Создайте `record Person` с полями `Name` и `Age`.

Выведите два объекта с одинаковыми данными и проверьте их на равенство. Используйте `with`-выражение для создания копии с изменённым именем.

2. Создайте `record struct Point` с полями `X` и `Y`.

Покажите, что это значимый тип (копирование по значению), проверьте равенство и измените копию через `with`.

РЕШЕНИЕ 1:

```
record Person(string Name, int Age);
```

```
var p1 = new Person("Alice", 30);
```

```

var p2 = new Person("Alice", 30);
var p3 = p1 with { Name = "Bob" };

Console.WriteLine(p1);           // Person { Name = Alice, Age = 30 }
Console.WriteLine(p1 == p2);     // True ← сравнение по значению
Console.WriteLine(ReferenceEquals(p1, p2)); // False ← разные объекты
Console.WriteLine(p3);           // Person { Name = Bob, Age = 30 }

```

РЕШЕНИЕ 2:

```

record struct Point(double X, double Y);

var p1 = new Point(1.0, 2.0);
var p2 = p1;           // копия по значению
p2 = p2 with { X = 99 };

Console.WriteLine(p1);   // Point { X = 1, Y = 2 } ← не изменился
Console.WriteLine(p2);   // Point { X = 99, Y = 2 }

var p3 = new Point(1.0, 2.0);
Console.WriteLine(p1 == p3); // True ← сравнение по значению

```

8 Объявление и наследование классов в C#

Если языки C++ и Java довольно схожи в плане синтаксиса основных конструкций, то в плане объектно-ориентированного программирования (ООП) они довольно сильно различаются.

Подход к ООП в языке C# в целом намного ближе к Java, чем к C++. Как и в Java, в языке C# нет множественного наследования классов, оно реализуется с помощью интерфейсов.

Все классы в языке C# неявно наследуются от базового класса System.Object (псевдоним object), который предоставляет такие методы, как ToString(), Equals(), GetHashCode(). Это означает, что любой объект в C# гарантированно обладает этими методами.

Разберем основы ООП в языке C# на основе фрагментов примера проекта «Classes», которые рассматриваются далее в этом разделе.

8.1 Объявление класса и его элементов

Классы в языке C# объявляются с использованием ключевого слова `class`.

Перед ключевым словом `class` могут указываться модификаторы. Модификатор доступа определяет видимость класса: классы верхнего уровня (объявленные непосредственно в пространстве имен) могут иметь модификатор `public` (виден из любой сборки) или `internal` (виден только в текущей сборке). Если модификатор доступа не указан, по умолчанию используется `internal`.

Помимо модификатора доступа, при объявлении класса могут использоваться следующие модификаторы:

- `static` — статический класс, все элементы которого должны быть статическими;
- `sealed` — запечатанный класс, от которого запрещено наследоваться;
- `abstract` — абстрактный класс;
- `partial` — частичный класс, объявление которого может быть разделено между несколькими файлами.

Примеры объявлений классов с различными модификаторами:

```
// Класс виден только в текущей сборке (по умолчанию)
class DefaultClass { }
```

```
// Класс виден из любой сборки
public class PublicClass { }
```

```
// От этого класса нельзя наследоваться
public sealed class FinalClass { }
```

```
// Объявление частичного класса в двух файлах
// Файл Part1.cs
partial class LargeClass
{
    public void Method1() { }
}
// Файл Part2.cs
```



```
partial class LargeClass
{
    public void Method2() { }
}
```

Рассмотрим более детально базовый класс примера – BaseClass:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace Classes
{
    internal class BaseClass
    {
        private int i;

        //Конструктор
        public BaseClass(int param) { i = param; }
        //Методы с различными сигнатурами
        public int MethodReturn(int a) { return i; }
        public string MethodReturn(string a) { return i.ToString(); }
    }

    //Свойство
    //private-значение, которое хранит данные для свойства
    private int _property1 = 0;
    //объявление свойства
    public int Property1
    {
        //возвращаемое значение
        get { return _property1; }
        //установка значения, value - ключевое слово
        set { _property1 = value; }
        //private set { _property1 = value; }
    }

    /// <summary>
    /// Вычисляемое свойство
    /// </summary>
    public int Property1mul2
    {
        get { return Property1 * 2; }
    }

    //Автоматически реализуемые свойства
    //поддерживающая переменная создается автоматически
    public string Property2 { get; set; } = "Строка";
    public float Property3 { get; private set; }
```

```

    }
}

```

ЗАДАНИЕ

В этом разделе мы будем проектировать систему библиотечного каталога. Система будет постепенно увеличиваться по мере изучения новых подразделов.

Создайте класс Book с четырьмя закрытыми полями: title (название), author (автор), year (год издания), pageCount (количество страниц). Добавьте открытый метод GetInfo(), который возвращает строку с информацией о книге. В основной программе создайте два-три объекта класса Book и выведите информацию о них в консоль.

РЕШЕНИЕ:

```

// ===== Файл: Book.cs =====
class Book
{
    // Закрытые поля класса
    private string title;
    private string author;
    private int year;
    private int pageCount;

    // Временный метод для заполнения данных (до изучения конструкторов)
    public void SetData(string title, string author, int year, int pageCount)
    {
        this.title = title;
        this.author = author;
        this.year = year;
        this.pageCount = pageCount;
    }

    // Метод, возвращающий информацию о книге
    public string GetInfo()
    {
        return $"«{title}» – {author} ({year}), {pageCount} стр.";
    }
}

// ===== Основная программа =====
Book book1 = new Book();
book1.SetData("Война и мир", "Л. Толстой", 1869, 1225);

Book book2 = new Book();
book2.SetData("Мастер и Маргарита", "М. Булгаков", 1967, 480);

Book book3 = new Book();
book3.SetData("1984", "Дж. Оруэлл", 1949, 328);

Console.WriteLine(book1.GetInfo());
Console.WriteLine(book2.GetInfo());
Console.WriteLine(book3.GetInfo());

```

8.2 Объявление конструктора

Класс содержит конструктор:

```
public BaseClass(int param) { this.i = param; }
```

Имя конструктора совпадает с именем класса. Конструктор принимает один параметр и присваивает его переменной класса, доступ к которой выполняется с помощью ключевого слова `this`. Если конструктор не определен в классе явно, то считается, что у него есть пустой конструктор без параметров.

Обратите внимание, если в классе определён хотя бы один конструктор с параметрами, конструктор без параметров автоматически не создаётся. В этом случае попытка создать объект без аргументов приведёт к ошибке компиляции:

```
class BaseClass
{
    private int i;
    public BaseClass(int param) { this.i = param; }
}

// Ошибка компиляции: нет конструктора без параметров
// BaseClass bc = new BaseClass();

// Правильно:
BaseClass bc = new BaseClass(333);
```

Если требуется, чтобы объекты класса можно было создавать как с параметрами, так и без них, необходимо явно определить оба конструктора:

```
class BaseClass
{
    private int i;
    public BaseClass() {}
    public BaseClass(int param) { this.i = param; }
}
```

При создании объекта в языке C# 9 и выше можно использовать сокращённую форму оператора new, если тип переменной известен из контекста (target-typed new):

```
// Полная форма:
BaseClass bc1 = new BaseClass(333);

// Сокращённая форма (C# 9):
BaseClass bc2 = new(333);
```

Помимо обычных конструкторов, класс может содержать статический конструктор. Статический конструктор вызывается автоматически средой выполнения не более одного раза — перед первым обращением к классу (создание объекта, обращение к статическому элементу). Статический конструктор не имеет параметров и модификатора доступа:

```
class MyClass
{
    private static int counter;

    // Статический конструктор
    static MyClass()
    {
        // Выполняется один раз при первом обращении к классу
        counter = 0;
        Console.WriteLine("Статический конструктор вызван");
    }

    // Обычный конструктор
    public MyClass()
    {
        counter++;
    }
}
```

Статические конструкторы, как правило, используются для инициализации статических полей класса, загрузки конфигурации или выполнения иных однократных подготовительных действий, а также при реализации паттерна проектирования Singleton (одиночка).

В версии C# 12 появились первичные конструкторы (primary constructors) для классов. Параметры первичного конструктора

указываются непосредственно после имени класса (как будто это метод) и доступны во всём теле класса. Сам конструктор как метод при этом отсутствует. Такой подход более характерен для некоторых объектно-функциональных языков программирования, в частности для F#.

Пример:

```
// C# 12 – первичный конструктор
// Как будто класс это метод с параметрами
class BaseClass(int param)
{
    private int i = param;

    public int GetValue() => i;
    public void PrintValue()
    {
        // Параметр param доступен в методах класса
        Console.WriteLine($"param = {param}");
    }

    // Дополнительный конструктор должен вызывать первичный
    public BaseClass() : this(0) { }
}

// Создание объекта через первичный конструктор
// аналогично обычному конструктору:
BaseClass bc = new BaseClass(333);
```

Первичные конструкторы позволяют значительно сократить объём шаблонного кода, особенно в классах, которые преимущественно хранят данные, переданные через конструктор.

Параметры первичного конструктора не становятся автоматически полями или свойствами класса. Если параметр не используется в инициализаторах полей или в теле методов, он не сохраняется в объекте.

При необходимости определить дополнительные конструкторы в классе с первичным конструктором, они должны вызывать первичный конструктор с помощью ключевого слова `this`.

ЗАДАНИЕ

Удалите метод `SetData` из класса `Book` и замените его конструкторами:

- Основной конструктор с четырьмя параметрами (title, author, year, pageCount).
- Конструктор с двумя параметрами (title, author), который вызывает основной через this(...) со значениями по умолчанию (year = 2024, pageCount = 0).
- Конструктор без параметров, который вызывает конструктор с двумя параметрами через this(...).

Создайте объекты с использованием всех трёх конструкторов и выведите информацию.

РЕШЕНИЕ:

```
// ===== Файл: Book.cs =====
class Book
{
    private string title;
    private string author;
    private int year;
    private int pageCount;

    // Основной конструктор
    public Book(string title, string author, int year, int pageCount)
    {
        this.title = title;
        this.author = author;
        this.year = year;
        this.pageCount = pageCount;
    }

    // Конструктор с двумя параметрами – вызывает основной
    public Book(string title, string author)
        : this(title, author, DateTime.Now.Year, 0)
    {
    }

    // Конструктор без параметров – вызывает конструктор с двумя параметрами
    public Book() : this("Без названия", "Неизвестен")
    {
    }

    public string GetInfo()
    {
        return $"«{title}» – {author} ({year}), {pageCount} стр.";
    }
}

// ===== Основная программа =====
Book book1 = new Book("Война и мир", "Л. Толстой", 1869, 1225);
Book book2 = new Book("Новая книга", "А. Автор");
Book book3 = new Book();

// Сокращённая форма (C# 9)
Book book4 = new("Евгений Онегин", "А. Пушкин", 1833, 224);

Console.WriteLine(book1.GetInfo());
Console.WriteLine(book2.GetInfo());
Console.WriteLine(book3.GetInfo());
```

```
Console.WriteLine(book4.GetInfo());
```

8.3 Объявление методов

Также класс содержит методы с одинаковыми именами, но различными сигнатурами, что допустимо в языке C#: Такой приём называется перегрузкой методов (method overloading). Перегруженные методы должны различаться количеством или типами параметров; различие только в типе возвращаемого значения не является допустимой перегрузкой:

```
// Полная форма:
public int MethodReturn(int a) { return i; }
public string MethodReturn(string a) { return i.ToString(); }
```

Начиная с версии C# 6, методы, состоящие из одного выражения, можно записывать в сокращённой форме с помощью оператора => (expression-bodied methods):

```
// Сокращённая форма (C# 6):
public int MethodReturn(int a) => i;
public string MethodReturn(string a) => i.ToString();
```

Сокращённая форма особенно удобна для коротких методов и часто используется в современном коде на C#.

8.3.1 Модификаторы видимости

Как и в Java, в языке C# модификаторы видимости указываются перед каждым элементом класса. В языке C++ они задаются в виде секций с двоеточием, например «public:».

В языке C# используются следующие модификаторы видимости:

- public – элемент виден и в классе и снаружи класса;
- private – элемент виден только в классе;
- protected – элемент виден только в классе и наследуемых классах;

- `internal` – элемент виден в текущей сборке;
- `protected internal` – элемент виден в классах текущей сборки или в наследуемых классах, даже если наследуемые классы находятся в другой сборке;
- `private protected` – элемент виден только в наследуемых классах текущей сборки;
- `file` – применяется только к типам верхнего уровня (классам, структурам, интерфейсам), а не к членам класса. Делает этот тип видимым только в текущем файле.

Если модификатор видимости для элемента класса не указан, по умолчанию используется `private`. Это отличается от поведения по умолчанию для самих классов, где по умолчанию используется `internal`.

8.3.2 Параметры по умолчанию и именованные аргументы

В языке C# методы могут иметь параметры со значениями по умолчанию (по аналогии с C++, но в отличие от Java, где такая возможность отсутствует). Параметры со значениями по умолчанию должны располагаться после обязательных параметров:

```
static void PrintMessage(string text, int count = 1,
                        string separator = ", ")
{
    for (int j = 0; j < count; j++)
    {
        if (j > 0) Console.Write(separator);
        Console.Write(text);
    }
    Console.WriteLine();
}

// Вызов с одним аргументом (count = 1, separator = ", "):
PrintMessage("Hello");

// Вызов с двумя аргументами (separator = ", "):
PrintMessage("Hello", 3);

// Вызов со всеми аргументами:
PrintMessage("Hello", 3, " | ");
```


При вызове метода аргументы можно передавать по имени параметра. Именованные аргументы позволяют указывать параметры в произвольном порядке, а также пропускать параметры со значениями по умолчанию:

```
// Пропуск параметра count, указание только separator:
PrintMessage("Hello", separator: " | ");

// Именованные аргументы в произвольном порядке:
PrintMessage(count: 2, text: "World");
```

Обратите внимание, что параметр text не имел значения по умолчанию, но при вызове может использоваться как именованный аргумент.

Именованные аргументы особенно повышают читаемость кода в случаях, когда метод принимает несколько параметров одного типа:

```
// Без именованных аргументов
CreateUser("Иванов", "Иван", "Петрович");

// С именованными аргументами
CreateUser(lastName: "Иванов", firstName: "Иван",
           middleName: "Петрович");
```

8.3.3 Локальные функции

В языке C# начиная с версии 7 метод может содержать вложенные функции, называемые локальными. Локальные функции объявляются внутри тела метода и видны только в этом методе. Они удобны для выделения вспомогательной логики, которая не нужна за пределами метода:

```
static void ProcessData(int[] data)
{
    int k = 10;
    // Локальные функции
    int Square(int x)
    {
        return x * x + k;
    }

    int Square2(int x) => x * x + k;
```

```

    for (int j = 0; j < data.Length; j++)
    {
        data[j] = Square(data[j]) + Square2(data[j]);
    }
}

int[] data = { 1, 2, 3 };
ProcessData(data);
foreach (var d in data) Console.WriteLine(d);

```

Обратите внимание что переменная `int k = 10;` не передается как параметр в локальные функции, но видна в этих функциях, такой механизм захвата переменных из внешней области видимости называется замыканием (closure). Важно понимать, что замыкание захватывает именно переменную, а не её текущее значение:

```

static void ClosureDemo()
{
    int k = 10;
    int F(int x) => x + k;

    Console.WriteLine(F(0)); // 10
    k = 20;
    Console.WriteLine(F(0)); // 20 – функция видит изменение k!
}

```

Локальная функция может быть объявлена как до, так и после кода, который её вызывает.

Начиная с C# 8, локальные функции могут быть объявлены как статические с помощью ключевого слова `static`. Статическая локальная функция не может обращаться к локальным переменным и параметрам содержащего метода, что позволяет избежать непреднамеренных замыканий и повысить производительность:

```

static void ProcessData2(int[] data)
{
    int k = 10;

    for (int j = 0; j < data.Length; j++)
    {
        data[j] = Square(data[j]);
    }
}

```

```

    }

    // Статическая локальная функция –
    // не захватывает переменные из ProcessData
    static int Square(int x) => x * x;
    // Раскомментирование этой функции приведет к ошибке
    // на этапе компиляции, так как в статической функции
    // локальную переменную k
    // нельзя использовать через замыкания
    //static int Square2(int x) => x * x + k;
}

int[] data = { 1, 2, 3 };
ProcessData2(data);
foreach (var d in data) Console.WriteLine(d);

```

ЗАДАНИЕ

Расширьте класс Book следующими методами:

1. Перегрузка метода GetInfo(): версия без параметров (уже есть) и версия GetInfo(bool showPages) — если showPages == false, количество страниц не выводится.
2. Метод IsOlderThan(int years = 50) с параметром по умолчанию — возвращает true, если книга старше указанного количества лет.
3. Метод GetFormattedInfo(string format = "short") с использованием локальной функции для форматирования строки.

В основной программе вызовите все методы, в том числе с именованными аргументами.

РЕШЕНИЕ:

```

// ===== Добавить в класс Book =====

// Перегрузка метода GetInfo
public string GetInfo(bool showPages)
{
    if (showPages)
        return $"«{title}» – {author} ({year}), {pageCount} стр.";
    else
        return $"«{title}» – {author} ({year})";
}

// Метод с параметром по умолчанию
public bool IsOlderThan(int years = 50)
{
    return (DateTime.Now.Year - year) > years;
}

```

```
// Метод с локальной функцией
public string GetFormattedInfo(string format = "short")
{
    // Локальная функция
    string FormatShort() => $"{title} ({year})";
    string FormatFull() => $"«{title}»\n Автор: {author}\n Год: {year}\n Страниц: {pageCount}";

    return format switch
    {
        "short" => FormatShort(),
        "full" => FormatFull(),
        _ => GetInfo()
    };
}

// ===== Основная программа =====
Book book1 = new Book("Война и мир", "Л. Толстой", 1869, 1225);

// Перегрузка
Console.WriteLine(book1.GetInfo());
Console.WriteLine(book1.GetInfo(showPages: false));

// Параметр по умолчанию
Console.WriteLine($"Старше 50 лет: {book1.IsOlderThan()}");
Console.WriteLine($"Старше 200 лет: {book1.IsOlderThan(years: 200)}");

// Локальная функция
Console.WriteLine(book1.GetFormattedInfo());
Console.WriteLine(book1.GetFormattedInfo(format: "full"));
```

8.4 Объявление свойств

Важное понятие языка C# – свойство. Оно отсутствует в языках C++ и Java в явном виде, вернее реализуется в этих языках с помощью данных и методов.

Если бы в языке C# не было свойств, также как и в C++ и Java, то можно было бы написать следующий код:

```
//объявление переменной
private int i;
//метод чтения
public int get_i() { return this.i; }
//метод записи
public void set_i(int value) { this.i = value; }
```

Смысл этого кода вполне понятен – к закрытой переменной можно обратиться на чтение и запись с помощью открытых методов. Но в языке C# для реализации такой задачи существует специальный вид конструкции – свойство.

Стоит отметить, что данный пример не случаен: при компиляции свойства языка C# действительно преобразуются в пары методов вида `get_Имя()` и `set_Имя()` в промежуточном коде (IL). Таким образом, свойства — это синтаксическая конструкция, упрощающая работу с широко распространённым паттерном «метод чтения / метод записи».

Пример объявления простого свойства:

```
//private-значение, которое хранит данные для свойства
private int _property1 = 0;

//объявление свойства
public int Property1
{
    //возвращаемое значение
    get { return _property1; }
    //установка значения, value - ключевое слово
    set { _property1 = value; }
}
```

Переменная `_property1` является закрытой (`private`) и содержит данные для свойства. Такую переменную принято называть опорной переменной свойства. Как правило, опорные переменные всегда имеют область видимости `private` или `protected`, чтобы к ним не было внешнего доступа. Для опорных переменных принято следующее соглашение по наименованию — опорная переменная имеет то же имя что и свойство, но перед ним ставится подчеркивание «`_`».

Далее следует объявление свойства «`public int property1`». Внешне оно очень похоже на объявление переменной, но после него ставятся фигурные скобки и указываются секции `get` и `set`, которые принято называть аксессорами (`accessors`), поскольку они обеспечивают доступ к свойству.

Get-аксессор (аксессор чтения) — метод, который вызывается при чтении значения свойства. Как правило, он содержит конструкцию `return`, возвращающую значение опорной переменной. Конечно, в `get`-аксессоре может выполняться и более сложный код.

Тип возвращаемого значения `get`-аксессора, который не задается в коде в явном виде, совпадает с типом свойства.

Set-аксессор (аксессор записи) является методом, который вызывается при присвоении значения свойству. Как правило, он содержит оператор присваивания значения опорной переменной «_property1 = value». В этом случае ключевое слово value обозначает то выражение, которое стоит в правой части от оператора присваивания.

Тип параметра set-аксессора, который не задается в коде в явном виде, совпадает с типом свойства. Тип возвращаемого значения set-аксессора — это тип void.

Объявленное свойство ведет себя как переменная, которой можно присвоить или прочесть значение.

Пример использования свойства:

```
//Создание объекта класса для работы со свойством
BaseClass bc = new BaseClass(333);
//Присвоение значения свойству (обращение к set-аксессору)
bc.property1 = 334;
//Чтение значения свойства (обращение к get-аксессору)
int temp = bc.property1;
```

Таким образом, свойство «кажется» обычной переменной, которой можно присвоить или прочесть значение. Однако при этом происходит обращение к соответствующим аксессорам свойства.

Для свойств существуют правила именования. В соответствии с официальными рекомендациями свойства в C# принято именовать с заглавной буквы (PascalCase). При этом опорная переменная именуется с подчёркиванием _property1.

8.4.1 Области видимости аксессоров

Области видимости аксессоров по умолчанию совпадают с областью видимости свойства, как правило, используется область видимости public. Однако для каждого аксессора можно указать свою область видимости, например:

```
public int property1
{
```

```

    get { return _property1; }
    private set { _property1 = value; }
}

```

В этом случае область видимости аксессуара чтения совпадает с областью видимости свойства (public), а область видимости аксессуара записи ограничена текущим классом (private). То есть прочитать значение такого свойства можно из любого места программы, а присвоить – только в текущем классе.

Область видимости аксессуара не может быть шире области видимости самого свойства. Например, для свойства с модификатором `internal` аксессуару нельзя указать модификатор `public`.

8.4.2 Автоопределяемые свойства

Рассмотренный код аксессуаров используется очень часто. Чтобы облегчить написание кода, в языке C# для таких «стандартных» свойств принята упрощенная форма синтаксиса, называемая автоопределяемым свойством:

```
public string property2 { get; set; }
```

Такая упрощенная форма синтаксиса эквивалентна следующему описанию:

```

private string _property2;

public string property2
{
    get { return _property2; }
    set { _property2 = value; }
}

```

В автоопределяемом свойстве опорная переменная «`_property2`» в исходном коде явно не задается, а автоматически генерируется на этапе компиляции и позволяет хранить значение свойства на этапе выполнения программы.

Для аксессуаров как автоопределяемого свойства, так и обычного свойства, можно явно указывать собственные области видимости:

```
public float property3 { get; private set; }
```

8.4.3 Вычисляемые свойства

Свойство может содержать только один из аксессоров. Например, можно создавать вычисляемые свойства, которые используются для вычисления значений и базируются на опорных переменных других свойств:

```
/// <summary>
/// Вычисляемое свойство
/// </summary>
public int Property1mul2
{
    get { return property1 * 2; }
}
```

Вычисляемые свойства, как правило, содержат только аксессоры чтения. Следует отметить, что для данного свойства также задан XML-комментарий.

8.4.4 Роль свойств в языке C#

Для программистов, работающих на C++ или Java, может быть не совсем понятно, почему в языке C# свойствам придается такое важное значение. В языке C# в классах вообще не принято использовать public переменные, вместо них употребляются автоопределяемые свойства.

Свойства позволяют придавать программам на языке C# дополнительную гибкость, особенно при модификации кода. Например, если изменились правила вычисления какой-либо переменной (допустим, нужно возвращать значение переменной, умноженной на 2), то можно модифицировать get-аксессор не внося изменений в вызывающий код. Если же при присвоении переменной нужно производить дополнительный контроль (допустим, целочисленной переменной можно присваивать значения только от 1 до 1000), его можно поместить в set-аксессор (если

условие не выполняется, то генерируется исключение) не внося изменений в вызывающий код.

Поэтому в языке C# принято объявлять public-переменные класса как автоопределяемые свойства, а при необходимости свойство может быть переписано в полной форме с расширением кода аксессоров.

Кроме того, свойства играют важную роль в технологиях привязки данных (data binding), используемых, в частности в разработке оконных и веб-интерфейсов (WPF, MAUI и Blazor). Механизм привязки работает именно со свойствами, а не с полями. Это ещё одна причина, по которой в C# принято использовать свойства вместо публичных полей.

8.4.5 Возможности свойств в C# 6 и C# 7

В версии C# 6 появились новые возможности работы со свойствами.

При объявлении свойства возможна его непосредственная инициализация:

```
public string String1 { get; set; } = "строка 1";
```

Возможно объявление автоопределяемых свойств, предназначенных только для чтения. Такие свойства могут быть инициализированы в конструкторе класса или непосредственно при объявлении свойства.

```
public int Int1 { get; } = 333;
public int Int2 { get; }
```

```
/// <summary>
/// Конструктор класса
/// </summary>
public PropertyExample()
{
    this.Int2 = 45;
}
```

Попытка изменить такое свойство, например, в методе класса приводит к ошибке:

```
public void Method1()
{
    //Ошибка
    this.Int1 = 123;
```

```
}
```

Компилятор выдает следующий текст ошибки: *«Невозможно присвоить значение свойству или индексактору "PropertyExample.Int1" — доступ только для чтения»*.

Следует отметить различие между свойством только для чтения (`{ get; }`) и свойством с закрытым аксессором записи (`{ get; private set; }`). Свойство `{ get; }` является неизменяемым (immutable) — после инициализации в конструкторе его значение не может быть изменено никаким методом класса. Свойство `{ get; private set; }` может быть изменено любым методом внутри класса.

Expression-bodied свойства. Вычисляемые свойства, состоящие из одного выражения, можно записывать в сокращённой форме:

```
// Вместо:
public int property1mul3
{
    get { return property1 * 2; }
}

// Можно написать:
public int property1mul3 => property1 * 2;
```

В версии C# 7 появились expression-bodied аксессоры:

```
private int _value;

public int Value
{
    get => _value;
    // Валидация
    set => _value = value < 0 ? 0 : value;
}
```

8.4.6 Init-аксессоры

В версии C# 9 появились Init-аксессоры, которые позволяют устанавливать значение только при инициализации объекта:

```
public class Person
{
    public string FirstName { get; init; }
    public string LastName { get; init; }
```

```

}

// Использование:
var person = new Person
{
    FirstName = "Иван",
    LastName = "Иванов"
};

// Ошибка компиляции:
// person.FirstName = "Петр";

```

Обратите внимание на конструирование объекта класса в примере выше. В современном C# при создании объекта класса можно не только передать аргументы в конструктор, но и сразу присвоить значения доступным свойствам с помощью инициализатора объекта. Инициализатор указывается в фигурных скобках после вызова конструктора:

```

BaseClass bc = new BaseClass(333)
{
    property1 = 10,
    property2 = "Hello"
};

```

Этот код эквивалентен следующему «классическому» коду, который напоминает C++:

```

BaseClass bc = new BaseClass(333);
bc.property1 = 10;
bc.property2 = "Hello";

```

Таким образом, `init`-аксессоры позволяют устанавливать значение только при инициализации объекта — в конструкторе или в инициализаторе объекта. После создания объекта изменить такое свойство невозможно.

`Init`-аксессоры занимают промежуточное положение между `set` и `{ get; }`. В отличие от `set`, они запрещают изменение после создания объекта. В отличие от свойства только для чтения (`{ get; }`), они позволяют устанавливать значение в инициализаторе объекта, а не только в конструкторе.

Сравнение различных видов свойств:

```

public class Comparison
{

```

```

// Можно изменять в любом месте
public string A { get; set; }

// Можно изменять только внутри класса
public string B { get; private set; }

// Можно задать только в конструкторе
public string C { get; }

// Можно задать в конструкторе или инициализаторе объекта
public string D { get; init; }
}

```

8.4.7 Required-свойства

С# 11 появились Required-свойства, которые обязывают инициализировать свойство при создании объекта:

```

public class Book
{
    public required string Title { get; set; }
    public int Year { get; set; }
}

//=====
//Required свойства
//=====

// Ошибка компиляции - не указано обязательное свойство Title:
// var book = new Book { Year = 2024 };

// Правильно:

var book1 = new Book
{
    Title = "Книга1",
    Year = 2026
};

var book2 = new Book
{
    Title = "Книга2",
};

```

Модификатор `required` может использоваться как с `set`-аксессорами, так и с `init`-аксессорами. Сочетание `required` и `init` позволяет создать свойство, которое обязательно должно быть задано при создании объекта и не может быть изменено впоследствии:

```
public class Person
{
    // Обязательное и неизменяемое после создания
    public required string Name { get; init; }
    // Обязательное, но может быть изменено
    public required int Group { get; set; }
}
```

ЗАДАНИЕ

Выполните рефакторинг класса Book: замените закрытые поля на свойства. Реализуйте:

1. Title и Author — автоопределяемые свойства с init-аксессором.
2. Year — обычное свойство с опорной переменной и валидацией в set (год не может быть больше текущего и не может быть меньше 1450).
3. PageCount — автоопределяемое свойство { get; set; }.
4. Genre — свойство с модификатором required.
5. AgeInYears — вычисляемое свойство (expression-bodied).
6. ShortDescription — вычисляемое свойство только для чтения.

Адаптируйте конструкторы и методы к новому коду.

РЕШЕНИЕ:

```
// ===== Файл: Book.cs (после рефакторинга) =====
class Book
{
    // Init-свойства — задаются только при создании объекта
    public string Title { get; init; }
    public string Author { get; init; }

    // Свойство с валидацией
    private int _year;
    public int Year
    {
        get => _year;
        set
        {
            if (value < 1450 || value > DateTime.Now.Year)
                Console.WriteLine($"Предупреждение: некорректный год {value}");
            else
                _year = value;
        }
    }

    // Автоопределяемое свойство
    public int PageCount { get; set; }

    // Обязательное свойство (C# 11)
    public required string Genre { get; set; }

    // Вычисляемые свойства (expression-bodied)
```

```

public int AgeInYears => DateTime.Now.Year - Year;
public string ShortDescription => $"{Title} ({Year})";

// Конструкторы адаптированы под свойства
public Book(string title, string author, int year, int pageCount)
{
    Title = title;
    Author = author;
    Year = year;
    PageCount = pageCount;
}

public Book(string title, string author)
    : this(title, author, DateTime.Now.Year, 0)
{
}

public Book() : this("Без названия", "Неизвестен")
{
}

public string GetInfo() =>
    $"«{Title}» - {Author} ({Year}), {PageCount} стр. [{Genre}]";

public string GetInfo(bool showPages) =>
    showPages ? GetInfo() : $"«{Title}» - {Author} ({Year}) [{Genre}]";

public bool IsOlderThan(int years = 50) =>
    AgeInYears > years;

public string GetFormattedInfo(string format = "short")
{
    string FormatShort() => ShortDescription;
    string FormatFull() =>
        $"«{Title}»\n Автор: {Author}\n Год: {Year}" +
        $" \n Страниц: {PageCount}\n Жанр: {Genre}" +
        $" \n Возраст: {AgeInYears} лет";

    return format switch
    {
        "short" => FormatShort(),
        "full" => FormatFull(),
        _ => GetInfo()
    };
}

// ===== Основная программа =====
// Инициализатор объекта + required-свойство
Book book1 = new Book("Война и мир", "Л. Толстой", 1869, 1225)
{
    Genre = "Роман"
};

    Book book2 = new Book("Новая книга", "А. Автор")
    {
        Genre = "Фантастика"
    };

Console.WriteLine(book1.GetInfo());
Console.WriteLine($"Возраст: {book1.AgeInYears} лет");
Console.WriteLine($"Краткое: {book1.ShortDescription}");
Console.WriteLine();
Console.WriteLine(book2.GetFormattedInfo("full"));

// Попытка присвоить некорректный год
book2.Year = 3000; // Предупреждение

```

8.5 Финализаторы

Финализатор (finalizer) — это специальный метод класса, который вызывается сборщиком мусора (Garbage Collector) перед освобождением памяти, занимаемой объектом. В документации и литературе финализаторы также иногда называются деструкторами, по аналогии с языком C++, однако их поведение существенно отличается от деструкторов C++.

Финализатор объявляется с использованием символа тильды ~ перед именем класса. Финализатор не имеет модификатора доступа, не принимает параметров и не возвращает значения:

```
class BaseClass
{
    private int i;

    public BaseClass(int param) { i = param; }

    // Финализатор
    ~BaseClass()
    {
        Console.WriteLine(
            $"Финализатор BaseClass вызван для объекта с i={i}");
    }
}
```

Синтаксис финализатора идентичен синтаксису деструктора в C++. Однако между ними есть принципиальные различия:

- Момент вызова. Деструктор в C++ вызывается детерминированно — при выходе объекта из области видимости или при вызове оператора delete. Финализатор в C# вызывается недетерминированно — сборщик мусора сам определяет, когда освободить память, и момент вызова финализатора заранее неизвестен.
- Гарантия вызова. Вызов деструктора в C++ гарантируется. Вызов финализатора в C# не гарантируется — например, при аварийном завершении программы финализатор может быть не вызван.

- Порядок вызова. В С++ порядок вызова деструкторов строго определён — деструкторы вызываются в порядке, обратном порядку вызова конструкторов. В С# порядок вызова финализаторов различных объектов не определён — сборщик мусора не гарантирует какую-либо последовательность.
- Явный вызов. Деструктор в С++ может быть вызван явно (хотя это и не рекомендуется). Финализатор в С# невозможно вызвать явно из кода — он вызывается только сборщиком мусора.

На практике финализаторы в языке С# используются крайне редко. Это связано с рядом причин:

- сборщик мусора автоматически освобождает управляемую память, поэтому для большинства классов финализатор не требуется;
- объекты с финализаторами обрабатываются сборщиком мусора медленнее — они помещаются в специальную очередь финализации, что увеличивает время жизни объекта и нагрузку на систему;
- момент вызова финализатора непредсказуем, поэтому на него нельзя полагаться при работе с ресурсами, требующими своевременного освобождения.

Для детерминированного освобождения неуправляемых ресурсов (файловых дескрипторов, сетевых соединений и т.д.) в языке С# используется интерфейс `IDisposable` (с методом `Dispose`) и оператор `using`.

Финализатор в таких случаях выступает лишь как «страховочный механизм» на случай, если метод `Dispose()` не был вызван.

ЗАДАНИЕ

Добавьте в класс `Book` финализатор, который выводит сообщение вида "Финализатор: книга «{Title}» удалена из памяти". В основной программе создайте несколько объектов `Book` в отдельном методе, после

возврата из метода вызовите `GC.Collect()` и `GC.WaitForPendingFinalizers()`, чтобы продемонстрировать вызов финализатора.

РЕШЕНИЕ:

```
// ===== Добавить в класс Book =====
~Book()
{
    Console.WriteLine($" Финализатор: книга «{Title}» удалена из памяти");
}
```

8.6 Объявление статических элементов класса

Элемент класса в языке C# может быть объявлен как статический с помощью ключевого слова `static`. Это означает, что данный элемент (поле данных, свойство, метод) принадлежит не объекту класса, а классу в целом, и для работы с такими элементами не нужно создавать объект класса.

Статические элементы существуют в единственном экземпляре независимо от того, сколько объектов класса создано (и даже если ни одного объекта не создано). Это отличает их от обычных (нестатических) элементов, у которых каждый объект имеет собственную копию. Рассмотрим пример класса, содержащего статические элементы:

```
class CountedClass
{
    // Статическое поле – общее для всех объектов
    private static int _count = 0;

    // Статическое свойство
    public static int Count => _count;

    // Статический метод
    public static void ResetCount() { _count = 0; }

    // Обычное (нестатическое) свойство
    public int Id { get; }

    // Конструктор – увеличивает общий счётчик
    public CountedClass()
    {
        _count++;
        Id = _count;
    }
}
```

Вызов статического метода выполняется в формате «Имя_класса.имя_метода(параметры)». Таким образом, для вызова как статических, так и нестатических методов используется символ «.». Но в случае статического метода перед символом «.» ставится имя класса, а в случае нестатического метода перед символом «.» ставится имя объекта. Аналогично создаются и вызываются статические методы в Java, а в языке C++ вместо точки используется символ удвоенного двоеточия «::».

Важно: статические методы не имеют доступа к нестатическим (обычным) элементам класса, поскольку они не связаны с конкретным объектом. При этом обратное неверно — нестатические (обычные) методы могут обращаться к статическим элементам.

Статический конструктор также является статическим элементом класса. Он используется для инициализации статических полей и вызывается автоматически при первом обращении к классу.

ЗАДАНИЕ

Добавьте в класс Book:

1. Статическое поле `_totalCount` — счётчик созданных книг (увеличивается в каждом конструкторе).
2. Статическое свойство `TotalCount` только для чтения.
3. Статический метод `PrintStatistics()` — выводит общее количество созданных книг.

В основной программе создайте несколько книг и вызовите `Book.PrintStatistics()`.

РЕШЕНИЕ:

```
// ===== Добавить в класс Book =====
// Статическое поле
private static int _totalCount = 0;
// Статическое свойство
public static int TotalCount => _totalCount;
// Статический метод
public static void PrintStatistics()
```

```

{
    Console.WriteLine($"Всего создано книг: {_totalCount}");
}

// В каждый конструктор добавить:
// _totalCount++;

// Обновлённый основной конструктор:
public Book(string title, string author, int year, int pageCount)
{
    Title = title;
    Author = author;
    Year = year;
    PageCount = pageCount;
    _totalCount++;
}

// ===== Основная программа =====
Console.WriteLine($"Книг до создания: {Book.TotalCount}");

Book b1 = new("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" };
Book b2 = new("1984", "Оруэлл", 1949, 328) { Genre = "Антиутопия" };
Book b3 = new() { Genre = "Без жанра" };

Book.PrintStatistics();

```

8.7 Статические классы

В языке C# весь класс может быть объявлен как статический. В этом случае ключевое слово `static` указывается перед именем класса, а все элементы класса должны быть статическими.

Статический класс не может быть источником для создания объектов (нельзя создать объект статического класса) и не может быть базовым классом при наследовании. Статические классы, как правило, используются для группировки вспомогательных методов и констант:

```

static class MathHelper
{
    // Статическое свойство
    public static double Pi { get; } = 3.14159265;

    // Статические методы
    public static double Square(double x) => x * x;
    public static double CircleArea(double radius)
        => Pi * Square(radius);
}

// Использование – без создания объекта:
double area = MathHelper.CircleArea(5.0);
Console.WriteLine($"Площадь круга: {area}");

```

Примерами статических классов в стандартной библиотеке .NET являются Math, Console, File, Directory, Convert.

ЗАДАНИЕ

Создайте статический класс LibraryUtils, содержащий:

1. Статический метод PrintSeparator(char symbol = '-', int length = 40) — выводит разделительную линию.
2. Статический метод FormatBookList(Book[] books) — возвращает форматированный нумерованный список книг.
3. Статический метод FindOldest(Book[] books) — возвращает самую старую книгу из массива.

РЕШЕНИЕ:

```
// ===== Файл: LibraryUtils.cs =====
static class LibraryUtils
{
    public static void PrintSeparator(char symbol = '-', int length = 40)
    {
        Console.WriteLine(new string(symbol, length));
    }

    public static string FormatBookList(Book[] books)
    {
        var sb = new System.Text.StringBuilder();
        for (int i = 0; i < books.Length; i++)
        {
            sb.AppendLine($" {i + 1}. {books[i].GetInfo()}");
        }
        return sb.ToString();
    }

    public static Book FindOldest(Book[] books)
    {
        Book oldest = books[0];
        foreach (Book b in books)
        {
            if (b.Year < oldest.Year)
                oldest = b;
        }
        return oldest;
    }
}

// ===== Основная программа =====
Book[] books = {
    new("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" },
    new("1984", "Оруэлл", 1949, 328) { Genre = "Антиутопия" },
    new("Мастер и Маргарита", "Булгаков", 1967, 480) { Genre = "Роман" }
};

LibraryUtils.PrintSeparator('-', 50);
Console.WriteLine(" КАТАЛОГ БИБЛИОТЕКИ");
LibraryUtils.PrintSeparator('-', 50);
Console.WriteLine(LibraryUtils.FormatBookList(books));
LibraryUtils.PrintSeparator();
```

```
Book oldest = LibraryUtils.FindOldest(books);
Console.WriteLine($"Самая старая книга: {oldest.ShortDescription}");
```

8.8 Константы и статические поля только для чтения

В языке C# существует два способа объявить неизменяемое значение, принадлежащее классу: константа (`const`) и статическое поле только для чтения (`static readonly`). Несмотря на внешнее сходство, между ними есть важные различия.

Константа объявляется с помощью ключевого слова `const`. Её значение определяется на этапе компиляции и подставляется непосредственно в код во всех местах использования (аналогично макроподстановке). Константа неявно является статической — указывать ключевое слово `static` для неё не нужно и недопустимо:

```
class Settings
{
    // Константа — значение определяется при компиляции
    public const int MaxUsers = 4;
    public const string Version = "1.0";
}

// Обращение — как к статическому элементу:
Console.WriteLine(Settings.MaxUsers); // 4
```

Статическое поле только для чтения объявляется с помощью ключевых слов `static readonly`. Его значение определяется на этапе выполнения — при объявлении или в статическом конструкторе. Это позволяет использовать значения, которые невозможно вычислить на этапе компиляции:

```
class AppSettings
{
    // static readonly — значение определяется при выполнении
    public static readonly DateTime StartTime = DateTime.Now;
    public static readonly string MachineName;

    static AppSettings()
    {
        MachineName = Environment.MachineName;
    }
}
```

```
}
```

Основные различия между `const` и `static readonly`:

- Момент вычисления. Значение `const` вычисляется на этапе компиляции, значение `static readonly` — на этапе выполнения.
- Допустимые типы. `const` может использоваться только с примитивными типами (`int`, `double`, `bool`, `string` и т.д.). `static readonly` может использоваться с любыми типами, включая объекты классов.
- Поведение при изменении. Если значение `const` изменено в библиотеке, все зависимые сборки необходимо перекомпилировать, поскольку старое значение было подставлено в их код при компиляции. При изменении `static readonly` перекомпиляция зависимых сборок не требуется.

Для значений, которые почти никогда не изменяются (математические константы, фиксированные размеры), лучше использовать `const`. Для значений, которые могут отличаться в разных средах или определяются при запуске программы, лучше использовать `static readonly`.

ЗАДАНИЕ

Добавьте в класс `Book`:

- Константу `MaxPageCount = 10000`.
- Константу `MinYear = 1450`.

Добавьте в класс `LibraryUtils`:

- `static readonly` поле `StartupTime` (инициализируется `DateTime.Now`).
- Константу `LibraryName = "Городская библиотека"`.

Используйте константы для валидации в свойстве `Year` и в свойстве `PageCount`. Выведите информацию о библиотеке при запуске.

РЕШЕНИЕ:

```
// ===== Добавить в класс Book =====
public const int MaxPageCount = 10000;
public const int MinYear = 1450;

// Обновить валидацию Year:
public int Year
{
    get => _year;
    set
    {
        if (value < MinYear || value > DateTime.Now.Year)
            Console.WriteLine($"Предупреждение: некорректный год {value}");
        else
            _year = value;
    }
}

// ===== Добавить в LibraryUtils =====
public const string LibraryName = "Городская библиотека";
public static readonly DateTime StartupTime = DateTime.Now;

public static void PrintHeader()
{
    PrintSeparator('=', 50);
    Console.WriteLine($" {LibraryName}");
    Console.WriteLine($" Система запущена: {StartupTime:dd.MM.yyyy HH:mm}");
    Console.WriteLine($" Макс. страниц: {Book.MaxPageCount}");
    PrintSeparator('=', 50);
}
```

8.9 Наследование классов

В языке C# наследовать класс можно только от одного класса. В большинстве современных ООП-языков (за исключением C++ и Python) не реализован механизм множественного наследования.

Для реализации механизма, подобного множественному наследованию, используются интерфейсы, которые разбираются в следующих разделах.

Если базовый класс явно не указан, класс неявно наследуется от System.Object. В данном примере класс ExtendedClass1 наследуется от класса BaseClass:

```
/// <summary>
/// Наследуемый класс 1
/// </summary>
class ExtendedClass1 : BaseClass
{
    private int i2;
    private int i3;

    //Конструкторы
```

```

//base(pi) - вызов конструктора базового класса
public ExtendedClass1(int pi, int pi2) : base(pi) { i2 = pi2; }

//this(pi, pi2) - вызов другого конструктора этого класса
public ExtendedClass1(int pi, int pi2, int pi3) : this(pi, pi2)
{ i3 = pi3; }

/// <summary>
/// Метод виртуальный, так как он объявлен в самом базовом классе
/// object
/// поэтому чтобы его переопределить добавлено ключевое слово
/// override
/// </summary>
public override string ToString()
{
    return "i=" + MethodReturn("1")
        + " i2=" + i2.ToString() + " i3=" + i3.ToString();
}
}

```

Синтаксис наследования в языке C# аналогичен синтаксису языка C++. Для обозначения наследования используется символ двоеточия при объявлении класса, а затем указывается имя базового класса «class ExtendedClass1 : BaseClass».

В отличие от C++, в языке C# не указываются модификаторы доступа при наследовании (в C++ используются конструкции public, protected, private перед именем базового класса). В C# наследование всегда работает аналогично public-наследованию в C++.

Наследуемый класс получает доступ к элементам базового класса с модификаторами public, protected, internal и protected internal. Элементы с модификатором private существуют в объекте наследуемого класса, но недоступны напрямую — доступ к ним возможен только через методы, свойства и конструкторы базового класса.

Если необходимо запретить наследование от класса, используется ключевое слово sealed:

```

sealed class FinalClass : BaseClass
{
    public FinalClass(int param) : base(param) { }
    // От FinalClass наследоваться нельзя
}

```


8.9.1 Вызов конструкторов при наследовании

При наследовании может возникнуть проблема, связанная с доступом к `private` полям базового класса, и в первую очередь она может появиться в конструкторе. Наследуемый класс имеет доступ к `protected` полям базового класса, но не имеет доступа к `private` полям. Инициализация `private` полей может быть реализована с помощью вызова конструктора базового класса:

```
//Конструктор
//base(pi) - вызов конструктора базового класса
public ExtendedClass1(int pi, int pi2) : base(pi) { i2 = pi2; }
```

Ключевое слово `base` обозначает вызов конструктора базового класса. В качестве параметров в круглых скобках указываются параметры, передаваемые в конструктор базового класса. После вызова конструктора базового класса (инициализация `private` полей базового класса) выполняются действия, указанные в фигурных скобках в теле текущего конструктора.

Порядок выполнения при создании объекта наследуемого класса всегда следующий: сначала выполняется конструктор базового класса, затем — тело конструктора наследуемого класса. Это гарантирует, что базовая часть объекта полностью инициализирована до начала инициализации наследуемой части.

Обратите внимание, что конструкторы базового класса не наследуются. Наследуемый класс должен определить собственные конструкторы. Если конструктор наследуемого класса не содержит явного вызова `base(...)`, компилятор автоматически подставляет вызов конструктора базового класса без параметров `base()`. Если в базовом классе нет конструктора без параметров, произойдёт ошибка компиляции:

```
class BaseClass
{
    private int i;
    // Единственный конструктор — с параметром
    public BaseClass(int param) { i = param; }
```

```

}

class DerivedClass : BaseClass
{
    // Ошибка компиляции: BaseClass не содержит конструктора
    // без параметров, а явный вызов base(...) отсутствует
    // public DerivedClass() { }

    // Правильно – явный вызов конструктора базового класса:
    public DerivedClass() : base(0) { }
}

```

8.9.2 Вызов других конструкторов текущего класса

Аналогично вызову конструктора базового класса может быть вызван другой конструктор текущего класса, но в этом случае вместо ключевого слова `base` используется ключевое слово `this`.

Пример использования ключевого слова `this`:

```

public ExtendedClass1(int pi, int pi2, int pi3) : this(pi, pi2)
{
    i3 = pi3;
}

```

В этом случае посредством ключевого слова `this` вызывается рассмотренный ранее конструктор класса с двумя параметрами «`public ExtendedClass1(int pi, int pi2)`», затем выполняются действия, указанные в фигурных скобках в теле текущего конструктора.

Таким образом, при создании объекта выражением `new ExtendedClass1(1, 2, 3)` цепочка вызовов выглядит следующим образом:

```

ExtendedClass1(1, 2, 3)
→ this(1, 2): вызов ExtendedClass1(int, int)
  → base(1): вызов BaseClass(int)           ← выполняется первым
  → тело ExtendedClass1(int, int): i2=2      ← выполняется вторым
→ тело ExtendedClass1(int, int, int): i3=3   ← выполняется третьим

```

Следует учитывать, что в одном конструкторе можно указать либо `base(...)`, либо `this(...)`, но не оба одновременно. Конструктор, вызываемый через `this(...)`, в свою очередь, может содержать вызов `base(...)` или другой вызов `this(...)`.

8.9.3 Ключевое слово `base` для доступа к элементам базового класса

Ключевое слово `base` используется не только для вызова конструктора базового класса, но и для обращения к его методам и свойствам из наследуемого класса. Это особенно полезно, когда наследуемый класс определяет элемент с тем же именем, что и в базовом классе:

```
class DerivedClass : BaseClass
{
    public new int Value { get; set; } = 20;

    public new void Print()
    {
        // Обращение к методу базового класса
        base.Print();
        Console.WriteLine($"DerivedClass: Value = {Value}");
    }

    public void ShowBoth()
    {
        Console.WriteLine($"base.Value = {base.Value}");
        Console.WriteLine($"this.Value = {this.Value}");
    }
}
```

В данном примере используется ключевое слово `new` перед свойством и методом наследуемого класса. Оно указывает, что элемент наследуемого класса намеренно скрывает одноимённый элемент базового класса. Если ключевое слово `new` не указано, компилятор выдаст предупреждение. Механизм сокрытия отличается от механизма переопределения виртуальных методов (`virtual/override`), который рассматривается в следующих разделах.

8.9.4 Проверка и приведение типов

При работе с иерархией наследования переменная базового типа может хранить ссылку на объект наследуемого класса. Для определения фактического типа объекта и безопасного приведения типов в языке C# используются операторы `is` и `as`:

```

class BaseClass
{
    public int Value { get; set; } = 10;

    public void Print()
    {
        Console.WriteLine($"BaseClass: Value = {Value}");
    }
}

BaseClass obj = new ExtendedClass1(1, 2);

// Оператор is – проверка типа
if (obj is ExtendedClass1)
{
    Console.WriteLine("obj является ExtendedClass1");
}

// Оператор is с объявлением переменной (C# 7)
if (obj is ExtendedClass1 ext)
{
    // ext доступна как ExtendedClass1
    Console.WriteLine(ext.ToString());
}

// Оператор as – приведение с проверкой
// Возвращает null, если приведение невозможно
ExtendedClass1? ext2 = obj as ExtendedClass1;
if (ext2 != null)
{
    Console.WriteLine(ext2.ToString());
}

```

Оператор `as` отличается от явного приведения типов в круглых скобках. Явное приведение генерирует исключение `InvalidCastException`, если приведение невозможно, тогда как `as` возвращает `null`.

Рекомендуется использовать оператор `is` с объявлением переменной (C# 7), поскольку он выполняет проверку и приведение в одном выражении, что является наиболее лаконичным и безопасным способом.

ЗАДАНИЕ

Выполните рефакторинг проекта:

1. Создайте базовый класс `LibraryItem` с общими свойствами: `Title` (init), `Year`, статический счётчик `TotalItems`, метод `GetInfo()`.

2. Класс Book теперь наследуется от LibraryItem. Книга добавляет свойства Author, PageCount, Genre.
3. Создайте класс Magazine, наследующийся от LibraryItem, со свойствами IssueNumber (номер выпуска) и Publisher (издательство).
4. В основной программе создайте массив LibraryItem[], содержащий и книги, и журналы. Используйте is и as для проверки типов.

РЕШЕНИЕ:

```
// ===== Файл: LibraryItem.cs =====
class LibraryItem
{
    private static int _totalItems = 0;
    public static int TotalItems => _totalItems;

    public string Title { get; init; }

    private int _year;
    public int Year
    {
        get => _year;
        set
        {
            if (value >= 1450 && value <= DateTime.Now.Year)
                _year = value;
        }
    }

    public int AgeInYears => DateTime.Now.Year - Year;
    public string ShortDescription => $"{Title} ({Year})";

    public LibraryItem(string title, int year)
    {
        Title = title;
        Year = year;
        _totalItems++;
    }

    public LibraryItem() : this("Без названия", DateTime.Now.Year) { }

    public string GetInfo()
    {
        return $"«{Title}» ({Year})";
    }
}

// ===== Файл: Book.cs =====
class Book : LibraryItem
{
    public string Author { get; init; }
    public int PageCount { get; set; }
    public required string Genre { get; set; }

    public Book(string title, string author, int year, int pageCount)
        : base(title, year)
    {
        Author = author;
        PageCount = pageCount;
    }
}
```

```

    public Book(string title, string author)
        : this(title, author, DateTime.Now.Year, 0) { }

    public Book() : this("Без названия", "Неизвестен") { }
}

// ===== Файл: Magazine.cs =====
class Magazine : LibraryItem
{
    public int IssueNumber { get; set; }
    public string Publisher { get; set; }

    public Magazine(string title, int year, int issueNumber, string publisher)
        : base(title, year)
    {
        IssueNumber = issueNumber;
        Publisher = publisher;
    }
}

// ===== Основная программа =====
// Массив базового типа с объектами разных наследников
LibraryItem[] items =
{
    new Book("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" },
    new Magazine("Наука и жизнь", 2025, 3, "Пресса"),
    new Book("1984", "Оруэлл", 1949, 328) { Genre = "Антиутопия" },
    new Magazine("National Geographic", 2024, 12, "NatGeo")
};

Console.WriteLine($"Всего элементов: {LibraryItem.TotalItems}\n");

foreach (LibraryItem item in items)
{
    Console.WriteLine($" {item.GetInfo()} - ");

    // Проверка типа с помощью is (C# 7)
    if (item is Book book)
    {
        Console.WriteLine($"Книга, автор: {book.Author}, жанр: {book.Genre}");
    }
    else if (item is Magazine mag)
    {
        Console.WriteLine($"Журнал, выпуск #{mag.IssueNumber}, изд-во: {mag.Publisher}");
    }
}

```

8.10 Виртуальные и абстрактные методы

Подход к виртуальным методам в C# ближе к Java, чем к C++. В Java все методы по умолчанию являются виртуальными, а в C++ и C# для этого требуется явное указание. Однако в отличие от C++, в C# переопределение виртуального метода также должно быть явно помечено ключевым словом `override`, что исключает случайное переопределение.

8.10.1 Виртуальные методы и их переопределение

Метод базового класса объявляется виртуальным с помощью ключевого слова `virtual`. Это означает, что наследуемый класс может переопределить данный метод, предоставив собственную реализацию с помощью ключевого слова `override`.

Рассмотрим пример. Добавим в класс `BaseClass` виртуальный метод:

```
class BaseClass
{
    private int i;

    public BaseClass(int param) { i = param; }

    /// <summary>
    /// Виртуальный метод — может быть переопределён
    /// в наследуемых классах
    /// </summary>
    public virtual string GetInfo()
    {
        return $"BaseClass: i = {i}";
    }
}
```

В наследуемом классе метод переопределяется с помощью ключевого слова `override`:

```
class ExtendedClass1 : BaseClass
{
    private int i2;

    public ExtendedClass1(int pi, int pi2) : base(pi)
    {
        i2 = pi2;
    }

    /// <summary>
    /// Переопределение виртуального метода
    /// </summary>
    public override string GetInfo()
    {
        return base.GetInfo() + $", i2 = {i2}";
    }
}
```

В переопределённом методе с помощью `base.GetInfo()` вызывается реализация базового класса, к результату которой добавляется информация наследуемого класса. Вызов базовой реализации не является обязательным — переопределённый метод может полностью заменить поведение базового метода.

Можно также отметить, что метод `ToString()`, переопределённый в классе `ExtendedClass1`, является виртуальным именно потому, что он объявлен как `virtual` в классе `System.Object` — базовом классе для всех классов C#:

```
public override string ToString()
{
    return "i=" + MethodReturn("1")
        + " i2=" + i2.ToString()
        + " i3=" + i3.ToString();
}
```

Таким образом, ключевое слово `override` в этом примере используется потому, что метод `ToString()` был объявлен как `virtual` в классе `object`. Аналогично могут быть переопределены другие виртуальные методы класса `object`: `Equals()` и `GetHashCode()`, которые используются для проверки объектов на равенство.

8.10.2 Полиморфизм

Главная цель виртуальных методов — обеспечение полиморфизма. Полиморфизм означает, что при вызове виртуального метода через переменную базового типа выполняется та реализация метода, которая соответствует фактическому типу объекта, а не типу переменной:

```
BaseClass obj1 = new BaseClass(1);
BaseClass obj2 = new ExtendedClass1(2, 20);

Console.WriteLine(obj1.GetInfo());
// Вывод: BaseClass: i = 1

Console.WriteLine(obj2.GetInfo());
// Вывод: BaseClass: i = 2, i2 = 20
```


Несмотря на то, что обе переменные имеют тип `BaseClass`, для объекта `obj2` вызывается переопределённый метод класса `ExtendedClass1`, поскольку фактический тип объекта — `ExtendedClass1`.

Полиморфизм особенно полезен при работе с коллекциями объектов разных типов, объединённых общим базовым классом:

```
class ExtendedClass2 : BaseClass
{
    private string name;

    public ExtendedClass2(int pi, string pName) : base(pi)
    {
        name = pName;
    }

    public override string GetInfo()
    {
        return base.GetInfo() + $", name = {name}";
    }
}

// Массив объектов базового типа, содержащий различные
// наследуемые типы
BaseClass[] items = new BaseClass[]
{
    new BaseClass(1),
    new ExtendedClass1(2, 20),
    new ExtendedClass2(3, "объект")
};

// Для каждого объекта вызывается его собственная реализация
foreach (BaseClass item in items)
{
    Console.WriteLine(item.GetInfo());
}

// Вывод:
// BaseClass: i = 1
// BaseClass: i = 2, i2 = 20
// BaseClass: i = 3, name = объект
```

Такой подход, называемый также динамическим связыванием (dynamic binding), позволяет писать универсальный код, не зависящий от

конкретного типа объекта. Добавление новых наследуемых классов не требует изменения существующего кода, работающего с базовым типом.

8.10.3 Переопределение и сокрытие методов (override и new)

Важно понимать принципиальное различие между сокрытием (new) и переопределением (override), поскольку они приводят к разному поведению при вызове через переменную базового типа:

```
class Base
{
    public virtual string Method() => "Base";
}

class DerivedOverride : Base
{
    // Переопределение — заменяет реализацию базового метода
    public override string Method() => "DerivedOverride";
}

class DerivedNew : Base
{
    // Сокрытие — создаёт новый метод, не связанный с базовым
    public new string Method() => "DerivedNew";
}

Base obj1 = new DerivedOverride();
Base obj2 = new DerivedNew();

Console.WriteLine(obj1.Method()); // DerivedOverride (полиморфизм)
Console.WriteLine(obj2.Method()); // Base (сокрытие — нет
полиморфизма)

// При обращении через переменную наследуемого типа:
DerivedNew obj3 = new DerivedNew();
Console.WriteLine(obj3.Method()); // DerivedNew
```

Ключевое различие между override и new — в том, работает ли полиморфизм:

- override переопределяет виртуальный метод базового класса. При вызове через переменную базового типа будет выполнен метод производного класса — это и есть полиморфизм.

Соответствующий метод в базовом классе обязательно должен быть помечен как `virtual`, `abstract` или `override`.

- `new` скрывает метод (или любой другой член) базового класса, причём базовый член не обязан быть виртуальным. Полиморфизм при этом не работает: какой метод будет вызван — определяется тем типом, с которым объявлена переменная объекта класса, а не реальным типом объекта. Если ключевое слово `new` не указать явно, компилятор выдаст предупреждение, однако код скомпилируется и будет вести себя так же, как при сокрытии.

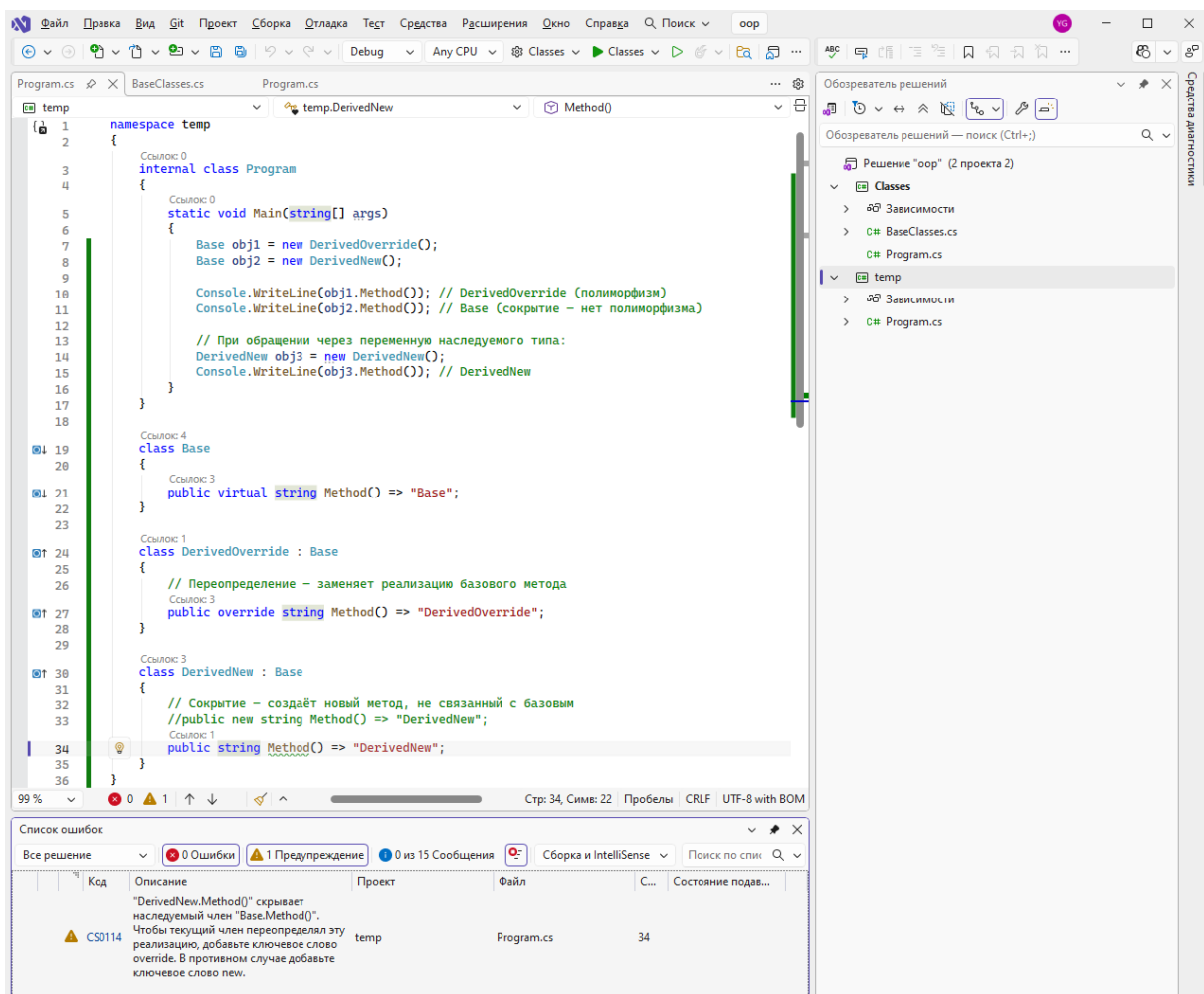


Рис. 25. Предупреждение компилятора при отсутствии ключевого слова `new`.

В подавляющем большинстве случаев используется ключевое слово `override`. Необходимость в `new` возникает редко и, как правило, указывает на проблему в проектировании иерархии классов.

8.10.4 Абстрактные классы и абстрактные методы

Виртуальный метод содержит реализацию по умолчанию, которую наследуемый класс может переопределить, а может и не переопределять. Однако иногда базовый класс не может предоставить осмысленную реализацию метода — она зависит от конкретного наследуемого класса. В этом случае используются абстрактные методы.

Абстрактный метод объявляется с помощью ключевого слова `abstract` и не содержит тела (реализации). Класс, содержащий хотя бы один абстрактный метод, должен быть объявлен как абстрактный:

```

/// <summary>
/// Абстрактный класс – нельзя создать объект напрямую
/// </summary>
abstract class Shape
{
    public string Name { get; }

    public Shape(string name) { Name = name; }

    /// <summary>
    /// Абстрактный метод – не имеет реализации.
    /// Каждый наследуемый класс обязан его реализовать.
    /// </summary>
    public abstract double Area();

    /// <summary>
    /// Обычный (не абстрактный) метод – имеет реализацию,
    /// доступную для всех наследуемых классов
    /// </summary>
    public void PrintInfo()
    {
        Console.WriteLine($"{Name}: площадь = {Area():F2}");
    }
}

```

Наследуемые классы обязаны реализовать все абстрактные методы базового класса с помощью ключевого слова `override` (так же, как при переопределении виртуальных методов):

```
class Circle : Shape
{
    public double Radius { get; }

    public Circle(double radius) : base("Круг")
    {
        Radius = radius;
    }

    public override double Area() => Math.PI * Radius * Radius;
}

class Rectangle : Shape
{
    public double Width { get; }
    public double Height { get; }

    public Rectangle(double width, double height)
        : base("Прямоугольник")
    {
        Width = width;
        Height = height;
    }

    public override double Area() => Width * Height;
}

// Ошибка компиляции – нельзя создать объект абстрактного класса:
// Shape shape = new Shape("фигура");

// Можно создавать объекты наследуемых классов:
Shape circle = new Circle(5.0);
Shape rect = new Rectangle(3.0, 4.0);

circle.PrintInfo(); // Круг: площадь = ..
rect.PrintInfo();   // Прямоугольник: площадь = ...

// Полиморфизм работает – вызывается реализация конкретного класса
Shape[] shapes = { circle, rect };
double totalArea = 0;
foreach (Shape s in shapes)
{
    totalArea += s.Area();
}
Console.WriteLine($"Общая площадь: {totalArea:F2}");
```

Основные правила работы с абстрактными классами и методами:

- Объект абстрактного класса не может быть создан напрямую (нельзя создать объект с помощью `new`), но может использоваться как тип переменной для хранения ссылок на объекты наследуемых классов.
- Абстрактный метод не имеет тела и может быть объявлен только в абстрактном классе. Абстрактные методы неявно являются виртуальными — указывать ключевое слово `virtual` не нужно и недопустимо. Аналог чистых виртуальных методов в C++.
- Наследуемый класс обязан реализовать все абстрактные методы базового класса, если сам не является абстрактным. Если наследуемый класс не реализует хотя бы один абстрактный метод, он тоже должен быть объявлен как `abstract`.
- Абстрактный класс может содержать как абстрактные, так и обычные методы с реализацией, а также свойства, поля и конструкторы. Конструкторы абстрактного класса могут вызываться при создании объектов наследуемых классов.

Сравнение виртуальных и абстрактных методов:

`virtual` — метод имеет реализацию по умолчанию. Наследуемый класс может переопределить метод, но не обязан это делать. Объект класса с виртуальными методами может быть создан с помощью `new`.

`abstract` — метод не имеет реализации. Наследуемый класс обязан реализовать метод. Класс с абстрактными методами должен быть абстрактным. Объект абстрактного класса НЕ может быть создан с помощью `new`.

8.10.5 Запечатывание переопределённых методов

Переопределённый виртуальный или абстрактный метод сам является виртуальным — классы, наследуемые далее по цепочке, могут

переопределить его повторно. Если необходимо запретить дальнейшее переопределение метода, используется сочетание ключевых слов `sealed override`:

```
class Circle : Shape
{
    public double Radius { get; }

    public Circle(double radius) : base("Круг")
    {
        Radius = radius;
    }

    // Метод запечатан — наследники Circle не смогут
    // его переопределить
    public sealed override double Area()
        => Math.PI * Radius * Radius;
}

class SpecialCircle : Circle
{
    public SpecialCircle(double radius) : base(radius) { }

    // Ошибка компиляции: нельзя переопределить
    // запечатанный метод
    // public override double Area() => 0;
}
```

Ключевое слово `sealed` при переопределении метода выполняет ту же роль, что и `sealed` при объявлении класса — запрещает дальнейшее наследование, но применительно к отдельному методу, а не ко всему классу.

8.10.6 Виртуальные свойства

Виртуальными могут быть объявлены не только методы, но и свойства. Виртуальные свойства объявляются и переопределяются по тем же правилам, что и виртуальные методы:

```
abstract class Shape
{
    public string Name { get; }

    public Shape(string name) { Name = name; }
```

```

// Абстрактное свойство
public abstract double Area { get; }

// Виртуальное свойство с реализацией по умолчанию
public virtual string Description => $"{Name}: площадь =
{Area:F2}";
}

class Circle : Shape
{
    public double Radius { get; }

    public Circle(double radius) : base("Круг")
    {
        Radius = radius;
    }

    // Реализация абстрактного свойства
    public override double Area => Math.PI * Radius * Radius;

    // Переопределение виртуального свойства
    public override string Description
        => $"{Name} (R={Radius}): площадь = {Area:F2}";
}

var c = new Circle(5);
Console.WriteLine(c.Area);
Console.WriteLine(c.Description);

```

Обратите внимание, что в данном примере Area объявлено не как метод, а как вычисляемое свойство только для чтения. Выбор между виртуальным методом и виртуальным свойством определяется общими рекомендациями: свойства используются для характеристик объекта, методы — для действий.

ЗАДАНИЕ

Модифицируйте иерархию классов:

1. Сделайте LibraryItem абстрактным классом.
2. Добавьте абстрактный метод GetCardInfo() — возвращает строку для каталожной карточки. Каждый наследник реализует его по-своему.

3. Метод GetInfo() сделайте виртуальным с реализацией по умолчанию. Переопределите его в Book и Magazine.
4. Добавьте виртуальное свойство Description в LibraryItem.
5. В основной программе продемонстрируйте полиморфизм: в цикле foreach вызовите методы для массива LibraryItem[].

РЕШЕНИЕ:

```
// ===== Файл: LibraryItem.cs (обновлённый) =====
abstract class LibraryItem
{
    private static int _totalItems = 0;
    public static int TotalItems => _totalItems;

    public string Title { get; init; }

    private int _year;
    public int Year
    {
        get => _year;
        set
        {
            if (value >= 1450 && value <= DateTime.Now.Year)
                _year = value;
        }
    }

    public int AgeInYears => DateTime.Now.Year - Year;
    public string ShortDescription => $"{Title} ({Year})";

    // Виртуальное свойство
    public virtual string Description => $"{Title} ({Year})";

    public LibraryItem(string title, int year)
    {
        Title = title;
        Year = year;
        _totalItems++;
    }

    // Абстрактный метод – каждый наследник обязан реализовать
    public abstract string GetCardInfo();

    // Виртуальный метод – наследник может переопределить
    public virtual string GetInfo()
    {
        return $"«{Title}» ({Year})";
    }
}

// ===== Файл: Book.cs (обновлённый) =====
class Book : LibraryItem
{
    public string Author { get; init; }
    public int PageCount { get; set; }
    public required string Genre { get; set; }

    public override string Description =>
        $"«{Title}» – {Author}, {Genre}";

    public Book(string title, string author, int year, int pageCount)
        : base(title, year)
    {
    }
}
```

```

{
    Author = author;
    PageCount = pageCount;
}

public Book(string title, string author)
    : this(title, author, DateTime.Now.Year, 0) { }

public Book() : this("Без названия", "Неизвестен") { }

// Реализация абстрактного метода
public override string GetCardInfo()
{
    return $"[КНИГА] {Title} / {Author}. - {Year}. - {PageCount} с. - ({Genre})";
}

// Переопределение виртуального метода
public override string GetInfo()
{
    return $"«{Title}» - {Author} ({Year}), {PageCount} стр. [{Genre}];
}

}

// ===== Файл: Magazine.cs (обновлённый) =====
class Magazine : LibraryItem
{
    public int IssueNumber { get; set; }
    public string Publisher { get; set; }

    public override string Description =>
        $"«{Title}» №{IssueNumber}, изд. {Publisher}";

    public Magazine(string title, int year, int issueNumber, string publisher)
        : base(title, year)
    {
        IssueNumber = issueNumber;
        Publisher = publisher;
    }

    public override string GetCardInfo()
    {
        return $"[ЖУРНАЛ] {Title}. - {Year}. - №{IssueNumber}. - Изд.: {Publisher}";
    }

    public override string GetInfo()
    {
        return $"«{Title}» ({Year}), выпуск №{IssueNumber}, изд-во: {Publisher}";
    }
}

// ===== Основная программа =====
LibraryItem[] catalog =
{
    new Book("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" },
    new Magazine("Наука и жизнь", 2025, 3, "Пресса"),
    new Book("1984", "Оруэлл", 1949, 328) { Genre = "Антиутопия" },
    new Magazine("National Geographic", 2024, 12, "NatGeo")
};

// Полиморфизм – один цикл, разное поведение
Console.WriteLine("=== Каталожные карточки ===");
foreach (LibraryItem item in catalog)
{
    Console.WriteLine(item.GetCardInfo());
}

Console.WriteLine("\n=== Описания ===");
foreach (LibraryItem item in catalog)

```

```
{
    Console.WriteLine(item.Description);
}
```

8.11 Методы расширения

В языке C# существует уникальный механизм, который позволяет добавлять новые методы к уже реализованным классам, в том числе классам стандартной библиотеки. Сами разработчики .NET активно его используют, многие методы стандартных библиотек реализованы как методы расширения. Аналогичная возможность отсутствует в языках C++ и Java.

Причем методы стандартной библиотеки могут находиться в одной сборке (файле .dll), а методы расширения – в другой сборке. Важно, чтобы пространство имён, содержащее класс с методами расширения, было подключено с помощью директивы using в файле, где эти методы используются. При этом методы расширения не обязаны находиться в том же пространстве имён, что и расширяемый класс.

Предположим, что нужно расширить класс ExtendedClass2 новым методом, однако по каким-либо причинам невозможно внести изменения в исходный код класса.

Тогда можно создать метод расширения следующим образом:

```
static class ExtendedClass2Extension
{
    public static int ExtendedClass2NewMethod(this ExtendedClass2 ec2,
    int i)
    {
        return i + 1;
    }
}
```

Данный класс является обычным классом, однако у него есть некоторые особенности. Метод расширения должен быть объявлен в статическом классе и должен быть статическим методом. Первый параметр метода расширения – объект расширяемого класса, перед которым указывается ключевое слово this.

При вызове метода расширения первый параметр (помеченный `this`) не указывается — вместо него используется объект, для которого вызывается метод:

```
ExtendedClass2 obj = new ExtendedClass2(1, "test");

// Вызов метода расширения — как обычного метода объекта
int result = obj.ExtendedClass2NewMethod(2); // вернёт 3

// Эквивалентный вызов через статический метод
int result2 = ExtendedClass2Extension.ExtendedClass2NewMethod(obj,
5);
```

Если откомпилировать класс `ExtendedClass2Extension`, то IntelliSense для класса `ExtendedClass2` будет работать так, как показано на рис. 26.

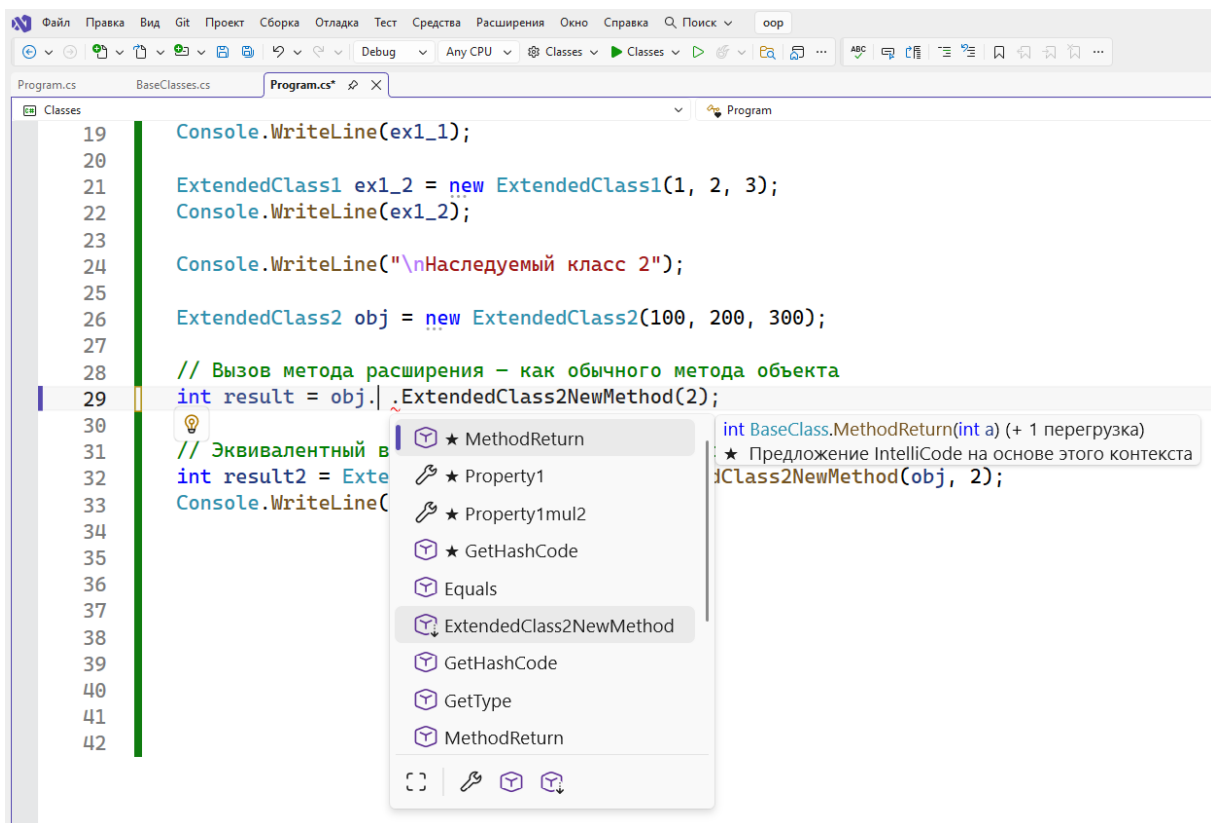


Рис. 26. Работа IntelliSense для метода расширения.

В этом случае метод расширения обозначен как обычный метод, но с дополнительной стрелкой, указывающей на то, что это метод расширения.

Таким образом, разработчику «кажется», что у класса `ExtendedClass2` появился новый метод `ExtendedClass2NewMethod`, однако, данный метод

объявлен в отдельном классе и возможно даже в отдельной сборке. Для использования метода расширения необходимо подключить пространство имён класса, содержащего метод расширения, с помощью директивы `using`.

При работе с методами расширения необходимо учитывать следующие правила:

- Статический класс. Метод расширения должен быть объявлен в статическом классе верхнего уровня (не во вложенном классе).
- Приоритет. Если у типа уже есть метод экземпляра с такой же сигнатурой, он имеет приоритет над методом расширения. Метод расширения никогда не перекрывает существующий метод экземпляра.
- Доступ к членам. Метод расширения не имеет доступа к `private` и `protected` элементам расширяемого класса, поскольку фактически является внешним статическим методом.
- Отсутствие полиморфизма. Методы расширения не являются виртуальными. Вызываемый метод определяется типом переменной на этапе компиляции, а не типом объекта на этапе выполнения.

Методы расширения можно создавать не только для классов, но и для интерфейсов и встроенных типов. Метод расширения для интерфейса становится доступен для всех классов, реализующих данный интерфейс:

```
static class StringExtensions
{
    /// <summary>
    /// Метод расширения для встроенного типа string
    /// </summary>
    public static int WordCount(this string str)
    {
        if (string.IsNullOrEmpty(str))
            return 0;
        return str.Split(' ',
            StringSplitOptions.RemoveEmptyEntries).Length;
    }
}
```

```
// Использование:
string text = "Язык программирования C#";
int words = text.WordCount(); // 3
```

Именно этот подход лежит в основе технологии LINQ: методы Where(), Select(), OrderBy() и другие являются методами расширения для интерфейса IEnumerable<T>, определёнными в статическом классе System.Linq.Enumerable. Поэтому они доступны для массивов, списков, словарей и любых других коллекций, реализующих IEnumerable<T>. Для их использования необходима директива using System.Linq.

ЗАДАНИЕ

Создайте статический класс LibraryItemExtensions с методами расширения для класса LibraryItem:

1. IsNew(int years = 5) — возвращает true, если элемент опубликован не более years лет назад.
2. ToCsvLine() — возвращает строку в формате CSV (разделитель — точка с запятой): Тип;Название;Год;Доп.информация.
3. PrintCard() — выводит в консоль оформленную каталожную карточку.

В основной программе вызовите методы расширения для объектов каталога.

РЕШЕНИЕ:

```
// ===== Файл: LibraryItemExtensions.cs =====
static class LibraryItemExtensions
{
    public static bool IsNew(this LibraryItem item, int years = 5)
    {
        return item.AgeInYears <= years;
    }

    public static string ToCsvLine(this LibraryItem item)
    {
        string type = item is Book ? "Книга" : item is Magazine ? "Журнал" :
"Элемент";

        string extra = item switch
        {
            Book b => $"{b.Author};{b.PageCount};{b.Genre}",
            Magazine m => $"{m.Publisher};{m.IssueNumber};",
            _ => ";;"
        };
    }
}
```

```

        return $"{type};{item.Title};{item.Year};{extra}";
    }

    public static void PrintCard(this LibraryItem item)
    {
        Console.WriteLine("┌" + new string('-', 48) + "┐");
        Console.WriteLine($"| {item.GetCardInfo(),-47}|");
        string status = item.IsNew() ? "НОВИНКА" : $"Возраст: {item.AgeInYears} лет";
        Console.WriteLine($"| {status,-47}|");
        Console.WriteLine("└" + new string('-', 48) + "┘");
    }
}

// ===== Основная программа =====
Book book1 = new("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" };
Magazine mag1 = new("Наука и жизнь", 2025, 3, "Пресса");

// Методы расширения вызываются как обычные методы объекта
Console.WriteLine($"«{book1.Title}» новинка? {book1.IsNew()}");
Console.WriteLine($"«{mag1.Title}» новинка? {mag1.IsNew()}");

Console.WriteLine("\n=== CSV ===");
Console.WriteLine(book1.ToCsvLine());
Console.WriteLine(mag1.ToCsvLine());

Console.WriteLine("\n=== Каталогные карточки ===");
book1.PrintCard();
mag1.PrintCard();

```

8.12 Частичные классы

Еще один уникальный механизм, существующий в языке C#, это механизм частичных классов. Этот механизм позволяет объявить класс в нескольких файлах.

В языке C# этот механизм используется вместе со средствами автоматической генерации кода. В Visual Studio существует много средств, которые автоматически генерируют код, например для обращения к базе данных используется технология Entity Framework.

Если класс автоматически генерируется с помощью какого-либо средства, то он по умолчанию создается как частичный с помощью ключевого слова `partial`. Это позволяет создать другую «часть» данного класса в отдельном файле и вручную дописать необходимые методы к автоматически сгенерированному классу.

Казалось бы, что эта возможность есть в языке C++, ведь в нем предусмотрено разделение класса на заголовочный файл и реализацию. Но в языке C++ невозможно разделить заголовочный файл на несколько

файлов. В Java это принципиально невозможно, поскольку действует правило один класс – один файл. Поэтому, когда в Java возникла необходимость создания заглушек для сетевого взаимодействия, то для этого создали специальный шаблон проектирования из нескольких классов.

Пример объявления частичного класса. Файл «PartialClass1.cs»:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Classes
{
    partial class PartialClass
    {
        int i1;

        public PartialClass(int pi1, int pi2) { i1 = pi1; i2 = pi2;}

        public int MethodPart1(int i1, int i2)
        {
            return i1 + i2;
        }
    }
}
```

Данная часть класса содержит закрытую переменную класса, метод MethodPart1 и конструктор.

Пример объявления частичного класса. Файл «PartialClass2.cs»:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Classes
{
    partial class PartialClass
    {
        int i2;

        public override string ToString()
        {
            return "Частичный класс. i1=" + i1.ToString()
                + " i2=" + i2.ToString();
        }
    }
}
```



```

    }

    public string MethodPart2(string i1, string i2)
    {
        return i1 + i2;
    }
}

```

Данная часть класса содержит закрытую переменную класса, метод MethodPart2 и переопределение виртуального метода ToString.

Работа механизма IntelliSense для частичного класса показана на рис. 27.

Компилятор успешно соединил части частичного класса в нескольких файлах. Методы MethodPart1 и MethodPart2, объявленные в разных файлах, показаны в едином откомпилированном классе. Все элементы, объявленные в любой части класса, доступны из всех остальных частей. В приведённом примере конструктор в файле «PartialClass1.cs» обращается к переменной i2, объявленной в файле «PartialClass2.cs».

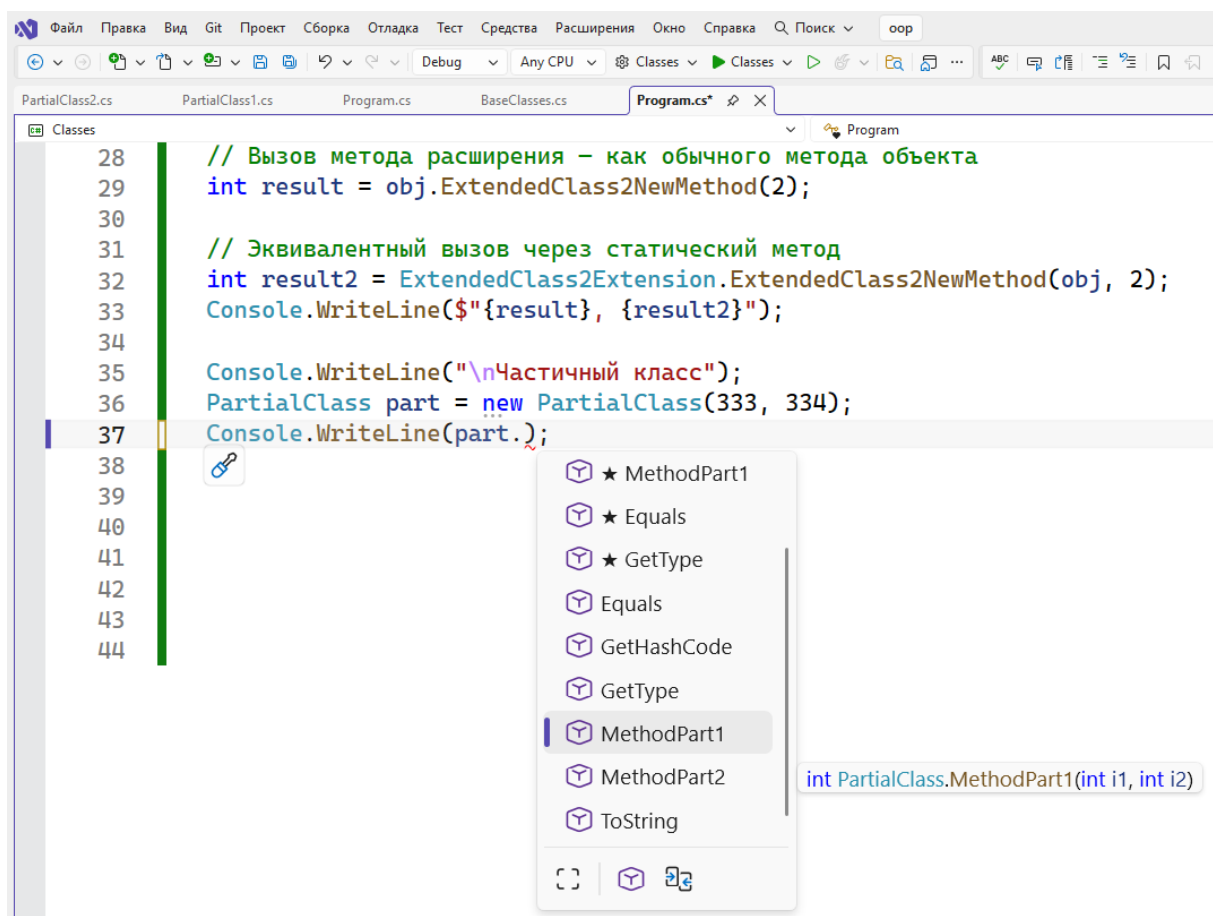


Рис. 27. Работа IntelliSense для частичного класса.

Элементы, помеченные звездочкой (★) в меню IntelliSense на скриншоте, — это рекомендации функции, которая называется Visual Studio IntelliCode. Вместо того чтобы просто показывать список всех доступных методов по алфавиту, ИИ предсказывает, какие методы или свойства вы, скорее всего, захотите использовать именно в этой строке кода. Самые вероятные варианты (в данном случае `MethodPart1`, `Equals`, `GetType`) выносятся в самый верх списка и помечаются звездочкой, чтобы вы могли выбрать их быстрее. Ниже в этом же окне те же самые методы (`MethodPart1` и т.д.) дублируются уже без звездочек — там начинается обычный стандартный список всех доступных членов класса по алфавиту.

При объявлении частичных классов необходимо соблюдать следующие правила:

- Ключевое слово `partial` должно быть указано во всех частях класса. Если хотя бы в одной части оно пропущено, произойдет ошибка компиляции.
- Все части класса должны находиться в одном пространстве имен и в одной сборке.
- Модификаторы доступа всех частей должны совпадать. Если одна часть объявлена как `public`, все остальные тоже должны быть `public`.
- Базовый класс может быть указан в любой из частей, но должен быть одним и тем же.
- Интерфейсы могут быть указаны в разных частях — они объединяются. Это позволяет реализовывать каждый интерфейс в отдельном файле.
- Модификаторы `abstract` и `sealed` могут быть указаны в любой из частей — они применяются ко всему классу.

Ключевое слово `partial` также применимо к структурам (`partial struct`), интерфейсам (`partial interface`) и записям (`partial record`).

8.12.1 Частичные методы

Частичные классы могут содержать частичные методы (`partial methods`). Частичный метод состоит из двух частей: объявления (сигнатуры) и реализации, размещённых в разных частях класса. Базовый синтаксис частичных методов появился в C# 3:

```
// Файл: Order.Generated.cs (сгенерированный код)
partial class Order
{
    public int Id { get; set; }

    public void Process()
    {
        // Вызов частичного метода
        OnProcessing();
        // ... основная логика
    }

    // Объявление частичного метода (без тела)
    partial void OnProcessing();
}

// Файл: Order.cs (код программиста)
partial class Order
{
    // Реализация частичного метода
    partial void OnProcessing()
    {
        Console.WriteLine($"Обработка заказа #{Id}");
    }
}
```

Если реализация частичного метода не предоставлена, компилятор полностью удаляет все вызовы этого метода из скомпилированного кода. Это позволяет генераторам кода предусматривать «точки расширения», которые не влияют на производительность, если программист их не использует.

Базовые частичные методы (C# 3) имеют ограничения: они должны возвращать `void`, не могут иметь модификаторов доступа и не могут иметь параметров `out`. Эти ограничения связаны с тем, что реализация может отсутствовать.

Расширенные частичные методы появились в версии C# 9, в которой были сняты ограничения для частичных методов с явным модификатором доступа. Такие методы обязаны иметь реализацию. Пример на C# 9:

```
// Файл: Calculator.Generated.cs
partial class Calculator
{
    // Расширенный частичный метод (C# 9):
    // имеет модификатор доступа и возвращаемый тип
    public partial int Compute(int a, int b);
}

// Файл: Calculator.cs
partial class Calculator
{
    // Реализация обязательна, иначе — ошибка компиляции
    public partial int Compute(int a, int b)
    {
        return a + b;
    }
}
```

В версии C# 13 поддержка ключевого слова `partial` была распространена на свойства. Частичные свойства следуют тем же принципам, что и расширенные частичные методы — объявление и реализация размещаются в разных частях класса:

```
// Файл: Person.Generated.cs
partial class Person
{
    // Объявление частичного свойства
    public partial string DisplayName { get; }
}

// Файл: Person.cs
partial class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

```
// Реализация частичного свойства
public partial string DisplayName
    => $"{FirstName} {LastName}";
}
```

ЗАДАНИЕ

Создайте класс Library (каталог библиотеки), разделённый на два файла с помощью partial:

- Файл 1 (Library.Data.cs) — хранение данных: список элементов List<LibraryItem>, метод Add(LibraryItem item), метод PrintAll().
- Файл 2 (Library.Search.cs) — методы поиска: FindByTitle(string keyword), FindBooks(), FindMagazines().

В основной программе создайте библиотеку, заполните её и используйте методы из обоих файлов.

РЕШЕНИЕ:

```
// ===== Файл: Library.Data.cs =====
partial class Library
{
    private List<LibraryItem> _items = new();

    public string Name { get; }
    public int Count => _items.Count;

    public Library(string name)
    {
        Name = name;
    }

    public void Add(LibraryItem item)
    {
        _items.Add(item);
        Console.WriteLine($" + Добавлено: {item.ShortDescription}");
    }

    public void PrintAll()
    {
        Console.WriteLine($"\\n=== {Name}: все элементы ({Count}) ===");
        for (int i = 0; i < _items.Count; i++)
        {
            Console.WriteLine($" {i + 1}. {_items[i].GetInfo()}");
        }
    }
}

// ===== Файл: Library.Search.cs =====
partial class Library
{
    public List<LibraryItem> FindByTitle(string keyword)
    {
        List<LibraryItem> results = new();
        foreach (var item in _items)
        {
```

```

        if (item.Title.Contains(keyword,
            StringComparison.OrdinalIgnoreCase))
            results.Add(item);
    }
    return results;
}

public List<Book> FindBooks()
{
    List<Book> books = new();
    foreach (var item in _items)
    {
        if (item is Book b)
            books.Add(b);
    }
    return books;
}

public List<Magazine> FindMagazines()
{
    List<Magazine> magazines = new();
    foreach (var item in _items)
    {
        if (item is Magazine m)
            magazines.Add(m);
    }
    return magazines;
}
}

// ===== Основная программа =====
Library lib = new Library(LibraryUtils.LibraryName);

lib.Add(new Book("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" });
lib.Add(new Magazine("Наука и жизнь", 2025, 3, "Пресса"));
lib.Add(new Book("1984", "Оруэлл", 1949, 328) { Genre = "Антиутопия" });
lib.Add(new Magazine("National Geographic", 2024, 12, "NatGeo"));
lib.Add(new Book("Война и мир. Том 2", "Толстой", 1869, 900) { Genre = "Роман" });

lib.PrintAll();

Console.WriteLine("\n=== Поиск по слову «война» ===");
foreach (var item in lib.FindByTitle("война"))
    Console.WriteLine($" {item.GetInfo()}");

Console.WriteLine($" \nКниг: {lib.FindBooks().Count}");
Console.WriteLine($" \nЖурналов: {lib.FindMagazines().Count}");

```

8.13 Паттерн «текущий интерфейс» (Fluent Interface) и цепочки вызовов методов

При работе с объектами нередко возникает ситуация, когда требуется последовательно вызвать несколько методов одного и того же объекта. В классическом стиле это выглядит следующим образом:

```

var builder = new ReportBuilder();
builder.SetTitle("Отчёт");
builder.AddLine("Строка 1");
builder.AddLine("Строка 2");
builder.SetFooter("Итого");
string report = builder.Build();

```

Имя переменной `builder` повторяется в каждой строке, что увеличивает объём кода и затрудняет его чтение. Для решения этой проблемы применяется паттерн проектирования `Fluent Interface` (текущий интерфейс), впервые описанный Мартином Фаулером и Эриком Эвансом в 2005 году.

Паттерн `Fluent Interface` широко применяется в стандартных библиотеках и фреймворках `.NET`.

Идея паттерна состоит в том, что метод объекта, выполнив полезное действие, возвращает ссылку на текущий объект (то есть `this`). Благодаря этому вызовы методов можно выстраивать в цепочку (`method chaining`):

```
string report = new ReportBuilder()
    .SetTitle("Отчёт")
    .AddLine("Строка 1")
    .AddLine("Строка 2")
    .SetFooter("Итого")
    .Build();
```

Каждый метод в цепочке возвращает объект типа `ReportBuilder`, поэтому к результату можно сразу применить следующий метод того же класса.

Примером паттерна `Fluent Interface` в стандартной библиотеке `.NET` является рассмотренный ранее класс `StringBuilder`. Методы `Append`, `AppendLine`, `AppendFormat`, `Insert`, `Replace` и другие возвращают ссылку на текущий объект `StringBuilder`, что позволяет записывать вызовы в цепочку:

```
string result = new StringBuilder()
    .Append("Привет")
    .Append(", ")
    .Append("мир!")
    .ToString();
```

Для реализации паттерна `Fluent Interface` достаточно, чтобы методы класса возвращали `this`:

```
class ReportBuilder
{
    private string _title = "";
    private readonly List<string> _lines = new();
    private string _footer = "";
```

```

public ReportBuilder SetTitle(string title)
{
    _title = title;
    return this; // возвращаем текущий объект
}

public ReportBuilder AddLine(string line)
{
    _lines.Add(line);
    return this;
}

public ReportBuilder SetFooter(string footer)
{
    _footer = footer;
    return this;
}

/// <summary>
/// Терминальный метод – завершает цепочку и возвращает результат
/// </summary>
public string Build()
{
    var sb = new StringBuilder();
    sb.AppendLine($"=== {_title} ===");
    foreach (var line in _lines)
        sb.AppendLine($" {line}");
    sb.AppendLine($"--- {_footer} ---");
    return sb.ToString();
}
}

```

Обратите внимание, что метод `Build()` является терминальным — он завершает цепочку и возвращает итоговый результат другого типа (`string`). В паттерне `Fluent Interface` принято разделять методы на две категории: **промежуточные** (возвращают `this` для продолжения цепочки) и **терминальные** (возвращают конечный результат и завершают цепочку).

Текущий интерфейс имеет глубокую связь с фундаментальным понятием алгебры — замкнутостью операций на носителе.

Алгеброй называется множество (носитель) с набором операций, результат которых также принадлежит этому множеству. Иными словами, операции замкнуты на носителе. Это означает, что результат любой

операции можно использовать как вход для следующей операции, что и позволяет выстраивать цепочки.

При реализации Fluent Interface следует учитывать ряд рекомендаций:

- **Неизменяемость.** Предпочтительно, чтобы промежуточные методы возвращали новый объект, а не изменяли текущий. Это делает код безопаснее и предсказуемее. Если же метод изменяет текущий объект и возвращает `this` (изменяемый текущий интерфейс), это должно быть чётко задокументировано. Примером изменяемого текучего интерфейса является `StringBuilder`.
- **Терминальные методы.** Цепочка должна завершаться терминальным методом, который возвращает конечный результат, например `ToString()`.
- **Читаемость.** Fluent Interface оправдан, когда он повышает читаемость кода. Если цепочка становится слишком длинной или запутанной, лучше использовать промежуточные переменные.
- **Отладка.** При возникновении ошибок (исключений) в длинной цепочке может быть затруднено определение конкретного метода, вызвавшего ошибку. В таких случаях полезно разбить цепочку на отдельные вызовы с промежуточными переменными.

ЗАДАНИЕ

Примените подход Fluent Interface к любому созданному Вами классу библиотечной системы.

8.14 Создание диаграммы классов в Visual Studio

Для удобства разработки в Visual Studio существует возможность создания диаграммы классов.

Для добавления диаграммы классов необходимо добавить в проект элемент «диаграмма классов», как показано на рисунках в этом разделе.

Нужно нажать правую кнопку мыши на проекте в дереве проектов и выбрать в контекстном меню пункты «Добавить/Создать элемент».

Затем в диалоге добавления нового элемента следует выбрать пункт меню «диаграмма классов» (файл с расширением .cd).

Схема классов открывается в виде белого «холста», на который нужно переносить файлы из обозревателя решений. При этом все классы данного файла автоматически попадают на диаграмму. Пример такой диаграммы приведен на рис. 30.

Нотация диаграмм классов в Visual Studio практически полностью соответствует нотации диаграмм классов UML (Unified Modelling Language). Наследование классов показано незаштрихованной треугольной стрелкой, реализация интерфейсов – стрелкой в виде незаштрихованной окружности.

При нажатии на холсте правой кнопки мыши предоставляется возможность выбрать в контекстном меню пункт «экспорт схемы как изображения». В этом случае вся диаграмма классов экспортируется как изображение в графическом формате (jpeg, png, другие форматы), что удобно для оформления документации.

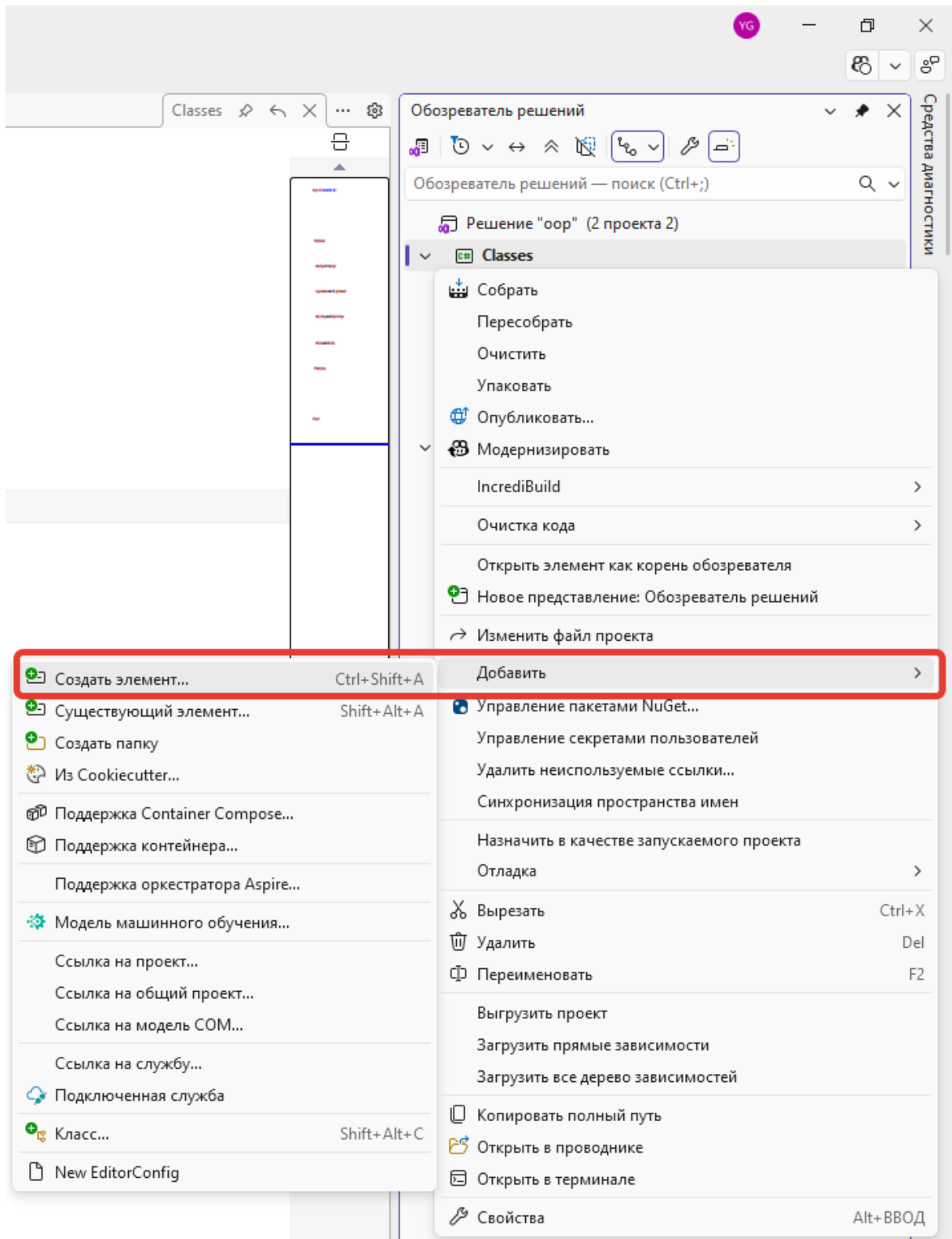


Рис. 28. Добавление диаграммы классов (шаг 1).

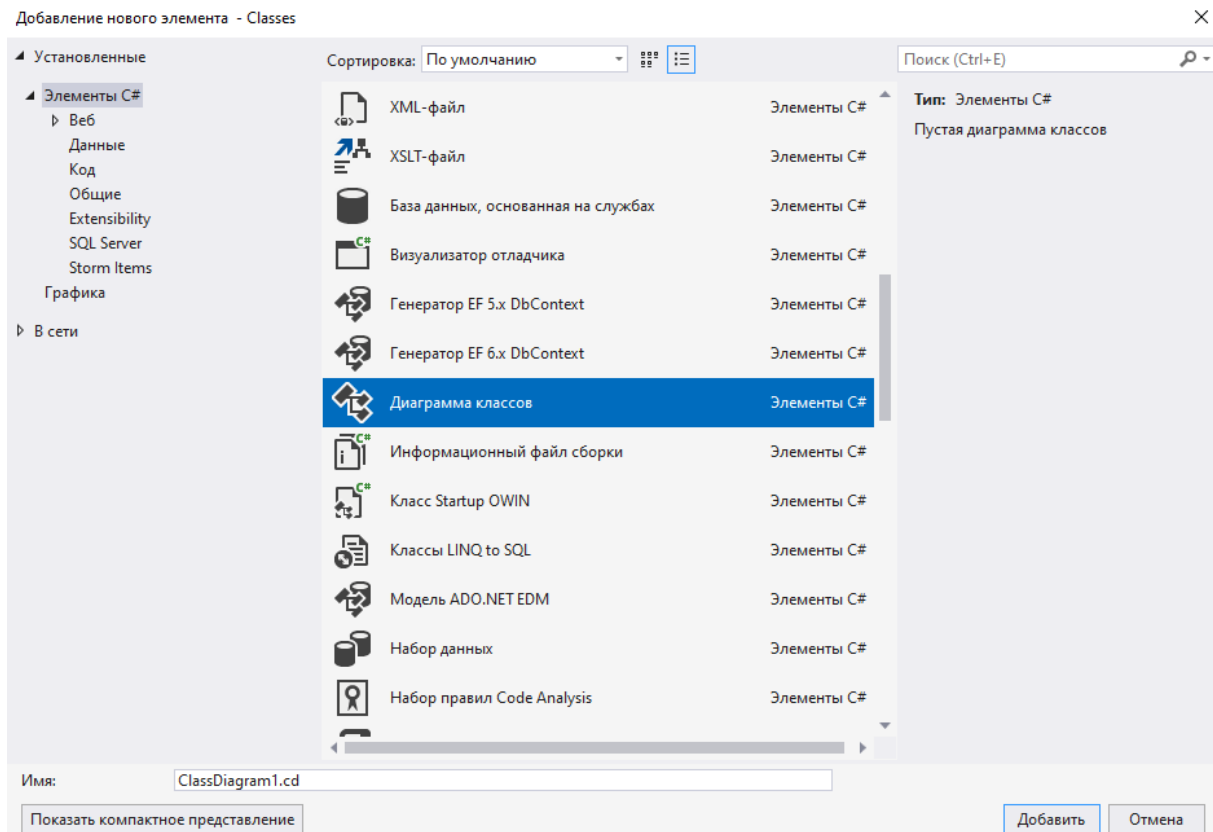


Рис. 29. Добавление диаграммы классов (шаг 2).

В панели кнопок диаграммы классов предусмотрены различные варианты действий (рис. 31):

- 
 – автоматическое размещение классов на диаграмме;
- 
 – растянуть выбранный класс на полную ширину;
- 
 – отобразить только имена полей;
- 
 – отобразить имена и типы полей;
- 
 – отобразить полную сигнатуру элементов класса.

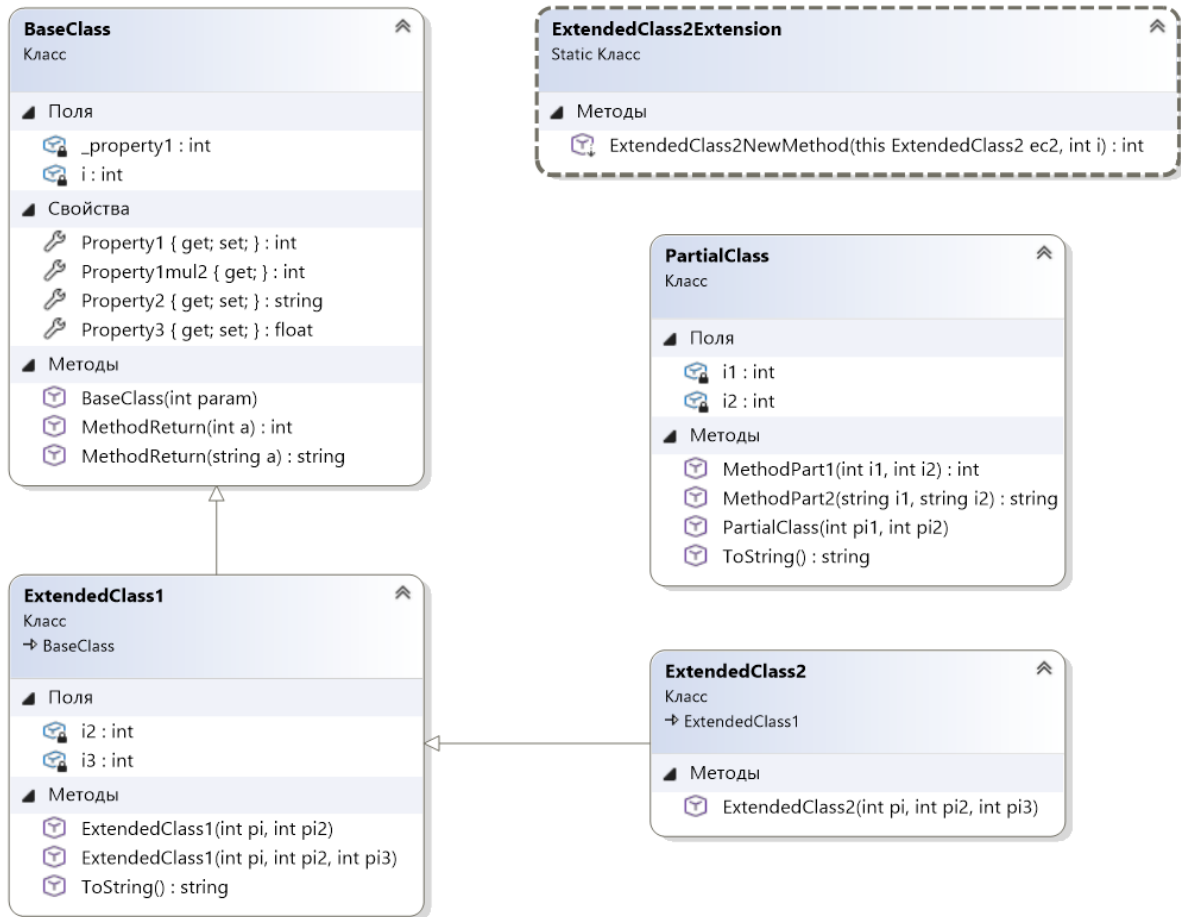


Рис. 30. Фрагмент диаграммы классов.

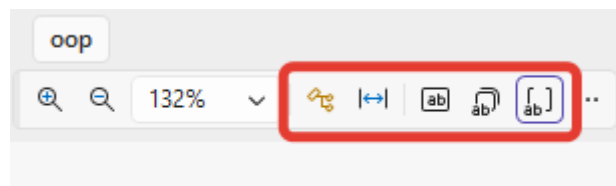


Рис. 31. Кнопки отображения информации о полях классов.

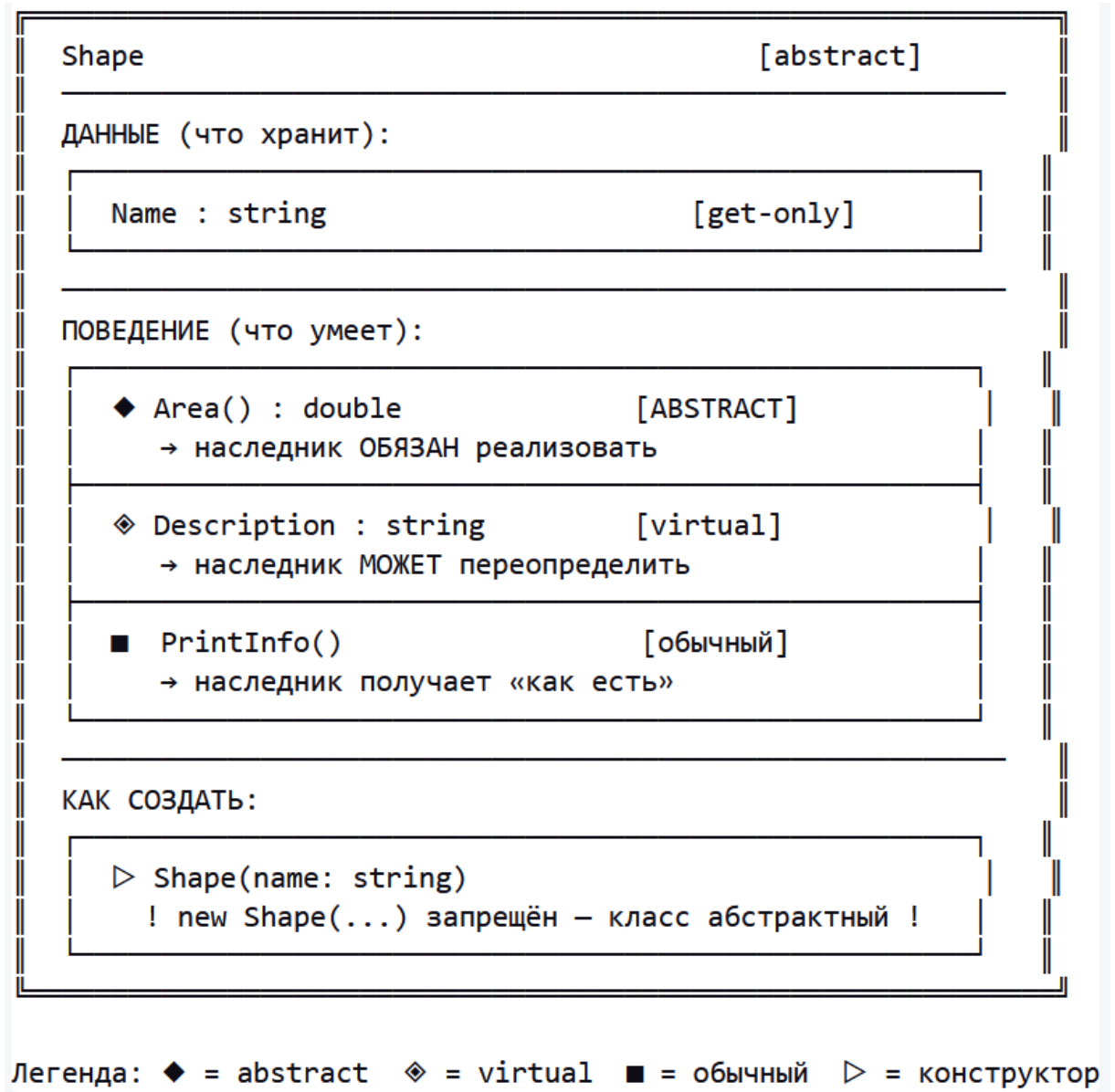
ЗАДАНИЕ

Создайте диаграмму классов для Вашего проекта, отразите полные сигнатуры для всех классов.

9 Ситуационное метаграфовое описание ООП в С#

9.1 Класс как ситуация

Рассмотрим класс Shape. Класс — это описание ситуации: он фиксирует, что существует некая «фигура», какие у неё данные и что она умеет делать. Представим его как «коробку», внутри которой лежат вложенные «карточки»:

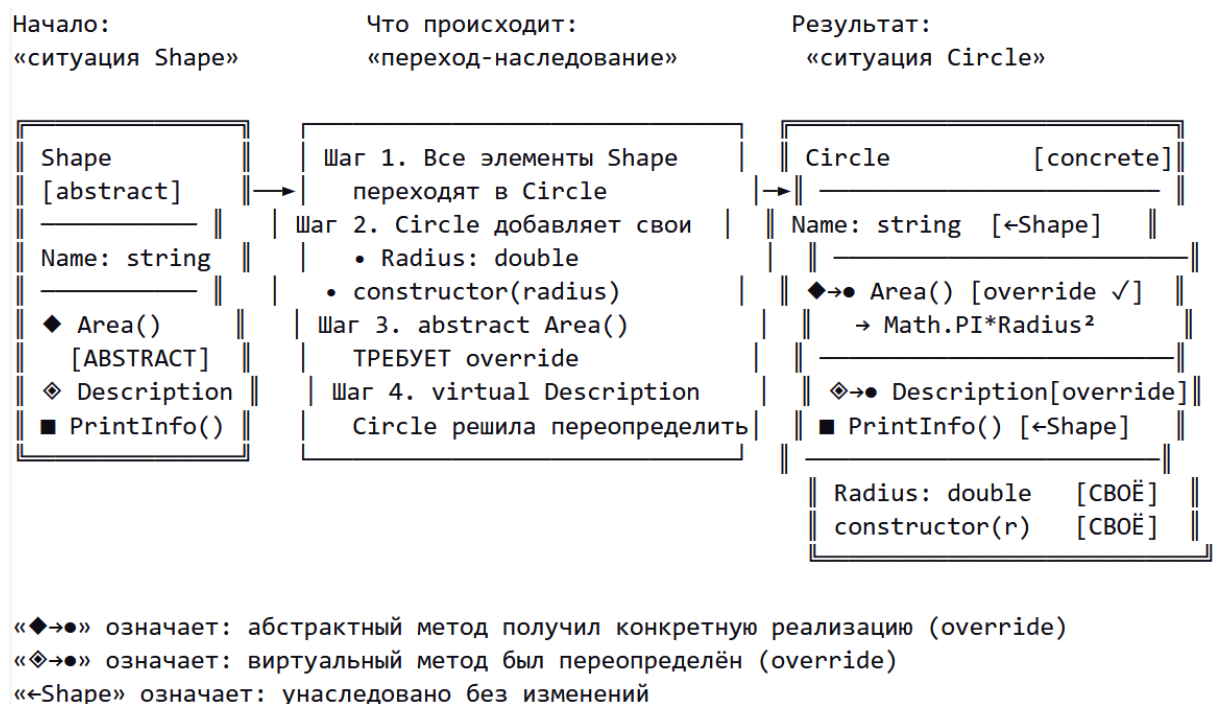


Главное наблюдение: коробка Shape сама по себе является целостным описанием ситуации «фигура существует». При этом коробка имеет

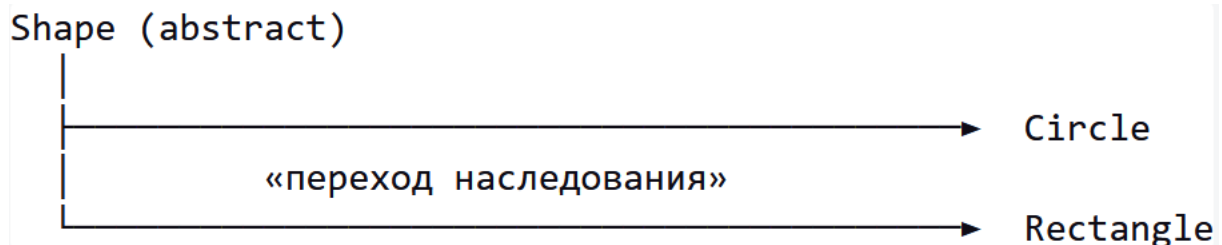
собственные свойства (abstract, видимость) — они принадлежат всей коробке, а не её содержимому.

9.2 Наследование как переход между ситуациями

Теперь объявляем `class Circle : Shape`. Это не просто «копирование» — это процесс, в котором исходная ситуация (Shape) трансформируется в новую, более конкретную (Circle). Ниже показан этот процесс как цепочка шагов:



Для `Rectangle : Shape` подход аналогичен. В итоге мы получаем семейство ситуаций, объединённых общим предком:

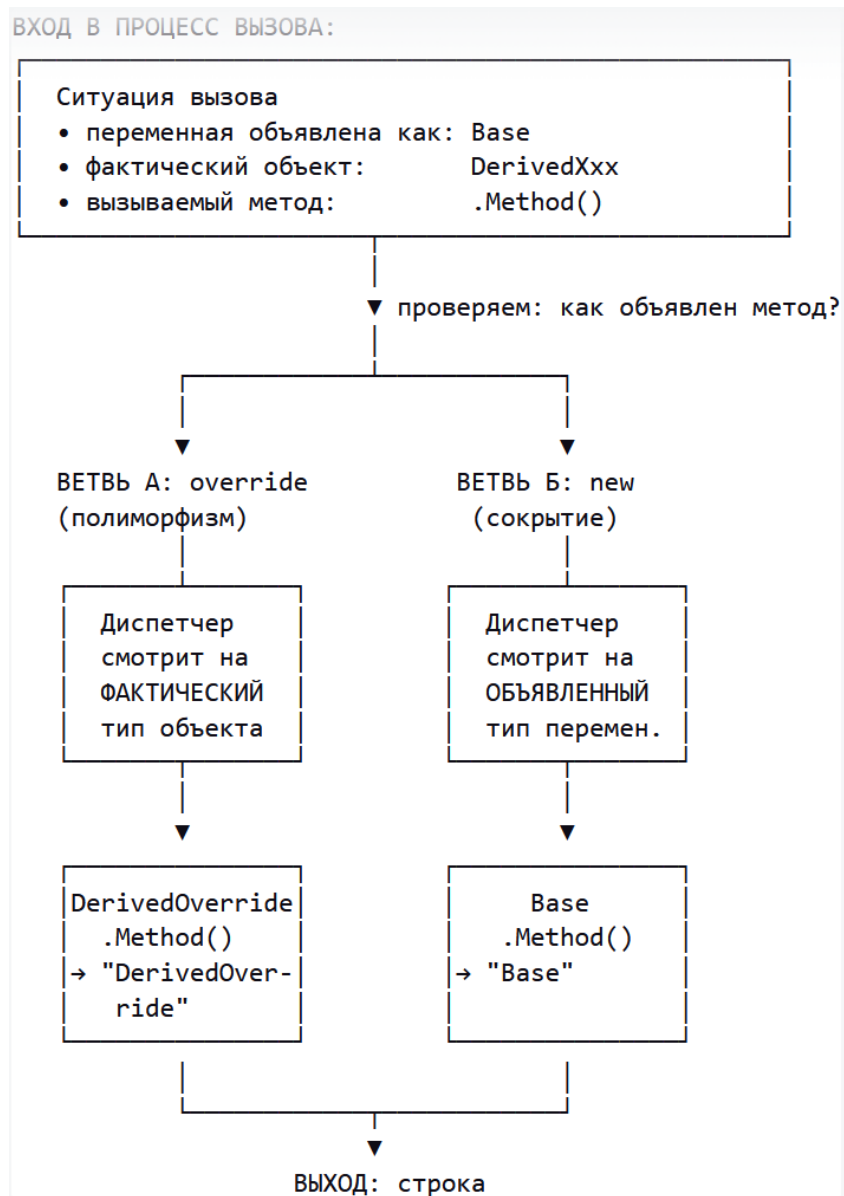


9.3 Цепочка виртуализации как процесс с ветвлением И/ИЛИ

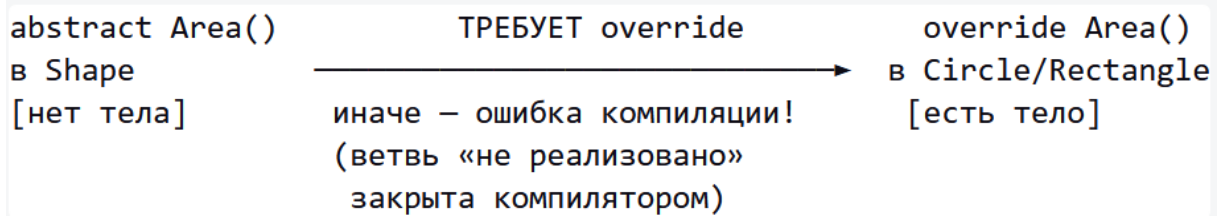
Когда мы вызываем метод через переменную базового типа, возникает ситуация диспетчеризации. В зависимости от того, как метод был объявлен, процесс идёт по разным ветвям.

```
Base obj1 = new DerivedOverride(); // virtual + override
Base obj2 = new DerivedNew();      // virtual + new (сокрытие)
```

Процесс вызова obj.Method() выглядит так:



Для `abstract` метода — подход ещё строже: ветви «не реализовано» в принципе нет (компилятор не позволит создать объект, не реализовавший `abstract`):



9.4 Основы метаграфового подхода для описания ситуаций

Дополним рисунки предыдущего раздела правильными понятиями:

Что нарисовали	Метаграфовый термин	Формально
«Коробка» <code>Shape</code> как целое	Метавершина mv_{Shape}	Имеет собственные атрибуты + вложенный фрагмент
Карточки внутри: <code>Name</code> , <code>Area()</code> , <code>Radius</code>	Вершины $v_i \in V$	Характеризуются атрибутами (тип, модификатор)
Свойства всей коробки: <code>abstract</code> , <code>visibility</code>	Атрибуты метавершины $\{atr_k\}$	Принадлежат метавершине, не входящим в неё вершинам
Связи внутри: $\blacklozenge \rightarrow \bullet$ (<code>override</code>), вызовы	Рёбра $e_j \in E$	Ненаправленные (структура) или направленные (вызов)
Стрелка наследования <code>Shape</code> $\rightarrow$ <code>Circle</code>	Метаребро $me_{inherit}$	Структурный переход между метавершинами

Перерисуем класс `Shape` с соответствующими аннотациями:

<code>mv: Shape</code>	← МЕТАВЕРШИНА
<code>atr: name = "Shape"</code>	← атрибут метавершины
<code>atr: modifier = "abstract"</code>	← атрибут метавершины
<code>atr: visibility = "internal"</code>	← атрибут метавершины
Вложенный фрагмент <code>MG_struct</code> :	
<code>v: Name</code>	← вершина
<code>v: Area()</code>	← вершина
<code>v: Description</code>	← вершина
<code>v: PrintInfo()</code>	← вершина
<code>v: ctor</code>	← вершина
<code>e: Area() —[requires_override]→ (наследники)</code>	← ребро

Рассмотрим формальное описание метаграфовой модели:

Определим **метаграф** следующим образом: $MG = \langle V, MV, E, ME \rangle$,

где MG – метаграф; V – множество вершин метаграфа; MV – множество метавершин метаграфа; E – множество ребер метаграфа; ME – множество метаребер метаграфа.

Вершина метаграфа характеризуется **множеством атрибутов**:

$v_i = \{atr_k\}, v_i \in V$, где v_i – вершина метаграфа; atr_k – атрибут.

Ребро метаграфа характеризуется множеством атрибутов, исходной и конечной вершиной и признаком направленности:

$e_i = \langle v_s, v_e, eo, \{atr_k\} \rangle, e_i \in E, eo = true | false$, где e_i – ребро метаграфа; v_s – исходная вершина (метавершина) ребра; v_e – конечная вершина (метавершина) ребра; eo – признак направленности ребра ($eo=true$ – направленное ребро, $eo=false$ – ненаправленное ребро); atr_k – атрибут.

Фрагмент метаграфа в общем виде может содержать произвольные вершины (метавершины) и ребра (метаребра): $MG_i = \{ev_j\}$, $ev_j \in (V \cup E \cup MV \cup ME)$, где MG_i – фрагмент метаграфа; ev_j – элемент, принадлежащий объединению множеств вершин (метавершин) и ребер (метаребер) метаграфа.

Метавершина метаграфа в дополнение к свойствам вершины включает вложенный фрагмент метаграфа: $mv_i = \langle \{atr_k\}, \{ev_j\} \rangle$, $mv_i \in MV, ev_j \in (V \cup E \cup MV \cup ME)$, где mv_i – вершина метаграфа; atr_k – атрибут, ev_j – элемент, принадлежащий объединению множеств вершин (метавершин) и ребер (метаребер) метаграфа.

Наличие у метавершин собственных атрибутов и связей с другими вершинами является важной особенностью метаграфов. Это соответствует принципу **эмерджентности**, то есть приданию понятию нового качества,

несводимости понятия к сумме его составных частей. Фактически, как только вводится новое понятие в виде метавершины, оно «получает право» на собственные свойства, связи и т.д., так как в соответствии с принципом эмерджентности новое понятие обладает новым качеством и не может быть сведено к подграфу базовых понятий.

Таким образом, метаграф можно охарактеризовать как «сложный граф с эмерджентностью» или «сложную сеть с эмерджентностью», то есть фрагмент сети, состоящий из вершин и связей, может выступать как отдельное целое.

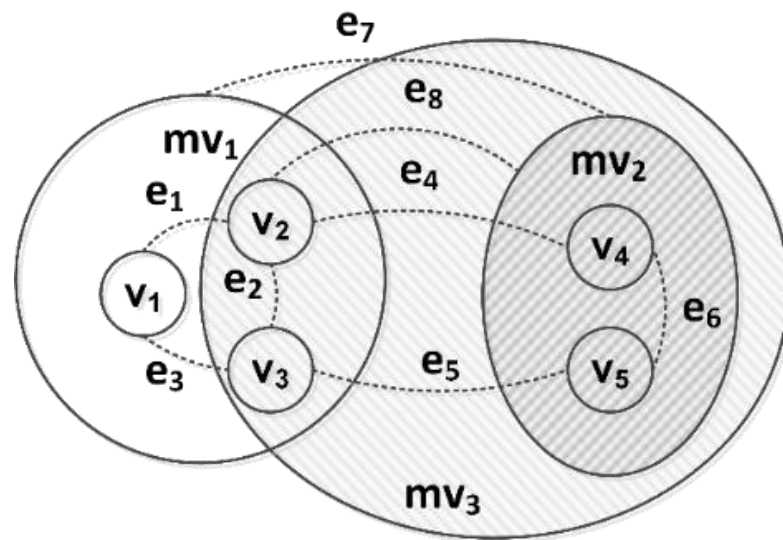


Рис. 32. Пример описания метаграфа в метаграфовой модели.

Пример описания метаграфа показан на рис. 32. Данный метаграф содержит вершины, метавершины и ребра. На рис. 32 показаны три метавершины: mv_1 , mv_2 и mv_3 . Метавершина mv_1 включает вершины v_1 , v_2 , v_3 и связывающие их ребра e_1 , e_2 , e_3 . Метавершина mv_2 включает вершины v_4 , v_5 и связывающее их ребро e_6 . Ребра e_4 , e_5 являются примерами ребер, соединяющих вершины v_2 - v_4 и v_3 - v_5 , включенные в различные метавершины mv_1 и mv_2 . Ребро e_7 является примером ребра, соединяющего метавершины mv_1 и mv_2 . Ребро e_8 является примером ребра, соединяющего вершину v_2 и метавершину mv_2 . Метавершина mv_3 включает метавершину

mv_2 , вершины v_2 , v_3 и ребро e_2 из метавершины mv_1 а также ребра e_4 , e_5 , e_8 , что показывает холоническую структуру метаграфа.

Важно, что в отличие от обычных графов метавершина может включать как вершины, так и ребра.

Метаребро метаграфа в дополнение к свойствам ребра включает вложенный фрагмент метаграфа:

$$me_i = \langle v_s, v_e, eo, \{atr_k\}, \{ev_j\} \rangle, e_i \in E, eo = true \mid false, ev_j \in (V \cup E \cup MV \cup ME),$$

где me_i – метаребро метаграфа; v_s – исходная вершина (метавершина) ребра; v_e – конечная вершина (метавершина) ребра; eo – признак направленности метаребра ($eo=true$ – направленное метаребро, $eo=false$ – ненаправленное метаребро); atr_k – атрибут; ev_j – элемент, принадлежащий объединению множеств вершин (метавершин) и ребер (метаребер) метаграфа.

Пример описания метаребра метаграфа представлен на рис. 3. Метаребро содержит метавершины $v_s, \dots, v_i, \dots, v_e$ и связывающие их ребра. Исходная метавершина содержит фрагмент метаграфа. В процессе преобразования исходной метавершины v_s в конечную метавершину v_e происходит дополнение содержимого метавершины, добавляются новые вершины, связи, вложенные метавершины.

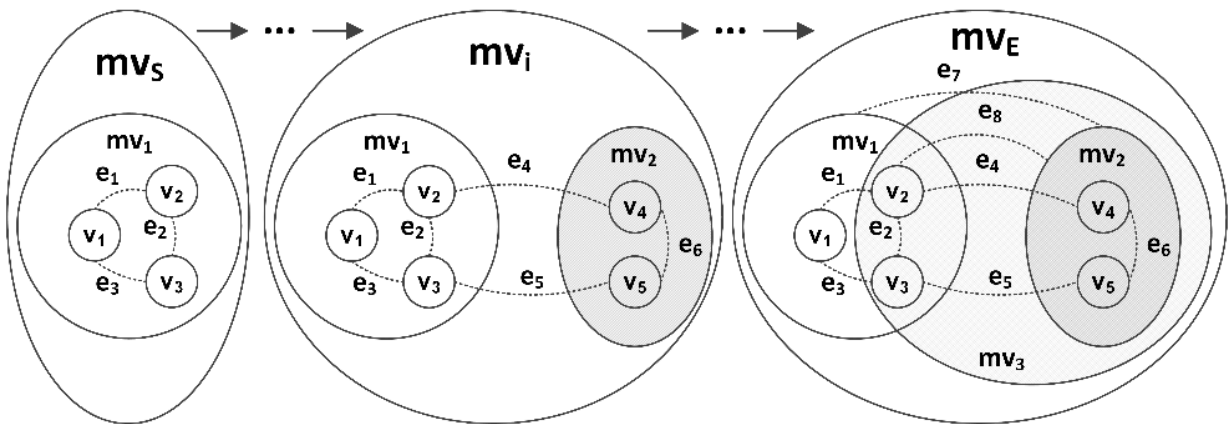


Рис. 33. Пример описания метаребра метаграфа в метаграфовой модели.

Если метавершины предназначены прежде всего для описания данных и знаний, то метаребра предназначены в большей степени для описания процессов. Таким образом, метаграфовая модель позволяет в рамках единой модели описывать данные, знания и процессы.

Поэтому метаграфовая модель подходит для описания как ситуаций, так и переходов между ними.

10 Расширенные возможности объектно-ориентированного программирования в языке C#

10.1 Обработка исключений

Если потребовалось бы написать на чистом языке C приложение, выполняющее критически важные действия, то оно было бы написано в следующем стиле: выполняется вызов функции, функция возвращает код возврата (код ошибки), производится анализ кода возврата с помощью оператора `switch...case`, в зависимости от кода возврата выполняются действия по обработке ошибок. Не возникает сомнений в том, что такое программирование будет очень трудоемко.

Для решения данной проблемы в современных языках программирования (C#, Java, современные версии C++) существуют конструкции обработки исключений, которые достаточно похожи в трех языках.

10.1.1 Блоки `try`, `catch` и `finally`

В блоке `try` записываются операторы, которые могут привести к возникновению ошибок. Собственно, термин «ошибка» употреблять не совсем корректно – то, что в программе на чистом языке C было

критической ошибкой, теперь называют «исключением», исключительной ситуацией, которая может быть обработана в программе.

Если возникает ошибка (исключение) определенного вида, то она обрабатывается в блоке `catch`. Все исключения в C# — классы, унаследованные от класса `Exception`. В скобках после `catch` указывают класс исключения и переменную этого класса, из которой можно получить конкретные сведения о произошедшем исключении.

Наиболее часто используемые свойства класса `Exception`:

- `Message` — текстовое описание ошибки;
- `StackTrace` — строка, содержащая стек вызовов в момент возникновения исключения (полезна для диагностики);
- `InnerException` — ссылка на внутреннее исключение, которое стало причиной текущего (может быть `null`);
- `Source` — имя приложения или объекта, вызвавшего исключение.

Блоков `catch` может быть несколько, каждый из них осуществляет перехват исключения определенного вида. В этом случае срабатывает только один, первый по порядку блок `catch`, поэтому последовательность обработки исключений очень важна. Необходимо, чтобы первыми располагались наиболее детальные исключения, те, которые размещаются на наиболее детальном уровне в дереве наследования от класса `Exception`. Если первым в списке поставить наиболее общий класс `Exception`, то любое исключение будет приведено к этому типу, и остальные блоки `catch` никогда не будут выполнены. Такое приведение типов является следствием полиморфизма классов в ООП — дочерний тип (класс) может быть приведен к родительскому типу (классу).

Необязательный блок `finally` располагается после блоков `catch`. Действия в блоке `finally` выполняются всегда, вне зависимости от того, произошло исключение в блоке `try` или нет. Если, например, в блоке `try` происходит работа с файлами, то в блоке `finally` можно расположить

оператор закрытия файла, который будет закрыт вне зависимости от того, были ошибки при работе с файлом или нет.

Обратите внимание, что файлы здесь упоминаются только для примера, как обсуждалось ранее работа с файлами должна происходить с помощью конструкции `using`.

Пример блока обработки исключений:

```
Console.WriteLine("\n\nДеление на 0:");
try
{
    int num1 = 1;
    int num2 = 1;

    string zero = "0";
    int.TryParse(zero, out num2);

    int num3 = num1 / num2;
}
catch (DivideByZeroException e)
{
    Console.WriteLine("Попытка деления на 0");
    Console.WriteLine(e.Message);
}
catch (Exception e)
{
    Console.WriteLine("Собственное исключение");
    Console.WriteLine(e.Message);
}
finally
{
    Console.WriteLine("Это сообщение выводится в блоке finally");
}
```

В данном примере сначала обрабатывается детальное исключение деления на ноль (класс `DivideByZeroException`), а затем все остальные исключения, которые приводятся к наиболее общему классу `Exception`.

Допустимы также конструкции `try/finally` без блока `catch` — в этом случае исключение не перехватывается, но блок `finally` гарантированно выполняется перед дальнейшим распространением исключения вверх по стеку вызовов:

```
FileStream file = new FileStream("data.txt", FileMode.Open);
try
```

```

{
    // Работа с файлом – исключение не перехватывается
    // ...
}
finally
{
    // Файл будет закрыт в любом случае
    file.Close();
}

```

10.1.2 Генерация исключений и оператор throw

Кроме стандартных классов исключений, предусмотренных в .NET, разработчик может создавать собственные исключения, унаследованные от класса `Exception`. Однако такие исключения не будут вызываться автоматически, ведь никаких критичных действий вроде деления на ноль не происходит. Поэтому разработчик должен искусственно сгенерировать ошибку с помощью команды `throw`, причем можно генерировать и стандартные исключения, существующие в .NET.

Пример генерации исключения:

```

Console.WriteLine("\nСобственное исключение:");
try
{
    throw new Exception("!!! Новое исключение !!!");
}
catch (DivideByZeroException e)
{
    Console.WriteLine("Попытка деления на 0");
    Console.WriteLine(e.Message);
}
catch (Exception e)
{
    Console.WriteLine("Собственное исключение");
    Console.WriteLine(e.Message);
}
finally
{
    Console.WriteLine("Это сообщение выводится в блоке finally");
}

```

Если оператор `throw` используется без параметров в форме «`throw;`», то он повторно генерирует текущее исключение, сохраняя исходный `stack`

trace, что важно для детальной диагностики исключений. Пример повторной генерации исключения:

```
try
{
    // ... код, который может вызвать исключение
}
catch (Exception e)
{
    // Логирование ошибки
    Console.WriteLine($"Ошибка: {e.Message}");
    // Повторная генерация – сохраняет исходный stack trace
    throw;

    // НЕПРАВИЛЬНО: throw e; – это сбросит stack trace,
    // и информация о месте возникновения будет потеряна
}
```

10.1.3 Часто используемые стандартные исключения

Таблица 6

Часто используемые стандартные исключения

Класс исключения	Описание
ArgumentNullException	Метод получил аргумент null, хотя null недопустим
ArgumentOutOfRangeException	Значение аргумента выходит за допустимый диапазон
ArgumentException	Недопустимое значение аргумента (общий случай)
InvalidOperationException	Вызов метода недопустим для текущего состояния объекта
NullReferenceException	Попытка обращения к члену объекта, равного null
IndexOutOfRangeException	Индекс за пределами массива или коллекции
DivideByZeroException	Деление целого числа на ноль
FileNotFoundException	Файл не найден

IOException	Общая ошибка ввода-вывода
FormatException	Некорректный формат строки при преобразовании
NotSupportedException	Метод или операция не реализованы
NotSupportedException	Вызванный метод не поддерживается

10.1.4 Создание собственных исключений

Для создания собственного исключения необходимо объявить класс, унаследованный от Exception (или от более конкретного стандартного класса исключений). Рекомендуется в методах класса вызывать конструкторы базового класса:

```

/// <summary>
/// Собственный класс исключения
/// </summary>
class InsufficientFundsException : Exception
{
    /// <summary>
    /// Сумма, которой не хватает
    /// </summary>
    public decimal Shortage { get; }

    public InsufficientFundsException()
        : base() { }

    public InsufficientFundsException(string message)
        : base(message) { }

    public InsufficientFundsException(
        string message, Exception innerException)
        : base(message, innerException) { }

    public InsufficientFundsException(
        string message, decimal shortage)
        : base(message)
    {
        Shortage = shortage;
    }
}

```

}

Пример использования собственного исключения:

```
class BankAccount
{
    public decimal Balance { get; private set; }

    public BankAccount(decimal initialBalance)
    {
        Balance = initialBalance;
    }

    public void Withdraw(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentOutOfRangeException(
                nameof(amount),
                "Сумма должна быть положительной");

        if (amount > Balance)
            throw new InsufficientFundsException(
                $"Недостаточно средств на счёте. "
                + $"Баланс: {Balance}, запрошено: {amount}",
                amount - Balance);

        Balance -= amount;
    }
}

// Использование:
var account = new BankAccount(1000);
try
{
    account.Withdraw(1500);
}
catch (InsufficientFundsException e)
{
    Console.WriteLine(e.Message);
    Console.WriteLine($"Не хватает: {e.Shortage}");
}
```

Обратите внимание, что именовать классы собственных исключений принято с суффиксом Exception.

ЗАДАНИЕ

Создайте собственный класс исключения LibraryException и его наследника InvalidBookDataException. Добавьте валидацию в конструкторы и свойства класса Book, генерируя исключения при

некорректных данных. В основной программе обработайте их с помощью try/catch/finally.

РЕШЕНИЕ:

```
// ===== Файл: LibraryException.cs =====

/// <summary>
/// Базовое исключение библиотечной системы
/// </summary>
class LibraryException : Exception
{
    public LibraryException() : base() { }
    public LibraryException(string message) : base(message) { }
    public LibraryException(string message, Exception inner)
        : base(message, inner) { }
}

/// <summary>
/// Исключение при некорректных данных элемента каталога
/// </summary>
class InvalidBookDataException : LibraryException
{
    public string FieldName { get; }

    public InvalidBookDataException(string fieldName, string message)
        : base(message)
    {
        FieldName = fieldName;
    }
}

// ===== Обновление конструктора Book =====
public Book(string title, string author, int year, int pageCount)
: base(title, year)
{
    if (string.IsNullOrEmpty(author))
        throw new InvalidBookDataException(
            nameof(Author), "Автор не может быть пустым");

    if (pageCount < 0 || pageCount > MaxPageCount)
        throw new InvalidBookDataException(
            nameof(PageCount),
            $"Количество страниц должно быть от 0 до {MaxPageCount}");

    Author = author;
    PageCount = pageCount;
}

// ===== Обновление конструктора LibraryItem =====
public LibraryItem(string title, int year)
{
    if (string.IsNullOrEmpty(title))
        throw new InvalidBookDataException(
            nameof(Title), "Название не может быть пустым");

    if (year < 1450 || year > DateTime.Now.Year)
        throw new ArgumentOutOfRangeException(
            nameof(year),
            $"Год должен быть от 1450 до {DateTime.Now.Year}");

    Title = title;
    Year = year;
    _totalItems++;
}
```

```
// ===== Основная программа (этап 1) =====
Console.WriteLine("=== Этап 1: Валидация и собственные исключения ===\n");

// Тест 1: корректные данные
try
{
    Book ok = new("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" };
    Console.WriteLine($"Успешно: {ok.GetInfo()}");
}
catch (LibraryException e)
{
    Console.WriteLine($"Ошибка: {e.Message}");
}

// Тест 2: пустой автор
try
{
    Book bad1 = new("Книга", "", 2024, 100) { Genre = "Тест" };
}
catch (InvalidBookDataException e)
{
    Console.WriteLine($"Ошибка в поле '{e.FieldName}': {e.Message}");
}

// Тест 3: некорректный год
try
{
    Book bad2 = new("Книга", "Автор", 3000, 100) { Genre = "Тест" };
}
catch (ArgumentOutOfRangeException e)
{
    Console.WriteLine($"Ошибка аргумента: {e.Message}");
}
finally
{
    Console.WriteLine("Блок finally: проверка завершена");
}

// Тест 4: отрицательное количество страниц
try
{
    Book bad3 = new("Книга", "Автор", 2024, -50) { Genre = "Тест" };
}
catch (InvalidBookDataException e)
{
    Console.WriteLine($"Ошибка в поле '{e.FieldName}': {e.Message}");
}
}
```

10.1.5 Фильтры исключений

В версии C# 6 появились фильтры исключений (exception filters) — оператор `when`, который позволяет указать дополнительное условие при перехвате исключения. Блок `catch` срабатывает только если условие `when` истинно.

Пример без фильтров исключений:

```
catch (DivideByZeroException ex)
{
}
```

```

    if (attempt < 3)
        Console.WriteLine("Предупреждение 1");
    else
        Console.WriteLine("Предупреждение 2");
}

```

Пример с фильтрами исключений:

```

catch (DivideByZeroException) when (attempt < 3)
{
    Console.WriteLine("Предупреждение 1");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Предупреждение 2");
}

```

Принципиальное различие между этими подходами состоит в том, что фильтр `when` проверяется до входа в блок `catch`. Если условие `when` ложно, блок `catch` пропускается и проверяется следующий. Это означает, что при использовании фильтров сохраняется исходный `stack trace`, а также можно фильтровать значение в блоке `when` по свойствам исключения:

```

try
{
    // ... сетевая операция
}
catch (HttpRequestException e) when (e.StatusCode ==
    System.Net.HttpStatusCode.NotFound)
{
    Console.WriteLine("Ресурс не найден (404)");
}
catch (HttpRequestException e) when (e.StatusCode ==
    System.Net.HttpStatusCode.Unauthorized)
{
    Console.WriteLine("Требуется авторизация (401)");
}
catch (HttpRequestException e)
{
    Console.WriteLine($"Ошибка HTTP: {e.StatusCode}");
}

```

10.1.6 Выражение `throw`

Начиная с версии `C# 7` оператор `throw` может использоваться как выражение (`expression`), а не только как отдельная инструкция. Это

позволяет генерировать исключения в тернарных операторах, в операторе ?? (null-coalescing) и в expression-bodied методах:

```
class Person
{
    private string _name;

    public string Name
    {
        get => _name;
        // throw в операторе ??
        set => _name = value
            ?? throw new ArgumentNullException(nameof(value));
    }

    // throw в тернарном операторе
    public int GetAge(int age) => age >= 0
        ? age
        : throw new ArgumentOutOfRangeException(nameof(age));
}
```

До C# 7 для таких проверок требовался отдельный оператор if:

```
// До C# 7:
public string Name
{
    get => _name;
    set
    {
        if (value == null)
            throw new ArgumentNullException(nameof(value));
        _name = value;
    }
}
```

ЗАДАНИЕ

Добавьте фильтры when в блоки catch и используйте throw-выражения в свойствах класса Library.

РЕШЕНИЕ:

```
// ===== Добавить в Library.Data.cs =====

// Свойство с throw-выражением (C# 7)
public LibraryItem this[int index] =>
    index >= 0 && index < _items.Count
        ? _items[index]
        : throw new ArgumentOutOfRangeException(
            nameof(index),
            $"Индекс {index} вне диапазона [0..{_items.Count - 1}]");

// Метод Add с валидацией через throw-выражение
public void Add(LibraryItem item)
{
    _ = item ?? throw new ArgumentNullException(nameof(item));
}
```

```

        _items.Add(item);
        Console.WriteLine($" + Добавлено: {item.ShortDescription}");
    }

    // ===== Основная программа (этап 2) =====
    Console.WriteLine("\n==== Этап 2: Фильтры исключений ===\n");

    Library lib = new Library(LibraryUtils.LibraryName);
    lib.Add(new Book("Война и мир", "Толстой", 1869, 1225) { Genre = "Роман" });

    // Фильтр when по свойству исключения
    string[] testData = { "", " ", "Нормальный автор" };
    foreach (string author in testData)
    {
        try
        {
            Book b = new("Тест", author, 2024, 100) { Genre = "Тест" };
            Console.WriteLine($" Создано: {b.GetInfo()}");
        }
        catch (InvalidBookDataException e) when(e.FieldName == "Author")
        {
            Console.WriteLine($" Проблема с автором: «{author}» – {e.Message}");
        }
        catch (InvalidBookDataException e)
        {
            Console.WriteLine($" Другая ошибка данных: {e.Message}");
        }
    }

    // Демонстрация throw-выражения в индексаторе
    try
    {
        var item = lib[99]; // несуществующий индекс
    }
    catch (ArgumentOutOfRangeException e)
    {
        Console.WriteLine($" \nОшибка доступа по индексу: {e.Message}");
    }

    // Попытка добавить null
    try
    {
        lib.Add(null!);
    }
    catch (ArgumentNullException e)
    {
        Console.WriteLine($" Ошибка: {e.Message}");
    }
}

```

10.1.7 Рекомендации по обработке исключений

При работе с исключениями рекомендуется придерживаться следующих рекомендаций:

- Исключения не следует использовать для управления потоком выполнения программы, так как они предназначены исключительно для нештатных ситуаций. В случаях, когда развитие событий можно предвидеть (например, при вводе

некорректных данных пользователем), рекомендуется применять обычную проверку условий.

- Желательно перехватывать только конкретные типы исключений. Перехвата базового класса `Exception` лучше избегать, за исключением тех ситуаций, когда ошибку необходимо лишь залогировать и передать дальше по стеку (с помощью `throw;`).
- Для повторной генерации исключения рекомендуется использовать конструкцию «`throw;`» вместо «`throw e;`», поскольку это позволяет сохранить исходную трассировку стека (`stack trace`).
- Важно всегда заботиться об освобождении ресурсов. Для гарантированной очистки следует применять блок `finally` или конструкцию `using`.
- По возможности стоит отдавать предпочтение стандартным исключениям платформы .NET. Создание собственных классов исключений оправдано в тех случаях, когда встроенные варианты не подходят по смыслу.
- В объект исключения полезно включать исчерпывающую информацию. Текст сообщения должен понятно объяснять суть и причины произошедшего. При проектировании собственных исключений рекомендуется добавлять свойства с дополнительным контекстом (например, как свойство `Shortage` в примере выше).

ЗАДАНИЕ

Добавьте в класс `Library` метод `AddFromConsole()`, который запрашивает у пользователя данные для создания книги, обрабатывает все возможные ошибки ввода и повторяет запрос при некорректных данных. Этот метод объединяет все изученные механизмы обработки исключений.

РЕШЕНИЕ:

```
// ===== Добавить в Library.Search.cs (или новый файл) =====
partial class Library
```

```

{
    /// <summary>
    /// Интерактивное добавление книги с полной обработкой ошибок
    /// </summary>
    public void AddBookFromConsole()
    {
        Console.WriteLine("\n--- Добавление новой книги ---");

        string title = ReadNonEmpty("Название: ");
        string author = ReadNonEmpty("Автор: ");
        int year = ReadInt("Год издания: ", 1450, DateTime.Now.Year);
        int pages = ReadInt("Страниц: ", 0, Book.MaxPageCount);
        string genre = ReadNonEmpty("Жанр: ");

        try
        {
            Book book = new(title, author, year, pages) { Genre = genre };
            Add(book);
            Console.WriteLine("Книга успешно добавлена!");
        }
        catch (LibraryException e)
        {
            Console.WriteLine($"Не удалось создать книгу: {e.Message}");
        }

        // Локальные функции для безопасного ввода
        static string ReadNonEmpty(string prompt)
        {
            while (true)
            {
                Console.Write(prompt);
                string? input = Console.ReadLine()?.Trim();
                if (!string.IsNullOrEmpty(input))
                    return input;
                Console.WriteLine(" Значение не может быть пустым, повторите
ввод.");
            }
        }

        static int ReadInt(string prompt, int min, int max)
        {
            while (true)
            {
                Console.Write(prompt);
                if (int.TryParse(Console.ReadLine(), out int value)
                    && value >= min && value <= max)
                    return value;
                Console.WriteLine($" Введите целое число от {min} до {max}.");
            }
        }
    }
}

// ===== Основная программа (этап 3) =====
Console.WriteLine("\n=== Этап 3: Интерактивное добавление ===");
lib.AddBookFromConsole();
lib.PrintAll();

```

10.2 Интерфейсы

10.2.1 Основы работы с интерфейсами

Абстрактный класс может содержать как абстрактные, так и обычные методы. Обычные методы содержат реализацию в виде кода языка C# и могут быть вызваны в наследуемом классе.

В традиционном понимании интерфейс является «предельным случаем» абстрактного класса: он не содержит реализации и определяет только сигнатуры методов и свойств, которые обязан реализовать класс-наследник. Начиная с версии C# 8, это правило было смягчено — интерфейсы получили возможность содержать реализации методов по умолчанию (default interface methods). Данная возможность рассматривается ниже.

В языке C# допустимо наследование от нескольких интерфейсов, но только от одного класса.

Наследование от нескольких классов может порождать ряд проблем, из которых наиболее известной является проблема «ромбовидного наследования». На рис. 34 приведена диаграмма наследования, представляющая собой ромб. В этом случае классы B и C наследуются от класса A, а класс D наследуется от классов B и C.

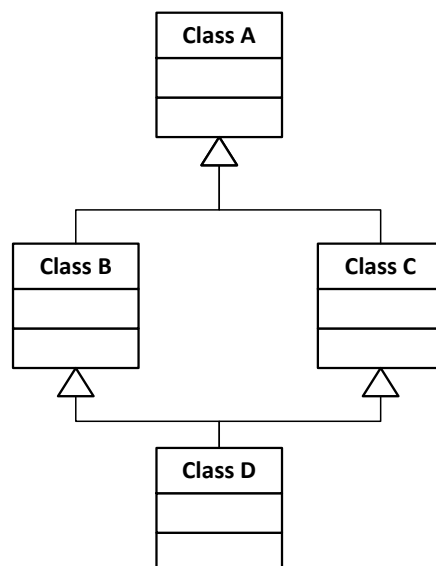


Рис. 34. Пример ромбовидного наследования.

При этом возникает вопрос о том, что делать с полями класса А. Они дублируются при наследовании в классах В и С и могут изменяться различным образом. Класс D может получить доступ к полям класса А, но к какой копии — к копии класса В или класса С?

Чтобы избежать подобных проблем, в языках Java и С# допустимо наследование только от одного класса. Почему же при этом возможно наследование от нескольких интерфейсов?

Наследование от интерфейсов принципиально отличается от наследования классов. При наследовании классов класс-наследник получает все реализованные методы от базового класса. При наследовании от интерфейсов класс-наследник обязуется реализовать методы, объявленные в интерфейсе. Поэтому в ряде объектно-ориентированных языков программирования интерфейсы называют контрактами: класс-наследник заключает контракт и берёт на себя обязательство реализовать нужную функциональность.

Следует отметить, что термин примесь (mixin), который также встречается применительно к интерфейсам, следует употреблять осторожно. В таких языках, как Ruby или Scala, примеси содержат готовую реализацию и подмешиваются к классу вместе с ней. Классические интерфейсы С# — это исключительно контракт без реализации. Начиная с С# 8 реализации по умолчанию сближают интерфейсы с примесями, однако это дополнительная возможность, а не основное назначение интерфейсов.

При проектировании больших программных систем интерфейсам отводится особая роль. Они рассматриваются не просто как средство для устранения ромбовидного наследования, но как высокоуровневое средство, позволяющее независимо проектировать отдельные подсистемы.

Производится декомпозиция системы на подсистемы, интерфейсы определяют контракты взаимодействия между ними — и после этого проектирование подсистем может вестись независимо.

Пример объявления интерфейса:

```
interface I1
{
    string I1_method();
}
```

В интерфейсе I1 объявлен метод I1_method, который не принимает параметров и возвращает строковое значение. Данный метод можно рассматривать как аналог абстрактного метода.

В традиционной форме члены интерфейса не имеют явного модификатора доступа — они неявно считаются public. Область видимости реализации определяется в классе-наследнике.

Имена интерфейсов в языке C# принято начинать с заглавной буквы I — такое соглашение закреплено в официальных рекомендациях Microsoft и позволяет в цепочке наследования сразу отличать интерфейсы от классов.

Интерфейсы могут наследоваться от интерфейсов. Пример наследования интерфейса от интерфейса:

```
interface I2 : I1
{
    string I2_method();
}
```

В этом случае интерфейс I2 содержит объявление метода I2_method, а также объявление метода I1_method, унаследованного от интерфейса I1.

10.2.2 Возможности интерфейсов в современном C#

Начиная с C# 8 интерфейс может содержать реализацию метода по умолчанию (default interface method). Это позволяет добавлять новые

методы в уже существующий интерфейс, не нарушая совместимость с классами, которые его реализуют:

```
interface IDescribable
{
    string GetTitle();

    // Метод по умолчанию – реализующий класс
    // может его не переопределять
    string GetDescription() => $"Объект: {GetTitle()}";
}

class Book : IDescribable
{
    public string Title { get; init; }
    public Book(string title) { Title = title; }

    // Обязательный метод – реализуется в классе
    public string GetTitle() => Title;

    // GetDescription() можно не реализовывать:
    // будет использована реализация интерфейса
}
```

Важно учитывать, что реализация по умолчанию доступна только через переменную интерфейсного типа:

```
Book b = new("Война и мир");
IDescribable d = b;

Console.WriteLine(b.GetTitle());           // Война и мир
// b.GetDescription() – ошибка компиляции:
// метод недоступен напрямую через Book
Console.WriteLine(d.GetDescription());     // Объект: Война и мир
```

10.2.3 Наследование классов от интерфейсов

Рассмотрим пример класса, который наследуется как от класса, так и от нескольких интерфейсов:

```
class ExtendedClass2 : ExtendedClass1, I1, I2
{
    // В конструкторе вызывается конструктор базового класса
    public ExtendedClass2(int pi, int pi2, int pi3)
        : base(pi, pi2, pi3) { }

    // Реализация методов, объявленных в интерфейсах
    public string I1_method() { return ToString(); }
```

```

    public string I2_method() { return ToString(); }
}

```

В языке C# для наследования как от классов, так и от интерфейсов используется двоеточие; базовый класс и интерфейсы перечисляются через запятую. В Java для наследования от классов используется ключевое слово `extends`, а для наследования от интерфейсов — `implements`. Аналогичный код на Java выглядел бы следующим образом:

```
class ExtendedClass2 extends ExtendedClass1 implements I1, I2
```

Обратите внимание: для реализации методов интерфейса не используются ключевые слова `virtual` и `override`. Реализация по умолчанию является неvirtуальной. Если потребуется разрешить её переопределение в классах-наследниках, метод следует явно пометить ключевым словом `virtual`.

10.2.4 Автоматическая генерация заглушек

Как и в случае абстрактных классов, Visual Studio позволяет автоматически генерировать заглушки для методов интерфейсов. При создании класса `ClassForI1`, наследуемого от интерфейса `I1`, достаточно нажать правую кнопку мыши на имени интерфейса — в контекстном меню появятся пункты для автоматической реализации интерфейса (рис. 11).

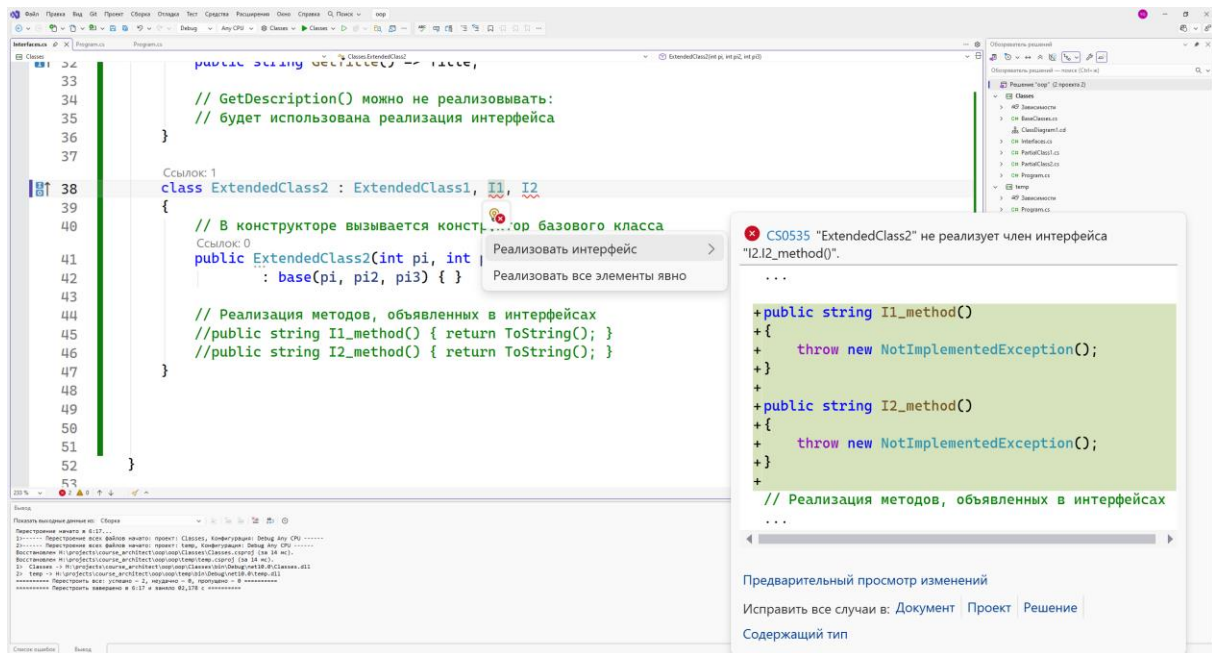


Рис. 35. Реализация интерфейса.

В отличие от абстрактного класса, подменю содержит два пункта: «Реализовать интерфейс» и «Реализовать интерфейс явно». При выборе обоих пунктов будет сгенерирован следующий код:

```
class ClassForI1 : I1
{
    // реализовать интерфейс
    public string I1_method()
    {
        throw new NotImplementedException();
    }

    // реализовать интерфейс явно
    string I1.I1_method()
    {
        throw new NotImplementedException();
    }
}
```

10.2.5 Неявная и явная реализация интерфейса

Возникает вопрос: чем неявная реализация отличается от явной и когда вызывается каждый из методов?

Сформулируем его в виде практического примера. Дана следующая реализация класса:

```
class ClassForI1 : I1
{
    // Неявная реализация
    public string I1_method()
    {
        return "1";
    }

    // Явная реализация
    string I1.I1_method()
    {
        return "2";
    }
}
```

Каким образом, используя данную реализацию, можно вывести в консоль число «12»?

```
ClassForI1 c1 = new ClassForI1();
string str1 = c1.I1_method(); // неявная реализация → "1"

I1 i1 = (I1)c1;
string str2 = i1.I1_method(); // явная реализация → "2"

Console.WriteLine(str1 + str2); // 12
```

Рассмотрим данный код подробнее:

1. Создаётся объект класса ClassForI1.
2. Через переменную типа ClassForI1 вызывается неявная реализация I1_method — метод возвращает "1".
3. Объект приводится к интерфейсному типу I1.
4. Через переменную интерфейсного типа вызывается явная реализация I1.I1_method — метод возвращает "2".

Таким образом, чтобы вызвать метод явной реализации интерфейса, необходимо привести объект к интерфейсному типу. Такой метод недоступен через переменную класса.

Явная реализация интерфейса применяется в следующих ситуациях:

- класс реализует два интерфейса с методами, имеющими одинаковые сигнатуры, но разные смысловые значения;
- разработчик хочет «скрыть» метод интерфейса от прямого вызова через класс, сделав его доступным только через интерфейсный тип;
- необходимо предоставить разные реализации одного метода — одну для открытого API класса и другую для интерфейсного контракта.

ЗАДАНИЕ

В рамках проекта «библиотечная система» создайте два интерфейса и реализуйте их в классах Book и Magazine.

```
// Элемент, поддерживающий поиск по ключевому слову
interface ISearchable
{
    bool ContainsKeyword(string keyword);
}

// Элемент, поддерживающий экспорт в различные форматы
interface IExportable
{
    string ToCsv();
    string ToJson();
}
```

Реализуйте интерфейсы в классах Book и Magazine:

- ContainsKeyword — возвращает true, если keyword встречается в названии, авторе (для книги) или издательстве (для журнала). Использовать StringComparison.OrdinalIgnoreCase.
- ToCsv — возвращает строку в формате CSV (поля через ;).
- ToJson — реализовать явно (через IExportable.ToJson()), чтобы метод был доступен только через интерфейсный тип. Вернуть упрощённое JSON-представление без использования сторонних библиотек.

В классе Library добавьте метод:

```
public List<LibraryItem> Search(string keyword)
```

Метод возвращает все элементы каталога, для которых ContainsKeyword вернул true. Для проверки использовать приведение к ISearchable через is.

В основной программе продемонстрируйте:

- Поиск по ключевому слову и вывод результатов.
- CSV-экспорт результатов поиска через интерфейс IExportable.
- JSON-экспорт через явную реализацию: показать, что прямой вызов book.ToJson() вызывает ошибку компиляции, а вызов через IExportable работает.

РЕШЕНИЕ:

```
// ===== Файл: ISearchable.cs =====

/// <summary>
/// Интерфейс поиска по ключевому слову
/// </summary>
interface ISearchable
{
    bool ContainsKeyword(string keyword);
}

// ===== Файл: IExportable.cs =====

/// <summary>
/// Интерфейс экспорта в различные форматы
/// </summary>
interface IExportable
{
    string ToCsv();
    string ToJson();
}

// ===== Обновление Book.cs =====

class Book : LibraryItem, ISearchable, IExportable
{
    // ... (существующий код без изменений) ...

    // — Реализация ISearchable —————

    /// <summary>
    /// Возвращает true, если keyword встречается в названии,
    /// авторе или жанре книги
    /// </summary>
    public bool ContainsKeyword(string keyword) =>
        Title.Contains(keyword, StringComparison.OrdinalIgnoreCase) ||
```

```

        Author.Contains(keyword, StringComparison.OrdinalIgnoreCase)
    ||
        Genre.Contains(keyword, StringComparison.OrdinalIgnoreCase);

    // — Реализация IExportable —————

    /// <summary>
    /// Неявная реализация – доступна через переменную Book
    /// </summary>
    public string ToCsv() =>
        $"Книга;{Title};{Author};{Year};{PageCount};{Genre}";

    /// <summary>
    /// Явная реализация – доступна только через IExportable
    /// </summary>
    string IExportable.ToJson() =>
        $"{{\"type\": \"book\", \" +
        $\"title\": \"{Title}\", \" +
        $\"author\": \"{Author}\", \" +
        $\"year\": {Year}}}\";
}

// ===== Обновление Magazine.cs =====

class Magazine : LibraryItem, ISearchable, IExportable
{
    // ... (существующий код без изменений) ...

    // — Реализация ISearchable —————

    public bool ContainsKeyword(string keyword) =>
        Title.Contains(keyword, StringComparison.OrdinalIgnoreCase) ||
        Publisher.Contains(keyword,
StringComparison.OrdinalIgnoreCase);

    // — Реализация IExportable —————

    public string ToCsv() =>
        $"Журнал;{Title};{Publisher};{Year};{IssueNumber};";

    string IExportable.ToJson() =>
        $"{{\"type\": \"magazine\", \" +
        $\"title\": \"{Title}\", \" +
        $\"issue\": {IssueNumber}, \" +
        $\"year\": {Year}}}\";
}

// ===== Добавить в Library.Search.cs =====

partial class Library
{
    /// <summary>
    /// Возвращает все элементы каталога, содержащие
    /// ключевое слово (используется интерфейс ISearchable)
    /// </summary>

```

```

/// <param name="keyword">Ключевое слово для поиска</param>
/// <returns>Список найденных элементов</returns>
public List<LibraryItem> Search(string keyword)
{
    List<LibraryItem> results = new();
    foreach (var item in _items)
    {
        // Приведение к интерфейсному типу через is
        if (item is ISearchable searchable &&
            searchable.ContainsKeyword(keyword))
        {
            results.Add(item);
        }
    }
    return results;
}

// ===== Основная программа =====

Library lib = new Library(LibraryUtils.LibraryName);

lib.Add(new Book("Война и мир", "Толстой", 1869, 1225) { Genre =
"Роман" });
lib.Add(new Book("Мастер и Маргарита", "Булгаков", 1967, 480) { Genre
= "Роман" });
lib.Add(new Book("1984", "Оруэлл", 1949, 328) { Genre = "Антиутопия"
});
lib.Add(new Magazine("Наука и жизнь", 2025, 3, "Наука"));
lib.Add(new Magazine("Вокруг света", 2024, 11, "Вокруг света"));

// 1. Поиск по ключевому слову
Console.WriteLine("=== Поиск по слову «роман» ===");
List<LibraryItem> found = lib.Search("роман");
foreach (var item in found)
    Console.WriteLine($" {item.GetInfo()}");

Console.WriteLine($"\\nНайдено элементов: {found.Count}");

// 2. CSV-экспорт результатов поиска через интерфейс IExportable
Console.WriteLine("\\n=== CSV-экспорт результатов поиска ===");
foreach (var item in found)
{
    if (item is IExportable exportable)
        Console.WriteLine(exportable.ToCsv());
}

// 3. Явная реализация – доступна только через IExportable
Console.WriteLine("\\n=== JSON-экспорт (явная реализация) ===");
Book book = new("Евгений Онегин", "Пушкин", 1833, 224) { Genre =
"Роман" };

// Следующая строка вызовет ошибку компиляции,
// так как ToJson() реализован явно:
// Console.WriteLine(book.ToJson()); // ← ошибка!

```

```
// Правильно — через интерфейсный тип:
IExportable exp = book;
Console.WriteLine(exp.ToJson());

// Также можно использовать приведение типа:
Console.WriteLine(((IExportable)book).ToJson());
```

10.3 Структуры (struct)

10.3.1 Структура как тип-значение

Ключевое отличие структуры от класса — способ размещения в памяти. Класс является ссылочным типом: объект класса создаётся в динамически распределяемой памяти (куче), а переменная хранит ссылку (указатель) на него. Структура является типом-значением: её данные хранятся непосредственно там, где объявлена переменная — в стеке, либо встроено внутри содержащего объекта или массива.

Синтаксис объявления структуры практически совпадает с синтаксисом класса — ключевое слово `class` заменяется на `struct`:

```
struct Point
{
    public double X;
    public double Y;

    public Point(double x, double y)
    {
        X = x;
        Y = y;
    }

    public double DistanceTo(Point other)
    {
        double dx = X - other.X;
        double dy = Y - other.Y;
        return Math.Sqrt(dx * dx + dy * dy);
    }

    public override string ToString() => $"({X}, {Y})";
}

Point p1 = new Point(1.0, 2.0);
```

```
Point p2 = new Point(4.0, 6.0);
Console.WriteLine(p1.DistanceTo(p2)); // 5
```

Поскольку структура является типом-значением, при присваивании и передаче в метод создаётся полная копия данных, а не копия ссылки:

```
Point a = new Point(1.0, 2.0);
Point b = a;           // создаётся копия данных

b.X = 99.0;

Console.WriteLine(a); // (1, 2) ← не изменился
Console.WriteLine(b); // (99, 2)
```

Для сравнения: в случае класса переменная `b` указывала бы на тот же объект, что и `a`, и изменение `b.X` затронуло бы оба.

10.3.2 Неизменяемые структуры (readonly struct)

Если структура используется как неизменяемый тип данных, рекомендуется объявить её с ключевым словом `readonly`. Компилятор гарантирует, что ни один метод структуры не изменяет её поля, и применяет дополнительные оптимизации при передаче таких структур: вместо полного копирования может использоваться передача по ссылке только для чтения (`in`-параметр):

```
readonly struct Vector2D
{
    public double X { get; }
    public double Y { get; }

    public Vector2D(double x, double y)
    {
        X = x;
        Y = y;
    }

    public double Length => Math.Sqrt(X * X + Y * Y);

    // Метод возвращает новую структуру, не изменяя текущую
    public Vector2D Add(Vector2D other) =>
        new Vector2D(X + other.X, Y + other.Y);

    public override string ToString() => $"({X}, {Y})";
}
```

10.3.3 Производительность: массив структур и массив классов

Главная область применения структур в современном С# — оптимизация производительности при работе с большими массивами однотипных данных. Рассмотрим разницу в организации памяти.

Массив объектов класса — это массив ссылок. Каждый элемент хранится отдельно в куче, и для доступа к нему необходим дополнительный переход по ссылке (разыменование указателя). При обходе такого массива процессор вынужден считывать данные из разных, потенциально далеко расположенных участков памяти, что приводит к промахам кэша:

Массив структур — это непрерывный блок памяти. Все данные расположены последовательно, без промежуточных ссылок. Процессор считывает их одним блоком в кэш, что существенно ускоряет обработку.

10.4 Перечисления

Перечисление (enumeration, enum) — это тип данных, объединяющий множество именованных целочисленных констант. Каждой константе присваивается целое числовое значение, а работа с ней в коде ведётся через имя, а не через число, что делает код более читаемым и защищённым от ошибок. Перечисление позволяет перечислить множество допустимых значений какого-либо свойства.

В языке С ключевое слово `enum` существовало начиная со стандарта С89, однако С-перечисления не являлись типобезопасными: значение перечисления можно было свободно смешивать с обычным целым числом без явного приведения типов. До появления `enum` в С программисты использовали директиву препроцессора `#define` для объявления именованных констант, нередко формируя «иерархические» имена вида `DIRECTION_NORTH`, `DIRECTION_SOUTH`. В языках С++, Java и С#

перечисления стали полноценными типами данных: компилятор строго контролирует совместимость типов при присваивании и сравнении.

```
/// <summary>
/// Перечисление сторон света
/// </summary>
public enum СтороныСвета
{
    Север, Юг, Запад, Восток
}
```

При таком объявлении именованным константам Север, Юг, Запад, Восток присваиваются целые значения, начиная с нуля.

Поскольку в языке C# используется кодировка Unicode, имена перечислений и их значений можно задавать на русском языке, как в данном примере. На практике, однако, для перечислений принято использовать английские идентификаторы — это облегчает совместную разработку и интеграцию с внешними системами.

Значения именованным константам можно присваивать явно (изм.: это полезно, например, когда числовые значения имеют внешнее значение — хранятся в базе данных или передаются по протоколу):

```
/// <summary>
/// Перечисление с явным присвоением значений
/// </summary>
public enum СтороныСвета2
{
    Север = 0,
    Юг = 1,
    Запад = 3,
    Восток = 4
}
```

По умолчанию базовым типом перечисления является int. При необходимости его можно изменить — это актуально для экономии памяти или при взаимодействии с внешними протоколами:

```
// Базовый тип byte занимает 1 байт вместо 4
public enum Direction : byte
{
    North = 0,
    South = 1,
```

```

    West = 2,
    East = 3
}

```

Допустимы типы: byte, sbyte, short, ushort, int, uint, long, ulong.

Использование перечислений облегчает понимание кода при проверке условий:

```
СтороныСвета temp1 = СтороныСвета.Восток;
```

```

if (temp1 == СтороныСвета.Восток)
{
    Console.WriteLine("Восток");
}

```

Естественным и современным способом ветвления по значению перечисления является switch expression (появился в C# 8):

```

string description = temp1 switch
{
    СтороныСвета.Север => "Движение на север",
    СтороныСвета.Юг => "Движение на юг",
    СтороныСвета.Запад => "Движение на запад",
    СтороныСвета.Восток => "Движение на восток",
    _ => "Неизвестное направление"
};
Console.WriteLine(description); // Движение на восток

```

Перечисления в языке C# позволяют получить значение по его строковому представлению. (изм.) Для этого предусмотрены два способа: Enum.Parse и безопасный Enum.TryParse<T>.

Метод Enum.Parse выполняет преобразование, но при неудаче генерирует исключение:

```

СтороныСвета temp2 =
    (СтороныСвета)Enum.Parse(typeof(СтороныСвета), "СЕВЕР", true);

```

Первый параметр — тип перечисления, второй — строка для преобразования, третий — флаг игнорирования регистра. В данном случае строка "СЕВЕР" не совпадает по регистру с Север, но преобразование выполнится успешно. В результате переменная temp2 будет содержать значение СтороныСвета.Север.

В современном коде предпочтительнее использовать обобщённый метод `Enum.TryParse<T>`, который не генерирует исключение при неудаче, а возвращает `false`:

```
// Успешное преобразование
if (Enum.TryParse<СтороныСвета>("СЕВЕР", ignoreCase: true,
                                out СтороныСвета result))
{
    Console.WriteLine($"Распознано: {result}"); // Распознано: Север
}

// Обработка некорректного ввода без исключения
if (!Enum.TryParse<СтороныСвета>("НеверноеЗначение",
                                out СтороныСвета unknown))
{
    Console.WriteLine("Не удалось распознать значение");
}
```

Особой разновидностью перечислений является перечисление-флаг. Оно применяется, когда одна переменная должна хранить несколько значений одновременно — например, набор разрешений или опций. Для этого значениям присваиваются степени двойки, что позволяет комбинировать их побитовыми операциями, а само перечисление помечается атрибутом `[Flags]`:

```
[Flags]
public enum SearchOptions
{
    None = 0,
    ByTitle = 1,           // 001 в двоичном
    ByAuthor = 2,          // 010 в двоичном
    ByGenre = 4,           // 100 в двоичном
    All = ByTitle | ByAuthor | ByGenre
}

// Комбинирование флагов оператором |
SearchOptions options = SearchOptions.ByTitle |
SearchOptions.ByAuthor;

// Проверка наличия флага методом HasFlag
if (options.HasFlag(SearchOptions.ByTitle))
    Console.WriteLine("Поиск по названию включён");

if (!options.HasFlag(SearchOptions.ByGenre))
    Console.WriteLine("Поиск по жанру исключён");

Console.WriteLine(options); // ByTitle, ByAuthor
```

ЗАДАНИЕ

В проекте «библиотечная система» добавьте несколько перечислений и используйте их для типизации данных вместо строк. Объявите следующие перечисления:

```
// Жанр книги
public enum BookGenre
{
    Unknown,
    Novel,           // Роман
    SciFi,           // Фантастика / Антиутопия
    Detective,       // Детектив
    History,         // История / Историческая литература
    Science          // Научная / Популярная наука
}

// Статус элемента библиотеки
public enum ItemStatus
{
    Available,       // Доступен
    OnLoan,          // Выдан читателю
    Reserved         // Зарезервирован
}

// Опции поиска (флаги – можно комбинировать)
[Flags]
public enum SearchOptions
{
    None = 0,
    ByTitle = 1,
    ByAuthor = 2,
    ByGenre = 4,
    All = ByTitle | ByAuthor | ByGenre
}
```

Выполните следующие изменения:

1. В классе Book замените свойство Genre типа string на свойство Genre типа BookGenre. Уберите required — вместо него задайте значение по умолчанию BookGenre.Unknown. Добавьте вычисляемое свойство GenreRu, возвращающее русское название жанра с помощью switch expression.
2. В оба класса Book и Magazine добавьте свойство Status типа ItemStatus со значением по умолчанию ItemStatus.Available.
3. В метод Search класса Library добавьте необязательный параметр SearchOptions options = SearchOptions.All и учитывайте его при

поиске: проверяйте заголовок только если установлен флаг ByTitle, автора — только если установлен ByAuthor, и т.д.

4. В основной программе переберите все значения BookGenre с помощью Enum.GetValues<BookGenre>() и выведите их в консоль. Продемонстрируйте поиск с разными комбинациями флагов SearchOptions.

РЕШЕНИЕ:

```
// ===== Файл: Enums.cs =====

/// <summary>Жанр книги</summary>
public enum BookGenre
{
    Unknown,
    Novel,
    SciFi,
    Detective,
    History,
    Science
}

/// <summary>Статус элемента каталога</summary>
public enum ItemStatus
{
    Available,
    OnLoan,
    Reserved
}

/// <summary>Опции поиска (флаги)</summary>
[Flags]
public enum SearchOptions
{
    None = 0,
    ByTitle = 1,
    ByAuthor = 2,
    ByGenre = 4,
    All = ByTitle | ByAuthor | ByGenre
}

// ===== Обновление Book.cs =====
// Заменить свойство Genre и добавить Status

// Было: public required string Genre { get; set; }
// Стало:
public BookGenre Genre { get; set; } = BookGenre.Unknown;

// Статус экземпляра
public ItemStatus Status { get; set; } = ItemStatus.Available;

// Вычисляемое свойство – русское название жанра
public string GenreRu => Genre switch
{
    BookGenre.Novel => "Роман",
    BookGenre.SciFi => "Фантастика",
    BookGenre.Detective => "Детектив",
    BookGenre.History => "История",
}
```

```

    BookGenre.Science => "Наука",
    _ => "Не указан"
};

// Обновить GetInfo():
public override string GetInfo() =>
    $"«{Title}» – {Author} ({Year}), {PageCount} стр." +
    $" [{GenreRu}] [{Status}]";

// ===== Обновление Magazine.cs =====

// Добавить статус
public ItemStatus Status { get; set; } = ItemStatus.Available;

public override string GetInfo() =>
    $"«{Title}» ({Year}), выпуск №{IssueNumber}, " +
    $" изд-во: {Publisher} [{Status}]";

// ===== Обновление Library.Search.cs =====

public List<LibraryItem> Search(string keyword,
    SearchOptions options = SearchOptions.All)
{
    List<LibraryItem> results = new();

    foreach (var item in _items)
    {
        bool match = false;

        // Поиск по названию
        if (options.HasFlag(SearchOptions.ByTitle) &&
            item.Title.Contains(keyword,
                StringComparison.OrdinalIgnoreCase))
            match = true;

        // Поиск по автору (только для книг)
        if (!match &&
            options.HasFlag(SearchOptions.ByAuthor) &&
            item is Book b &&
            b.Author.Contains(keyword,
                StringComparison.OrdinalIgnoreCase))
            match = true;

        // Поиск по жанру (только для книг)
        if (!match &&
            options.HasFlag(SearchOptions.ByGenre) &&
            item is Book bk)
        {
            // Попытка распознать строку как значение BookGenre
            if (Enum.TryParse<BookGenre>(keyword,
                ignoreCase: true, out BookGenre genre) &&
                bk.Genre == genre)
                match = true;
        }

        if (match) results.Add(item);
    }

    return results;
}

```

10.5 Перегрузка операторов

По возможностям перегрузки операторов язык C# занимает промежуточное положение между Java и C++.

В языке Java перегрузка операторов не используется. Предполагается, что программист на языке Java должен применять методы классов вместо перегруженных операторов.

Язык C++ обладает богатыми возможностями перегрузки операторов. Перегрузка операторов в C++ разделяется на внутреннюю (когда перегружаемый оператор объявляется как метод экземпляра класса) и внешнюю (когда перегружаемый оператор объявляется как статический метод класса).

В языке C# есть только аналог внешней перегрузки операторов C++.

Рассмотрим перегрузку операторов на примере класса Number.

Пример класса, в котором перегружены основные операторы:

```

/// <summary>
/// Пример класса для перегрузки операторов.
/// Реализует IEquatable<Number> и IComparable<Number>
/// для интеграции с коллекциями и LINQ.
/// </summary>
public class Number : IEquatable<Number>, IComparable<Number>
{
    /// <summary>
    /// Целое число
    /// </summary>
    public int Num { get; protected set; }

    /// <summary>
    /// Конструктор
    /// </summary>
    public Number(int param)
    {
        Num = param;
        Console.WriteLine($"Вызов конструктора для {param}"); //
(изм.) интерполяция
    }

    /// <summary>
    /// Приведение к строке
    /// </summary>
    public override string ToString() => Num.ToString(); // (изм.)
expression-bodied

```

```
// — Унарные операторы —————

/// <summary>
/// Перегрузка унарного оператора ++.
/// Префиксную и постфиксную формы компилятор различает
автоматически.
/// </summary>
public static Number operator ++(Number lhs)
{
    // Возвращаем НОВЫЙ объект – не изменяем переданный операнд.
    // Это правильная практика: компилятор сам присвоит результат
    // обратно в переменную при выполнении a++ или ++a.
    return new Number(lhs.Num + 1);
}

// — Бинарные операторы —————

/// <summary>
/// Перегрузка бинарного оператора для (Number + Number)
/// </summary>
public static Number operator +(Number lhs, Number rhs)
    => new Number(lhs.Num + rhs.Num); // (изм.) expression-bodied

/// <summary>
/// Перегрузка бинарного оператора для (Number + int).
/// Методы перегрузки операторов поддерживают полиморфизм по типу
операндов.
/// </summary>
public static Number operator +(Number lhs, int rhs)
    => new Number(lhs.Num + rhs);

/// <summary>
/// Контрпример: оператор * изменяет левый операнд вместо создания
нового объекта.
/// Это плохая практика – оператор неявно мутирует один из
операндов.
/// </summary>
public static Number operator *(Number lhs, int rhs)
{
    lhs.Num = lhs.Num * rhs; // изменяем переданный объект –
плохая практика
    return lhs;
}

// — Операторы равенства —————

public static bool operator ==(Number lhs, Number rhs)
    => lhs.Num == rhs.Num;

public static bool operator !=(Number lhs, Number rhs)
    => !(lhs == rhs);

// Реализация IEquatable<Number> –
// обеспечивает корректную работу с коллекциями и LINQ
/// <summary>
```



```

/// Типобезопасное сравнение (IEquatable)
/// </summary>
public bool Equals(Number? other)
    => other is not null && Num == other.Num;

public override bool Equals(object? obj)
    => obj is Number other && Equals(other);

public override int GetHashCode()
    => Num.GetHashCode();

// — Операторы сравнения —————

// Реализация IComparable<Number> –
// позволяет использовать OrderBy, Min, Max, Array.Sort и т.д.
/// <summary>
/// Сравнение объектов (IComparable)
/// </summary>
public int CompareTo(Number? other)
    => other is null ? 1 : Num.CompareTo(other.Num);

public static bool operator <(Number lhs, Number rhs)
    => lhs.CompareTo(rhs) < 0;

public static bool operator >(Number lhs, Number rhs)
    => lhs.CompareTo(rhs) > 0;

public static bool operator <=(Number lhs, Number rhs)
    => lhs.CompareTo(rhs) <= 0;

public static bool operator >=(Number lhs, Number rhs) // (изм.)
было => (ошибка)
    => lhs.CompareTo(rhs) >= 0;

// — Операторы приведения типов —————

/// <summary>
/// Явное приведение типа Number к типу int
/// </summary>
public static explicit operator int(Number lhs)
    => lhs.Num;

/// <summary>
/// Неявное приведение типа Number к типу double
/// </summary>
public static implicit operator double(Number lhs)
    => (double)lhs.Num;

// — Индексатор —————

/// <summary>
/// Пример индексатора
/// </summary>
public int this[int i]
{

```

```

        get => Num + i;
        set => Num = value + i;
    }
}

```

Класс содержит свойство Num, которое используется в качестве основы для перегруженных операторов, и конструктор, инициализирующий данное свойство. Далее следуют метод приведения к строке ToString и методы, соответствующие перегруженным операторам.

В большинстве перегруженных операторов для обозначения параметров применяют сокращения: lhs — left hand side (левый операнд (изм.)), rhs — right hand side (правый операнд (изм.)).

Как и в языке C++, для перегрузки операторов используется ключевое слово `operator`. Например, объявление:

```
public static Number operator ++(Number lhs)
```

означает, что данный метод реализует перегруженный оператор ++. Перегрузка операторов в C# может быть реализована только в виде статического метода.

Интересно то, что компилятор автоматически различает префиксную и постфиксную форму оператора ++. В языке C++ для этого используется специальный фиктивный параметр типа `int`. Аналогично реализуется перегрузка оператора --.

При перегрузке операторов в C# предполагается, что возвращаемое значение является новым объектом, а параметры-операнды не изменяются. Контрпример реализован при перегрузке оператора *: результат сохраняется в левый операнд, который затем возвращается. Это плохая практика — если левый операнд является ссылочным типом, то оператор неявно меняет один из операндов. Для наглядности оба подхода приведены в примере, однако использовать паттерн из `operator *` не рекомендуется.

Операторы сравнения принимают два операнда, и в этом смысле их перегрузка не отличается от рассмотренных примеров. Однако их необходимо перегружать попарно:

операторы `==` (равно) и `!=` (не равно);

операторы `>` (больше) и `<` (меньше);

операторы `>=` (больше или равно) и `<=` (меньше или равно).

Если один из пары перегружен, а другой нет, компилятор выдаёт ошибку.

Если перегружены операторы равенства и неравенства, необходимо также переопределить виртуальные методы `Equals` и `GetHashCode`. Без этого компилятор выдаёт предупреждения:

"Number" определяет `operator ==` или `operator !=`, но не переопределяет `Object.GetHashCode()`;

"Number" определяет `operator ==` или `operator !=`, но не переопределяет `Object.Equals(object o)`.

Помимо `Equals(object?)` рекомендуется также реализовывать обобщённый интерфейс `IEquatable<T>`. Это позволяет коллекциям и LINQ использовать типобезопасный метод `Equals(Number? other)` без упаковки (boxing) при работе с типами-значениями:

```
// IEquatable<T> — корректно используется в List.Contains, LINQ и т.д.
public bool Equals(Number? other) => other is not null && Num ==
other.Num;
```

```
// Equals(object?) — исправленный вариант с безопасной проверкой
типа
```

```
public override bool Equals(object? obj) => obj is Number other &&
Equals(other);
```

Аналогично, при перегрузке операторов `<`, `>`, `<=`, `>=` рекомендуется реализовывать интерфейс `IComparable<T>`. Его реализация позволяет

применять к объектам методы `Array.Sort`, `List.Sort`, LINQ-методы `OrderBy`, `Min`, `Max`:

```
csharp
// IComparable<T> — реализуется один раз, операторы сравнения
// делегируют к нему, что исключает дублирование логики
public int CompareTo(Number? other) =>
    other is null ? 1 : Num.CompareTo(other.Num);

public static bool operator <(Number lhs, Number rhs) =>
lhs.CompareTo(rhs) < 0;
public static bool operator >(Number lhs, Number rhs) =>
lhs.CompareTo(rhs) > 0;
// и т.д.
```

Операторы приведения типов переопределяются следующим образом:

```
csharp
// Явное приведение — требуется явный оператор (int)obj
public static explicit operator int(Number lhs) => lhs.Num;
```

// Неявное приведение — применяется автоматически в выражениях

```
public static implicit operator double(Number lhs) => (double)lhs.Num;
```

Вместо имени перегружаемого оператора указывается тип данных, к которому производится приведение. Ключевое слово `explicit` означает, что приведение должно быть указано явно в коде, `implicit` — что приведение может выполняться автоматически при вычислении выражений.

Особым случаем является перегрузка оператора квадратных скобок. В C# для этого предусмотрена отдельная конструкция — индексатор. Он напоминает свойство, однако принимает параметры, указываемые в квадратных скобках. Вместо имени свойства используется ключевое слово `this`:

```

public int this[int i]
{
    get => Num + i;
    set => Num = value + i;
}

```

Параметров может быть произвольное количество. Поскольку программисты интуитивно воспринимают квадратные скобки как обращение к коллекции, в качестве параметра обычно передают числовой или строковый ключ.

Форма индексатора позволяет определить сразу два аксессора — чтения и записи, что является наиболее привычным поведением при работе с коллекциями.

Пример использования перегруженных операторов:

```

Number a = new Number(1);

Console.WriteLine("\nУнарный оператор");
a++;
Console.WriteLine($"a={a}");
++a;
Console.WriteLine($"a={a}");

Console.WriteLine("\nБинарный оператор");
Number b = new Number(100);
Number c = a + b;
Console.WriteLine($"c={c}");

int i = 333;
Number d = a + i;
Console.WriteLine($"d={d}");

// Ошибка компиляции – не перегружен оператор (int + Number):
// Number d1 = i + d;

Console.WriteLine("\nКонтрпример – «внутренняя перегрузка» оператора
*");
Number mul1 = new Number(5);
Number mul2 = mul1 * 4;
Console.WriteLine($"mul1={mul1}"); // 20 – mul1 изменён! плохая
практика
Console.WriteLine($"mul2={mul2}"); // 20 – тот же объект

Console.WriteLine("\nПерегрузка операторов сравнения");
Number eq1 = new Number(3);

```

```

Number eq2 = new Number(2);
Console.WriteLine($"{eq1} == {eq2} ----> {eq1 == eq2}");
Console.WriteLine($"{eq1} != {eq2} ----> {eq1 != eq2}");
Console.WriteLine($"{eq1} > {eq2} ----> {eq1 > eq2}");
Console.WriteLine($"{eq1} < {eq2} ----> {eq1 < eq2}");
Console.WriteLine($"{eq1} >= {eq2} ----> {eq1 >= eq2}");
Console.WriteLine($"{eq1} <= {eq2} ----> {eq1 <= eq2}");

// (доб.) Демонстрация IComparable<T> через LINQ
var numbers = new List<Number> { new(5), new(2), new(8), new(1) };
var sorted = numbers.OrderBy(n => n).ToList();
Console.WriteLine("\nСортировка через IComparable<T>:");
foreach (var n in sorted)
    Console.Write($"{n} "); // 1 2 5 8
Console.WriteLine();

Console.WriteLine("\nОператоры приведения типов");
int intEq1 = (int)eq1; // явное
double doubleEq1 = eq1; // неявное
Console.WriteLine($"{eq1} ----> {intEq1}");
Console.WriteLine($"{eq1} ----> {doubleEq1}");

Console.WriteLine("\nИндексатор");
Number index = new Number(1);
index[330] = 3; // set: Num = 3 + 330 = 333
Console.WriteLine($"index[0] = {index[0]}"); // get: 333 + 0 = 333

```

10.6 Обобщения

Обобщения (generics) являются аналогом механизма языка C++ который называется шаблонами (templates).

Как и шаблоны, обобщения позволяют одинаковым способом производить одинаковые действия для различных типов данных.

Однако реализация обобщений в .NET отличается от реализации шаблонов в компиляторе языка C++. Если программа на языке C# содержит обобщённый класс, то он будет скомпилирован в MSIL с сохранением обобщённого типа. Подстановка реальных типов вместо обобщённого типа будет произведена уже на этапе JIT-компиляции. Таким образом, работу с обобщениями поддерживает не только компилятор, но и сама среда .NET runtime. Это принципиальное отличие от шаблонов C++, где для каждой подстановки типа на этапе компиляции создаётся отдельный экземпляр кода.

В языке Java существует механизм, аналогичный тому, что и в языке C#, который также называется обобщениями. Однако в Java обобщения реализованы так, что на этапе выполнения информация об обобщённом типе отсутствует, в отличие от .NET.

Рассмотрим работу с обобщениями в C#.

Пример обобщенного класса:

```
/// <summary>
/// Обобщенный класс
/// </summary>
/// <typeparam name="T">Обобщенный тип</typeparam>
internal class GenericClass1<T>
{
    private T i;

    /// <summary>
    /// Конструктор
    /// </summary>
    public GenericClass1()
    {
        this.i = default(T);
        // Короткая форма default (C# 7.1+)
        // this.i = default;
    }

    /// <summary>
    /// Метод инициализации значения
    /// </summary>
    public void SetValue(T param)
    {
        this.i = param;
    }

    /// <summary>
    /// Приведение к строке
    /// </summary>
    public override string ToString()
    {
        // Используем ?. – если i == null
        // вернётся пустая строка вместо NullReferenceException
        return i?.ToString() ?? string.Empty;
    }
}
```

Как и шаблоны в языке C++, обобщённые типы в языке C# указываются в треугольных скобках после имени класса. Обобщённые

типы принято обозначать одной прописной буквой T или именем, начинающимся с префикса T (например, TKey, TValue).

В данном примере обобщённым является тип T.

В классе `GenericClass1` объявлено поле `i` обобщённого типа T. Класс содержит конструктор без параметров, в котором полю `i` присваивается значение по умолчанию.

Для присваивания начальных значений переменным обобщённых типов в языке C# используется оператор `default`. Начиная с C# 7.1, тип в операторе `default` можно не указывать, если он выводится из контекста (в данном случае из типа поля `i`). Оператор возвращает значение по умолчанию в зависимости от того, какой тип будет подставлен в обобщённый тип: для числовых типов это 0, для `bool` — `false`, для ссылочных типов — `null`.

Метод `SetValue` присваивает значение полю `i`. Метод `ToString` возвращает строковое представление поля `i`.

Этот обобщенный класс фактически является контейнером для одного поля обобщенного типа. На практике использование такого контейнера не имеет большой необходимости, данный пример является учебным.

Пример создания объектов класса `GenericClass1` на основе типов `int` и `string`:

```
GenericClass1<int> g1 = new GenericClass1<int>();
g1.SetValue(333);
Console.WriteLine("Обобщенный класс 1: " + g1);

GenericClass1<string> g1Str = new GenericClass1<string>();
g1Str.SetValue("строка");
Console.WriteLine("Обобщенный класс 1: " + g1Str);
```

Здесь объект `g1` создан в результате подстановки в обобщенный тип типа `int`, а объект `g1Str` — в результате подстановки в обобщенный тип типа `string`.

В языке C# реализован механизм ограничений (constraints), позволяющий накладывать условия на обобщённый тип T (табл. 7).

Ограничения дают компилятору информацию о том, какие операции можно выполнять с обобщённым типом.

Таблица 7

Список ограничений, поддерживаемых обобщениями

Ограничение	Описание
where T: struct	Тип T должен быть типом-значением (не может быть nullable)
where T: class	Тип T должен быть ссылочным типом
where T : class?	Тип T должен быть ссылочным типом (допускает nullable, C# 8+)
where T: Class1	Тип T должен наследоваться от класса Class1
where T: Interface1	Тип T должен реализовывать интерфейс Interface1
where T1: T2	При подстановке реальных типов в обобщенные типы T1 и T2, тип подставленный в T1 должен наследоваться от типа, подставленного в T2
where T: new()	Тип T должен иметь конструктор без параметров
where T : Enum	Тип T должен быть перечислением (C# 7.3+)
where T : Delegate	Тип T должен быть делегатом (C# 7.3+)

При использовании нескольких ограничений они перечисляются через запятую, причём их порядок строго регламентирован: сначала идёт ограничение по типу (class, struct, имя базового класса), затем интерфейсы, и в конце — new(). Пример комбинации ограничений:

```
class Repository<T> where T : class, I1, new()
{
    public T Create() => new T();
}
```

Рассмотрим пример ограничения на основе реализации интерфейса.

Создадим интерфейс:

```
interface I1
{
    string I1_method();
}
```

Создадим обобщенный класс, содержащий ограничение:

```
class GenericClass2<T> where T : I1
{
    private T i;

    //Конструктор
    public GenericClass2(T param) { this.i = param; }

    //Приведение к строке
    public override string ToString()
    {
        return i.I1_method();
    }
}
```

Ограничение задается непосредственно при объявлении класса:

```
class GenericClass2<T> where T : I1
```

Для того чтобы создать объект класса GenericClass2 необходимо сначала создать класс, реализующий интерфейс I1:

```
class I1_class : I1
{
    string str;

    public I1_class(string param)
    {
        this.str = param;
    }

    public string I1_method() { return this.str; }
}
```

Теперь можно создать объект класса GenericClass2:

```
GenericClass2<I1_class> g2 =
    new GenericClass2<I1_class>(
        new I1_class("Обобщенный класс с ограничением"));
```

Кроме обобщенных классов в языке C# можно создавать обобщенные методы.

Пример обобщенного метода с ограничением:

```
static void GenericMethod1<T>(T param) where T : I1
{
    Console.WriteLine(param.I1_method());
}
```

Пример вызова обобщенного метода:

```
GenericMethod1<I1_class>(new I1_class("Вызов обобщенного метода"));
```

Обобщенные классы и методы могут содержать несколько обобщенных типов.

Пример использования нескольких обобщенных типов:

```
static void GenericMethod2<T, Q>(T param1, Q param2)
{
    Console.WriteLine("Обобщенный метод с двумя типами");
}
```

Пример вызова метода с несколькими обобщенными типами:

```
GenericMethod2<I1_class, int>(new I1_class("строка"), 333);
```

Обобщения являются важным и часто используемым механизмом в программах на C#. Наряду с обобщенными классами и методами можно создавать другие обобщенные структуры, например интерфейсы и делегаты.

ЗАДАНИЕ

Продолжим разработку системы библиотечного каталога. В этом разделе мы создадим обобщённый каталог, способный хранить любые библиотечные единицы (книги, журналы, диски и т.п.), а также обобщённые методы для поиска и обработки.

1. Создайте интерфейс `ILibraryItem` с методами:
 - `string GetInfo()` — информация о единице;
 - `int GetYear()` — год издания/выпуска.
2. Модифицируйте класс `Book` так, чтобы он реализовывал интерфейс `ILibraryItem`.
3. Создайте класс `Magazine` (журнал) со свойствами `Title`, `IssueNumber` (номер выпуска), `Year`. Класс также должен реализовывать `ILibraryItem`.

4. Создайте обобщённый класс `Catalog<T>` с ограничениями `where T : ILibraryItem, new()` и следующими элементами:
 - закрытое поле — список единиц (`List<T>`);
 - свойство `Count` (только для чтения);
 - метод `Add(T item)` — добавить единицу;
 - метод `CreateEmpty()` — создать и добавить пустую единицу через `new T()` (демонстрация ограничения `new()`);
 - метод `PrintAll()` — вывести информацию обо всех единицах;
 - обобщённый метод `FindOlderThan<U>(int years)` с ограничением `where U : T`, возвращающий список единиц подтипа `U`, которые старше указанного количества лет.
5. Создайте статический класс `CatalogUtils` с обобщённым методом `PrintInfo<T>(T item)` `where T : ILibraryItem`, демонстрирующим вывод типов у обобщённого метода.
6. В основной программе:
 - создайте `Catalog<Book>` и `Catalog<Magazine>`;
 - добавьте несколько единиц в каждый каталог;
 - выведите их содержимое;
 - используйте `FindOlderThan` для поиска старых книг;
 - вызовите `CatalogUtils.PrintInfo` без явного указания типа (вывод типа).

РЕШЕНИЕ:

```
// ===== Файл: ILibraryItem.cs =====
interface ILibraryItem
{
    string GetInfo();
    int GetYear();
}
// ===== Файл: Book.cs (дополнение из предыдущих разделов) =====
class Book : ILibraryItem
{
    public string Title { get; init; }
    public string Author { get; init; }
    private int _year;
    public int Year
    {
        get => _year;
        set
```

```

    {
        if (value < 1450 || value > DateTime.Now.Year)
            Console.WriteLine($"Предупреждение: некорректный год {value}");
        else
            _year = value;
    }
}

public int PageCount { get; set; }
public required string Genre { get; set; }
public int AgeInYears => DateTime.Now.Year - Year;
// Конструктор без параметров нужен для ограничения new()
// Genre помечено required, поэтому инициализируем его здесь
public Book()
{
    Title = "Без названия";
    Author = "Неизвестен";
    Genre = "Не указан";
    Year = DateTime.Now.Year;
}

[SetsRequiredMembers]
public Book(string title, string author, int year, int pageCount)
{
    Title = title;
    Author = author;
    Year = year;
    PageCount = pageCount;
    Genre = "Не указан";
}

// Реализация интерфейса
public string GetInfo() =>
    $"Книга: «{Title}» – {Author} ({Year}), {PageCount} стр. [{Genre}]";
public int GetYear() => Year;
}

// ===== Файл: Magazine.cs =====
class Magazine : ILibraryItem
{
    public string Title { get; init; } = "Без названия";
    public int IssueNumber { get; init; }
    public int Year { get; init; }
    // Конструктор без параметров (требуется для new())
    public Magazine() { }
    public Magazine(string title, int issueNumber, int year)
    {
        Title = title;
        IssueNumber = issueNumber;
        Year = year;
    }
    public string GetInfo() =>
        $"Журнал: «{Title}», выпуск №{IssueNumber} ({Year})";
    public int GetYear() => Year;
}

// ===== Файл: Catalog.cs =====
class Catalog<T> where T : ILibraryItem, new()
{
    private readonly List<T> items = new();
    public int Count => items.Count;
    public void Add(T item) => items.Add(item);
    // Демонстрация ограничения new()
    public T CreateEmpty()
    {
        T item = new T();
        items.Add(item);
        return item;
    }
}

public void PrintAll()
{
    foreach (var item in items)

```

```

        Console.WriteLine(item.GetInfo());
    }
    // Обобщённый метод внутри обобщённого класса
    // Ограничение where U : T означает, что U должен наследовать от T
    public List<U> FindOlderThan<U>(int years) where U : T
    {
        var result = new List<U>();
        int currentYear = DateTime.Now.Year;
        foreach (var item in items)
        {
            if (item is U u && (currentYear - item.GetYear()) > years)
                result.Add(u);
        }
        return result;
    }
}
// ===== Файл: CatalogUtils.cs =====
static class CatalogUtils
{
    // Обобщённый метод с ограничением
    public static void PrintInfo<T>(T item) where T : ILibraryItem
    {
        Console.WriteLine($"{typeof(T).Name} {item.GetInfo()}");
    }
}
// ===== Основная программа =====
// Каталог книг
var bookCatalog = new Catalog<Book>();
bookCatalog.Add(new Book("Война и мир", "Л. Толстой", 1869, 1225));
bookCatalog.Add(new Book("1984", "Дж. Оруэлл", 1949, 328));
bookCatalog.Add(new Book("Мастер и Маргарита", "М. Булгаков", 1967, 480));
Console.WriteLine($"Книг в каталоге: {bookCatalog.Count}");
bookCatalog.PrintAll();
Console.WriteLine();
// Каталог журналов
var magazineCatalog = new Catalog<Magazine>();
magazineCatalog.Add(new Magazine("Наука и жизнь", 5, 2024));
magazineCatalog.Add(new Magazine("Вокруг света", 12, 2023));
Console.WriteLine($"Журналов в каталоге: {magazineCatalog.Count}");
magazineCatalog.PrintAll();
Console.WriteLine();
// Демонстрация ограничения new()
Book emptyBook = bookCatalog.CreateEmpty();
Console.WriteLine($"Создана пустая книга: {emptyBook.GetInfo()}");
Console.WriteLine();
// Поиск старых книг (старше 100 лет)
List<Book> oldBooks = bookCatalog.FindOlderThan<Book>(100);
Console.WriteLine($"Книг старше 100 лет: {oldBooks.Count}");
foreach (var b in oldBooks)
    Console.WriteLine($" - {b.GetInfo()}");
Console.WriteLine();
// Демонстрация вывода типов в обобщённом методе
// Тип T выводится автоматически
CatalogUtils.PrintInfo(new Book("Евгений Онегин", "А. Пушкин", 1833, 224));
CatalogUtils.PrintInfo(new Magazine("Техника — молодёжи", 3, 2024));

```

10.7 Делегаты

Концепция делегата может показаться несколько незнакомой программисту на языке C++. Пожалуй, наиболее близким аналогом в C и C++ являются указатели на функцию.

Однако эта концепция будет понятна программисту на языке Паскаль. Это практически точная копия процедурных типов языка Паскаль.

Преимущество процедурных типов Паскаля и делегатов языка C# перед указателями на функции языка C++ состоит в том, что корректность процедурных типов и делегатов проверяется компилятором, который, в частности, способен выявить ошибки, связанные с несоответствием параметров. В языке C++ при работе с указателем на функцию подобные проверки не выполняются, что создает возможность для возникновения ошибок.

Если кратко описывать сущность делегата, то он является аналогом типа данных для методов. Класс является типом данных, экземпляром класса является объект, содержащий конкретные данные. Делегат является типом методов, экземпляром делегата является метод, соответствующий делегату.

Пример объявления делегата:

```
delegate int PlusOrMinus(int p1, int p2);
```

Данное описание очень напоминает описание метода (без реализации). В его начале ставится ключевое слово `delegate`, далее указывается тип возвращаемого значения, затем имя делегата, потом в круглых скобках указываются входные параметры. Имена входных параметров могут быть произвольными, и при работе с делегатами они не используются.

Пример методов, соответствующих рассмотренному делегату:

```
class Program
{
    static int Plus(int p1, int p2)
```

```

{
    return p1 + p2;
}

static int Minus(int p1, int p2)
{
    return p1 - p2;
}
}

```

Методы `Plus` и `Minus` соответствуют делегату `PlusOrMinus`, потому что их сигнатуры совпадают с сигнатурой делегата. То есть они имеют набор параметров тех же типов, и тот же тип возвращаемого значения, которые объявлены в делегате. Для входных параметров важно, чтобы именно их типы совпадали с типами входных параметров делегата. Имена входных параметров делегата могут быть другими, но это не имеет значения при проверке.

Однако при создании объектов классов с помощью ключевого слова `new` программист явно указывает, экземпляр какого класса он создает, то есть при создании объектов явно указывается имя класса.

При объявлении методов `Plus` и `Minus` не задается никаких ссылок на делегат `PlusOrMinus`. Как же компилятор определяет, что методы соответствуют делегату? Это делается с помощью так называемой неявной типизации.

Компилятор сравнивает сигнатуры (типы входных параметров и тип возвращаемого значения) метода и делегата. Если они совпадают, то считается, что метод соответствует делегату.

В некоторых языках программирования, например в `Ruby`, неявная типизация используется также для классов и объектов. Ее иногда называют «утиной» в соответствии с высказыванием, именуемым «утиным тестом»: «если это выглядит как утка, плавает как утка, и крякает как утка, то вероятно это утка». То есть если экземпляр имеет те же признаки что и тип, то он подобен этому типу и может считаться экземпляром этого типа.

Следующий вопрос состоит в том, где можно применять делегаты. Особенность языка C# состоит в том, что делегатный тип может быть использован в качестве параметра метода.

Пример объявления метода с делегатным параметром:

```
static void PlusOrMinusMethod(
    string str,
    int i1,
    int i2,
    PlusOrMinus PlusOrMinusParam)
{
    int Result = PlusOrMinusParam(i1, i2);
    Console.WriteLine(str + Result.ToString());
}
```

У метода `PlusOrMinusMethod` есть несколько интересных особенностей. Его последний параметр – это параметр делегатного типа. В теле метода имя этого параметра используется как функция, производится ее вызов:

```
int Result = PlusOrMinusParam(i1, i2);
```

Однако метода `PlusOrMinusParam` в программе нет. Реально будет вызван метод, переданный в качестве параметра в функцию `PlusOrMinusMethod`.

Таким образом, параметр делегатного типа вызывается как еще не известный метод, который в будущем будет передан в качестве параметра.

Сигнатура вызова `PlusOrMinusParam` соответствует сигнатуре делегата: передаются два входных параметра целого типа, возвращается значение целого типа.

Пример вызова метода с делегатным параметром:

```
int i1 = 3;
int i2 = 2;
PlusOrMinusMethod("Плюс: ", i1, i2, Plus);
PlusOrMinusMethod("Минус: ", i1, i2, Minus);
```

Результат вывода в консоль:

```
Плюс: 5
Минус: 1
```

В данном примере в качестве последнего параметра `PlusOrMinusMethod` передаются имена функций `Plus` и `Minus`.

Рассмотрим способы создания переменных делегатного типа. С использованием конструктора делегатного типа:

```
PlusOrMinus pm1 = new PlusOrMinus(Plus);
```

Или в сокращенной форме, которую называют «предположением делегата»:

```
PlusOrMinus pm2 = Plus;
```

Создание анонимного метода на основе делегата:

```
PlusOrMinus pm3 = delegate(int param1, int param2)
{
    return param1 + param2;
};
```

Создание анонимных методов применялось в языке C# до версии 3.0, хотя рассмотренный код будет корректно компилироваться и в текущих версиях. Но в языке C# версии 3.0 для работы с делегатными типами появилось новое средство – лямбда-выражения.

10.8 Лямбда-выражения

Лямбда-выражения или лямбда-функции пришли в язык C# из функционального программирования. Потребность в них появилась в связи с реализацией технологии LINQ.

Если описывать суть лямбда-выражений кратко, то это «одноразовые функции». Казалось бы, что такое определение противоречиво. Ведь в любом классическом учебнике информатики можно прочитать, что методы (процедуры, функции) пишутся один раз и многократно вызываются по имени, что обеспечивает повторное использование кода. Зачем же могут потребоваться одноразовые функции? И почему функцию нельзя объявить традиционным способом и вызывать только один раз, а если потребуется, то и большее количество раз?

Традиционный метод обладает двумя важными свойствами:

- именованностью – имеет имя, по которому метод может быть вызван;
- сигнатурностью – имеет сигнатуру, то есть набор входных параметров с указанием их типов и тип возвращаемого значения.

Лямбда-выражение обладает свойством сигнатурности, но у него нет свойства именованности. То есть у лямбда-выражения есть набор входных параметров и тип возвращаемого значения, но нет имени. Оно записывается в том месте, где должно вызываться.

Пример объявления лямбда-выражения (лямбда-выражение выделено подчеркиванием):

```
PlusOrMinus pm4 = (int x, int y) =>
{
    int z = x + y;
    return z;
};
```

Лямбда-выражение напоминает по объявлению обыкновенный метод. Объявление начинается со списка входных параметров, которые записываются в скобках. Имя метода в случае лямбда-выражения не указывается. Тип возвращаемого значения автоматически определяется компилятором, используется так называемый механизм вывода типов (англ. type inference).

После скобок указывается стрелка «`=>`», которая отделяет список параметров от тела лямбда-выражения. Стрелка является характерным синтаксическим элементом, по которому всегда можно определить лямбда-выражение. Необходимо отметить, что оператор сравнения «больше-или-равно» записывается как «`>=`» и его синтаксис не совпадает с синтаксисом лямбда-выражения.

За стрелкой записывается тело метода, которое в данном примере ничем не отличается от тела обычного метода.

После такого объявления лямбда-выражение может быть вызвано как метод через определенную переменную pm4:

```
int test = pm4(1, 2);
```

Однако чаще всего лямбда-выражение передают в качестве параметра делегатного типа.

Пример передачи лямбда-выражения в качестве параметра (лямбда-выражение выделено подчеркиванием):

```
PlusOrMinusMethod(
    "Создание экземпляра делегата на основе лямбда-выражения 1: ",
    i1,
    i2,
    (int x, int y) =>
    {
        int z = x + y;
        return z;
    }
);
```

В примере лямбда-выражение передается в качестве параметра делегатного типа PlusOrMinus.

Соответствие сигнатуры делегата и сигнатуры лямбда-выражения проверяется путем неявной типизации.

Например, зададим в лямбда-выражении третий входной параметр, что нарушит соответствие между сигнатурой лямбда-выражения и сигнатурой делегата (третий параметр подчеркнут):

```
PlusOrMinusMethod(
    "Создание экземпляра делегата на основе лямбда-выражения 1: ",
    i1,
    i2,
    (int x, int y, int k) =>
    {
        int z = x + y ;
        return z;
    }
);
```

При этом компилятор выдаст в строке определения лямбда-выражения следующую ошибку: «Делегат "Delegates.PlusOrMinus" не принимает "3" аргументов».

Язык C# предоставляет возможность упростить синтаксис лямбда-выражений. Например, вместо исходного лямбда-выражения:

```
// Statement lambda
(int x, int y) =>
{
    int z = x + y;
    return z;
}
```

можно записать:

```
(x, y) =>
{
    return x + y;
}
```

В данном случае у входных параметров не указываются типы, компилятор определяет их автоматически с помощью механизма вывода типов. Возвращаемое значение вычисляется в операторе return. В том случае, когда результат лямбда-функции может быть задан в виде единого выражения, синтаксис можно еще более упростить:

```
(x, y) => x + y //Expression lambda
```

Тогда компилятор автоматически определяет, что выражение $x + y$ является возвращаемым значением лямбда-функции.

Еще одна интересная особенность лямбда-выражений – возможность использования в них внешних переменных. Такая особенность в языках программирования называется замыканием (closure). Суть замыкания состоит в том, что в каком-то блоке кода (чаще всего это обычная функция или лямбда-выражение) используются ссылки на переменные, которые объявлены снаружи этого блока кода, при этом такие внешние переменные не передаются явным образом в блок кода как параметры. То есть блок кода непосредственно «видит» переменные из внешнего контекста.

Пример замыкания на основе лямбда-выражения:

```
int outer = 100;
PlusOrMinus pm5 = (int x, int y) =>
{
    int z = x + y + outer;
    return z;
}
```

```
};
```

Переменная `outer`, объявленная на уровне внешней функции, видна в лямбда-выражении, и она может быть использована в вычислениях.

Таким образом, лямбда-выражение можно рассматривать как фрагмент кода, которым прикладной программист может дополнить функциональность какого-либо библиотечного метода. При этом данный код не пишется произвольно, а имеет интерфейс с библиотечным методом в виде соответствующего делегатного типа.

10.9 Обобщенные делегаты *Func* и *Action* и (устаревший) *Predicate*

Появление лямбда-выражения в языке C# облегчило создание параметров делегатного типа. Однако сами делегаты при этом по-прежнему необходимо создавать отдельно. Нельзя ли облегчить решение и этой задачи? Это сделано в версии 3.5 языка C# с использованием обобщенных делегатов `Func` и `Action`.

Ранее был объявлен обычный делегат:

```
delegate int PlusOrMinus(int p1, int p2);
```

Ему соответствует такое объявление делегата `Func`:

```
Func<int, int, int>
```

Рассмотренный ранее метод:

```
static void PlusOrMinusMethod(
    string str,
    int i1,
    int i2,
    PlusOrMinus PlusOrMinusParam)
{
    int Result = PlusOrMinusParam(i1, i2);
    Console.WriteLine(str + Result.ToString());
}
```

может быть переписан с использованием обобщенного делегата Func следующим образом (параметры делегатного типа подчеркнуты в примерах):

```
static void PlusOrMinusMethodFunc(
    string str,
    int i1,
    int i2,
    Func<int, int, int> PlusOrMinusParam)
{
    int Result = PlusOrMinusParam(i1, i2);
    Console.WriteLine(str + Result.ToString());
}
```

Вызов делегатного параметра в теле обоих методов производится совершенно одинаково: параметр вызывается как функция, которой передаются два целочисленных параметра и возвращается целочисленный параметр.

В качестве значения параметра, соответствующего обобщенному делегату, может быть использовано имя обычного метода (в примере применяется ранее рассмотренный метод Plus):

```
PlusOrMinusMethodFunc(
    "Создание экземпляра делегата на основе метода: ",
    i1,
    i2,
    Plus
);
```

Однако чаще для этих целей используется лямбда-выражение (подчеркнуто в примере):

```
PlusOrMinusMethodFunc(
    "Создание экземпляра делегата на основе лямбда-выражения 3:",
    i1,
    i2,
    (x, y) => x + y
);
```

Таким образом, применение обычного делегата и обобщенного делегата Func позволяет получить одинаковые результаты. Однако преимущество использования Func состоит в том, что его, в отличие от обычного делегата, не нужно объявлять в тексте программы, он уже объявлен в библиотечных классах языка C#.

Если нажать правую кнопку мыши на ключевом слове Func и выбрать в контекстном меню пункт «перейти к определению», то Visual Studio осуществит переход к следующему определению, объявленному в пространстве имен System:

```
public delegate TResult Func<in T1, in T2, out TResult>(T1 arg1, T2 arg2);
```

В языке C# делегаты могут быть обобщенными, и Func – имя обобщенного делегата. При объявлении обобщенного делегата обобщенные типы в треугольных скобках задаются после его имени. В данном случае используются три обобщенных типа: два первых типа – это типы входных параметров, а последний – тип возвращаемого значения.

Таким образом, в объявлении Func<int, int, int> последний параметр задает тип возвращаемого значения, а предыдущие – задают входные типы. Общее правило при использовании обобщенного делегата Func – последний параметр определяет тип выходного значения.

Если перейти к определению аналогичного делегата Action

```
Action<int, int, int>
```

то оно будет следующим:

```
public delegate void Action<in T1, in T2, in T3>(T1 arg1, T2 arg2, T3 arg3);
```

В отличие от делегата Func, у которого последний обобщенный параметр задает тип выходного значения, все обобщенные параметры делегата Action задают типы входных параметров. Обобщенный делегат Action возвращает значение типа void.

По сравнению с разбиравшимися выше примерами обобщенных классов и методов у данных обобщенных делегатов есть еще одна

особенность – перед обобщенными типами указаны ключевые слова `in` и `out`.

Данная особенность обобщенных делегатов, появившаяся в версии 4.0 языка *C#*, и называется ковариантностью и контравариантностью. Кроме обобщенных делегатов эта особенность может также применяться в обобщенных интерфейсах.

Обобщенный делегат или интерфейс ковариантен, если обобщенный тип аннотирован ключевым словом `out`. Обобщенный тип `T` ковариантен, если для подставленного в него конкретного типа `t`, в параметр типа `T` может быть передано значение типа `t` или любого типа, производного от `t`, то есть объект более конкретного типа, чем изначально заданный. Использование ключевого слова `out` также означает, что тип `T` разрешен только в качестве типа возвращаемого значения.

Обобщенный делегат или интерфейс контравариантен, если обобщенный тип аннотирован ключевым словом `in`. Обобщенный тип `T` контравариантен, если для подставленного в него конкретного типа `t`, в параметр типа `T` может быть передано значение типа `t` или любого типа, базового для `t`, то есть объект более общего типа, чем изначально заданный. Использование ключевого слова `in` также означает, что тип `T` разрешен лишь в качестве типа входного параметра.

Если ключевые слова `in` или `out` не заданы, то обобщенный тип `T` инвариантен, то есть для подставленного в него конкретного типа `t`, в параметр типа `T` может быть передано значение только типа `t` и никакого другого типа.

Помимо `Func` и `Action`, в пространстве имён `System` объявлен ещё один стандартный обобщённый делегат — `Predicate<T>`. Он предназначен для представления функции проверки условия: принимает один параметр и возвращает `bool`.

Определение делегата в библиотеке `.NET`:

```
public delegate bool Predicate<in T>(T obj);
```

Обратите внимание, что параметр `T` помечен как `in` — делегат `Predicate<T>` является контравариантным.

Функционально `Predicate<T>` полностью эквивалентен `Func<T, bool>`:

```
// Две строки определяют, по сути, одну и ту же функциональность
Predicate<Book> isOldBook1 = b => b.Year < 1900;
Func<Book, bool> isOldBook2 = b => b.Year < 1900;
```

В данном примере оба делегата принимают один параметр типа `T` и возвращают `bool`. Лямбда-выражение может быть присвоено любому из них.

Несмотря на идентичную сигнатуру, `Predicate<T>` и `Func<T, bool>` являются разными типами делегатов и не могут быть присвоены друг другу напрямую.

`Predicate<T>` в настоящее время считается устаревшим, в современных библиотеках (в частности в LINQ) вместо него используется `Func<T, bool>`.

10.10 Групповые делегаты

Если делегат имеет тип возвращаемого значения `void`, то на его основе может быть создан групповой делегат, которому соответствуют сразу несколько методов. В частности, этому условию удовлетворяет обобщенный делегат `Action`.

Рассмотрим пример использования групповых делегатов. Определим методы в виде лямбда-выражений, соответствующих делегату `Action<int, int>`, то есть принимающих два входных параметра типа `int` и тип возвращаемого значения `void`:

```
Action<int, int> a1 =
    (x, y) =>
        { Console.WriteLine("{0} + {1} = {2}", x, y, x + y); };

Action<int, int> a2 =
    (x, y) =>
        { Console.WriteLine("{0} - {1} = {2}", x, y, x - y); };
```

Теперь на основе данных методов можно определить групповой делегат:

```
Action<int, int> group = a1 + a2;
```

Если вызвать данный групповой делегат:

```
group(5, 3);
```

то будут вызваны методы a1 и a2. В консоль будет выведено:

```
5 + 3 = 8
```

```
5 - 3 = 2
```

Необходимо учитывать, что очередность вызовов метода в групповом делегате не гарантируется. То есть разработчик может быть уверен, что оба метода a1 и a2 будут вызваны, но они могут быть вызваны в произвольном порядке.

Для добавления вызова метода к групповому делегату может быть использован оператор «+=», а для удаления вызова метода из группового делегата оператор «-=».

Пример добавления и удаления вызова методов для группового делегата:

```
Action<int, int> group2 = a1;
Console.WriteLine("Добавление вызова метода к групповому делегату");
group2 += a2;
group2(10, 5);

Console.WriteLine("Удаление вызова метода из группового делегата");
group2 -= a1;
group2(20, 10);
```

Результат вывода в консоль:

Добавление вызова метода к групповому делегату

```
10 + 5 = 15
```

```
10 - 5 = 5
```

Удаление вызова метода из группового делегата

```
20 - 10 = 10
```

Групповые делегаты являются основой для реализации механизма событий.

10.11 События

События в языке C# реализуют механизм подписки/публикации. Любой класс может объявить события и вызывать их в необходимые моменты времени. Внешние классы могут прикрепить методы в качестве обработчиков событий, иначе говоря «подписаться на событие».

События основаны на обобщенных делегатах, следовательно, обработчики событий должны соответствовать сигнатуре обобщенного делегата для события.

В языках C++ и Java существует большое количество библиотек, реализующих подход подписки/публикации, однако они реализованы в виде внешних библиотек, а не в виде конструкции, встроенной в язык программирования.

Особенность событий языка C# заключается в том, что они соответствуют механизму событий .NET-фреймворка. Например, событие нажатия на кнопку в технологии Windows Forms является событием языка C#, создание обработчика события нажатия на кнопку происходит с использованием стандартного механизма событий в языке C#.

Объявим делегат, на котором основано событие:

```
public delegate void NewEventDelegate(string str);
```

Делегат принимает один строковый параметр. Делегат, на котором основано событие, должен возвращать значение void.

Событие объявлено в классе консольного приложения Program:

```
public static event NewEventDelegate NewEvent;
```

Событие объявляется как обычный элемент класса, NewEvent фактически является специализированной переменной.

Для события указана область видимости (public). Объявление static означает, что данное поле статическое, события также могут быть объявлены в виде нестатических полей. Далее указывается ключевое слово event, затем идет наименование делегата, на котором основано событие, в

данном примере `NewEventDelegate`. После наименования делегата указывается имя события `NewEvent`, фактически, это имя переменной.

События отображаются в IntelliSense в виде пиктограммы с «молнией» (рис. 36).

Чтобы прикрепить обработчик к событию используется перегруженный оператор «+=», для открепления от события (удаления обработчика события) – перегруженный оператор «-=»:

Поскольку механизм событий основан на механизме групповых делегатов, к событию может быть прикреплено несколько обработчиков. Гарантируется, что при вызове события будут вызваны все обработчики, но порядок их вызова не гарантируется. Таким образом, при написании обработчиков событий, программист не должен рассчитывать, что обработчики событий обязательно будут вызваны в порядке их прикрепления к событию.

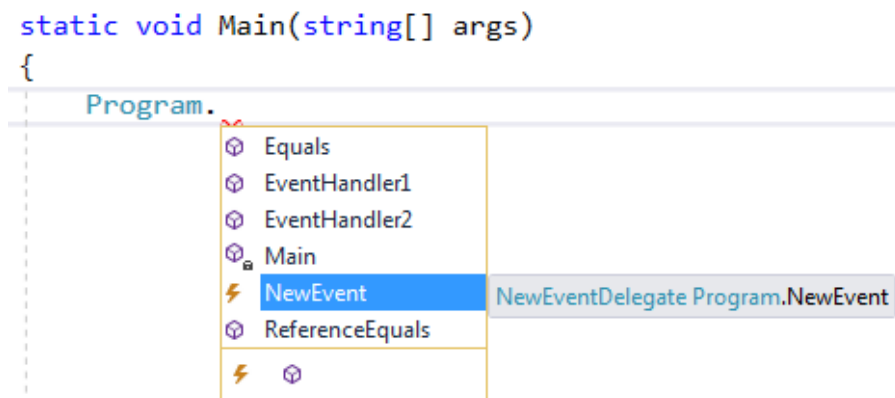


Рис. 36. Отображение события в IntelliSense.

Вызов события производится как вызов обыкновенного метода, в скобках указываются параметры вызова. Однако при вызове событий существует одна особенность: если вызвать событие, к которому не прикреплен ни один обработчик, то возникнет исключение `NullReferenceException`. Для решения этой проблемы у событий переопределено сравнение с `null`. Если оператор сравнения события с `null` возвращает истину, значит к нему не прикреплен ни один из обработчиков.

Поэтому перед вызовом события необходимо проводить проверку на сравнение с null.

Пример объявления обработчиков событий:

```
public static void EventHandler1(string str)
{
    Console.WriteLine("Первый обработчик события: " + str);
}

public static void EventHandler2(string str)
{
    Console.WriteLine("Второй обработчик события: " + str);
}
```

Пример прикрепления обработчиков и вызова события:

```
//Этот вызов не сработает, так как не прикреплены обработчики события
//NewEvent != null - проверка того, что к событию прикреплены обработчики
if (NewEvent != null) NewEvent("вызов события 0");

//Прикрепление обработчика события
Program.NewEvent += EventHandler1;
if (Program.NewEvent != null) Program.NewEvent("вызов события 1");

//Прикрепление второго обработчика события
//Вызываются оба обработчика
Program.NewEvent += EventHandler2;
if (Program.NewEvent != null) Program.NewEvent("вызов события 2");

//Удаление второго обработчика события
Program.NewEvent -= EventHandler2;
if (Program.NewEvent != null) Program.NewEvent("вызов события 3");
```

Результаты вывода в консоль:

```
Первый обработчик события: вызов события 1
Первый обработчик события: вызов события 2
Второй обработчик события: вызов события 2
Первый обработчик события: вызов события 3
```

Рассмотренный пример, содержащий основы работы с событиями, необходим для понимания основ работы с событиями, однако, нужно отметить следующее:

- как правило, событие объявляется внутри класса и не является статическим, а обработчики события прикрепляются снаружи класса;

- событие основывают на библиотечном делегате EventHandler или других специализированных делегатах;
- обработчики событий часто задают в виде лямбда-выражений.

Рассмотрим пример создания события с учетом данных особенностей. Он основан на стандартном делегате EventHandler, объявленном в пространстве имен System:

```
public delegate void EventHandler<TEventArgs>(object sender, TEventArgs e);
```

Данный делегат – обобщенный. Он возвращает тип void, что является необходимым условием для делегата события, и принимает два параметра. Первый параметр sender типа object содержит ссылку на объект, инициировавший событие (напомним, что object – ссылочный тип). Вторым параметром e обобщенного типа TEventArgs содержат произвольные параметры события. В качестве обобщенного типа TEventArgs может использоваться любой тип, включающий в себя описание параметров события.

Пример класса, предназначенного для описания параметров события:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Events
{
    /// <summary>
    /// Параметры события
    /// </summary>
    public class NewGenericEventArgs : EventArgs
    {
        /// <summary>
        /// Конструктор
        /// </summary>
        /// <param name="param"></param>
        public NewGenericEventArgs(string param)
        {
            this.NewGenericEventArgsParam = param;
        }

        /// <summary>
```

```

    /// Свойство, содержащее параметр
    /// </summary>
    public string NewGenericEventArgsParam { get; private set; }
}

```

Класс `NewGenericEventArgs` – наследник класса `EventArgs`. Библиотечный класс `EventArgs` базовый для классов, содержащих данные о событии, и объявлен в пространстве имен `System.Runtime.InteropServices`.

Класс `NewGenericEventArgs` содержит автоопределяемое свойство `NewGenericEventArgsParam` типа `string`, предназначенное для хранения строкового параметра. Также класс содержит конструктор, который инициализирует данное свойство.

Пример класса издателя события `NewGenericEventPublisher`:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Events
{
    /// <summary>
    /// Класс, содержащий событие
    /// </summary>
    public class NewGenericEventPublisher
    {
        /// <summary>
        /// Событие создается через обобщенный делегат
        /// </summary>
        public event EventHandler<NewGenericEventArgs> NewGenericEvent;

        /// <summary>
        /// Вызов события
        /// </summary>
        public void RaiseNewGenericEvent(string param)
        {
            //Если у события есть подписчики
            if (NewGenericEvent != null)
            {
                //Вызов события
                NewGenericEvent(this, new NewGenericEventArgs(param));
            }
        }
    }
}

```


Для объявления события применяется обобщенный делегат EventHandler. Класс NewGenericEventArgs используется в качестве класса-обобщения для хранения параметров события.

```
public event EventHandler<NewGenericEventArgs> NewGenericEvent;
```

Класс NewGenericEventPublisher не содержит конструктор, поэтому применяется конструктор по умолчанию, не содержащий аргументов.

Вызов события тестируется с помощью метода RaiseNewGenericEvent. Метод принимает строковый аргумент, который используется при создании параметра события, экземпляра класса NewGenericEventArgs. В методе проверяется, что для события заданы обработчики «NewGenericEvent != null», если обработчики заданы, то производится вызов события.

Данный класс является учебным примером, в нем вызов события выполняется с помощью тестового метода. В реальных проектах вызовы событий могут производиться внутри методов класса при достижении определенных условий.

Классом слушателя события будет основной класс Program консольного приложения. Работа с событием происходит в основном методе Main консольного приложения.

Для вызова события необходимо сначала создать объект класса:

```
NewGenericEventPublisher ne = new NewGenericEventPublisher();
```

Далее к событию, являющемуся полем объекта класса, прикрепляется обработчик, который здесь задается в виде лямбда-выражения:

```
ne.NewGenericEvent += new EventHandler<NewGenericEventArgs>(
    (sender, e) =>
    {
        Console.WriteLine(e.NewGenericEventArgsParam);
    }
);
```

Лямбда-выражение соответствует делегату EventHandler. В лямбда-выражении в консоль выводятся строки параметров, передаваемых через объект класса NewGenericEventArgs.

Для вызова события в этом случае используется метод `RaiseNewGenericEvent`:

```
ne.RaiseNewGenericEvent("Событие NewGenericEventPublisher");
```

В данном упрощенном примере вызов события происходит в результате внешнего вызова метода `RaiseNewGenericEvent`. Однако, это не совсем естественный путь использования событий. Предполагается, что событие будет вызываться внутри класса при срабатывании определенных условий, характеризующих, например, изменение состояния класса.

11 Рефлексия

В некоторых книгах термин «рефлексия» (reflection) иногда переводят с английского как «отражение». Но с таким переводом трудно согласиться. Дело в том, что данный термин в большей степени соответствует философскому термину «рефлексия». В соответствии с философским энциклопедическим словарем «рефлексия (от лат. Reflexio – обращение назад) – способность человеческого мышления к критическому самоанализу». Если спроецировать это определение на язык информатики, то можно определить рефлексия как способность языка программирования анализировать собственные программы различными способами.

В .NET рефлексия – это совокупность различных механизмов, которые позволяют инспектировать код и работать с кодом как со структурами данных.

11.1 Работа со сборками

Классы для работы с рефлексией расположены в пространстве имен `System.Reflection`.

Класс `Assembly` предназначен для получения информации о сборках. Метод `GetExecutingAssembly` возвращает информацию о текущей исполняемой сборке. Также существует группа методов `Load*` (`Load`,

LoadFile, LoadFrom), позволяющих загрузить сборку и получить информацию о ней.

Пример кода, выводящего в консоль информацию о текущей сборке:

```
Console.WriteLine("Вывод информации о сборке:");
Assembly i = Assembly.GetExecutingAssembly();
Console.WriteLine("Полное имя:" + i.FullName);
Console.WriteLine("Исполняемый файл:" + i.Location);
```

Результаты вывода в консоль:

Вывод информации о сборке:

```
Полное имя: Reflection, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
Исполняемый файл: C:\root\13\Reflection\Reflection\bin\Debug\Reflection.exe
```

11.2 Работа с типами данных

Работа с типами данных один из наиболее часто применяемых механизмов рефлексии.

Создадим класс ForInspection, который содержит свойства, методы и другие элементы для получения информации с помощью рефлексии.

```
using System;
```

```
namespace Reflection
{
    /// <summary>
    /// Класс для исследования с помощью рефлексии
    /// </summary>
    public class ForInspection : IComparable
    {
        public ForInspection() { }
        public ForInspection(int i) { }
        public ForInspection(string str) { }

        public int Plus(int x, int y) { return x + y; }
        public int Minus(int x, int y) { return x - y; }

        [NewAttribute("Описание для property1")]
        public string property1
        {
            get { return _property1; }
            set { _property1 = value; }
        }
        private string _property1;

        public int property2 { get; set; }
    }
}
```

```

[NewAttribute(Description = "Описание для property3")]
public double property3 { get; private set; }

public int field1;
public float field2;

/// <summary>
/// Реализация интерфейса IComparable
/// </summary>
public int CompareTo(object obj)
{
    return 0;
}
}

```

Класс реализует интерфейс `IComparable`, а также содержит конструкторы, методы, свойства, поля данных. Для получения информации о типе (классе) `ForInspection` необходимо создать объект класса `Type`.

Строго говоря, рефлексия не является объектно-ориентированным механизмом. Для получения информации о типах разработчики языка `C#` могли создать отдельный языковой механизм. Но поскольку `C#` – объектно-ориентированный язык, то и рефлексии типов разработчики языка `C#` реализовали на основе объектно-ориентированного подхода. `Type` – это специализированный класс, содержащий информацию о других классах. Объект класса `Type` содержит информацию о конкретном классе.

Получить информацию о типе можно двумя способами. Если создан объект класса, то получить информацию о классе можно с помощью метода `GetType`. Метод `GetType` унаследован от класса `Object`, поэтому он должен присутствовать у любого объекта языка `C#`.

Пример получения информации о типе с помощью метода `GetType`:

```

ForInspection obj = new ForInspection();
Type t = obj.GetType();

```

Если объект класса не создан, то можно применить оператор `typeof`.

Пример получения информации о типе с помощью оператора `typeof`:

```
Type t = typeof(ForInspection);
```

Пример получения информации о типе ForInspection с использованием класса Type:

```
ForInspection obj = new ForInspection();
Type t = obj.GetType();
//другой способ
//Type t = typeof(ForInspection);

Console.WriteLine("\nИнформация о типе:");
Console.WriteLine("Тип " + t.FullName + " унаследован от " +
t.BaseType.FullName);
Console.WriteLine("Пространство имен " + t.Namespace);
Console.WriteLine("Находится в сборке " + t.AssemblyQualifiedName);

Console.WriteLine("\nКонструкторы:");
foreach (var x in t.GetConstructors())
{
    Console.WriteLine(x);
}

Console.WriteLine("\nМетоды:");
foreach (var x in t.GetMethods())
{
    Console.WriteLine(x);
}

Console.WriteLine("\nСвойства:");
foreach (var x in t.GetProperties())
{
    Console.WriteLine(x);
}

Console.WriteLine("\nПоля данных (public):");
foreach (var x in t.GetFields())
{
    Console.WriteLine(x);
}

Console.WriteLine("\nForInspection реализует IComparable -> " +
t.GetInterfaces().Contains(typeof(IComparable))
);
```

Таким образом, класс Type содержит свойства и методы, позволяющие получить информацию о конструкторах, методах, свойствах, полях данных исследуемого класса, проверить от какого класса наследуется класс, какие реализует интерфейсы.

Результаты вывода в консоль:

Информация о типе:

Тип `Reflection.ForInspection` унаследован от `System.Object`

Пространство имен `Reflection`

Находится в сборке `Reflection.ForInspection, Reflection, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null`

Конструкторы:

`Void .ctor()`

`Void .ctor(Int32)`

`Void .ctor(System.String)`

Методы:

`Int32 Plus(Int32, Int32)`

`Int32 Minus(Int32, Int32)`

`System.String get_property1()`

`Void set_property1(System.String)`

`Int32 get_property2()`

`Void set_property2(Int32)`

`Double get_property3()`

`Int32 CompareTo(System.Object)`

`System.String ToString()`

`Boolean Equals(System.Object)`

`Int32 GetHashCode()`

`System.Type GetType()`

Свойства:

`System.String property1`

`Int32 property2`

`Double property3`

Поля данных (public):

`Int32 field1`

`Single field2`

`ForInspection` реализует `Comparable` -> `True`

По аналогии с конструкцией `typeof` в версии `C# 6` появилась конструкция `nameof`, которая возвращает имя объекта. Это полезная

особенность, так как при использовании рефлексии мы не всегда можем знать имя объекта.

Пример использования конструкции nameof:

```
PropertyExample pe1 = new PropertyExample();
Console.WriteLine(nameof(pe1));
```

Результаты вывода в консоль:

```
pe1
```

11.3 Динамические действия с объектами классов

Метод InvokeMember класса Type позволяет выполнять динамические действия с объектами классов: создавать объекты, вызывать методы, получать и присваивать значения свойств и др.

Его особенность заключается в том, что имена свойств, классов передаются методу InvokeMember в виде строковых параметров.

В следующем примере с использованием метода InvokeMember создается объект класса ForInspection и вызывается его метод:

```
Type t = typeof(ForInspection);
Console.WriteLine("\nВызов метода:");
//Создание объекта
//ForInspection fi = new ForInspection();
//Можно создать объект через рефлексия
ForInspection fi =
    (ForInspection)t.InvokeMember(
        null, BindingFlags.CreateInstance,
        null, null, new object[] { });
//Параметры вызова метода
object[] parameters = new object[] { 3, 2 };
//Вызов метода
object Result =
    t.InvokeMember("Plus", BindingFlags.InvokeMethod,
        null, fi, parameters);
Console.WriteLine("Plus(3,2)={0}", Result);
```

Метод InvokeMember принимает различные параметры: имя метода или свойства, к которому происходит обращение, список аргументов вызываемого метода и др.

Одним из важнейших параметров является параметр типа `BindingFlags` – это перечисление, которое определяет выполняемое действие: создание объекта, обращение к методу или свойству и т.д.

Результаты вывода в консоль:

Вызов метода:

`Plus(3,2)=5`

11.4 Работа с атрибутами

Работа с атрибутами, также как и с типами данных – один из наиболее часто применяемых механизмов рефлексии.

Поскольку в языке C# рефлексия реализована с использованием объектно-ориентированного подхода, то атрибуты являются классами.

Пример класса атрибута:

```
using System;

namespace Reflection
{
    /// <summary>
    /// Класс атрибута
    /// </summary>
    [AttributeUsage(AttributeTargets.Property, AllowMultiple =
false, Inherited = false)]
    public class NewAttribute : Attribute
    {
        public NewAttribute() { }
        public NewAttribute(string DescriptionParam)
        {
            Description = DescriptionParam;
        }

        public string Description { get; set; }
    }
}
```

Если класс применяется в качестве атрибута, то он должен быть унаследован от класса `System.Attribute`. Класс может содержать любые данные и методы. В данном случае у класса есть два конструктора (с параметром и без параметра) и автоопределяемое свойство `Description`.

Также класс атрибута должен быть, в свою очередь, помечен атрибутом `AttributeUsage`, который принимает три параметра:

- параметр типа перечисление `AttributeTargets`, которое указывает, к каким элементам класса может применяться атрибут `NewAttribute` (классам, свойствам, методам и т.д.); в данном случае только к свойствам (`Property`);
- логический параметр `AllowMultiple`, указывающий, может ли применяться к свойству несколько атрибутов `NewAttribute`; в данном случае это запрещено;
- логический параметр `Inherited`, указывающий, наследуется ли атрибут классами, производными от класса с атрибутами; обычно используется значение `false`.

Применять несколько атрибутов можно, например, в тех случаях, когда с их помощью помечают сделанные изменения, каждое изменение может помечаться атрибутом с указанием даты-времени и описания изменения.

Рассмотрим использование атрибута `NewAttribute` на примере класса `ForInspection`. Фрагмент класса, который содержит использование атрибутов:

```
[NewAttribute("Описание для property1")]
public string property1
{
    get { return _property1; }
    set { _property1 = value; }
}
private string _property1;

[NewAttribute(Description = "Описание для property3")]
public double property3 { get; private set; }
```

Атрибут необходимо записывать в квадратных скобках непосредственно перед тем элементом класса, к которому он относится; в данном случае атрибут применяется к свойствам.

Каждая запись в квадратных скобках – это объект соответствующего класса атрибута, в данном случае объект класса `NewAttribute`. В примере мы «приклеиваем стикер» в виде объекта класса `NewAttribute` к соответствующему свойству класса `ForInspection`, аннотируем свойства некоторой дополнительной информацией.

При создании аннотации механизм `IntelliSense` автоматически определяет конструкторы класса-атрибута. Все `public`-свойства автоматически превращаются в именованные параметры. Пример использования `IntelliSense` для аннотации показан на рис. 37.

Информация на рис. 37 полностью соответствует определению класса `NewAttribute`. Действительно, класс `NewAttribute` содержит два конструктора (пустой и с одним строковым параметром), а также `public`-свойство `Description`, которое `IntelliSense` определяет как именованный параметр аннотации.

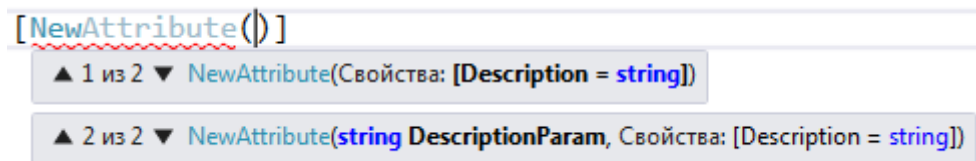


Рис. 37. Работа `IntelliSense` для аннотаций.

При компиляции аннотации также компилируются и записываются в соответствующую сборку. Затем с помощью механизма рефлексии можно проверить снабжен ли элемент класса какой-либо аннотацией. В рассматриваемом примере такая проверка вынесена в отдельную функцию:

```
/// <summary>
/// Проверка, что у свойства есть атрибут заданного типа
/// </summary>
/// <returns>Значение атрибута</returns>
public static bool GetPropertyAttribute(PropertyInfo checkType, Type
attributeType, out object attribute)
{
    bool Result = false;
    attribute = null;

    //Поиск атрибутов с заданным типом
    var isAttribute =
```

```

        checkType.GetCustomAttributes(attributeType, false);
    if (isAttribute.Length > 0)
    {
        Result = true;
        attribute = isAttribute[0];
    }

    return Result;
}

```

Метод принимает в качестве параметров информацию о проверяемом свойстве «PropertyInfo checkType» и тип проверяемого атрибута «Type attributeType». Если проверяемое свойство содержит атрибуты данного типа (метод checkType.GetCustomAttributes возвращает коллекцию isAttribute из более чем одного элемента), то метод возвращает истину, а в выходной параметр метода «out object attribute» помещается первый элемент коллекции isAttribute (в соответствии с определением, свойство NewAttribute может применяться к свойству не более одного раза).

Рассмотрим *пример, осуществляющий проверку того, что свойства снабжены атрибутом NewAttribute*. Если атрибут используется, то выводится содержимое поля Description:

```

Type t = typeof(ForInspection);
Console.WriteLine("\nСвойства, помеченные атрибутом:");
foreach (var x in t.GetProperties())
{
    object attrObj;
    if (GetPropertyAttribute(x, typeof(NewAttribute), out attrObj))
    {
        NewAttribute attr = attrObj as NewAttribute;
        Console.WriteLine(x.Name + " - " + attr.Description);
    }
}

```

В данном примере в цикле перебираются все свойства класса ForInspection. Если свойство снабжено атрибутом, то выводится название свойства и поле Description атрибута.

Информация о свойстве получается с помощью рассмотренной функции GetPropertyAttribute. Для приведения полученного значения типа object к требуемому типу NewAttribute используется оператор as в форме

«выражение as тип». Оператор as является другой формой приведения типов кроме рассмотренного ранее приведения типов с помощью оператора «круглые скобки» в форме «(тип)выражение».

Результаты вывода в консоль:

Свойства, помеченные атрибутом:

property1 - Описание для property1

property3 - Описание для property3

Атрибуты очень широко применяются в .NET. Например, в технологии ASP.NET MVC с помощью атрибутов задаются правила проверки полей ввода при заполнении форм данных.

Отметим, что в Java существует аналогичный атрибутам механизм, который называется аннотациями. Вместо квадратных скобок для выделения аннотаций используется символ «@».

11.5 Использование рефлексии на уровне откомпилированных инструкций

Как уже было сказано выше, .NET является программно-реализованной моделью микропроцессора. Поэтому язык машинных команд MSIL (IL) можно представить в виде команд специфического диалекта языка ассемблер.

Для дизассемблирования необходимо запустить утилиту ildasm - IL Disassembler. Данная утилита входит в комплект Visual Studio. Для запуска необходимо открыть «командную строку разработчика Visual Studio» (данный пункт есть в меню «пуск» в разделе Visual Studio) и набрать в командной строке команду «ildasm».

Данная утилита является оконной, хотя содержит ключи для запуска в командной строке. При запуске без ключей в командной строке открывается окно, в котором нужно выбрать пункт меню «Файл/Открыть», а затем выбрать соответствующий файл сборки.

В результате открывается окно, содержащее список классов сборки. Если раскрыть соответствующий класс (в нашем примере ForInspection), то отобразится список конструкторов, методов, свойств (рис. 38).

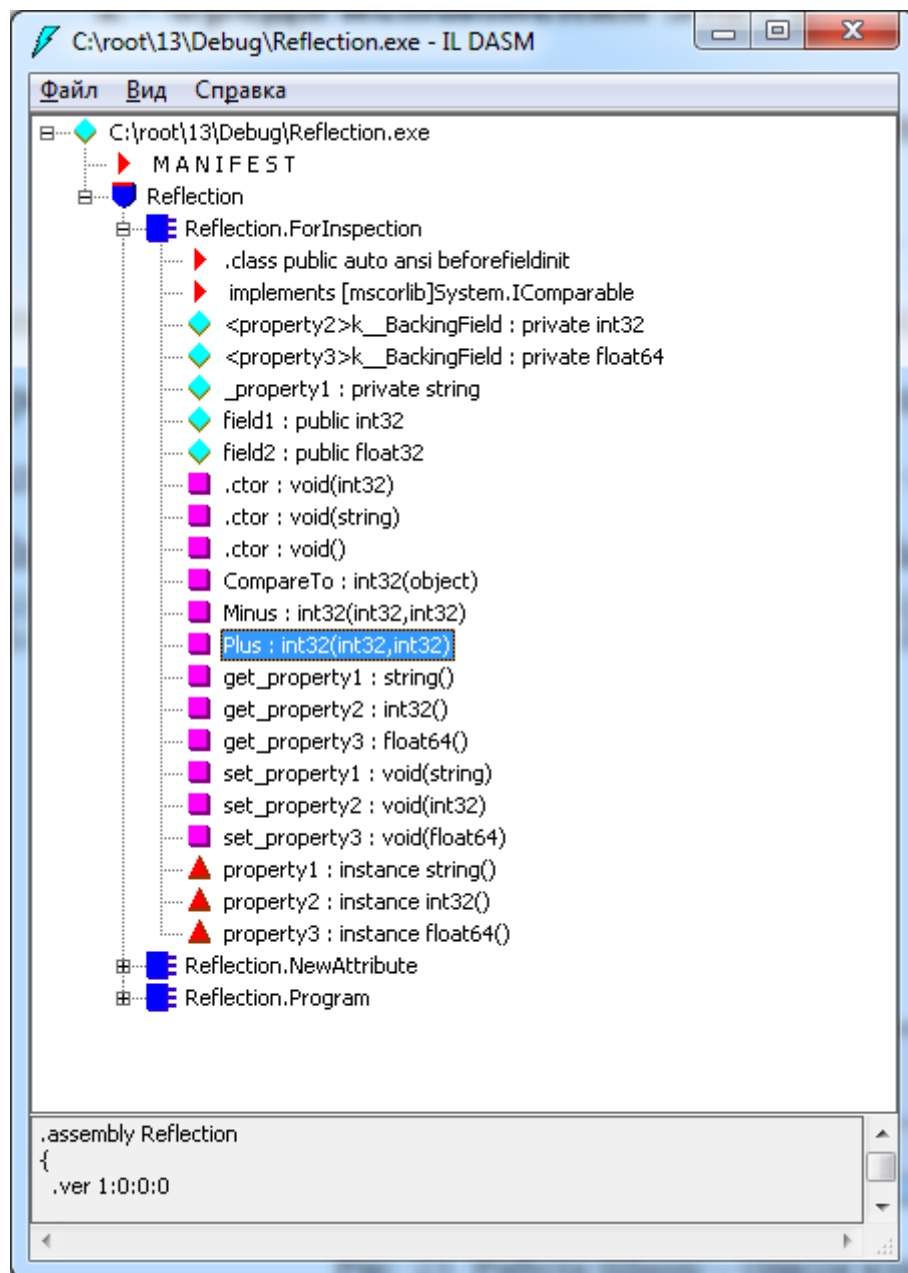


Рис. 38. Работа ildasm – список классов сборки.

Если дважды нажать левой клавишей мыши на определение любого элемента, то детальные команды языка IL отобразятся в отдельном окне. Нажмем дважды на определение метода Plus (рис. 39).

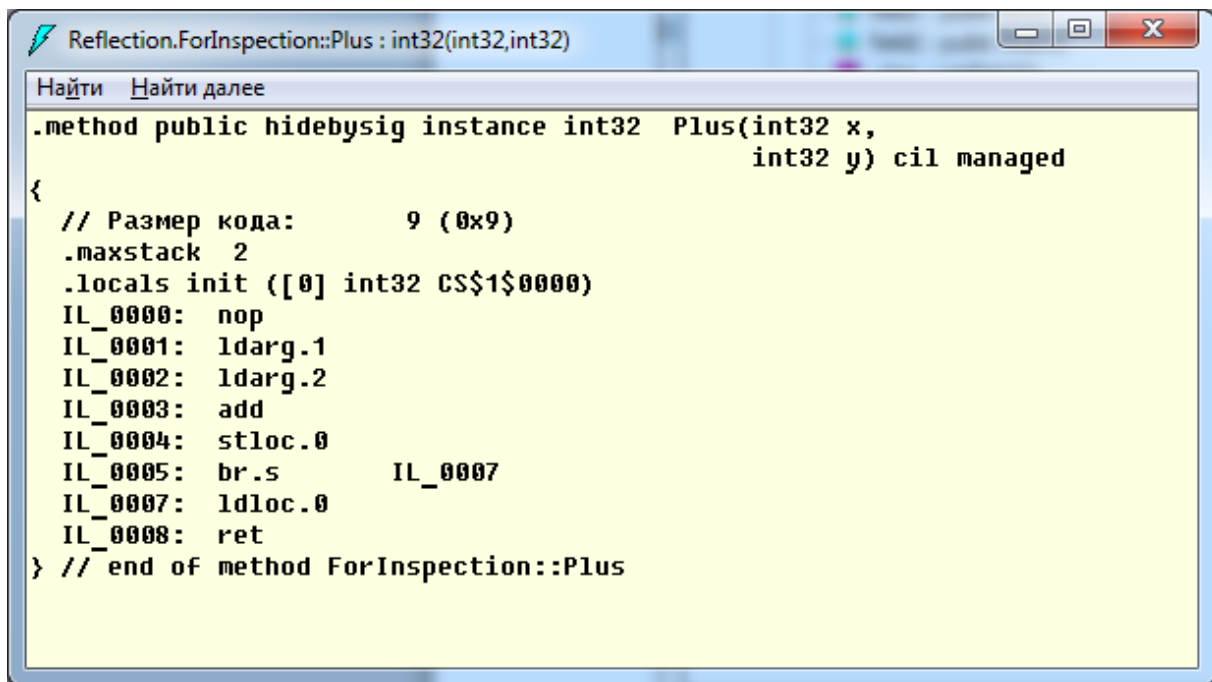


Рис. 39. Работа ildasm – код метода.

Метод Plus осуществляет сложение двух целых чисел.

Пример кода метода на языке C#:

```
public int Plus(int x, int y) { return x + y; }
```

Можно отметить некоторое сходство команд языка IL и команд ассемблера для Intel x86. Команда add выполняет сложение двух элементов стека виртуальной машины, ldarg загружает параметр метода в стек, ret осуществляет выход из метода.

Также необходимо отметить, что .NET предоставляет набор классов в пространстве имен System.Reflection.Emit, предназначенных для динамической генерации сборок. Эти классы могут быть полезны для разработчиков новых языков на платформе .NET а также для более детального знакомства с MSIL. В данное пространство имен входят классы для создания сборок, классов, методов и др. В частности, класс ILGenerator позволяет генерировать бинарный код MSIL. Класс OpCodes содержит полный список всех команд MSIL.

Существуют программы-декомпиляторы, которые пытаются по коду MSIL восстановить исходный код на C#, например проект .NET Reflector.

Но такому восстановлению могут препятствовать программы-обфускаторы, производящие обфускацию (запутывание) откомпилированного кода. Если сборка подвергнута обфускации, то она работает корректно. Однако команды MSIL расположены в сборке таким образом, что декомпилятор не может восстановить по ним более высокоуровневые команды на языке C#. Примером такого проекта является Obfuscator.

11.6 Самоотображаемость

Самоотображаемость (англ. homoiconicity) – это способность языка программирования анализировать программу на этом языке как структуру данных. В некоторых русскоязычных источниках этот термин дословно переводят как «гомоиконичность».

Термин «самоотображаемость» зачастую используют вместе с термином «метапрограммирование». Самоотображаемость предполагает анализ программ как структур данных, а метапрограммирование предполагает генерацию на основе этих структур данных других программ.

Можно сказать, что самоотображаемость – предельный случай рефлексии, так как при этом нет никаких ограничений для анализа. Нет необходимости помечать какие-то фрагменты программы атрибутами, читать список методов класса и т.д. – вся программа является структурой данных, в которой можно производить любой поиск и с которой можно выполнять любые действия. В том числе самоотображаемость предполагает работу с данными на уровне операторов языка, то есть на самом детальном уровне.

В настоящее время язык C# не поддерживает самоотображаемость в полной мере, и проблема состоит как раз в детальном уровне. Поскольку язык C# является компилируемым языком, то с помощью рефлексии

можно получить откомпилированные команды (как это делалось в предыдущем разделе), но по ним невозможно на лету восстановить исходные команды C#. Существуют библиотеки, которые позволяют восстановить и использовать AST-дерево для фрагментов кода.

Изначально самоотображаемость заявлялась разработчиками C# в числе основных приоритетов. Однако в настоящее время разработчики заявляют, что самоотображаемость планируется реализовать в отдаленной перспективе.

Необходимо отметить, что самоотображаемость может быть реализована поверх языка с использованием библиотек для работы с AST (Abstract Syntax Tree – абстрактное синтаксическое дерево). AST дает возможность представить текст программы в виде дерева, в котором внутренние вершины соответствуют операторам языка программирования или функциям, а листья соответствуют операндам. Таким образом, AST позволяет представить текст программы в виде структуры данных (дерева) и фактически реализует принцип самоотображаемости. В отличие от получения команд из откомпилированной сборки с использованием рефлексии, AST-дерево строится по исходному тексту программы. В .NET работа с AST-деревом для прикладных программистов реализуется в рамках проекта модульного компилятора Roslyn.

В настоящее время наиболее известным языком, имеющим поддержку самоотображаемости, является язык Lisp, у которого существует большое количество диалектов, и который исторически является одним из первых языков программирования. В Lisp программа представляется в виде иерархического списка (программа фактически и есть AST), и каждая программа может быть обработана другой программой. На платформе .NET реализован один из современных диалектов Lisp – clojure.

К самоотображаемым языкам также относятся Elixir (синтаксис которого похож на Ruby, но при этом реализована самоотображаемость в

стиле Lisp) и Prolog (специализированный язык логического программирования).

Самоотображаемость на уровне команд языка ближе к интерпретируемым языкам, которые интерпретируются и выполняются на уровне исходного кода. Изначально Lisp был интерпретируемым языком. Самоотображаемость для компилируемого языка требует разработки сложного специализированного компилятора. В настоящее время проводятся активные работы по разработке самоотображаемых языков на основе интерпретируемого языка JavaScript.

12 Технология LINQ

12.1 Мотивация: зачем нужен LINQ

На семинаре, посвящённом реляционной алгебре, мы реализовывали операции фильтрации, проекции и соединения вручную. Рассмотрим типичный фрагмент кода, в котором необходимо решить простую задачу: вывести имена сотрудников отдела «Разработка», отсортированные по убыванию зарплаты.

Определим классы данных и заполним списки:

```

/// <summary>
/// Отдел
/// </summary>
class Department
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
}

/// <summary>
/// Сотрудник
/// </summary>
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int DepartmentId { get; set; }
    public decimal Salary { get; set; }
}

```

// — Исходные данные

```
var departments = new List<Department>
{
    new Department { Id = 1, Name = "Разработка" },
    new Department { Id = 2, Name = "Тестирование" },
    new Department { Id = 3, Name = "Аналитика" }
};

var employees = new List<Employee>
{
    new Employee { Id = 1, Name = "Иванов", DepartmentId = 1, Salary = 150000 },
    new Employee { Id = 2, Name = "Петрова", DepartmentId = 1, Salary = 180000 },
    new Employee { Id = 3, Name = "Сидоров", DepartmentId = 2, Salary = 120000 },
    new Employee { Id = 4, Name = "Козлова", DepartmentId = 1, Salary = 160000 },
    new Employee { Id = 5, Name = "Николаев", DepartmentId = 3, Salary = 140000 },
    new Employee { Id = 6, Name = "Морозова", DepartmentId = 2, Salary = 130000 }
};
```

Решение без LINQ — вручную:

```
// Шаг 1. Найдти Id отдела «Разработка»
int devDepId = -1;
foreach (var dep in departments)
{
    if (dep.Name == "Разработка")
    {
        devDepId = dep.Id;
        break;
    }
}

// Шаг 2. Отобратить сотрудников этого отдела
var devEmployees = new List<Employee>();
foreach (var emp in employees)
{
    if (emp.DepartmentId == devDepId)
    {
        devEmployees.Add(emp);
    }
}

// Шаг 3. Отсортировать по убыванию зарплаты (пузырьковая сортировка)
for (int i = 0; i < devEmployees.Count - 1; i++)
{
    for (int j = 0; j < devEmployees.Count - 1 - i; j++)
    {
        if (devEmployees[j].Salary < devEmployees[j + 1].Salary)
        {
```

```

        var temp = devEmployees[j];
        devEmployees[j] = devEmployees[j + 1];
        devEmployees[j + 1] = temp;
    }
}

// Шаг 4. Вывести имена
Console.WriteLine("=== Без LINQ ===");
foreach (var emp in devEmployees)
{
    Console.WriteLine($" {emp.Name} – {emp.Salary:N0} руб.");
}

```

Код занимает около 30 строк, содержит четыре отдельных цикла и промежуточную переменную `devDepId`. Логика задачи — «найти сотрудников отдела и отсортировать» — растворяется в деталях реализации: ручной поиск, ручная сортировка, ручное копирование в новый список. При усложнении задачи (например, добавить группировку по должности или вычислить среднюю зарплату) объём такого кода быстро растёт.

Фактически, мы воспроизводим те же операции, которые в семинаре 3 были реализованы для работы с CSV-файлами, — выборку (Where), проекцию (Projection) и соединение (Join). Только теперь данные хранятся не в таблицах CSV, а в списках объектов C#.

Решение с LINQ — один запрос:

```

// Для работы LINQ необходимо подключить пространство имён:
// using System.Linq;

Console.WriteLine("=== С LINQ ===");

var result = from emp in employees
              join dep in departments on emp.DepartmentId equals dep.Id
              where dep.Name == "Разработка"
              orderby emp.Salary descending
              select new { emp.Name, emp.Salary };

foreach (var item in result)
{
    Console.WriteLine($" {item.Name} – {item.Salary:N0} руб.");
}

```

Результат вывода в обоих случаях одинаков:

Петрова — 180 000 руб.
 Козлова — 160 000 руб.
 Иванов — 150 000 руб.

Однако LINQ-версия решает задачу в одном выражении. Код читается почти как формулировка задачи на естественном языке: «из сотрудников, соединённых с отделами, где отдел — Разработка, упорядоченных по зарплате по убыванию, выбрать имя и зарплату». При этом нет ни промежуточных переменных, ни ручных циклов, ни реализации сортировки.

Тот же запрос можно записать в альтернативном синтаксисе — с помощью цепочки вызовов методов:

```
var result2 = employees
    .Join(departments,
        emp => emp.DepartmentId,
        dep => dep.Id,
        (emp, dep) => new { emp, dep })
    .Where(x => x.dep.Name == "Разработка")
    .OrderByDescending(x => x.emp.Salary)
    .Select(x => new { x.emp.Name, x.emp.Salary });
```

Оба варианта полностью эквивалентны — компилятор транслирует первый (query syntax) во второй (method syntax). Оба способа записи будут подробно рассмотрены далее.

Но главная сила LINQ — не просто лаконичность. Рассмотрим ещё один пример. Допустим, те же данные о сотрудниках хранятся не в списке, а в базе данных SQLite (с которой мы работали на семинаре 3). При использовании технологии Entity Framework запрос к базе данных выглядит практически так же, как запрос к списку в памяти:

```
// Запрос к базе данных SQLite через Entity Framework
// (подробно рассматривается в разделе 12.9)
using var db = new CompanyContext();

var result3 = from emp in db.Employees
    join dep in db.Departments on emp.DepartmentId equals dep.Id
    where dep.Name == "Разработка"
    orderby emp.Salary descending
    select new { emp.Name, emp.Salary };
```

Синтаксис запроса не изменился — изменился только источник данных: вместо `employees` (список в памяти) используется `db.Employees` (таблица в базе данных). При этом LINQ автоматически транслирует запрос в SQL:

```
SELECT e.Name, e.Salary
FROM Employees e
INNER JOIN Departments d ON e.DepartmentId = d.Id
WHERE d.Name = 'Разработка'
ORDER BY e.Salary DESC
```

Таким образом, LINQ (Language Integrated Query) — это технология языка C#, позволяющая писать запросы к данным в едином синтаксисе независимо от того, где эти данные находятся: в коллекциях в оперативной памяти, в XML-документах, в JSON-файлах или в реляционных базах данных. Программисту не нужно переключаться между C# и SQL, между C# и XPath — всё пишется на C#, проверяется компилятором и поддерживается IntelliSense.

В последующих разделах мы подробно рассмотрим, как LINQ устроен внутри, какие операции поддерживает, и как использовать его с различными источниками данных.

12.2 Краткая история LINQ и его место в .NET

Идея интеграции запросов к данным непосредственно в язык программирования возникла задолго до появления LINQ. Программисты всегда сталкивались с проблемой, которую называют несоответствием импедансов (*impedance mismatch*): объектно-ориентированные языки работают с объектами, иерархиями классов и коллекциями, а базы данных — с таблицами, строками и столбцами. При этом SQL-запросы записываются в виде строк, которые не проверяются компилятором, — опечатка в имени столбца обнаруживается только при выполнении программы.

Аналогичная проблема существует и при работе с XML: для навигации по XML-документам используются языки XPath и XQuery, которые также записываются в строковом виде и не контролируются компилятором C#.

Решение этой проблемы было предложено в проекте LINQ, архитектором которого стал Эрик Мейер (Erik Meijer) — специалист по функциональным языкам программирования, работавший в Microsoft Research. Мейер привнёс в C# идеи из языка Haskell, в частности концепцию монад — абстракции, позволяющей единообразно выстраивать цепочки операций над данными. Другим важным источником вдохновения послужила реляционная алгебра, рассмотренная на семинаре 3, — операции проекции, выборки, соединения и группировки, замкнутые на множестве отношений (таблиц). Андерс Хейлсберг (Anders Hejlsberg) — ведущий архитектор языка C# — играл ключевую роль в проектировании и интеграции LINQ в язык. Именно он представлял LINQ на публичных презентациях и рассказывал о концепциях, которые не успели войти в релиз 2005 года (C# 2.0) и были реализованы позднее в C# 3.0. Эрик Мейер работал над LINQ параллельно, привнося в проект теоретический фундамент из функционального программирования (монады, comprehension syntax из Haskell). Таким образом, LINQ — результат совместной работы, где Хейлсберг отвечал за интеграцию в язык C# в целом, а Мейер — за функциональные и теоретические основы.

LINQ был выпущен в 2007 году в составе C# 3.0 и .NET Framework 3.5. Однако для его реализации потребовалось одновременно добавить в язык C# целый ряд новых механизмов. Все они были рассмотрены в предыдущих разделах курса:

Механизм языка C#	Роль в LINQ	Раздел курса
Лямбда-выражения	Задают условия фильтрации, проекции, сортировки и другие операции в компактной	10.8

Механизм языка C#	Роль в LINQ	Раздел курса
	форме	
Методы расширения	Методы Where, Select, OrderBy и другие реализованы как методы расширения для интерфейса IEnumerable<T>	8.11
Обобщённые делегаты Func<> и Action<>	Используются в качестве типов параметров LINQ-методов (например, Where принимает Func<T, bool>)	10.9
Анонимные типы	Позволяют создавать объекты произвольной структуры в select new { ... } без предварительного объявления класса	—
Вывод типов (var)	Позволяет компилятору автоматически определить тип результата LINQ-запроса, в том числе анонимного типа	3
Паттерн Fluent Interface	Цепочки вызовов LINQ-методов (.Where(...).OrderBy(...).Select(...)) являются классическим примером текучего интерфейса	8.13

Таким образом, LINQ не является изолированной технологией — это результат целенаправленного развития языка C#, в котором каждый новый механизм выполняет свою роль.

Рассмотрим, как выглядит связь между этими механизмами на конкретном примере. Выражение:

```
var names = employees
    .Where(e => e.Salary > 100000)
    .Select(e => e.Name);
```

можно разобрать следующим образом:

- employees — объект типа List<Employee>, реализующий интерфейс IEnumerable<Employee>;
- .Where(...) — **метод расширения** (раздел 8.11), определённый в статическом классе System.Linq.Enumerable для интерфейса IEnumerable<T>;

- `e => e.Salary > 100000` — **лямбда-выражение** (раздел 10.8), соответствующее **обобщённому делегату** `Func<Employee, bool>` (раздел 10.9);
- `.Select(...)` — ещё один метод расширения, вызванный на результате `Where`, что образует **цепочку вызовов** в стиле *Fluent Interface* (раздел 8.13);
- `var` — **вывод типа**: компилятор определяет, что `names` имеет тип `IEnumerable<string>`.

Для использования LINQ-методов необходимо подключить пространство имён:

```
using System.Linq;
```

В современных проектах .NET (начиная с .NET 6) при использовании операторов верхнего уровня и шаблона проекта по умолчанию это пространство имён подключается автоматически через механизм глобальных `using`-директив. Однако при работе со старыми проектами или при явном управлении подключениями необходимо добавлять эту директиву вручную.

Со временем концепция LINQ вышла за рамки C#. В языке Java 8 (2014) появились *Stream API* — механизм, вдохновлённый LINQ, позволяющий работать с коллекциями в функциональном стиле. В языке Python аналогичную роль выполняют генераторы списков (*list comprehensions*) и библиотека *pandas* для работы с табличными данными. В языке Kotlin операции `filter`, `map`, `groupBy` над коллекциями также концептуально близки к LINQ. Однако именно в C# интеграция запросов в язык реализована наиболее глубоко — вплоть до специального синтаксиса `from ... where ... select` и механизма деревьев выражений, позволяющего транслировать запросы в другие языки.

12.3 Предметная область: сотрудники, отделы, проекты

В качестве сквозного примера для всех последующих разделов используется предметная область, продолжающая тему семинара 3 (сотрудники и отделы), но расширенная для демонстрации связи «много-ко-многим».


```

class Project
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public decimal Budget { get; set; }
}

/// <summary>
/// Связь «много-ко-многим» между сотрудниками и проектами.
/// Каждая запись означает, что сотрудник с EmployeeId
/// участвует в проекте с ProjectId.
/// </summary>
class EmployeeProject
{
    public int EmployeeId { get; set; }
    public int ProjectId { get; set; }
}

```

12.3.2 Заполнение тестовых данных

```

// — Отделы
var departments = new List<Department>
{
    new Department { Id = 1, Name = "Разработка" },
    new Department { Id = 2, Name = "Тестирование" },
    new Department { Id = 3, Name = "Аналитика" }
};

// — Сотрудники
var employees = new List<Employee>
{
    new Employee { Id = 1, Name = "Иванов", DepartmentId = 1, Salary = 150000 },
    new Employee { Id = 2, Name = "Петрова", DepartmentId = 1, Salary = 180000 },
    new Employee { Id = 3, Name = "Сидоров", DepartmentId = 2, Salary = 120000 },
    new Employee { Id = 4, Name = "Козлова", DepartmentId = 1, Salary = 160000 },
    new Employee { Id = 5, Name = "Николаев", DepartmentId = 3, Salary = 140000 },
    new Employee { Id = 6, Name = "Морозова", DepartmentId = 2, Salary = 130000 }
};

// — Проекты
var projects = new List<Project>
{
    new Project { Id = 101, Title = "Сайт", Budget = 500000 },
    new Project { Id = 102, Title = "Мобильное приложение", Budget = 800000 }
},

```

```

    new Project { Id = 103, Title = "Аналитическая платформа", Budget = 12
00000 }
};

// — Связи «сотрудник – проект» —————
// Один сотрудник может участвовать в нескольких проектах,
// один проект может выполняться несколькими сотрудниками.

var employeeProjects = new List<EmployeeProject>
{
    // Иванов → Сайт, Мобильное приложение
    new EmployeeProject { EmployeeId = 1, ProjectId = 101 },
    new EmployeeProject { EmployeeId = 1, ProjectId = 102 },
    // Петрова → Мобильное приложение, Аналитическая платформа
    new EmployeeProject { EmployeeId = 2, ProjectId = 102 },
    new EmployeeProject { EmployeeId = 2, ProjectId = 103 },
    // Сидоров → Сайт
    new EmployeeProject { EmployeeId = 3, ProjectId = 101 },
    // Козлова → Сайт, Мобильное приложение, Аналитическая платформа
    new EmployeeProject { EmployeeId = 4, ProjectId = 101 },
    new EmployeeProject { EmployeeId = 4, ProjectId = 102 },
    new EmployeeProject { EmployeeId = 4, ProjectId = 103 },
    // Николаев → Аналитическая платформа
    new EmployeeProject { EmployeeId = 5, ProjectId = 103 },
    // Морозова → Сайт, Мобильное приложение
    new EmployeeProject { EmployeeId = 6, ProjectId = 101 },
    new EmployeeProject { EmployeeId = 6, ProjectId = 102 }
};

```

12.3.3 Структура данных

Для наглядности приведём содержимое списков в табличном виде.

Отделы (departments):

Id	Name
1	Разработка
2	Тестирование
3	Аналитика

Сотрудники (employees):

Id	Name	DepartmentId	Salary
1	Иванов	1	150 000
2	Петрова	1	180 000
3	Сидоров	2	120 000
4	Козлова	1	160 000
5	Николаев	3	140 000
6	Морозова	2	130 000

Проекты (projects):

Id	Title	Budget
101	Сайт	500 000
102	Мобильное приложение	800 000
103	Аналитическая платформа	1 200 000

Связи «сотрудник ↔ проект» (employeeProjects):

EmployeeId	ProjectId
1	101
1	102
2	102
2	103
3	101
4	101
4	102
4	103
5	103
6	101
6	102

Обратите внимание на ключевые свойства данных:

- Связь «один-ко-многим»: в отделе «Разработка» (Id = 1) работают три сотрудника (Иванов, Петрова, Козлова). Каждый сотрудник принадлежит ровно одному отделу — это определяется полем DepartmentId.
- Связь «много-ко-многим»: Козлова участвует в трёх проектах, а проект «Сайт» выполняется четырьмя сотрудниками. Ни сотрудник не «принадлежит» проекту, ни проект — сотруднику; связь реализуется через промежуточный список employeeProjects.

Эти данные будут использоваться во всех последующих примерах — от базовых LINQ-запросов к коллекциям в памяти до запросов через Entity Framework к базе данных SQLite.

12.4 LINQ to Objects: основные операции

LINQ to Objects — это набор методов расширения, определённых в статическом классе System.Linq.Enumerable для интерфейса

`IEnumerable<T>`. Поскольку все стандартные коллекции .NET (массивы, `List<T>`, `Dictionary<TKey, TValue>`, `HashSet<T>` и др.) реализуют `IEnumerable<T>`, LINQ-методы доступны для любой из них.

В этом разделе рассматриваются основные операции LINQ to Objects на примере предметной области, определённой в разделе 12.3. Все примеры предполагают, что списки `departments`, `employees`, `projects` и `employeeProjects` уже заполнены данными.

12.4.1 Два синтаксиса: `query syntax` и `method syntax`

В языке C# существует два эквивалентных способа записи LINQ-запросов.

`Query syntax` (синтаксис запросов) напоминает SQL и использует ключевые слова `from`, `where`, `select`, `orderby`, `join`, `group`, `into`, `let`:

```
var result = from emp in employees
              where emp.Salary > 140000
              select emp.Name;
```

`Method syntax` (синтаксис методов) использует цепочки вызовов методов расширения с лямбда-выражениями в качестве параметров:

```
var result = employees
    .Where(emp => emp.Salary > 140000)
    .Select(emp => emp.Name);
```

Оба варианта полностью эквивалентны. Компилятор транслирует `query syntax` в `method syntax` на этапе компиляции. Таким образом, `query syntax` — это синтаксический сахар, который не добавляет новых возможностей, но повышает читаемость для определённых типов запросов.

На практике используют оба варианта, нередко комбинируя их в одном выражении. `Query syntax` удобнее для запросов с несколькими соединениями (`join`) и конструкцией `let`. `Method syntax` предпочтительнее для простых операций и для методов, не имеющих эквивалента в `query syntax` (например, `Distinct`, `Skip`, `Take`, `Count`).

В последующих примерах каждая операция будет показана в обоих синтаксисах.

12.4.2 Выборка и проекция (Select)

Простейшая операция LINQ — выборка всех элементов коллекции:

```
Console.WriteLine("=== Простая выборка ===");
var q1 = from emp in employees select emp;
foreach (var emp in q1)
    Console.WriteLine($" {emp.Id}: {emp.Name}");
```

Проекция — это выбор отдельных полей или создание новых объектов на основе существующих. Проекция позволяет извлечь из объекта только нужные данные:

```
Console.WriteLine("=== Проекция: только имена ===");
var q2 = from emp in employees select emp.Name;
foreach (var name in q2)
    Console.WriteLine($" {name}");
```

Результат:

```
Иванов
Петрова
Сидоров
Козлова
Николаев
Морозова
```

При необходимости извлечь несколько полей используются анонимные типы — объекты, класс которых не объявляется явно, а автоматически генерируется компилятором:

```
Console.WriteLine("=== Проекция: анонимный тип ===");
var q3 = from emp in employees
    select new { emp.Name, emp.Salary };

foreach (var item in q3)
    Console.WriteLine($" {item.Name} – {item.Salary:N0} руб.");
```

Результат:

```
Иванов – 150 000 руб.
Петрова – 180 000 руб.
Сидоров – 120 000 руб.
Козлова – 160 000 руб.
Николаев – 140 000 руб.
Морозова – 130 000 руб.
```

В конструкции `new { emp.Name, emp.Salary }` компилятор создаёт анонимный класс с двумя свойствами `Name` и `Salary`. Имена свойств определяются автоматически по именам исходных полей. При желании можно задать имена явно:

```
var q3b = from emp in employees
        select new { Сотрудник = emp.Name, Зарплата = emp.Salary };
```

Эквивалентная запись в `method syntax`:

```
var q3c = employees.Select(emp => new { emp.Name, emp.Salary });
```

12.4.3 Фильтрация (Where)

Оператор `where` позволяет отобрать элементы, удовлетворяющие условию. Условие задаётся в виде логического выражения, в котором можно использовать любые операторы C#:

```
Console.WriteLine("=== Фильтрация ===");
var q4 = from emp in employees
        where emp.Salary > 140000 && emp.DepartmentId == 1
        select emp;

foreach (var emp in q4)
    Console.WriteLine($" {emp.Name} — {emp.Salary:N0} руб.");
```

Результат:

```
Иванов — 150 000 руб.
Петрова — 180 000 руб.
Козлова — 160 000 руб.
```

Эквивалентная запись в `method syntax`:

```
var q4m = employees
    .Where(emp => emp.Salary > 140000 && emp.DepartmentId == 1);
```

Метод `Where` принимает параметр типа `Func<Employee, bool>` — обобщённый делегат (раздел 10.9), которому соответствует лямбда-выражение, возвращающее `true` для элементов, включаемых в результат.

Для фильтрации по типу объекта используется метод `OfType<T>`, который полезен при работе с коллекциями, содержащими объекты разных типов:

```
object[] mixed = { 1, "строка", 2, true, "ещё строка", 3.14 };
var strings = mixed.OfType<string>();
```

```
Console.WriteLine("=== Фильтрация по типу ===");
foreach (var s in strings)
    Console.WriteLine($" {s}");
```

Результат:

```
строка
ещё строка
```

12.4.4 Сортировка (OrderBy, ThenBy)

Оператор `orderby` задаёт порядок сортировки. По умолчанию сортировка выполняется по возрастанию; для сортировки по убыванию используется ключевое слово `descending`. При сортировке по нескольким полям поля перечисляются через запятую:

```
Console.WriteLine("=== Сортировка ===");
var q5 = from emp in employees
        orderby emp.DepartmentId, emp.Salary descending
        select emp;

foreach (var emp in q5)
    Console.WriteLine($" Отдел {emp.DepartmentId}: {emp.Name} – {emp.Salary:N0}");
```

Результат:

```
Отдел 1: Петрова – 180 000
Отдел 1: Козлова – 160 000
Отдел 1: Иванов – 150 000
Отдел 2: Морозова – 130 000
Отдел 2: Сидоров – 120 000
Отдел 3: Николаев – 140 000
```

Эквивалентная запись в `method syntax` использует методы `OrderBy` и `ThenByDescending`. Обратите внимание, что для вторичной сортировки используется именно `ThenBy` / `ThenByDescending`, а не повторный вызов `OrderBy` — повторный вызов `OrderBy` заменит предыдущую сортировку, а не дополнит её:

```
var q5m = employees
    .OrderBy(emp => emp.DepartmentId)
    .ThenByDescending(emp => emp.Salary);
```


12.4.5 Соединения (Join)

Соединение — одна из важнейших операций при работе с данными, связанными через идентификаторы. В реляционной алгебре ей соответствует операция соединения (join), рассмотренная на семинаре 3.

Inner Join — соединение, при котором в результат попадают только те пары элементов, для которых найдено совпадение ключей:

```
Console.WriteLine("=== Inner Join: сотрудники с названиями отделов ===");
var q6 = from emp in employees
        join dep in departments on emp.DepartmentId equals dep.Id
        select new { emp.Name, Department = dep.Name, emp.Salary };

foreach (var item in q6)
    Console.WriteLine($" {item.Name} - {item.Department} - {item.Salary:N0}");
```

Результат:

```
Иванов - Разработка - 150 000
Петрова - Разработка - 180 000
Сидоров - Тестирование - 120 000
Козлова - Разработка - 160 000
Николаев - Аналитика - 140 000
Морозова - Тестирование - 130 000
```

Обратите внимание, что в конструкции join ... on ... equals слово equals является ключевым словом языка C# и не может быть заменено оператором ==. Это связано с тем, что компилятор специальным образом обрабатывает ключевые поля соединения. Левая и правая части equals должны ссылаться на разные коллекции — левая часть на внешнюю (employees), правая часть на присоединяемую (departments).

Эквивалентная запись в method syntax:

```
var q6m = employees.Join(
    departments,
    emp => emp.DepartmentId, // ключ из левой коллекции
    dep => dep.Id,           // ключ из правой коллекции
    (emp, dep) => new { emp.Name, Department = dep.Name, emp.Salary }
);
```

Group Join — соединение, при котором для каждого элемента левой коллекции собирается группа связанных элементов правой коллекции. Результатом является «один-ко-многим»: один элемент слева и коллекция

совпадений справа. В отличие от Inner Join, элемент левой коллекции попадает в результат даже если в правой коллекции нет совпадений (в этом случае группа будет пустой):

```
Console.WriteLine("=== Group Join: отделы с сотрудниками ===");
var q7 = from dep in departments
        join emp in employees on dep.Id equals emp.DepartmentId into dept
        Employees
        select new { dep.Name, Employees = deptEmployees };

foreach (var item in q7)
{
    Console.WriteLine($" {item.Name}:");
    foreach (var emp in item.Employees)
        Console.WriteLine($" {emp.Name} – {emp.Salary:N0}");
}
```

Результат:

Разработка:

Иванов – 150 000

Петрова – 180 000

Козлова – 160 000

Тестирование:

Сидоров – 120 000

Морозова – 130 000

Аналитика:

Николаев – 140 000

Ключевым элементом является конструкция `into deptEmployees`, которая помещает все совпадения по ключу во временную переменную-группу `deptEmployees`. Эта переменная имеет тип `IEnumerable<Employee>` и может быть перебрана вложенным циклом или обработана агрегатными функциями.

Left Outer Join — соединение, при котором все элементы левой коллекции попадают в результат, а если в правой коллекции совпадений нет, подставляется значение по умолчанию. В LINQ outer join реализуется через комбинацию `group join` и метода `DefaultIfEmpty`:

```
// Добавим отдел без сотрудников для демонстрации
departments.Add(new Department { Id = 4, Name = "Маркетинг" });

Console.WriteLine("=== Left Outer Join ===");
var q8 = from dep in departments
        join emp in employees on dep.Id equals emp.DepartmentId into dept
        Employees
        from emp in deptEmployees.DefaultIfEmpty()
```

```

select new
{
    Department = dep.Name,
    Employee = emp?.Name ?? "(нет сотрудников)"
};

foreach (var item in q8)
    Console.WriteLine($" {item.Department} – {item.Employee}");

```

Результат:

```

Разработка – Иванов
Разработка – Петрова
Разработка – Козлова
Тестирование – Сидоров
Тестирование – Морозова
Аналитика – Николаев
Маркетинг – (нет сотрудников)

```

Конструкция `from emp in deptEmployees.DefaultIfEmpty()` «раскрывает» группу обратно в плоский список, подставляя `null` (значение по умолчанию для ссылочного типа) в тех случаях, когда группа пуста. Оператор `?.` и `??` используются для безопасной обработки `null`.

Соединение по составному ключу выполняется с помощью анонимных типов в конструкции `equals`. Анонимные типы автоматически реализуют сравнение по значениям свойств:

```

// Пример: соединение по двум полям одновременно
var q8b = from a in collection1
join b in collection2
on new { a.Field1, a.Field2 }
equals new { b.Field1, b.Field2 }
select new { a, b };

```

Имена свойств в обоих анонимных типах должны совпадать, иначе компилятор не сможет выполнить сравнение.

12.4.6 Связь «много-ко-многим»

Связь «много-ко-многим» между сотрудниками и проектами реализуется через промежуточный список `employeeProjects`. Для навигации по такой связи необходимо выполнить два последовательных соединения: сначала с промежуточной таблицей, затем с целевой.

Все проекты каждого сотрудника:

```

Console.WriteLine("=== Много-ко-многим: проекты сотрудников ===");
var q9 = from emp in employees
        join ep in employeeProjects on emp.Id equals ep.EmployeeId
        join prj in projects on ep.ProjectId equals prj.Id
        select new { emp.Name, Project = prj.Title };

foreach (var item in q9)
    Console.WriteLine($" {item.Name} → {item.Project}");

```

Результат:

```

Иванов → Сайт
Иванов → Мобильное приложение
Петрова → Мобильное приложение
Петрова → Аналитическая платформа
Сидоров → Сайт
Козлова → Сайт
Козлова → Мобильное приложение
Козлова → Аналитическая платформа
Николаев → Аналитическая платформа
Морозова → Сайт
Морозова → Мобильное приложение

```

Все сотрудники каждого проекта (с группировкой):

```

Console.WriteLine("=== Много-ко-многим: сотрудники по проектам ===");
var q10 = from prj in projects
        join ep in employeeProjects on prj.Id equals ep.ProjectId
        join emp in employees on ep.EmployeeId equals emp.Id
        group emp by prj.Title into projectGroup
        select new { Project = projectGroup.Key, Members = projectGroup
};

foreach (var item in q10)
{
    Console.WriteLine($" {item.Project}:");
    foreach (var emp in item.Members)
        Console.WriteLine($" {emp.Name}");
}

```

Результат:

```

Сайт:
    Иванов
    Сидоров
    Козлова
    Морозова
Мобильное приложение:
    Иванов
    Петрова
    Козлова
    Морозова
Аналитическая платформа:
    Петрова
    Козлова
    Николаев

```

Найти сотрудников, участвующих в проекте с определённым условием. Например, найти всех сотрудников, которые участвуют хотя бы в одном проекте с бюджетом более 1 000 000:

```
Console.WriteLine("=== Сотрудники в крупных проектах ===");
var q11 = from emp in employees
          join ep in employeeProjects on emp.Id equals ep.EmployeeId
          join prj in projects on ep.ProjectId equals prj.Id
          where prj.Budget > 1000000
          select emp;

// Distinct нужен, чтобы исключить дубликаты:
// сотрудник может участвовать в нескольких крупных проектах
foreach (var emp in q11.Distinct())
    Console.WriteLine($" {emp.Name}");
```

Результат:

```
Петрова
Козлова
Николаев
```

12.4.7 Группировка и агрегатные функции

Оператор `group ... by` разделяет коллекцию на группы по значению указанного ключа. Результатом является коллекция групп, где каждая группа реализует интерфейс `IGrouping<TKey, TElement>` — она содержит свойство `Key` (значение ключа) и саму коллекцию элементов группы.

Группировка сотрудников по отделам:

```
Console.WriteLine("=== Группировка по отделам ===");
var q12 = from emp in employees
          group emp by emp.DepartmentId into g
          select new { DepartmentId = g.Key, Employees = g };

foreach (var group in q12)
{
    Console.WriteLine($" Отдел {group.DepartmentId}:");
    foreach (var emp in group.Employees)
        Console.WriteLine($" {emp.Name} — {emp.Salary:N0}");
}
```

Результат:

```
Отдел 1:
Иванов — 150 000
Петрова — 180 000
Козлова — 160 000
Отдел 2:
Сидоров — 120 000
```

Морозова – 130 000
 Отдел 3:
 Николаев – 140 000

К группам можно применять агрегатные функции: Count(), Sum(), Min(), Max(), Average(). Эти методы вызываются непосредственно для переменной группы:

```
Console.WriteLine("=== Группировка с агрегатными функциями ===");
var q13 = from emp in employees
          group emp by emp.DepartmentId into g
          select new
          {
              DepartmentId = g.Key,
              Count = g.Count(),
              AvgSalary = g.Average(e => e.Salary),
              MaxSalary = g.Max(e => e.Salary),
              TotalSalary = g.Sum(e => e.Salary)
          };

foreach (var item in q13)
    Console.WriteLine(
        $" Отдел {item.DepartmentId}: " +
        $"сотр. {item.Count}, " +
        $"средняя {item.AvgSalary:N0}, " +
        $"макс. {item.MaxSalary:N0}, " +
        $"фонд {item.TotalSalary:N0}");
```

Результат:

Отдел 1: сотр. 3, средняя 163 333, макс. 180 000, фонд 490 000
 Отдел 2: сотр. 2, средняя 125 000, макс. 130 000, фонд 250 000
 Отдел 3: сотр. 1, средняя 140 000, макс. 140 000, фонд 140 000

Сравним этот запрос с его SQL-эквивалентом (с которым студенты работали на семинаре 3):

```
SELECT dep_id,
       COUNT(*)           AS cnt,
       AVG(salary)        AS avg_salary,
       MAX(salary)         AS max_salary,
       SUM(salary)         AS total_salary
FROM employees
GROUP BY dep_id
```

Структура запроса практически совпадает, однако LINQ-версия проверяется компилятором — опечатка в имени свойства приведёт к ошибке компиляции, а не к ошибке на этапе выполнения.

Фильтрация групп выполняется с помощью where после group ... into. Это аналог HAVING в SQL. Методы Any и All проверяют, удовлетворяет ли условию хотя бы один или все элементы группы соответственно:

```
Console.WriteLine("=== Отделы, где хотя бы один сотрудник" +
    " получает более 150 000 ===");
var q14 = from emp in employees
    group emp by emp.DepartmentId into g
    where g.Any(e => e.Salary > 150000)
    select new { DepartmentId = g.Key, Employees = g };

foreach (var group in q14)
{
    Console.WriteLine($" Отдел {group.DepartmentId}:");
    foreach (var emp in group.Employees)
        Console.WriteLine($" {emp.Name} – {emp.Salary:N0}");
}
```

Результат:

```
Отдел 1:
Иванов – 150 000
Петрова – 180 000
Козлова – 160 000
```

12.4.8 Операции над множествами

LINQ предоставляет стандартные теоретико-множественные операции, соответствующие операциям реляционной алгебры (семинар 3):

```
int[] a = { 1, 2, 3, 4, 5 };
int[] b = { 3, 4, 5, 6, 7 };

Console.WriteLine("=== Операции над множествами ===");

// Объединение (Union) – все уникальные элементы из обоих множеств
Console.Write(" Union: ");
foreach (var x in a.Union(b)) Console.Write($"{x} ");
Console.WriteLine(); // 1 2 3 4 5 6 7

// Пересечение (Intersect) – элементы, присутствующие в обоих множествах
Console.Write(" Intersect: ");
foreach (var x in a.Intersect(b)) Console.Write($"{x} ");
Console.WriteLine(); // 3 4 5

// Вычитание (Except) – элементы первого множества,
// отсутствующие во втором
Console.Write(" Except: ");
foreach (var x in a.Except(b)) Console.Write($"{x} ");
Console.WriteLine(); // 1 2

// Конкатенация (Concat) – объединение без исключения дубликатов
```

```

Console.Write("  Concat:  ");
foreach (var x in a.Concat(b)) Console.Write($"{x} ");
Console.WriteLine();                                     // 1 2 3 4 5 3 4 5 6 7

// Уникальные значения (Distinct)
Console.Write("  Distinct: ");
foreach (var x in a.Concat(b).Distinct()) Console.Write($"{x} ");
Console.WriteLine();                                     // 1 2 3 4 5 6 7

```

При работе с объектами пользовательских классов (а не с примитивными типами) операции Distinct, Union, Intersect и Except по умолчанию сравнивают объекты по ссылке. Для сравнения по значениям полей необходимо либо передать реализацию интерфейса IEqualityComparer<T> в качестве параметра метода, либо переопределить методы Equals и GetHashCode в классе (раздел 10.5):

```

// Без компаратора – дубликаты не исключаются
// (разные объекты, хоть и с одинаковыми данными)
var list1 = new List<Employee>
{
    new Employee { Id = 1, Name = "Иванов" },
    new Employee { Id = 1, Name = "Иванов" }
};
Console.WriteLine(list1.Distinct().Count()); // 2 (объекты разные!)

```

12.4.9 Разделение данных на страницы (пагинация) – Skip, Take

Методы Skip и Take позволяют реализовать постраничный вывод данных. Метод Skip(n) пропускает первые n элементов, Take(n) — берёт не более n элементов:

```

///

```



```
foreach (var emp in page)
    Console.WriteLine($" {emp.Id}: {emp.Name}");
```

Результат:

```
3: Сидоров
4: Козлова
```

Существуют также методы `SkipWhile` и `TakeWhile`, которые пропускают или берут элементы не по количеству, а до тех пор, пока выполняется условие:

```
int[] numbers = { 1, 2, 3, 4, 5, 6, 7, 8 };
// Пропустить, пока < 4, затем брать, пока <= 7
var slice = numbers.SkipWhile(x => x < 4).TakeWhile(x => x <= 7);

Console.Write(" SkipWhile/TakeWhile: ");
foreach (var x in slice) Console.Write($"{x} ");
Console.WriteLine(); // 4 5 6 7
```

12.4.10 Декартово произведение

Конструкция с двумя операторами `from` формирует декартово произведение — все возможные пары элементов из двух коллекций:

```
Console.WriteLine("=== Декартово произведение (фрагмент) ===");
var q15 = from dep in departments
          from prj in projects
          select new { dep.Name, prj.Title };

foreach (var item in q15.Take(4))
    Console.WriteLine($" {item.Name} x {item.Title}");
Console.WriteLine($" ... всего пар: {q15.Count()}");
```

Результат:

```
Разработка x Сайт
Разработка x Мобильное приложение
Разработка x Аналитическая платформа
Тестирование x Сайт
... всего пар: 9
```

Если к декартову произведению добавить условие `where`, получается `Inner Join`, записанный альтернативным способом:

```
// Inner Join через декартово произведение + where
var q16 = from emp in employees
          from dep in departments
          where emp.DepartmentId == dep.Id
          select new { emp.Name, Department = dep.Name };
```

Этот вариант функционально эквивалентен конструкции `join ... on ... equals`, однако менее эффективен для больших коллекций, так как сначала формируются все пары, а затем отбрасываются несовпадения. Конструкция `join` использует внутреннюю хеш-таблицу и работает значительно быстрее.

12.5 LINQ to Objects: расширенные возможности

12.5.1 Отложенное и немедленное выполнение

Одна из ключевых особенностей LINQ, которая часто вызывает затруднения у начинающих, — это отложенное выполнение (`deferred execution`). Суть его в том, что LINQ-запрос при объявлении не выполняется. Запрос представляет собой лишь описание операций, которые будут выполнены позднее — в момент перебора результатов (например, в цикле `foreach`).

Рассмотрим пример:

```
Console.WriteLine("=== Отложенное выполнение ===");

var query = from emp in employees
             where emp.DepartmentId == 1
             select emp;

// В этот момент запрос ещё НЕ выполнен.
// Переменная query содержит не данные, а объект-итератор.
Console.WriteLine($"Тип query: {query.GetType().Name}");

// Запрос выполняется здесь – при переборе в foreach
Console.WriteLine("Первый перебор:");
foreach (var emp in query)
    Console.WriteLine($" {emp.Name}");
```

Результат:

```
Тип query: WhereSelectListIterator`2
Первый перебор:
    Иванов
    Петрова
    Козлова
```

Тип переменной `query` — не `List<Employee>`, а внутренний класс-итератор. Данные извлекаются из источника только при обращении к элементам результата.

Из отложенного выполнения следует важное свойство: если изменить источник данных после объявления запроса, повторный перебор выдаст обновлённые результаты:

```
// Добавляем нового сотрудника в отдел 1
employees.Add(new Employee
    { Id = 7, Name = "Новиков", DepartmentId = 1, Salary = 170000 });

// Тот же запрос — но результат изменился!
Console.WriteLine("Второй перебор (после добавления сотрудника):");
foreach (var emp in query)
    Console.WriteLine($" {emp.Name}");
```

Результат:

```
Второй перебор (после добавления сотрудника):
Иванов
Петрова
Козлова
Новиков
```

Переменная `query` не пересоздавалась — но результат изменился, потому что запрос каждый раз выполняется заново над текущим состоянием источника.

Такое поведение может быть как преимуществом (запрос всегда отражает актуальные данные), так и источником ошибок (непредсказуемые результаты, если источник изменяется в процессе работы программы). Поэтому в LINQ предусмотрен механизм немедленного выполнения (`immediate execution`) — принудительная материализация результатов запроса в конкретную коллекцию.

12.5.2 Материализация результатов

Методы материализации выполняют запрос немедленно и сохраняют результат в коллекцию определённого типа:

```
Console.WriteLine("=== Немедленное выполнение ===");

// Материализация в список
```

```

var list = (from emp in employees
            where emp.DepartmentId == 1
            select emp).ToList();

Console.WriteLine($"ToList → {list.GetType().Name}, элементов: {list.Count}");

// Материализация в массив
var array = (from emp in employees
            where emp.DepartmentId == 1
            select emp).ToArray();

Console.WriteLine($"ToArray → {array.GetType().Name}, элементов: {array.Length}");

```

Результат:

```

ToList → List`1, элементов: 4
ToArray → Employee[], элементов: 4

```

После вызова `ToList()` или `ToArray()` результат сохранён в отдельной коллекции и не зависит от последующих изменений источника. Добавление или удаление элементов в `employees` не повлияет на переменные `list` и `array`.

Метод `ToDictionary` преобразует результат в словарь, где ключ определяется переданной лямбда-функцией. Если в данных окажутся дублирующиеся ключи, метод сгенерирует исключение `ArgumentException`:

```

// Словарь: ключ – Id сотрудника, значение – объект Employee
var dict = employees.ToDictionary(emp => emp.Id);

Console.WriteLine($"ToDictionary → {dict.GetType().Name}");
Console.WriteLine($" dict[2] = {dict[2].Name}"); // Петрова

```

Метод `ToLookup` аналогичен `ToDictionary`, но допускает несколько элементов с одинаковым ключом. Результатом является объект типа `ILookup<TKey, TElement>` — структура, подобная словарю, в котором каждому ключу соответствует коллекция элементов:

```

// Группировка по отделу: ключ – DepartmentId,
// значение – коллекция сотрудников
var lookup = employees.ToLookup(emp => emp.DepartmentId);

Console.WriteLine($"ToLookup → {lookup.GetType().Name}");
foreach (var group in lookup)
{

```

```

Console.WriteLine($" Отдел {group.Key}:");
foreach (var emp in group)
    Console.WriteLine($" {emp.Name}");
}

```

Результат:

```

Отдел 1:
    Иванов
    Петрова
    Козлова
    Новиков
Отдел 2:
    Сидоров
    Морозова
Отдел 3:
    Николаев

```

Отличие ToLookup от ToDictionary можно сформулировать следующим образом: ToDictionary создаёт словарь «ключ → один элемент» и требует уникальности ключей, а ToLookup создаёт словарь «ключ → коллекция элементов» и допускает повторяющиеся ключи. По назначению ToLookup близок к группировке (group ... by), но результат материализован немедленно и доступен по ключу за O(1).

Помимо методов преобразования в коллекции, немедленное выполнение запроса вызывают все агрегатные функции (Count, Sum, Min, Max, Average) и методы получения отдельных элементов (First, FirstOrDefault, Single, ElementAt), рассмотренные далее.

12.5.3 Получение отдельных элементов

LINQ предоставляет несколько методов для извлечения одного элемента из результата запроса:

```

Console.WriteLine("=== Получение отдельных элементов ===");

var devDept = employees.Where(e => e.DepartmentId == 1);

// First – первый элемент, удовлетворяющий условию.
// Генерирует InvalidOperationException, если элемент не найден.
var first = devDept.First(e => e.Salary > 150000);
Console.WriteLine($"First(Salary > 150000): {first.Name}"); // Петрова

// FirstOrDefault – первый элемент или значение по умолчанию (null).
// Не генерирует исключение, если элемент не найден.

```

```

var notFound = devDept.FirstOrDefault(e => e.Salary > 500000);
Console.WriteLine($"FirstOrDefault(Salary > 500000): " +
    $"{(notFound is null ? "null" : notFound.Name)}");           // null

// ElementAt – элемент по индексу (нумерация с нуля).
// Генерирует ArgumentOutOfRangeException при выходе за границы.
var second = devDept.ElementAt(1);
Console.WriteLine($"ElementAt(1): {second.Name}");               // Петрова

```

Результат:

```

First(Salary > 150000): Петрова
FirstOrDefault(Salary > 500000): null
ElementAt(1): Петрова

```

Метод `Single` возвращает единственный элемент, удовлетворяющий условию, и генерирует исключение, если таких элементов ноль или больше одного. Это полезно для поиска по уникальному ключу:

```

// Single – ровно один элемент. Исключение, если 0 или > 1.
var emp3 = employees.Single(e => e.Id == 3);
Console.WriteLine($"Single(Id == 3): {emp3.Name}");             // Сидоров

// SingleOrDefault – ровно один элемент или null.
// Исключение, если > 1. Не генерирует исключение, если 0.
var emp99 = employees.SingleOrDefault(e => e.Id == 99);
Console.WriteLine($"SingleOrDefault(Id == 99): " +
    $"{(emp99 is null ? "null" : emp99.Name)}");                 // null

```

Выбор между этими методами определяется ожиданиями разработчика:

Метод	Если 0 элементов	Если 1 элемент	Если > 1 элемента
<code>First</code>	Исключение	Возвращает элемент	Возвращает первый
<code>FirstOrDefault</code>	Возвращает null / default	Возвращает элемент	Возвращает первый
<code>Single</code>	Исключение	Возвращает элемент	Исключение
<code>SingleOrDefault</code>	Возвращает null / default	Возвращает элемент	Исключение
<code>ElementAt</code>	Исключение (индекс за границами)	Возвращает элемент	—

Все перечисленные методы вызывают немедленное выполнение запроса, так как для получения конкретного значения необходимо обратиться к данным.

12.5.4 Конструкция let для промежуточных вычислений

Ключевое слово `let` в `query syntax` позволяет сохранить промежуточный результат вычисления в переменную, которую можно использовать далее в запросе. Это избавляет от повторных вычислений и повышает читаемость:

```
Console.WriteLine("=== Использование let ===");

var q17 = from emp in employees
          // Промежуточное вычисление: количество проектов сотрудника
          let projectCount = employeeProjects
            .Count(ep => ep.EmployeeId == emp.Id)
          where projectCount >= 2
          orderby projectCount descending
          select new { emp.Name, ProjectCount = projectCount };

foreach (var item in q17)
    Console.WriteLine($" {item.Name}: {item.ProjectCount} проектов");
```

Результат:

```
Козлова: 3 проектов
Иванов: 2 проектов
Петрова: 2 проектов
Морозова: 2 проектов
```

Без `let` выражение `employeeProjects.Count(ep => ep.EmployeeId == emp.Id)` пришлось бы повторить и в `where`, и в `orderby`, и в `select`, что привело бы к трёхкратному вычислению одного и того же значения.

Конструкция `let` может также использоваться для хранения вложенных подзапросов:

```
var q18 = from emp in employees
          let empProjects = from ep in employeeProjects
                            where ep.EmployeeId == emp.Id
                            join prj in projects on ep.ProjectId equals pr
                            j.Id
                            select prj
          where empProjects.Any(p => p.Budget > 1000000)
          select new { emp.Name, BigProjects = empProjects.Where(
            p => p.Budget > 1000000) };

Console.WriteLine("=== let с подзапросом ===");
foreach (var item in q18)
{
    Console.Write($" {item.Name}: ");
    Console.WriteLine(string.Join(", ",
```

```

        item.BigProjects.Select(p => p.Title)));
    }

```

Результат:

Петрова: Аналитическая платформа
 Козлова: Аналитическая платформа
 Николаев: Аналитическая платформа

В method syntax прямого аналога let нет. Аналогичного результата можно добиться промежуточным вызовом Select, создающим анонимный тип с дополнительным вычисленным полем:

```

var q17m = employees
    .Select(emp => new
    {
        emp.Name,
        ProjectCount = employeeProjects.Count(ep => ep.EmployeeId == emp.Id)
    })
    .Where(x => x.ProjectCount >= 2)
    .OrderByDescending(x => x.ProjectCount);

```

12.5.5 Генерация последовательностей

Статический класс Enumerable содержит методы для генерации последовательностей, которые удобны для формирования тестовых данных, числовых диапазонов и повторяющихся элементов:

```

Console.WriteLine("=== Генерация последовательностей ===");

// Range – арифметическая последовательность целых чисел
// Range(start, count): начинается с start, содержит count элементов
Console.Write(" Range(1, 5): ");
foreach (var x in Enumerable.Range(1, 5))
    Console.Write($"{x} ");
Console.WriteLine(); // 1 2 3 4 5

// Repeat – последовательность из одинаковых элементов
// Repeat(element, count): повторяет element count раз
Console.Write(" Repeat(0, 4): ");
foreach (var x in Enumerable.Repeat(0, 4))
    Console.Write($"{x} ");
Console.WriteLine(); // 0 0 0 0

```

Метод Range часто используется в сочетании с Select для генерации коллекций объектов:

```

// Генерация 5 тестовых сотрудников
var testEmployees = Enumerable.Range(1, 5)
    .Select(i => new Employee

```



```

{
    Id = 100 + i,
    Name = $"Тест_{i}",
    DepartmentId = 1,
    Salary = 100000 + i * 10000
})
.ToList();

Console.WriteLine(" Сгенерированные сотрудники:");
foreach (var emp in testEmployees)
    Console.WriteLine($" {emp.Id}: {emp.Name} – {emp.Salary:N0}");

```

Результат:

```

Сгенерированные сотрудники:
101: Тест_1 – 110 000
102: Тест_2 – 120 000
103: Тест_3 – 130 000
104: Тест_4 – 140 000
105: Тест_5 – 150 000

```

12.5.6 Агрегатная функция Aggregate

Помимо стандартных агрегатных функций (Count, Sum, Min, Max, Average), LINQ предоставляет универсальный метод Aggregate, позволяющий описать произвольную логику агрегирования. Метод принимает начальное значение аккумулятора и функцию, которая на каждом шаге обновляет аккумулятор с учётом очередного элемента:

```

Console.WriteLine("=== Aggregate ===");

// Конкатенация имён всех сотрудников через запятую
var allNames = employees.Aggregate(
    "", // начальное значение
    (accumulator, emp) => // функция агрегирования
        accumulator == ""
        ? emp.Name
        : accumulator + ", " + emp.Name
);

Console.WriteLine($" Все сотрудники: {allNames}");

```

Результат:

```

Все сотрудники: Иванов, Петрова, Сидоров, Козлова, Николаев, Морозова, Н
овиков

```

Метод Aggregate является наиболее гибким, но и наименее читаемым из агрегатных методов. В большинстве случаев предпочтительнее

использовать специализированные методы (Sum, Max и др.), а Aggregate применять только тогда, когда стандартные методы не подходят.

Следует отметить, что для конкатенации строк в реальном коде удобнее использовать string.Join:

```
var allNames2 = string.Join(", ", employees.Select(e => e.Name));
```

В функциональных языках такую функцию принято называть Reduce или «левый Fold».

12.6 Как устроен LINQ: провайдеры и деревья выражений

В предыдущих разделах все LINQ-запросы выполнялись над коллекциями в оперативной памяти. Однако в разделе 12.1 был показан пример, в котором практически тот же запрос адресовался к базе данных SQLite. Как LINQ обеспечивает единый синтаксис для принципиально разных источников данных? Ответ кроется в архитектуре провайдеров и механизме деревьев выражений.

12.6.1 Два интерфейса: IEnumerable и IQueryable

В пространстве имён System.Linq определены два статических класса с методами расширения, имена которых совпадают (Where, Select, OrderBy и т.д.), но работают они с разными интерфейсами:

Класс System.Linq.Enumerable содержит методы расширения для интерфейса IEnumerable<T>. Это те методы, которые использовались во всех примерах разделов 12.4–12.5. Их параметры — обычные делегаты Func<T, bool>:

```
// Определение метода Where в классе Enumerable (упрощённо):
public static IEnumerable<T> Where<T>(
    this IEnumerable<T> source,
    Func<T, bool> predicate);
```

При вызове такого метода лямбда-выражение компилируется в обычный IL-код (как любой делегат) и выполняется непосредственно в оперативной памяти, перебирая элементы коллекции один за другим.

Класс `System.Linq.Queryable` содержит методы расширения для интерфейса `IQueryable<T>`. Их параметры — не делегаты, а деревья выражений `Expression<Func<>>`:

```
// Определение метода Where в классе Queryable (упрощённо):
public static IQueryable<T> Where<T>(
    this IQueryable<T> source,
    Expression<Func<T, bool>> predicate);
```

Интерфейс `IQueryable<T>` наследуется от `IEnumerable<T>`, поэтому результат запроса к `IQueryable<T>` также можно перебирать в цикле `foreach`. Однако механизм выполнения принципиально отличается.

12.6.2 Делегаты и деревья выражений

Рассмотрим одно и то же лямбда-выражение, присвоенное переменным двух разных типов:

```
using System.Linq.Expressions;
```

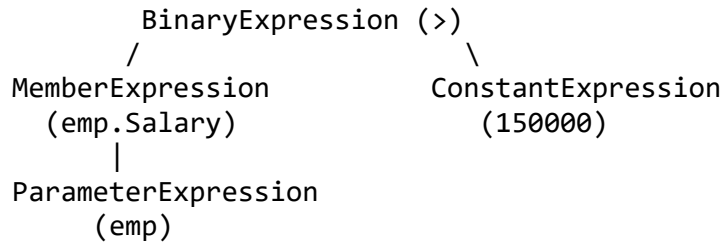
```
// 1. Делегат — скомпилированный исполняемый код
Func<Employee, bool> delegateExpr = emp => emp.Salary > 150000;
```

```
// 2. Дерево выражений — структура данных, описывающая код
Expression<Func<Employee, bool>> treeExpr = emp => emp.Salary > 150000;
```

Исходный код лямбда-выражения в обоих случаях одинаков: `emp => emp.Salary > 150000`. Однако компилятор обрабатывает их по-разному:

- Переменная `delegateExpr` содержит скомпилированный метод, который можно вызвать: `delegateExpr(someEmployee)` вернёт `true` или `false`. Это обычный делегат, рассмотренный в разделе 10.7.
- Переменная `treeExpr` содержит дерево объектов, описывающее структуру выражения: «обратиться к свойству `Salary` параметра `emp`, сравнить с константой `150000` оператором `>`». Это дерево можно анализировать программно, обходя его узлы.

Дерево выражений можно визуализировать следующим образом:



Каждый узел дерева — это объект одного из классов пространства имён `System.Linq.Expressions`: `BinaryExpression`, `MemberExpression`, `ConstantExpression`, `ParameterExpression` и др.

Убедиться в этом можно, выведя структуру дерева в консоль:

```

Console.WriteLine("=== Дерево выражений ===");
Console.WriteLine($"Тип узла: {treeExpr.NodeType}"); // Lambda
Console.WriteLine($"Тело: {treeExpr.Body}"); // (emp.Salary > 150000)
Console.WriteLine($"Тип тела: {treeExpr.Body.NodeType}"); // GreaterThan
Console.WriteLine($"Параметр: {treeExpr.Parameters[0]}"); // emp
Console.WriteLine($"Тип: {treeExpr.Body.GetType().Name}"); // BinaryExpression
  
```

Результат:

```

Тип узла: Lambda
Тело: (emp.Salary > 150000)
Тип тела: GreaterThan
Параметр: emp
Тип: BinaryExpression
  
```

Дерево выражений можно разобрать на составные части, извлекая левый и правый операнды:

```

if (treeExpr.Body is BinaryExpression binary)
{
    Console.WriteLine($"Левая часть: {binary.Left}"); // emp.Salary
    Console.WriteLine($"Оператор: {binary.NodeType}"); // GreaterThan
    Console.WriteLine($"Правая часть: {binary.Right}"); // 150000
}
  
```

Результат:

```

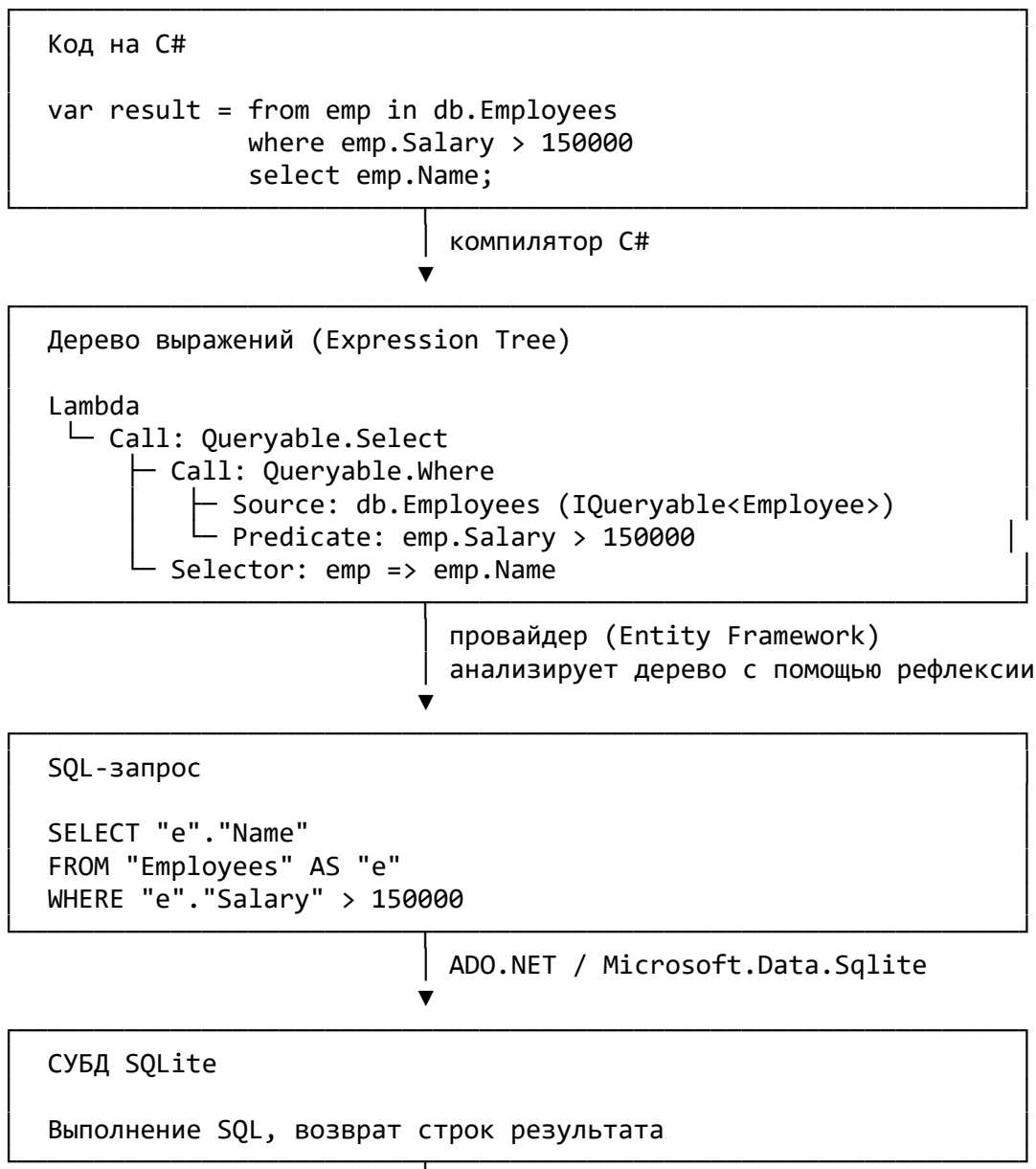
Левая часть: emp.Salary
Оператор: GreaterThan
Правая часть: 150000
  
```

Обратите внимание на связь с разделом 11 (Рефлексия): дерево выражений — это, по существу, представление фрагмента программы в виде структуры данных. В разделе 11.6 упоминалось понятие

самоотображаемости — способности языка анализировать собственный код как данные. Деревья выражений являются частичной реализацией этой идеи в C#: компилятор представляет лямбда-выражение не как исполняемый код, а как дерево объектов (по сути, AST — абстрактное синтаксическое дерево), которое можно обходить, анализировать и транслировать в другой язык.

12.6.3 12.6.3 Схема работы LINQ-провайдера

Теперь можно описать полную схему работы LINQ с внешним источником данных на примере Entity Framework и SQLite:



провайдер (Entity Framework)
материализация строк в объекты C#

```
IEnumerable<string>
```

```
"Петрова", "Козлова", "Иванов"
```

Процесс состоит из следующих шагов:

1. Компиляция. Компилятор C# видит, что `db.Employees` реализует `IQueryable<Employee>`, и вместо делегатов формирует деревья выражений для всех лямбда-выражений в запросе.
2. Трансляция. При обращении к результатам (например, в цикле `foreach`) LINQ-провайдер (в данном случае Entity Framework) получает дерево выражений и обходит его, анализируя узлы с помощью рефлексии. На основе этого анализа провайдер генерирует SQL-запрос, соответствующий исходному LINQ-выражению.
3. Выполнение. Сгенерированный SQL-запрос отправляется в СУБД (SQLite) через стандартный механизм ADO.NET — тот же `Microsoft.Data.Sqlite`, который использовался на семинаре 3.
4. Материализация. Строки, возвращённые СУБД, преобразуются обратно в объекты C# и предоставляются в виде `IEnumerable<T>`.

Таким образом, LINQ-провайдер выступает в роли транслятора: он принимает дерево выражений (представление C#-кода в виде структуры данных) и переводит его в язык, понятный целевому источнику данных.

12.6.4 Существующие LINQ-провайдеры

Архитектура провайдеров позволяет подключать различные источники данных через единый LINQ-синтаксис. Наиболее известные провайдеры:

Провайдер	Источник данных	Интерфейс	Механизм
LINQ to Objects	Коллекции в памяти	<code>IEnumerable<T></code>	Делегаты, прямое

Провайдер	Источник данных	Интерфейс	Механизм
LINQ to XML	XML-документы	<code>IEnumerable<T></code>	выполнение Делегаты (объекты <code>XElement</code> реализуют <code>IEnumerable</code>)
LINQ to JSON	JSON-документы	<code>IEnumerable<T></code>	Делегаты (<code>JToken</code> реализует <code>IEnumerable</code>)
Entity Framework	Реляционные СУБД	<code>IQueryable<T></code>	Деревья выражений → SQL
LINQ to SQL	SQL Server	<code>IQueryable<T></code>	Деревья выражений → T-SQL (устаревший, заменён EF)

Обратите внимание, что LINQ to XML и LINQ to JSON в этой таблице работают через `IEnumerable<T>`, а не через `IQueryable<T>`. Это означает, что они не транслируют запросы в другой язык, а выполняют их в памяти — как обычный LINQ to Objects. Их особенность заключается в удобных классах для работы с соответствующими форматами данных (`XElement`, `JObject`), которые реализуют `IEnumerable` и тем самым становятся доступными для LINQ-запросов.

Провайдеры, работающие через `IQueryable<T>` и деревья выражений, обладают важным практическим преимуществом: фильтрация, сортировка и другие операции выполняются на стороне источника данных (например, в СУБД), а не в оперативной памяти приложения. Это принципиально для работы с большими объёмами данных — из базы извлекаются только те строки, которые соответствуют условиям запроса.

12.6.5 Практическое следствие: ограничения `IQueryable`

Поскольку деревья выражений должны быть транслированы в целевой язык (например, SQL), не все конструкции C# допустимы в LINQ-запросах к `IQueryable<T>`. Провайдер может «не знать», как транслировать определённый метод C# в SQL.

Например, следующий запрос корректен для LINQ to Objects, но может вызвать ошибку при работе с Entity Framework:

```
// Пользовательский метод – работает в LINQ to Objects
static bool IsHighPaid(Employee emp) => emp.Salary > 150000;

// LINQ to Objects – работаем:
var r1 = employees.Where(e => IsHighPaid(e)).ToList();

// Entity Framework (IQueryable) – ошибка при выполнении!
// Провайдер не может транслировать вызов IsHighPaid в SQL.
// var r2 = db.Employees.Where(e => IsHighPaid(e)).ToList();
```

В подобных случаях необходимо либо переписать условие в виде простого выражения, понятного провайдеру, либо сначала загрузить данные в память (вызвав ToList() или AsEnumerable()), а затем применить фильтрацию средствами LINQ to Objects:

```
// Вариант 1: условие в виде простого выражения
// var r2 = db.Employees.Where(e => e.Salary > 150000).ToList();

// Вариант 2: загрузка в память, затем фильтрация
// var r2 = db.Employees.AsEnumerable().Where(e => IsHighPaid(e)).ToList()
;
```

Второй вариант загружает все строки таблицы в память, что может быть неэффективно для больших таблиц. Поэтому при работе с IQueryable<T> рекомендуется по возможности формулировать условия в виде простых выражений над свойствами сущностей.

12.7 LINQ для различных форматов данных

В разделе 12.6 было показано, что LINQ-провайдеры делятся на две категории: работающие через IEnumerable<T> (выполнение в памяти) и через IQueryable<T> (трансляция в целевой язык). Работа с форматами XML, JSON и YAML относится к первой категории: данные загружаются в объекты, реализующие IEnumerable<T>, после чего к ним применяется обычный LINQ to Objects. Преимущество заключается в удобных классах для работы с каждым форматом.

Во всех примерах данного раздела используется одна и та же предметная область — список сотрудников с информацией об отделах.

12.7.1 LINQ to XML

Пространство имён `System.Xml.Linq` входит в стандартную библиотеку `.NET` и не требует установки дополнительных пакетов. Ключевые классы — `XDocument`, `XElement`, `XAttribute`.

Создание XML-документа программно (functional construction):

```
using System.Xml.Linq;

var xml = new XDocument(
    new XElement("Company",
        new XElement("Employee",
            new XAttribute("Id", 1),
            new XElement("Name", "Иванов"),
            new XElement("Department", "Разработка"),
            new XElement("Salary", 150000)),
        new XElement("Employee",
            new XAttribute("Id", 2),
            new XElement("Name", "Петрова"),
            new XElement("Department", "Разработка"),
            new XElement("Salary", 180000)),
        new XElement("Employee",
            new XAttribute("Id", 3),
            new XElement("Name", "Сидоров"),
            new XElement("Department", "Тестирование"),
            new XElement("Salary", 120000))
    )
);

// Сохранение в файл и вывод в консоль
xml.Save("employees.xml");
Console.WriteLine(xml);
```

Результат:

```
<Company>
  <Employee Id="1">
    <Name>Иванов</Name>
    <Department>Разработка</Department>
    <Salary>150000</Salary>
  </Employee>
  <Employee Id="2">
    <Name>Петрова</Name>
    <Department>Разработка</Department>
    <Salary>180000</Salary>
  </Employee>
  <Employee Id="3">
```

```

    <Name>Сидоров</Name>
    <Department>Тестирование</Department>
    <Salary>120000</Salary>
  </Employee>
</Company>

```

LINQ-запрос к XML — метод `Descendants` возвращает `IEnumerable<XElement>`, к которому применяются стандартные LINQ-операции:

```

// Загрузка из файла: var xml = XDocument.Load("employees.xml");

Console.WriteLine("=== LINQ to XML: сотрудники отдела «Разработка» ===");

var xmlQuery = from emp in xml.Descendants("Employee")
               where (string)emp.Element("Department")! == "Разработка"
               orderby (int)emp.Element("Salary")! descending
               select new
               {
                 Name = (string)emp.Element("Name")!,
                 Salary = (int)emp.Element("Salary")!
               };

foreach (var item in xmlQuery)
    Console.WriteLine($" {item.Name} – {item.Salary:N0}");

```

Результат:

```

Петрова – 180 000
Иванов – 150 000

```

Обратите внимание, что для извлечения значений из XML-элементов используется явное приведение типов: `(string)emp.Element("Name")` и `(int)emp.Element("Salary")`. Классы `XElement` и `XAttribute` перегружают операторы приведения типов (раздел 10.5), что позволяет безопасно преобразовывать текстовое содержимое узлов в нужный тип.

12.7.2 LINQ to JSON

Для работы с JSON в .NET наиболее распространена библиотека `Newtonsoft.Json` (`Json.NET`), устанавливаемая через `NuGet`:

```
Install-Package Newtonsoft.Json
```

Классы `JObject`, `JArray`, `JToken` реализуют `IEnumerable<JToken>`, что позволяет применять LINQ-запросы к JSON-структурам:

```

using Newtonsoft.Json.Linq;

string json = @"
{
  ""employees"": [
    { ""id"": 1, ""name"": ""Иванов"", ""department"": ""Разработка"",
""salary"": 150000 },
    { ""id"": 2, ""name"": ""Петрова"", ""department"": ""Разработка"",
""salary"": 180000 },
    { ""id"": 3, ""name"": ""Сидоров"", ""department"": ""Тестирование"",
""salary"": 120000 }
  ]
}";

var doc = JObject.Parse(json);
// Также возможна загрузка из файла:
// var doc = JObject.Parse(File.ReadAllText("employees.json"));

Console.WriteLine("=== LINQ to JSON: сотрудники отдела «Разработка» ===");

var jsonQuery = from emp in doc["employees"]!
  where (string)emp["department"]! == "Разработка"
  orderby (int)emp["salary"]! descending
  select new
  {
    Name = (string)emp["name"]!,
    Salary = (int)emp["salary"]!
  };

foreach (var item in jsonQuery)
  Console.WriteLine($" {item.Name} – {item.Salary:N0}");

```

Результат:

```

Петрова – 180 000
Иванов – 150 000

```

Структура запроса практически идентична примеру с XML — различается только способ доступа к полям: `emp["name"]` вместо `emp.Element("Name")`.

В стандартной библиотеке .NET также существует пространство имён `System.Text.Json`, однако оно ориентировано на сериализацию/десериализацию объектов и не предоставляет LINQ-подобного API для навигации по JSON-документу. При использовании `System.Text.Json` типичный подход — десериализовать JSON в список объектов C# и применять к нему LINQ to Objects:

```
using System.Text.Json;

var emps = JsonSerializer.Deserialize<List<Employee>>(jsonString);
var result = emps!.Where(e => e.DepartmentId == 1)
    .OrderByDescending(e => e.Salary);
```

12.7.3 LINQ to YAML

Для работы с YAML в .NET используется библиотека YamlDotNet, устанавливаемая через NuGet:

Install-Package YamlDotNet

В отличие от XML и JSON, для YAML не существует отдельного LINQ-провайдера с классами, реализующими IEnumerable. Это связано с тем, что YAML чаще используется для конфигурационных файлов, а не для запросов к данным. Стандартный подход — десериализовать YAML в объекты C#, после чего применять обычный LINQ to Objects:

```
using YamlDotNet.Serialization;
using YamlDotNet.Serialization.NamingConventions;

string yaml = @"
employees:
  - id: 1
    name: Иванов
    department: Разработка
    salary: 150000
  - id: 2
    name: Петрова
    department: Разработка
    salary: 180000
  - id: 3
    name: Сидоров
    department: Тестирование
    salary: 120000
";

// Вспомогательные классы для десериализации
class YamlEmployee
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Department { get; set; } = "";
    public decimal Salary { get; set; }
}

class YamlRoot
{
    public List<YamlEmployee> Employees { get; set; } = new();
}
```

```

}

// Десериализация YAML → объекты C#
var deserializer = new DeserializerBuilder()
    .WithNamingConvention(CamelCaseNamingConvention.Instance)
    .Build();

var data = deserializer.Deserialize<YamlRoot>(yaml);

// Далее – обычный LINQ to Objects
Console.WriteLine("=== LINQ to YAML: сотрудники отдела «Разработка» ===");

var yamlQuery = from emp in data.Employees
    where emp.Department == "Разработка"
    orderby emp.Salary descending
    select new { emp.Name, emp.Salary };

foreach (var item in yamlQuery)
    Console.WriteLine($" {item.Name} – {item.Salary:N0}");

```

Результат:

Петрова – 180 000
Иванов – 150 000

12.7.4 Принцип обработки различных источников данных

Во всех трёх примерах результат одинаков, а LINQ-запросы различаются лишь способом доступа к данным. Это иллюстрирует центральную идею LINQ — единый язык запросов для различных источников:

Формат	Библиотека	Подход	Установка
XML	System.Xml.Linq	Классы XElement реализуют IEnumerable — LINQ-запросы применяются напрямую к дереву документа	Встроена в .NET
JSON	Newtonsoft.Json	Классы JToken реализуют IEnumerable — LINQ-запросы применяются к JSON-объектам	NuGet: Newtonsoft.Json
JSON	System.Text.Json	Десериализация в объекты C#, затем	Встроена в .NET

Формат	Библиотека	Подход	Установка
YAML	YamlDotNet	LINQ to Objects Десериализация в объекты C#, затем LINQ to Objects	NuGet: YamlDotNet

12.7.5 Другие LINQ-провайдеры и библиотеки

Архитектура LINQ является открытой — любой разработчик может создать провайдер для своего источника данных. За время существования технологии сообщество создало множество провайдеров, в том числе весьма неожиданных. Приведём обзор наиболее известных:

Провайдеры для форматов данных и файлов:

- LINQ to CSV — библиотеки CsvHelper и LinqToCsv позволяют читать CSV-файлы непосредственно в `IEnumerable<T>`, после чего к ним применяется LINQ. На семинаре 3 студенты реализовывали подобную функциональность вручную (парсинг CSV, операции Where, Projection, Join); библиотека CsvHelper решает ту же задачу в несколько строк кода.
- LINQ to Excel — библиотеки EPPlus, ClosedXML и LinqToExcel предоставляют доступ к строкам и ячейкам Excel-файлов как к коллекциям, к которым можно применять LINQ-запросы.

Провайдеры для баз данных (помимо Entity Framework):

- Dapper — micro-ORM, который не является полноценным LINQ-провайдером, но позволяет маппить результаты SQL-запросов на объекты C# с минимальными накладными расходами. Часто используется в сочетании с LINQ to Objects для пост-обработки результатов.
- LINQ to DB (linq2db) — легковесная альтернатива Entity Framework, транслирующая LINQ-запросы в SQL. Поддерживает SQLite и другие СУБД.

Провайдеры для параллельной и асинхронной обработки:

- PLINQ (Parallel LINQ) — встроен в .NET. Достаточно добавить .AsParallel() к запросу, чтобы операции Where, Select и другие выполнялись параллельно на нескольких ядрах процессора:

```
// Обычный LINQ
var result1 = employees.Where(e => e.Salary > 140000).ToList();

// Параллельный LINQ – автоматическое распределение по ядрам
var result2 = employees.AsParallel()
    .Where(e => e.Salary > 140000)
    .ToList();
```

12.8 ORM и Entity Framework Core

12.8.1 От ручного SQL к ORM: постановка проблемы

На семинаре 3 мы работали с базой данных SQLite, выполняя SQL-запросы через библиотеку Microsoft.Data.Sqlite. Рассмотрим фрагмент кода, загружающий сотрудников отдела «Разработка»:

```
using Microsoft.Data.Sqlite;

using var connection = new SqliteConnection("Data Source=company.db");
connection.Open();

var cmd = connection.CreateCommand();
cmd.CommandText = @"
    SELECT e.dev_id, e.dev_name, e.dev_commits
    FROM dev e
    INNER JOIN dep d ON e.dep_id = d.dep_id
    WHERE d.dep_name = @name
    ORDER BY e.dev_commits DESC;";
cmd.Parameters.AddWithValue("@name", "Разработка");

using var reader = cmd.ExecuteReader();

var result = new List<Employee>();
while (reader.Read())
{
    result.Add(new Employee
    {
        Id = reader.GetInt32(0),
        Name = reader.GetString(1),
        Salary = reader.GetDecimal(2)
    });
}
```

Этот код решает задачу, однако обладает рядом существенных недостатков:

1. SQL-запрос записан в виде строки. Компилятор C# не проверяет его синтаксис. Опечатка в имени таблицы или столбца обнаружится только при выполнении программы — возможно, уже в продуктивной среде.
2. Ручной маппинг результатов. Программист должен знать порядок столбцов в результате (`GetInt32(0)`, `GetString(1)`, `GetDecimal(2)`) и правильно сопоставить их со свойствами класса. При изменении SQL-запроса (например, добавлении столбца) необходимо синхронно изменить код маппинга.
3. Дублирование структуры данных. Структура таблицы описана в SQL-скрипте (`CREATE TABLE`), а структура объекта — в классе C#. Программист вынужден поддерживать согласованность двух описаний вручную.
4. Повторяющийся шаблонный код. Для каждой операции (чтение, вставка, обновление, удаление) необходимо создавать команду, заполнять параметры, обрабатывать результат — десятки строк типового кода.

Перечисленные проблемы становятся критичными в крупных проектах с десятками таблиц и сотнями запросов. Для их решения была создана концепция ORM.

12.8.2 Понятие ORM

ORM (Object-Relational Mapping) — это технология автоматического отображения объектов языка программирования на таблицы реляционной базы данных и обратно. ORM берёт на себя:

- Генерацию SQL-запросов по LINQ-выражениям или вызовам методов — программист пишет код на C#, а ORM транслирует его в SQL.

- Маппинг результатов — строки, возвращённые СУБД, автоматически преобразуются в объекты C#.
- Управление схемой базы данных — структура таблиц создаётся или обновляется на основе классов C# (миграции).
- Отслеживание изменений — ORM запоминает, какие объекты были изменены, и при сохранении генерирует только необходимые UPDATE- и INSERT-команды.

ORM-технологии существуют для большинства языков программирования. В Java наиболее известен Hibernate, в Python — SQLAlchemy и Django ORM. В .NET стандартным ORM является Entity Framework Core (EF Core).

Можно провести аналогию с семинаром 3: на семинаре мы реализовали простейший прототип реляционной СУБД на чистом C# (операции Projection, Where, Join над CSV-таблицами), а ORM — это промышленная реализация подобного «моста» между объектами C# и таблицами СУБД.

12.8.3 Установка Entity Framework Core для SQLite

Для работы с EF Core и SQLite необходимо установить два NuGet-пакета. В консоли диспетчера пакетов (Tools → NuGet Package Manager → Package Manager Console):

```
Install-Package Microsoft.EntityFrameworkCore.Sqlite
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Первый пакет содержит провайдер EF Core для SQLite. Второй — инструменты для работы с миграциями (создание и обновление схемы базы данных).

12.8.4 Классы модели данных

В EF Core таблицы базы данных описываются обычными классами C#, которые принято называть моделями или сущностями (entities). Каждый класс соответствует таблице, каждое свойство — столбцу.

Определим модели для нашей предметной области:

```

/// <summary>
/// Отдел – соответствует таблице Departments
/// </summary>
class Department
{
    public int Id { get; set; }
    public string Name { get; set; } = "";

    /// <summary>
    /// Навигационное свойство: коллекция сотрудников отдела.
    /// Позволяет обращаться к сотрудникам через объект отдела
    /// без явного написания join.
    /// </summary>
    public List<Employee> Employees { get; set; } = new();
}

/// <summary>
/// Сотрудник – соответствует таблице Employees
/// </summary>
class Employee
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Salary { get; set; }

    /// <summary>
    /// Внешний ключ – ссылка на отдел
    /// </summary>
    public int DepartmentId { get; set; }

    /// <summary>
    /// Навигационное свойство: объект отдела.
    /// EF Core автоматически заполняет его при загрузке данных.
    /// </summary>
    public Department Department { get; set; } = null!;

    /// <summary>
    /// Навигационное свойство для связи «много-ко-многим»:
    /// список связей с проектами
    /// </summary>
    public List<EmployeeProject> EmployeeProjects { get; set; } = new();
}

/// <summary>

```

```

/// Проект — соответствует таблице Projects
/// </summary>
class Project
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public decimal Budget { get; set; }

    /// <summary>
    /// Навигационное свойство для связи «много-ко-многом»
    /// </summary>
    public List<EmployeeProject> EmployeeProjects { get; set; } = new();
}

/// <summary>
/// Связующая таблица для связи «много-ко-многом»
/// между сотрудниками и проектами
/// </summary>
class EmployeeProject
{
    public int EmployeeId { get; set; }
    public Employee Employee { get; set; } = null!;

    public int ProjectId { get; set; }
    public Project Project { get; set; } = null!;
}

```

Обратите внимание на навигационные свойства — свойства, тип которых является другой сущностью или коллекцией сущностей (Department, List<Employee>, List<EmployeeProject>). Навигационные свойства не существуют в базе данных как столбцы — они существуют только в объектах C# и автоматически заполняются средствами EF Core. Именно они позволяют обращаться к связанным данным без явного написания join.

Свойство public Department Department { get; set; } = null!; инициализировано значением null! (null-forgiving operator, рассмотренный на семинаре 4). Это сообщает компилятору, что свойство не будет null в момент использования — EF Core заполнит его при загрузке данных из базы.

12.8.5 Контекст базы данных (DbContext)

DbContext — центральный класс EF Core, представляющий сеанс работы с базой данных. Он содержит свойства DbSet<T> для каждой таблицы и методы для настройки модели:

```
using Microsoft.EntityFrameworkCore;

class CompanyContext : DbContext
{
    /// <summary>
    /// Таблицы базы данных, представленные как DbSet.
    /// К ним применяются LINQ-запросы.
    /// </summary>
    public DbSet<Department> Departments { get; set; }
    public DbSet<Employee> Employees { get; set; }
    public DbSet<Project> Projects { get; set; }
    public DbSet<EmployeeProject> EmployeeProjects { get; set; }

    /// <summary>
    /// Настройка подключения к базе данных
    /// </summary>
    protected override void OnConfiguring(
        DbContextOptionsBuilder options)
    {
        // Файл базы данных SQLite будет создан в текущем каталоге
        options.UseSqlite("Data Source=company.db");

        // Включаем логирование SQL-запросов в консоль,
        // чтобы видеть, какой SQL генерирует EF Core
        options.LogTo(Console.WriteLine,
            Microsoft.Extensions.Logging.LogLevel.Information);
    }

    /// <summary>
    /// Настройка модели данных: связи, ключи, ограничения
    /// </summary>
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Настройка связующей таблицы для связи «много-ко-многом»
        // Составной первичный ключ из двух полей
        modelBuilder.Entity<EmployeeProject>()
            .HasKey(ep => new { ep.EmployeeId, ep.ProjectId });

        // Связь EmployeeProject → Employee
        modelBuilder.Entity<EmployeeProject>()
            .HasOne(ep => ep.Employee)
            .WithMany(e => e.EmployeeProjects)
            .HasForeignKey(ep => ep.EmployeeId);

        // Связь EmployeeProject → Project
        modelBuilder.Entity<EmployeeProject>()
```

```

        .HasOne(ep => ep.Project)
        .WithMany(p => p.EmployeeProjects)
        .HasForeignKey(ep => ep.ProjectId);
    }
}

```

Метод `OnConfiguring` задаёт строку подключения — в нашем случае это файл `company.db` в формате SQLite. Метод `OnModelCreating` позволяет тонко настроить модель данных. В данном примере настраивается связь «много-ко-многим»: составной первичный ключ связующей таблицы и внешние ключи.

Обратите внимание, что для связей «один-ко-многим» (`Department` ↔ `Employee`) явная настройка в `OnModelCreating` не требуется — EF Core автоматически распознаёт связь по соглашению об именовании: свойство `DepartmentId` в классе `Employee` совпадает с именем первичного ключа `Id` класса `Department` с добавлением имени класса.

12.8.6 Создание базы данных и заполнение данными

Для создания базы данных по описанной модели используется метод `EnsureCreated`, который создаёт базу данных и все таблицы, если они ещё не существуют:

```

using var db = new CompanyContext();

// Удаление и пересоздание базы данных (для демонстрации)
db.Database.EnsureDeleted();
db.Database.EnsureCreated();

Console.WriteLine("[OK] База данных создана");

```

Для заполнения данных объекты C# добавляются в `DbSet` с помощью метода `Add` или `AddRange`, а затем изменения сохраняются в базу вызовом `SaveChanges`:

```

// — Отделы
var dev = new Department { Name = "Разработка" };
var test = new Department { Name = "Тестирование" };
var analytics = new Department { Name = "Аналитика" };

db.Departments.AddRange(dev, test, analytics);
db.SaveChanges(); // INSERT INTO Departments ...

```

```
// — Сотрудники
var ivanov = new Employee
    { Name = "Иванов", DepartmentId = dev.Id, Salary = 150000 };
var petrova = new Employee
    { Name = "Петрова", DepartmentId = dev.Id, Salary = 180000 };
var sidorov = new Employee
    { Name = "Сидоров", DepartmentId = test.Id, Salary = 120000 };
var kozlova = new Employee
    { Name = "Козлова", DepartmentId = dev.Id, Salary = 160000 };
var nikolaev = new Employee
    { Name = "Николаев", DepartmentId = analytics.Id, Salary = 140000 };
var morozova = new Employee
    { Name = "Морозова", DepartmentId = test.Id, Salary = 130000 };

db.Employees.AddRange(
    ivanov, petrova, sidorov, kozlova, nikolaev, morozova);
db.SaveChanges();

// — Проекты
var site = new Project { Title = "Сайт", Budget = 500000 };
var app = new Project { Title = "Мобильное приложение", Budget = 800000 };
;
var plat = new Project { Title = "Аналитическая платформа", Budget = 120000 };

db.Projects.AddRange(site, app, plat);
db.SaveChanges();

// — Связи «сотрудник ↔ проект»
db.EmployeeProjects.AddRange(
    new EmployeeProject { EmployeeId = ivanov.Id, ProjectId = site.Id },
    new EmployeeProject { EmployeeId = ivanov.Id, ProjectId = app.Id },
    new EmployeeProject { EmployeeId = petrova.Id, ProjectId = app.Id },
    new EmployeeProject { EmployeeId = petrova.Id, ProjectId = plat.Id },
    new EmployeeProject { EmployeeId = sidorov.Id, ProjectId = site.Id },
    new EmployeeProject { EmployeeId = kozlova.Id, ProjectId = site.Id },
    new EmployeeProject { EmployeeId = kozlova.Id, ProjectId = app.Id },
    new EmployeeProject { EmployeeId = kozlova.Id, ProjectId = plat.Id },
    new EmployeeProject { EmployeeId = nikolaev.Id, ProjectId = plat.Id },
    new EmployeeProject { EmployeeId = morozova.Id, ProjectId = site.Id },
    new EmployeeProject { EmployeeId = morozova.Id, ProjectId = app.Id }
);
db.SaveChanges();

Console.WriteLine("[OK] Данные загружены");
```

Обратите внимание, что при добавлении сотрудников мы используем `dev.Id`, а не числовое значение. После вызова `SaveChanges()` для отделов EF Core автоматически заполняет свойство `Id` значениями, сгенерированными СУБД (автоинкремент).

12.8.7 LINQ-запросы через Entity Framework Core

Свойства `DbSet<T>` контекста реализуют интерфейс `IQueryable<T>`. Это означает, что к ним применяются LINQ-запросы, которые EF Core транслирует в SQL (как описано в разделе 12.6).

Простая выборка с фильтрацией:

```
Console.WriteLine("=== Сотрудники с зарплатой > 140 000 ===");

var q1 = from emp in db.Employees
        where emp.Salary > 140000
        orderby emp.Salary descending
        select emp;

foreach (var emp in q1)
    Console.WriteLine($" {emp.Name} — {emp.Salary:N0}");
```

При выполнении этого кода EF Core сгенерирует и отправит в SQLite следующий SQL-запрос (видимый в консоли благодаря `LogTo`):

```
SELECT "e"."Id", "e"."DepartmentId", "e"."Name", "e"."Salary"
FROM "Employees" AS "e"
WHERE "e"."Salary" > 140000
ORDER BY "e"."Salary" DESC
```

Сравните этот SQL с ручным SQL-запросом из раздела 12.8.1 — EF Core сгенерировал его автоматически, на основе LINQ-выражения.

Одно из главных преимуществ EF Core — возможность обращаться к связанным данным через навигационные свойства, без явного написания `join`. Метод `Include` указывает EF Core загрузить связанные данные вместе с основной сущностью (жадная загрузка, *eager loading*):

```
Console.WriteLine("=== Сотрудники с отделами (Include) ===");

var q2 = db.Employees
        .Include(emp => emp.Department) // загрузить связанный отдел
        .Where(emp => emp.Department.Name == "Разработка")
        .OrderByDescending(emp => emp.Salary);

foreach (var emp in q2)
    Console.WriteLine(
        $" {emp.Name} — {emp.Department.Name} — {emp.Salary:N0}");
```

Результат:

Петрова — Разработка — 180 000
 Козлова — Разработка — 160 000
 Иванов — Разработка — 150 000

EF Core сгенерирует SQL с INNER JOIN:

```
SELECT "e"."Id", "e"."DepartmentId", "e"."Name", "e"."Salary",
       "d"."Id", "d"."Name"
FROM "Employees" AS "e"
INNER JOIN "Departments" AS "d" ON "e"."DepartmentId" = "d"."Id"
WHERE "d"."Name" = 'Разработка'
ORDER BY "e"."Salary" DESC
```

Без вызова Include навигационное свойство Department не будет загружено и останется равным null. Это поведение называется отложенной загрузкой и позволяет избежать загрузки ненужных данных.

Соединение через явный join также возможно — синтаксис полностью совпадает с LINQ to Objects:

```
var q2b = from emp in db.Employees
          join dep in db.Departments on emp.DepartmentId equals dep.Id
          where dep.Name == "Разработка"
          orderby emp.Salary descending
          select new { emp.Name, Department = dep.Name, emp.Salary };
```

Результат будет тем же, однако в большинстве случаев навигационные свойства с Include предпочтительнее: они делают код короче и не требуют запоминать имена ключевых полей.

Группировка с агрегатными функциями:

```
Console.WriteLine("=== Средняя зарплата по отделам ===");

var q3 = from emp in db.Employees
          group emp by emp.DepartmentId into g
          join dep in db.Departments on g.Key equals dep.Id
          select new
          {
              Department = dep.Name,
              AvgSalary = g.Average(e => e.Salary),
              Count = g.Count()
          };

foreach (var item in q3)
    Console.WriteLine(
        $" {item.Department}: средняя {item.AvgSalary:N0}, " +
        $"сотрудников {item.Count}");
```

EF Core сгенерирует:


```
SELECT "d"."Name", AVG("e"."Salary"), COUNT(*)
FROM "Employees" AS "e"
INNER JOIN "Departments" AS "d" ON "e"."DepartmentId" = "d"."Id"
GROUP BY "e"."DepartmentId"
```

Связь «МНОГО-КО-МНОГИМ»:

```
Console.WriteLine("=== Проекты каждого сотрудника ===");

var q4 = db.Employees
    .Include(e => e.EmployeeProjects)
    .ThenInclude(ep => ep.Project) // вложенный Include
    .OrderBy(e => e.Name);

foreach (var emp in q4)
{
    var projectNames = emp.EmployeeProjects
        .Select(ep => ep.Project.Title);
    Console.WriteLine(
        $" {emp.Name}: {string.Join(", ", projectNames)}");
}
```

Результат:

```
Иванов: Сайт, Мобильное приложение
Козлова: Сайт, Мобильное приложение, Аналитическая платформа
Морозова: Сайт, Мобильное приложение
Николаев: Аналитическая платформа
Петрова: Мобильное приложение, Аналитическая платформа
Сидоров: Сайт
```

Метод `ThenInclude` используется для загрузки навигационных свойств второго уровня вложенности: сначала загружаются связи `EmployeeProject`, затем для каждой связи загружается объект `Project`.

Найти сотрудников, участвующих в проекте с бюджетом более 1 000 000:

```
Console.WriteLine("=== Сотрудники в крупных проектах ===");

var q5 = db.Employees
    .Where(e => e.EmployeeProjects
        .Any(ep => ep.Project.Budget > 1000000))
    .Select(e => e.Name);

foreach (var name in q5)
    Console.WriteLine($" {name}");
```

Результат:

```
Петрова
Козлова
Николаев
```

Обратите внимание, что в этом запросе Include не требуется — EF Core анализирует дерево выражений и генерирует SQL с подзапросом EXISTS, не загружая связанные объекты в память:

```
SELECT "e"."Name"
FROM "Employees" AS "e"
WHERE EXISTS (
    SELECT 1 FROM "EmployeeProjects" AS "ep"
    INNER JOIN "Projects" AS "p" ON "ep"."ProjectId" = "p"."Id"
    WHERE "e"."Id" = "ep"."EmployeeId" AND "p"."Budget" > 1000000
)
```

12.8.8 Сравнение подходов

Для наглядности сравним три подхода к решению одной задачи — «найти сотрудников отдела Разработка»:

Ручной SQL (семинар 3):

```
var cmd = connection.CreateCommand();
cmd.CommandText = @"SELECT e.dev_name, e.dev_commits
FROM dev e INNER JOIN dep d ON e.dep_id = d.dep_id
WHERE d.dep_name = @name ORDER BY e.dev_commits DESC;";
cmd.Parameters.AddWithValue("@name", "Разработка");
using var reader = cmd.ExecuteReader();
while (reader.Read())
    Console.WriteLine($" {reader.GetString(0)} – {reader.GetInt32(1)}");
```

LINQ to Objects (списки в памяти, разделы 12.4–12.5):

```
var result = from emp in employees
join dep in departments on emp.DepartmentId equals dep.Id
where dep.Name == "Разработка"
orderby emp.Salary descending
select new { emp.Name, emp.Salary };
```

Entity Framework Core (база данных SQLite):

```
var result = db.Employees
.Include(e => e.Department)
.Where(e => e.Department.Name == "Разработка")
.OrderByDescending(e => e.Salary)
.Select(e => new { e.Name, e.Salary });
```

Критерий	Ручной SQL	LINQ to Objects	EF Core
Проверка компилятором	Нет (строка)	Да	Да
Маппинг результатов	Ручной	Автоматический	Автоматический
Источник данных	СУБД	Память	СУБД

Критерий	Ручной SQL	LINQ to Objects	EF Core
Управление схемой	Ручное (CREATE TABLE)	—	Автоматическое
Производительность	Максимальная	—	Незначительные накладные расходы

12.9 Типичные ошибки и рекомендации

12.9.1 Многократное перечисление IEnumerable

Распространённая ошибка — многократный перебор отложенного LINQ-запроса. Каждый перебор выполняет запрос заново:

```
// ПЛОХО: запрос выполняется трижды
var query = employees.Where(e => e.Salary > 100000);
Console.WriteLine($"Количество: {query.Count()}"); // 1-й перебор
Console.WriteLine($"Первый: {query.First().Name}"); // 2-й перебор
foreach (var emp in query) { /* ... */ } // 3-й перебор

// ХОРОШО: запрос выполняется один раз, результат сохранён
var list = employees.Where(e => e.Salary > 100000).ToList();
Console.WriteLine($"Количество: {list.Count}");
Console.WriteLine($"Первый: {list[0].Name}");
foreach (var emp in list) { /* ... */ }
```

Для LINQ to Objects это приводит к снижению производительности. Для EF Core — к многократным обращениям к базе данных.

Общая рекомендация: если результат запроса будет использоваться более одного раза, его следует материализовать вызовом `ToList()` или `ToArray()`.

12.9.2 Загрузка всей таблицы вместо фильтрации на сервере

При работе с EF Core важно, чтобы фильтрация выполнялась на стороне СУБД, а не в памяти приложения:

```
// ПЛОХО: загружает ВСЮ таблицу в память, затем фильтрует
var result = db.Employees.ToList()
    .Where(e => e.Salary > 150000);

// ХОРОШО: фильтрация выполняется в СУБД,
// загружаются только подходящие строки
var result = db.Employees
```

```
.Where(e => e.Salary > 150000)
.ToList();
```

Разница — в порядке вызовов: `ToList()` должен стоять после всех фильтров. Вызов `ToList()` (или `AsEnumerable()`) переключает последующие операции с `IQueryable` на `IEnumerable`, то есть с выполнения в СУБД на выполнение в памяти.

12.9.3 Забытый Include в EF Core

Если навигационное свойство не загружено через `Include`, обращение к нему вернёт `null` или пустую коллекцию:

```
// ПЛОХО: Department не загружен – NullReferenceException
var emp = db.Employees.First();
Console.WriteLine(emp.Department.Name); // Ошибка!

// ХОРОШО: явная загрузка связанных данных
var emp = db.Employees
    .Include(e => e.Department)
    .First();
Console.WriteLine(emp.Department.Name); // Работает
```

12.9.4 Выбор между query syntax и method syntax

В качестве практической рекомендации: используйте `query syntax` для запросов с несколькими `join`, конструкцией `let` и сложной группировкой — он читается нагляднее. Используйте `method syntax` для простых цепочек `.Where().Select().OrderBy()` и для операций, не имеющих эквивалента в `query syntax` (`Distinct`, `Skip`, `Take`, `Count`, `Any`, `All`). Комбинирование обоих синтаксисов в одном выражении допустимо и часто используется на практике:

```
// Комбинирование: query syntax обернут в вызов метода Count
var count = (from emp in employees
    join dep in departments on emp.DepartmentId equals dep.Id
    where dep.Name == "Разработка"
    select emp).Count();
```

12.10 Итоги

В данном разделе была рассмотрена технология LINQ — механизм языка C#, позволяющий писать запросы к данным в едином синтаксисе независимо от источника данных. Были изучены следующие темы:

- LINQ to Objects — запросы к коллекциям в оперативной памяти: фильтрация, проекция, сортировка, соединения, группировка, агрегаты, операции над множествами, связь «много-ко-многим», отложенное и немедленное выполнение.
- Архитектура провайдеров — два интерфейса (IEnumerable<T> и IQueryable<T>), деревья выражений (Expression<Func<>>), трансляция LINQ-запросов в целевой язык через рефлексиию.
- LINQ для различных форматов — применение LINQ к XML-документам, JSON-файлам и YAML-конфигурациям; обзор существующих провайдеров.
- ORM и Entity Framework Core — понятие объектно-реляционного отображения, настройка модели данных, навигационные свойства, LINQ-запросы к базе данных SQLite, сравнение с ручным SQL.
- LINQ является одной из наиболее мощных и практически значимых технологий языка C#. Вместе с тем он построен на фундаменте уже изученных механизмов: методов расширения, лямбда-выражений, обобщённых делегатов, текущего интерфейса и рефлексии.

12.11 Задание для закрепления материала (LINQ to Objects)

Все задания выполняются без использования базы данных — только LINQ to Objects. Используйте предметную область из раздела 12.3 (сотрудники, отделы, проекты, связи «сотрудник ↔ проект»). При желании расширьте набор данных дополнительными записями.

Уровень 1. Базовые операции:

1. Выведите имена всех сотрудников с зарплатой от 130 000 до 160 000 включительно, отсортированных по алфавиту.
2. Выведите список уникальных отделов (только названия), в которых есть хотя бы один сотрудник с зарплатой выше 150 000.
3. Для каждого отдела выведите название отдела и количество сотрудников в нём.

Уровень 2. Соединения и группировка:

4. Выведите список сотрудников в формате «Имя — Название отдела — Зарплата», отсортированный по названию отдела, затем по убыванию зарплаты.
5. Для каждого отдела вычислите суммарный фонд оплаты труда и среднюю зарплату. Выведите только отделы, где средняя зарплата превышает 130 000.
6. Выведите отделы, в которых все сотрудники получают более 100 000 (используйте All).

Уровень 3. Связь «много-ко-многим»:

7. Для каждого сотрудника выведите список проектов, в которых он участвует.
8. Для каждого проекта выведите количество участников и их среднюю зарплату.
9. Найдите сотрудников, которые участвуют более чем в одном проекте, и выведите их имена вместе с количеством проектов.
10. (Повышенная сложность) Найдите пары сотрудников, которые работают хотя бы над одним общим проектом. Выведите пары без дублирования (если выведена пара «Иванов — Петрова», пара «Петрова — Иванов» выводиться не должна).

13 Разработка пользовательских интерфейсов

13.1 Паттерны пользовательского интерфейса: MVC, MVP, MVVM

Разработка пользовательского интерфейса — это не только расстановка кнопок и текстовых полей. Главный вопрос архитектуры UI: как организовать взаимодействие между данными, логикой и отображением так, чтобы код оставался понятным, тестируемым и расширяемым.

Ответом на этот вопрос служат архитектурные паттерны UI. Все три рассматриваемых паттерна разделяют систему на три части, но делают это по-разному.

13.1.1 Паттерн MVC (Model — View — Controller)

MVC — старейший из рассматриваемых паттернов, предложенный в 1970-х годах в среде Smalltalk. Сегодня он лежит в основе большинства серверных веб-фреймворков.

Три роли:

Model — данные и бизнес-логика. Не знает ни о View, ни о Controller.

View — отображение данных. Получает данные от Model, генерирует визуальное представление.

Controller — обрабатывает входящий запрос пользователя, обращается к Model, выбирает View для ответа.

Поток управления в веб-MVC:

Браузер → HTTP-запрос → Controller → обращение к Model → выбор View → HTML-ответ → Браузер

Ключевая особенность: Controller не обновляет View напрямую — он передаёт данные, а View сам формирует представление. В классическом

веб-MVC (ASP.NET Core MVC) View — это шаблон на стороне сервера (Razor), который рендерится в HTML.

13.1.2 Паттерн MVP (Model — View — Presenter)

MVP появился как эволюция MVC для настольных приложений, где View имеет прямой доступ к элементам управления. Ключевое отличие: Presenter полностью управляет View через интерфейс.

Три роли:

Model — данные и бизнес-логика (как в MVC).

View — пассивный элемент: не содержит логики, только отображает данные и делегирует все события Presenter. Реализует интерфейс IView.

Presenter — посредник: подписывается на события View, обращается к Model, обновляет View через его интерфейс.

Поток управления:

Пользователь → событие View → Presenter → обращение к Model → обновление View через IView

Ключевая особенность: View и Presenter связаны через интерфейс — это означает, что Presenter можно протестировать с mock-объектом View, не запуская оконное приложение. MVP часто применяется в Windows Forms, Android, а также устаревших ASP.NET WebForms.

13.1.3 Паттерн MVVM (Model — View — ViewModel)

MVVM разработан Microsoft в 2005 году специально для WPF. Его отличительная черта — механизм привязки данных (data binding), который позволяет View автоматически отображать изменения ViewModel без единой строки «соединяющего» кода в code-behind.

Три роли:

Model — данные и бизнес-логика (как в MVC и MVP).

View — декларативная разметка (XAML). Не содержит логики. Привязывается к ViewModel через data binding.

ViewModel — адаптер между Model и View: подготавливает данные для отображения, реализует команды, уведомляет View об изменениях через INotifyPropertyChanged.

Поток управления:

Пользователь → событие View (через Binding) → Command в ViewModel → обращение к Model

Model изменилась → ViewModel.PropertyChanged → Binding автоматически обновляет View

Ключевая особенность: View и ViewModel связаны только через соглашение о именах и интерфейсы (INotifyPropertyChanged, ICommand). ViewModel не содержит ссылок на элементы UI — это означает полную тестируемость бизнес-логики без запуска интерфейса.

13.2 Основы разработки пользовательского интерфейса с использованием технологии Windows Forms

Windows Forms (WinForms) — технология разработки настольных приложений, входящая в состав .NET с 2002 года. Несмотря на появление более современных технологий (WPF, MAUI), Windows Forms по-прежнему широко используется в корпоративных системах и является отличной отправной точкой для понимания событийно-ориентированного программирования UI.

В контексте паттернов UI, рассмотренных в разделе 15.1, Windows Forms наиболее естественно сочетается с паттерном MVP: форма реализует пассивное View, а отдельный класс Presenter содержит логику. Однако в учебных примерах логика часто помещается непосредственно в код формы, что является антипаттерном.

13.2.1 Создание проекта

Для создания оконного приложения необходимо создать в Visual Studio новый проект типа «приложение Windows Forms» (рис. 40).

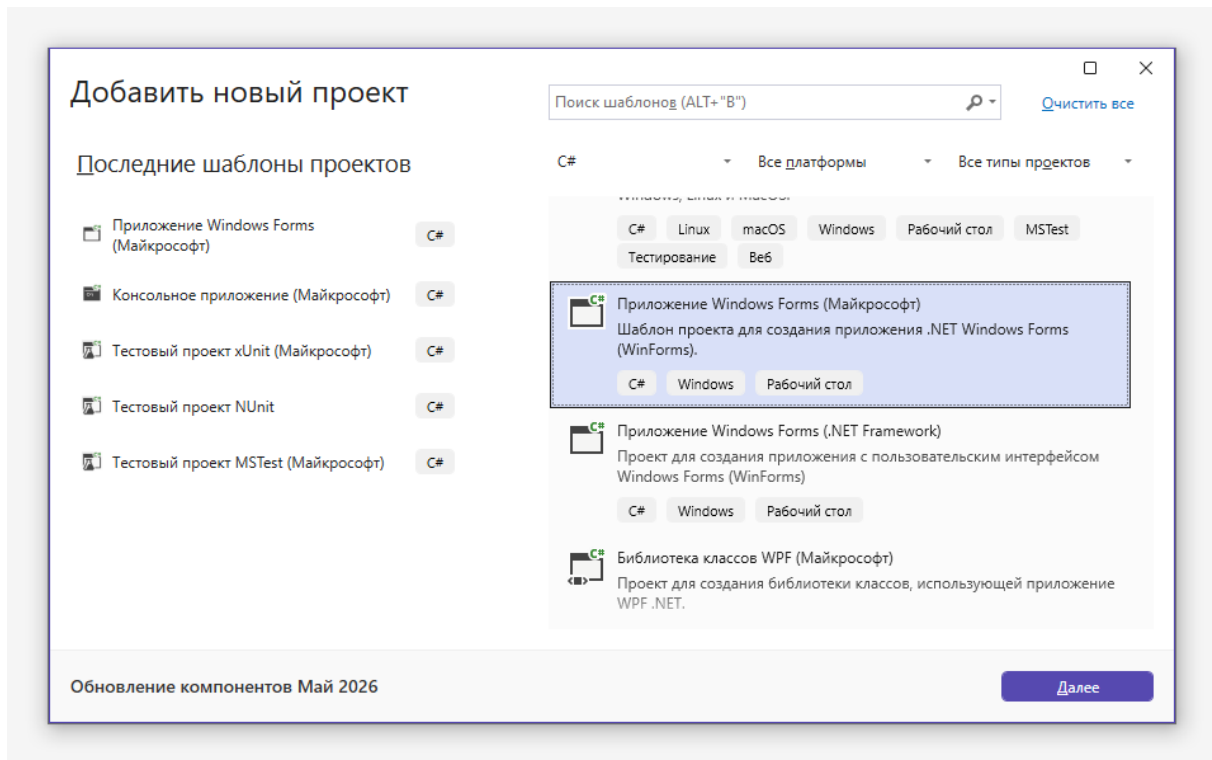


Рис. 40. Создание приложения Windows Forms.

После этого открывается не окно кода, как было ранее в случае консольных приложений, а редактор макета формы, и автоматически создается форма Form1.

При нажатии на вкладку «Панель элементов», расположенную в левой части окна, открывается панель, содержащая список элементов, которые можно перетащить на форму мышью (рис. 41).

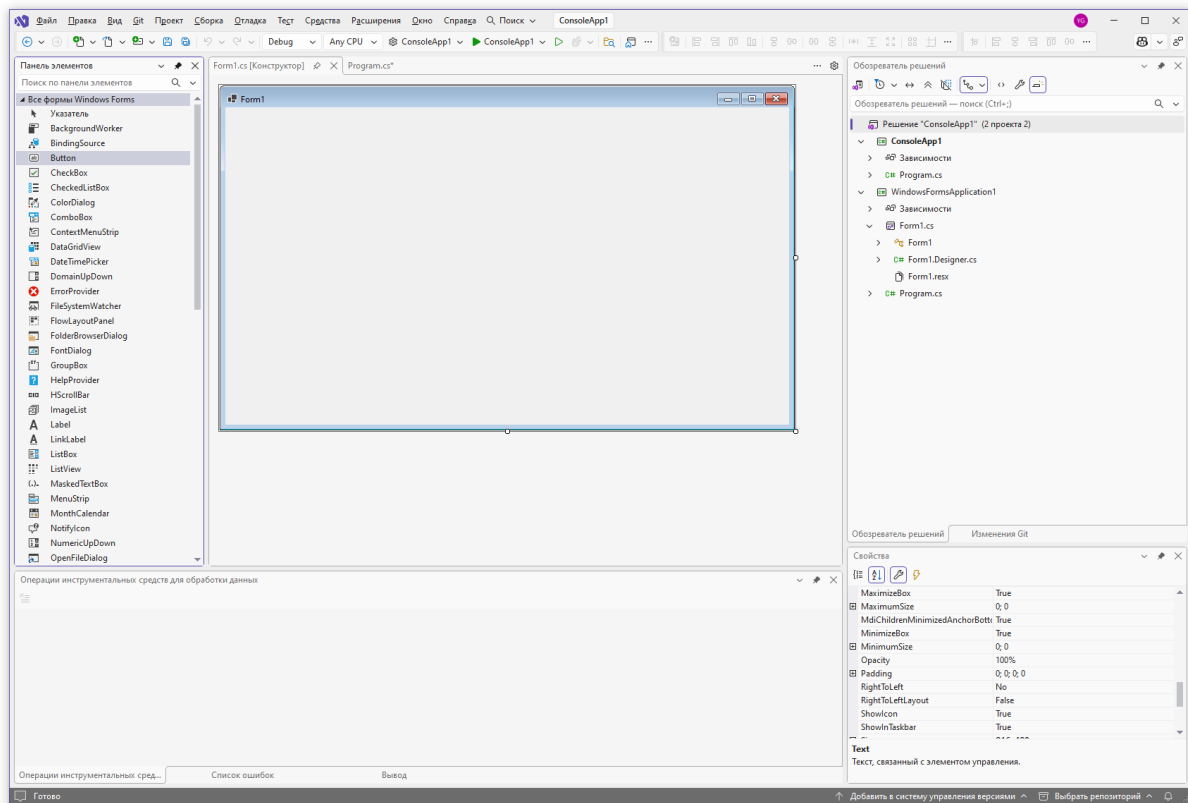


Рис. 41. Пример панели элементов.

Все элементы делятся на визуальные и невидимые. Визуальные элементы отображаются на форме в виде элементов управления. Это такие элементы как кнопка (Button), поле ввода (TextBox), текстовая надпись (Label) и другие.

Невидимые элементы также можно перетащить на форму, однако они отображаются в специальной области редактора макета формы. Примером такого элемента является таймер (Timer).

Если какая-либо из панелей не видна в текущей настройке Visual Studio, то ее можно включить в пункте меню «Вид» (в англоязычной версии «View»).

Панель элементов содержит десятки элементов. В том числе это элементы из группы «Данные», позволяющие достаточно быстро разрабатывать макеты приложений на основе реляционной СУБД.

Ниже мы разберем только основы работы с элементами управления на основе наиболее часто используемых элементов.

13.2.2 Пример работы с кнопкой и текстовым полем

Перетащим в текущее окно элементы Button, TextBox, Label и Timer. Их можно свободно перемещать мышью на форме. Элемент Timer отображается не на форме, а в отдельной нижней части конструктора. Выделим элемент Label. В правой нижней части окна Visual Studio отображается панель свойств (рис. 42).

В панели свойств отображаются свойства текущего выделенного элемента Label. Изменим свойство «Text» на «Текст по нажатию кнопки».

Для элемента Button изменим свойство «Text» на «Тест».

Отметим, что в данном примере не изменяются стандартные имена элементов управления, они называются button1, textBox1 и т.д. Имя элемента управления можно изменить с помощью свойства Name.

Имена элементов управления рекомендуется менять сразу после перетаскивания на форму, так как используются в автоматически генерируемых именах методов.

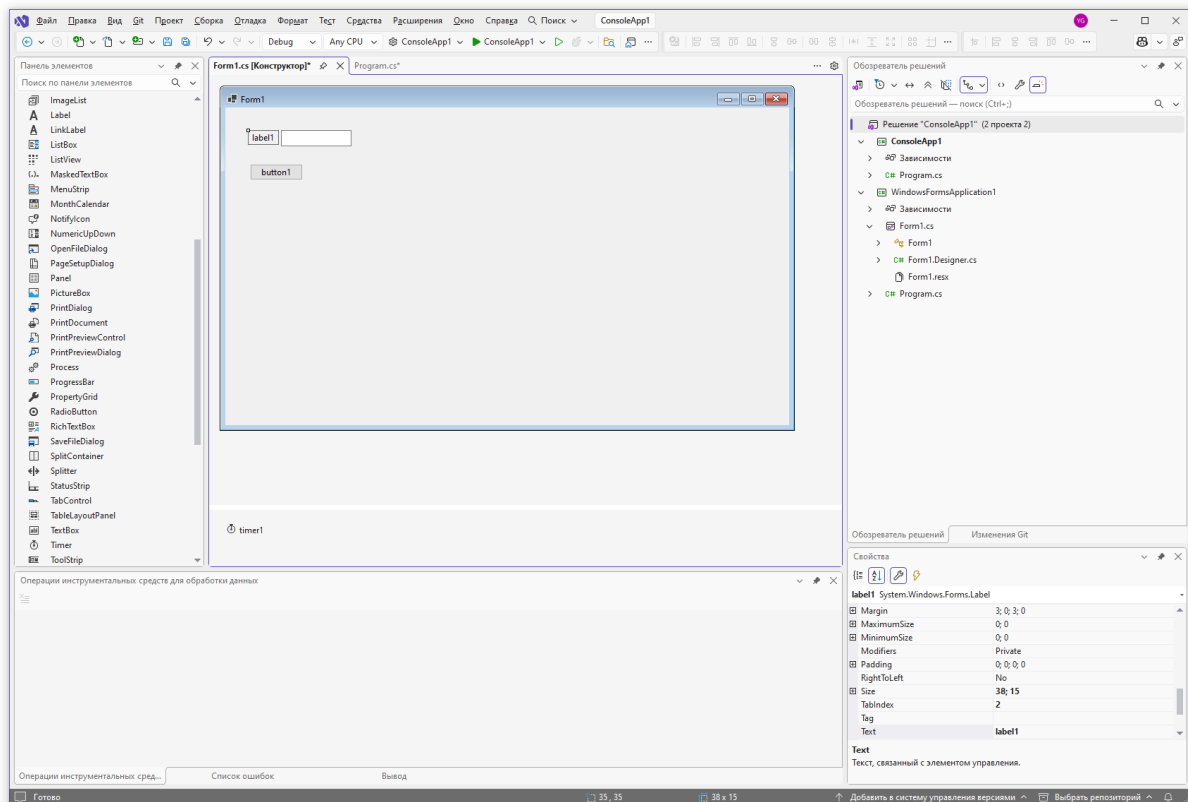


Рис. 42. Пример панели свойств.

В панели свойств используются кнопки, описание которых приведено в таблице 7.

Таблица 8

Кнопки панели свойств

Кнопка	Описание
	Переключение к списку свойств элемента
	Переключение к списку событий элемента
	Сортировка текущего списка свойств (событий) по категориям
	Сортировка текущего списка свойств (событий) по алфавиту по названиям свойств (событий)

Осуществим двойное нажатие левой клавишей мыши на кнопке Button. При этом будет автоматически сформирован обработчик события нажатия на кнопку (button1_Click) и откроется окно редактирования обработчика события (рис. 43).

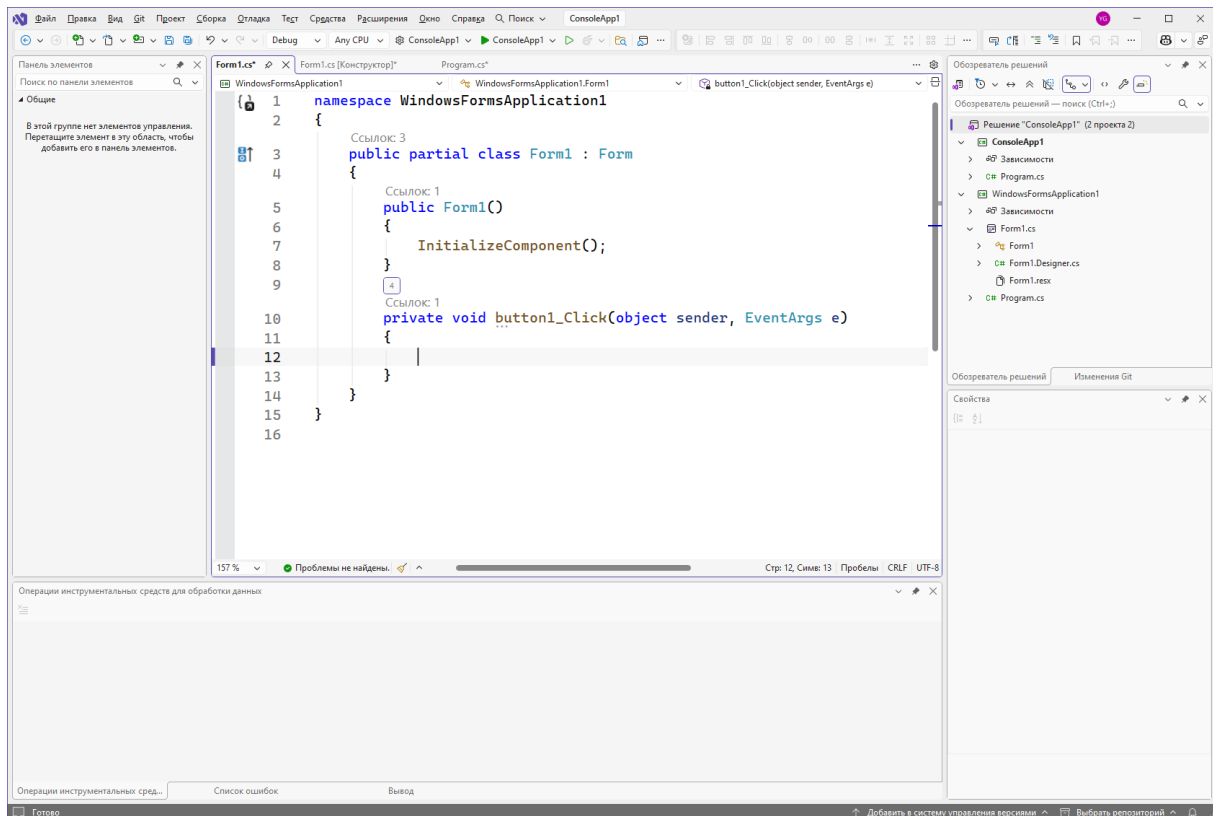


Рис. 43. Обработчик события нажатия на кнопку.

С точки зрения языка C# форма «Form1» представляет собой класс, унаследованный от класса Form (расположен в пространстве имен System.Windows.Forms). Обработчики событий добавляются в данный класс как методы. Отметим, что данный класс объявлен как частичный (partial), а в конструкторе вызывается метод InitializeComponent, который в данном файле не объявлен. Значит, в другом файле должна быть еще одна часть данного класса. Если на методе InitializeComponent нажать правую кнопку мыши и выбрать пункт меню «Перейти к определению», то будет открыт файл Form1.Designer.cs, который содержит вторую, автоматически сгенерированную часть класса Form1.

Использование частичных классов (partial) в Windows Forms — это один из ключевых архитектурных решений технологии. Класс формы разделён на два файла: Form1.cs (код, который пишет программист) и Form1.Designer.cs (код, который генерирует Visual Studio). Такое разделение задач соответствует принципу SRP: файл Designer.cs отвечает

за инициализацию и расположение элементов управления, файл Form1.cs — за логику обработки событий.

Текст файла Form1.Designer.cs:

```
namespace WindowsFormsApplication1
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should
        be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            components = new System.ComponentModel.Container();
            button1 = new Button();
            textBox1 = new TextBox();
            label1 = new Label();
            timer1 = new System.Windows.Forms.Timer(components);
            SuspendLayout();
            //
            // button1
            //
            button1.Location = new Point(35, 80);
            button1.Name = "button1";
            button1.Size = new Size(75, 23);
            button1.TabIndex = 0;
            button1.Text = "button1";
            button1.UseVisualStyleBackColor = true;
            button1.Click += button1_Click;
            //
            // textBox1
```

```

        //
        textBox1.Location = new Point(79, 32);
        textBox1.Name = "textBox1";
        textBox1.Size = new Size(100, 23);
        textBox1.TabIndex = 1;
        //
        // label1
        //
        label1.AutoSize = true;
        label1.Location = new Point(35, 35);
        label1.Name = "label1";
        label1.Size = new Size(38, 15);
        label1.TabIndex = 2;
        label1.Text = "label1";
        //
        // Form1
        //
        AutoScaleDimensions = new.SizeF(7F, 15F);
        AutoScaleMode = AutoScaleMode.Font;
        ClientSize = new Size(800, 450);
        Controls.Add(label1);
        Controls.Add(textBox1);
        Controls.Add(button1);
        Name = "Form1";
        Text = "Form1";
        ResumeLayout(false);
        PerformLayout();
    }

    #endregion

    private Button button1;
    private TextBox textBox1;
    private Label label1;
    private System.Windows.Forms.Timer timer1;
}
}

```

В данном файле в виде private-полей объявлены элементы управления, добавленные на форму: button1, timer1, textBox1, label1.

В методе `InitializeComponent` задаются свойства объектов, установленные в панели свойств: расположение на форме (`Location`), текстовая подпись (`Text`) и др.

Обработчик события `button1_Click` прикрепляется к настоящему событию, расположенному в классе `Button` (точнее в классе `Control`, от которого наследуется `Button`). Таким образом, рассмотренный ранее механизм событий используется в технологии `Windows Forms`.

С использованием файла Form1.Designer.cs связана одна неочевидная особенность, которая часто проблему в начале изучения технологии Windows Forms. Дело в том, что если для элемента управления были созданы свойства и события, а потом он был переименован, то файл Form1.Designer.cs не регенерируется. При компиляции это может приводить к ошибкам в файле Form1.Designer.cs, что требует исправления файла Form1.Designer.cs вручную.

Добавим в обработчик события button1_Click следующий код:

```
private void button1_Click(object sender, EventArgs e)
{
    //Запись текста в текстовое поле
    textBox1.Text = "Кнопка нажата";
    //Окно сообщения
    MessageBox.Show("Кнопка нажата");
}
```

При нажатии на кнопку выполняется запись текста «Кнопка нажата» в текстовое поле, при этом используется свойство Text текстового поля textBox1. Отметим, что возможно изменение практически любого свойства элемента, в том числе цвета, шрифта, положения на форме и т.д.

С помощью метода Show статического класса MessageBox осуществляется вывод окна сообщения. Окна сообщений в частности удобны для вывода сообщений об ошибках.

Нажмем на кнопку «Тест». Результат выполнения этого действия показан на рис 44.

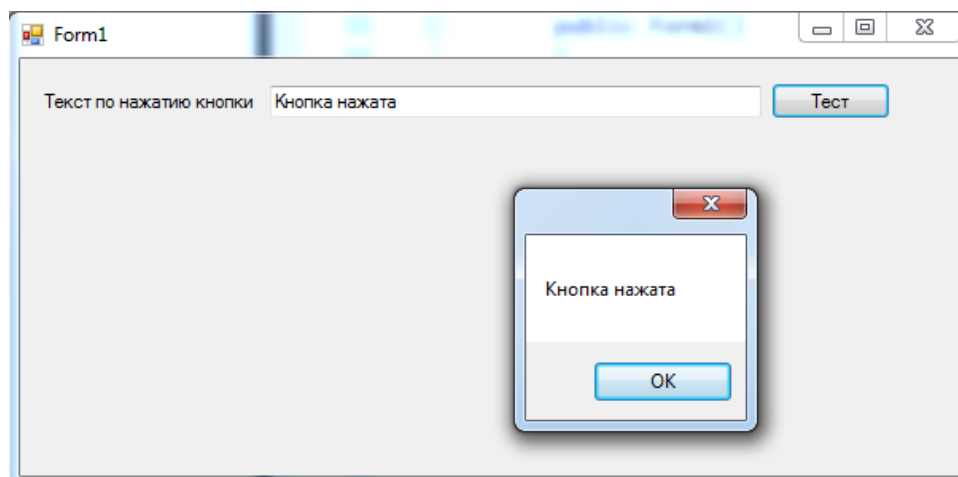


Рис. 44. Обработчик события нажатия на кнопку.

13.2.3 Пример работы с таймером

Добавим на форму следующие элементы (в скобках заданы значения свойств, которые требуется установить):

- Label (Text = «Таймер:»);
- TextBox (Name = «textTimer»);
- Button (Name = «buttonStartTimer», Text = «Старт»);
- Button (Name = «buttonStopTimer», Text = «Стоп»);
- Button (Name = «buttonClearTimer», Text = «Сброс»);
- Timer (Name = «timer1», Interval = «1000»).

Элемент Timer с именем timer1 был уже ранее добавлен на форму, для него необходимо установить только свойство Interval.

Общий вид формы в конструкторе форм показан на рис. 45.

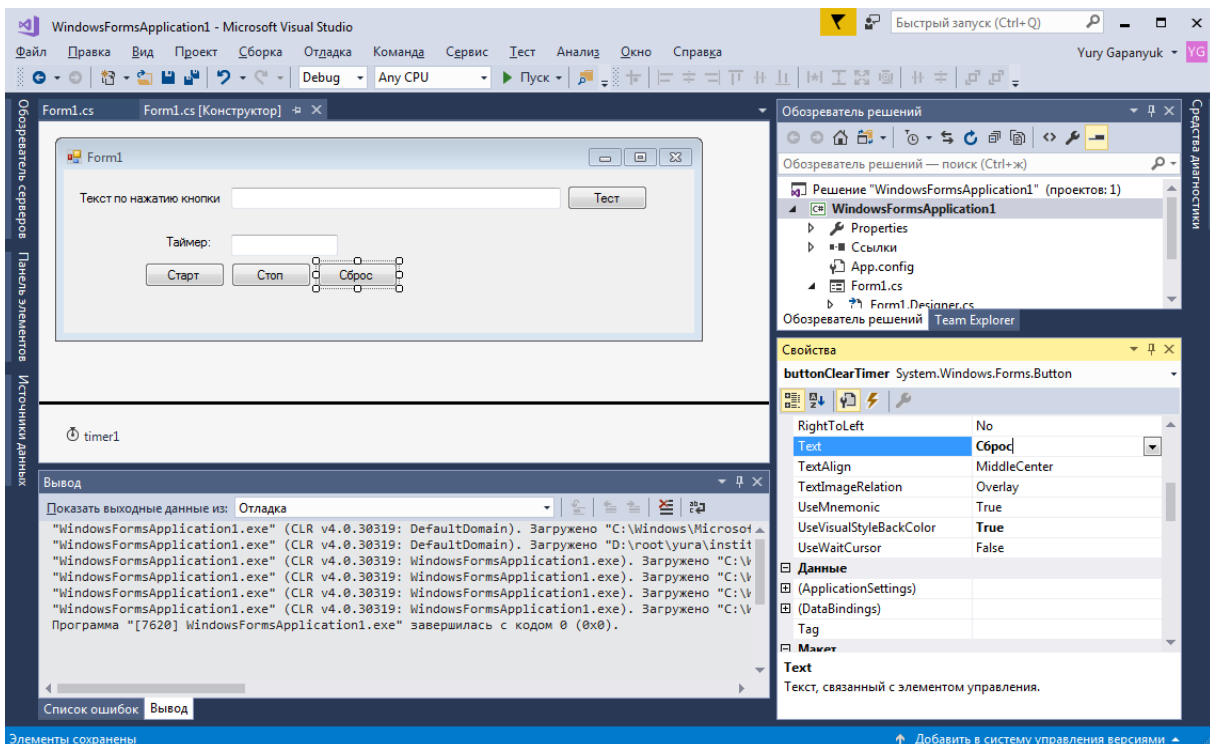


Рис. 45. Форма с добавлением таймера.

Свойство Interval=«1000» для таймера означает, что таймер будет срабатывать один раз в секунду (1000 мс).

Осуществим двойное нажатие левой клавишей мыши на таймере и трех добавленных кнопках, будут сгенерированы соответствующие обработчики событий, которые пока не содержат код:

```
private void timer1_Tick(object sender, EventArgs e)
{
}

private void buttonStartTimer_Click(object sender, EventArgs e)
{
}

private void buttonStopTimer_Click(object sender, EventArgs e)
{
}

private void buttonClearTimer_Click(object sender, EventArgs e)
{
}
```

С точки зрения языка C# файл с описанием обработчиков событий – обычный класс, поэтому в него можно добавлять поля данных, свойства, методы.

При разработке приложений необходимо реализовывать сложные алгоритмы в виде отдельных классов и из форм осуществлять только их вызов. Это облегчает тестирование алгоритмов и позволяет использовать их в различных приложениях, например в Windows Forms и ASP.NET MVC.

Добавим в файл, содержащий обработчики событий, поле для хранения текущего состояния таймера и метод обновления текущего состояния таймера:

```
/// <summary>
/// Текущее состояние таймера
/// </summary>
(TimeSpan currentTimer = new TimeSpan());

/// <summary>
/// Обновление текущего состояния таймера
/// </summary>
private void RefreshTimer()
```

```

{
    //Обновление поля таймера в форме
    textTimer.Text = currentTimer.ToString();
}

```

Для формы введем обработчик события по загрузке формы. Событие в окне свойств приведено на рис. 46.

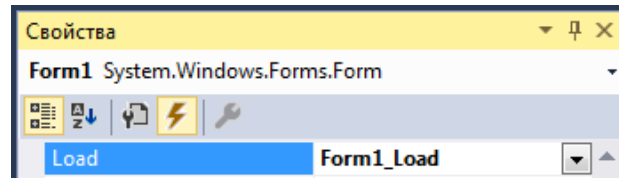


Рис. 46. Обработчик события по загрузке формы.

Для добавления обработчика события необходимо дважды щелкнуть левой клавишей мыши на строку Load.

При загрузке формы выводится начальное (нулевое) значение таймера:

```

private void Form1_Load(object sender, EventArgs e)
{
    //Обновление текущего состояния таймера
    RefreshTimer();
}

```

Обработчик события Load очень часто используется в формах. В нем выполняются действия, которые следует реализовать в только что отображенной форме.

Добавим код обработчиков событий таймера и кнопок (каждая строка кода откомментирована):

```

private void timer1_Tick(object sender, EventArgs e)
{
    //Добавление к текущему состоянию таймера
    //интервала в одну секунду
    currentTimer = currentTimer.Add(new TimeSpan(0, 0, 1));
    //Обновление текущего состояния таймера
    RefreshTimer();
}

private void buttonStartTimer_Click(object sender, EventArgs e)
{
    //Запуск таймера
    timer1.Start();
}

private void buttonStopTimer_Click(object sender, EventArgs e)

```

```

{
    //Остановка таймера
    timer1.Stop();
}

private void buttonClearTimer_Click(object sender, EventArgs e)
{
    //Остановка таймера
    timer1.Stop();
    //Сброс текущего состояния таймера
    currentTimer = new TimeSpan();
    //Обновление текущего состояния таймера
    RefreshTimer();
}

```

Пример работы таймера представлен на рис. 47. При загрузке формы в поле таймера отображаются нулевые значения: «00:00:00». Кнопка «Старт» запускает таймер, кнопка «Стоп» останавливает таймер (но не сбрасывает его значение), кнопка «Сброс» останавливает таймер и сбрасывает его значение.

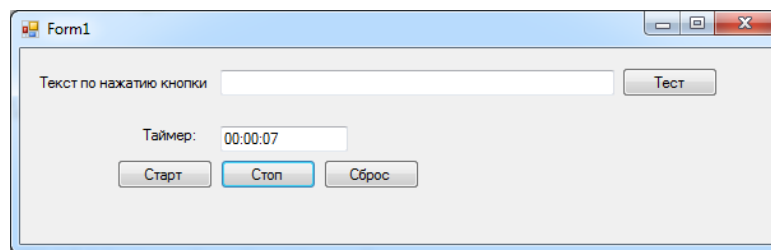


Рис. 47. Пример работы таймера.

13.2.4 Пример открытия дочерних окон

В большинстве случаев приложения Windows Forms содержат более одного окна. В данном разделе мы научимся добавлять и открывать новые окна.

Для добавления нового окна необходимо нажать правую кнопку мыши на проекте и выбрать пункт меню «Добавить/Создать элемент» (рис. 48).

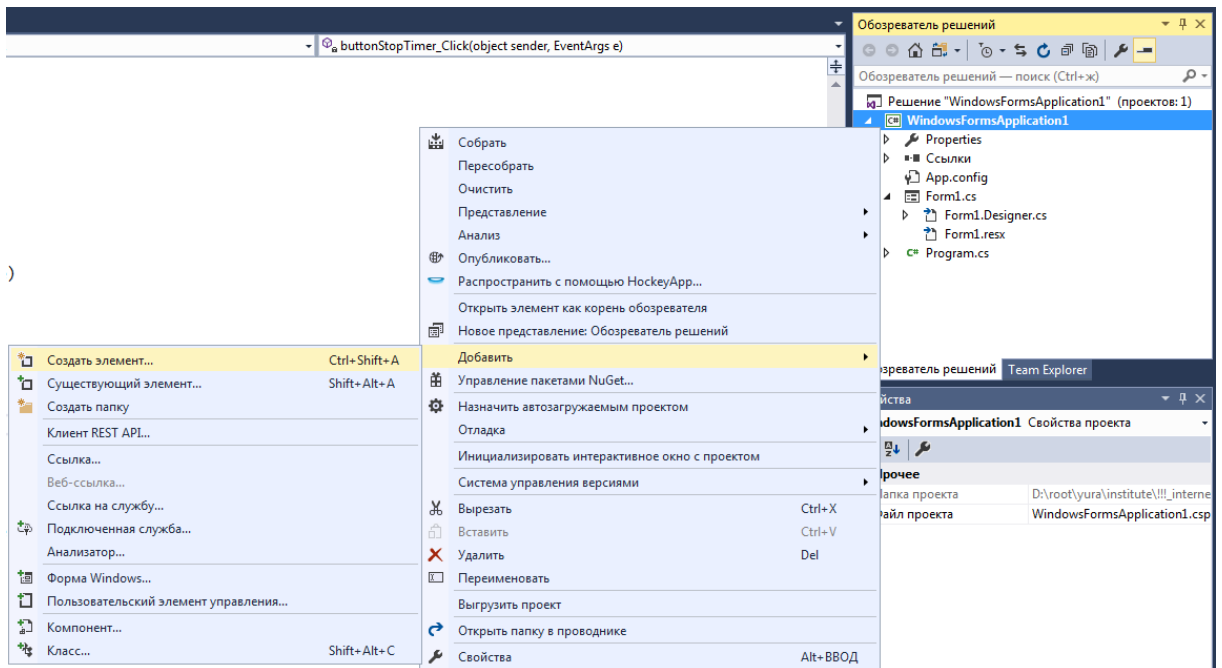


Рис. 48. Добавление формы – шаг 1.

В появившемся меню необходимо выбрать элемент «Форма Windows Forms» и нажать кнопку «Добавить» (на рис. 49).

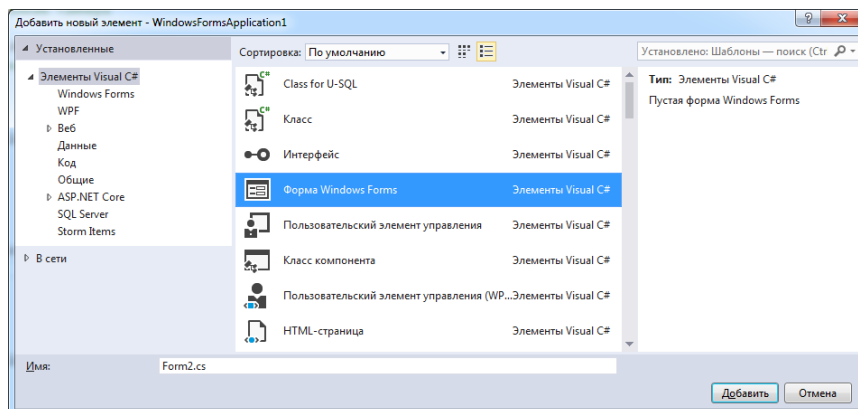


Рис. 49. Добавление формы – шаг 2.

Для добавляемой формы можно указать имя формы (имя класса формы, которое совпадает с именем файла), в данном примере Form2.cs.

Если приложение содержит несколько форм, то Windows Forms должен каким-то образом определить первую запускаемую форму. Это действие задается в файле Program.cs:

```
using System;
using System.Windows.Forms;

namespace WindowsFormsApplication1
{
```

```

static class Program
{
    /// <summary>
    /// Главная точка входа для приложения.
    /// </summary>
    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new Form1());
    }
}

```

Для приложений Windows Forms метод Main класса Program содержит код для запуска начальной формы. Если в методе Application.Run заменить Form1 на Form2, то при запуске приложения первой будет открываться Form2.

Добавим на форму 1 четыре кнопки:

- Button (Name = «buttonOpenNonModal», Text = «Открыть немодальное окно»);
- Button (Name = «buttonOpenModal», Text = «Открыть модальное окно»);
- Button (Name = «buttonClose», Text = «Закрыть окно»);
- Button (Name = «buttonExit», Text = «Закрыть приложение»).

Внешний вид формы 1 после добавления кнопок показан на рис. 50.

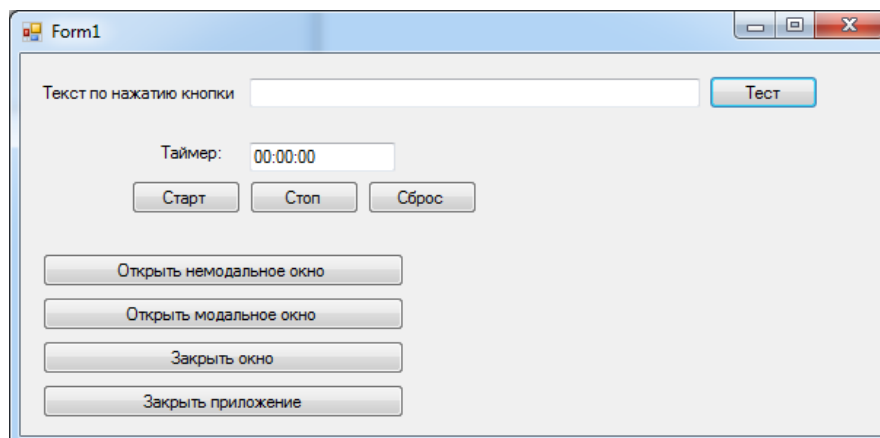


Рис. 50. Добавление кнопок открытия и закрытия окна.

Немодальное окно – это обычное окно приложения, с которым работает пользователь. Если открыть несколько немодальных окон, то между ними возможно переключение фокуса ввода.

Модальное окно – это окно, которое пользователь должен закрыть перед продолжением работы. Если открыто модальное окно, то переключение фокуса ввода в другие окна невозможно. Модальные окна также называют диалоговыми.

Рассмотренные ранее окна сообщений, создаваемые с помощью класса `MessageBox`, являются модальными окнами.

Добавим обработчики событий для кнопок.

Кнопка «Открыть немодальное окно»:

```
private void buttonOpenNonModal_Click(object sender, EventArgs e)
{
    Form2 f2 = new Form2();
    f2.Show();
}
```

Для открытия дочернего окна необходимо сначала создать новый объект соответствующего класса. При открытии нескольких окон нужно каждый раз создавать новый объект класса, потому что в каждое окно могут быть введены различные данные.

Далее с помощью метода `Show` можно открыть окно в немодальном режиме. При многократном нажатии на кнопку «Открыть немодальное окно» можно открыть произвольное количество дочерних окон.

Кнопка «Открыть модальное окно»:

```
private void buttonOpenModal_Click(object sender, EventArgs e)
{
    Form2 f2 = new Form2();
    f2.ShowDialog();
}
```

С помощью метода `ShowDialog` можно открыть окно в модальном режиме. При этом невозможно вернуться в форму 1 (она не получит фокус) до тех пор, пока не будет закрыта форма 2.

Многократное нажатие на кнопку «Открыть модальное окно» невозможно, так как при первом же нажатии открывается модальное окно, которое необходимо закрыть перед продолжением работы.

Кнопка «Закрыть окно»:

```
private void buttonClose_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Эта кнопка вызывает метод Close для текущего объекта, что приводит к закрытию окна.

Кнопка «Закрыть приложение»:

```
private void buttonExit_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

Данная кнопка вызывает метод Exit для статического класса Application, что приводит к закрытию приложения. Класс Application отвечает за приложение Windows Forms.

Отметим, что кнопки «Закрыть окно» и «Закрыть приложение» работают одинаково для главной формы и приводят к закрытию приложения.

Добавим данные кнопки на форму 2. Внешний вид формы 2 после добавления кнопок показан на рис. 51.

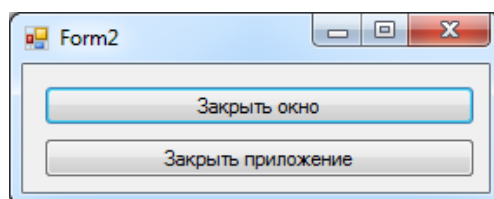


Рис. 51. Добавление кнопок открытия и закрытия окна.

Обработчики событий кнопок совпадают с обработчиками событий в форме 1:

```
private void buttonClose_Click(object sender, EventArgs e)
{
    this.Close();
}
```

```
private void buttonExit_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

Однако поведение кнопок в дочерней форме будет отличаться от их поведения в родительской форме.

Ключевое слово `this` в данном случае означает текущий объект формы 2. Поэтому при нажатии на кнопку «Заккрыть окно» будет закрываться форма 2.

Но при нажатии на кнопку «Заккрыть приложение» будет закрываться все приложение.

Добавим для формы 2 обработчики событий `FormClosing` и `FormClosed`, как показано на рис. 52.

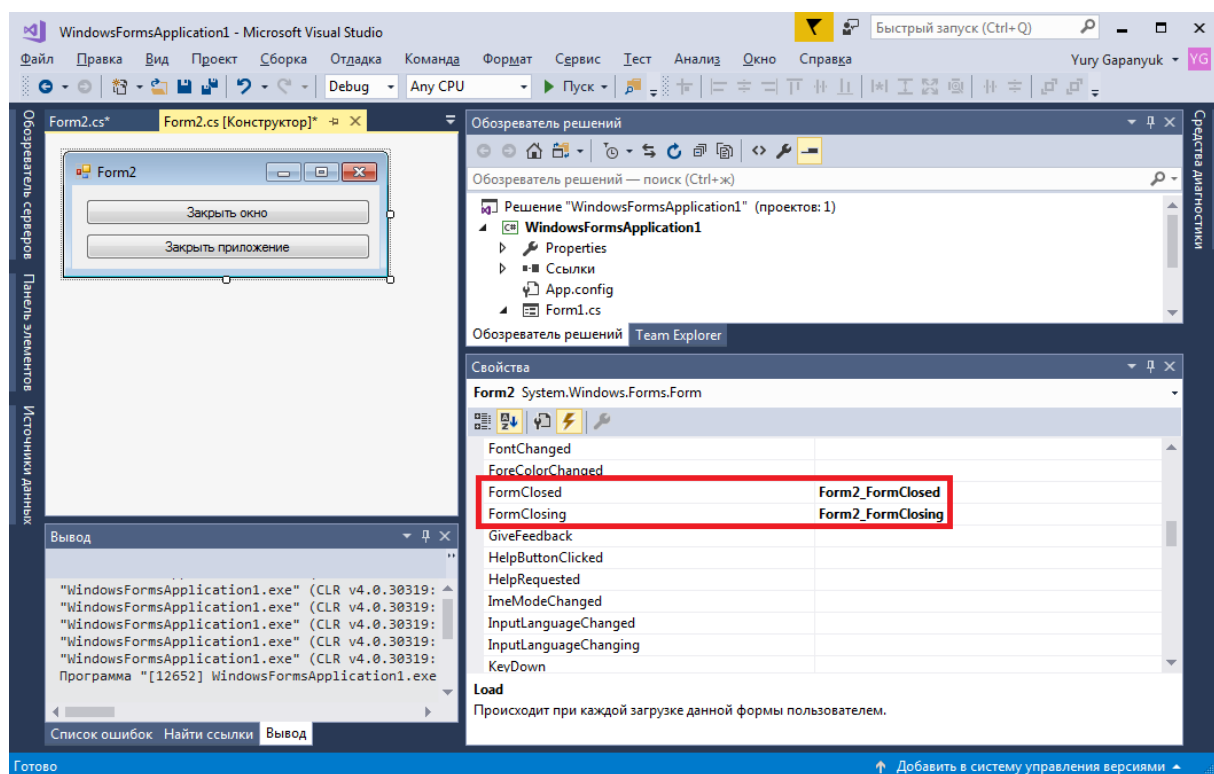


Рис. 52. Обработчики событий `FormClosing` и `FormClosed`.

Данные события также довольно часто используются в Windows Forms.

Событие `FormClosed` возникает, когда окно уже считается закрытым. Обработчик события:

```
private void Form2_FormClosed(object sender, FormClosedEventArgs e)
{
    MessageBox.Show("Форма 2 закрылась!");
}
```

Событие `FormClosing` возникает перед закрытием окна, в данном обработчике можно отменить закрытие окна. Обработчик события:

```
private void Form2_FormClosing(object sender, FormClosingEventArgs e)
{
    //Вывод диалогового окна
    DialogResult result = MessageBox.Show("Вы действительно хотите закрыть форму 2?",
        "Уважаемый пользователь", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
    //Отмена закрытия окна
    if (result == DialogResult.No)
    {
        e.Cancel = true;
    }
}
```

В данном примере метод `MessageBox.Show` открывает диалоговое окно в виде вопроса. Пользователь может выбрать кнопки «Да» или «Нет». Если он выбирает «Нет» то свойство `e.Cancel` (`e` – параметр метода) устанавливается в `true`, что приводит к отмене стандартного обработчика события и окно не закрывается.

Результаты работы показаны на рис. 53 и 54.

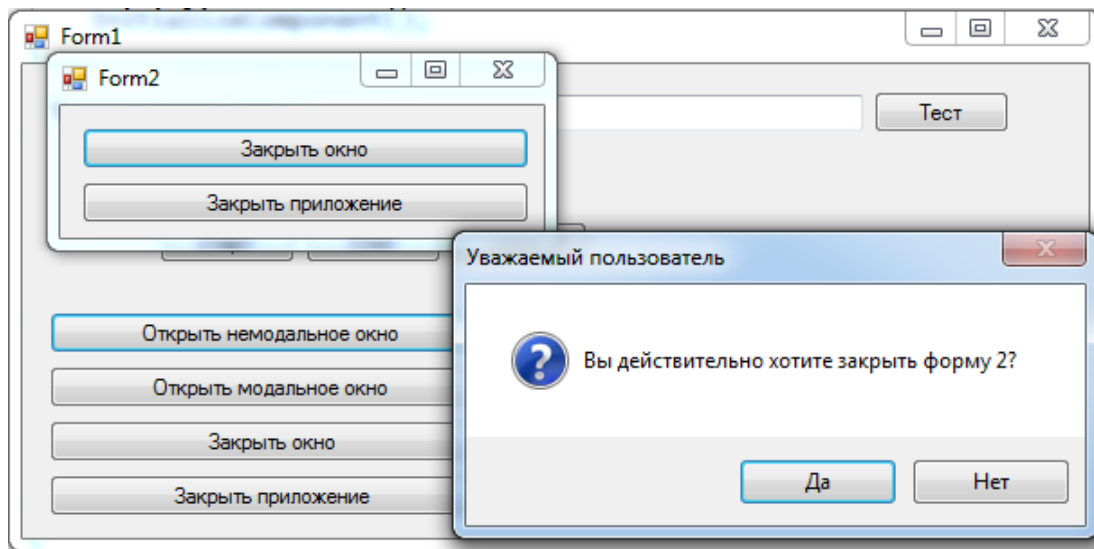


Рис. 53. Результат работы обработчика событий `FormClosing`.

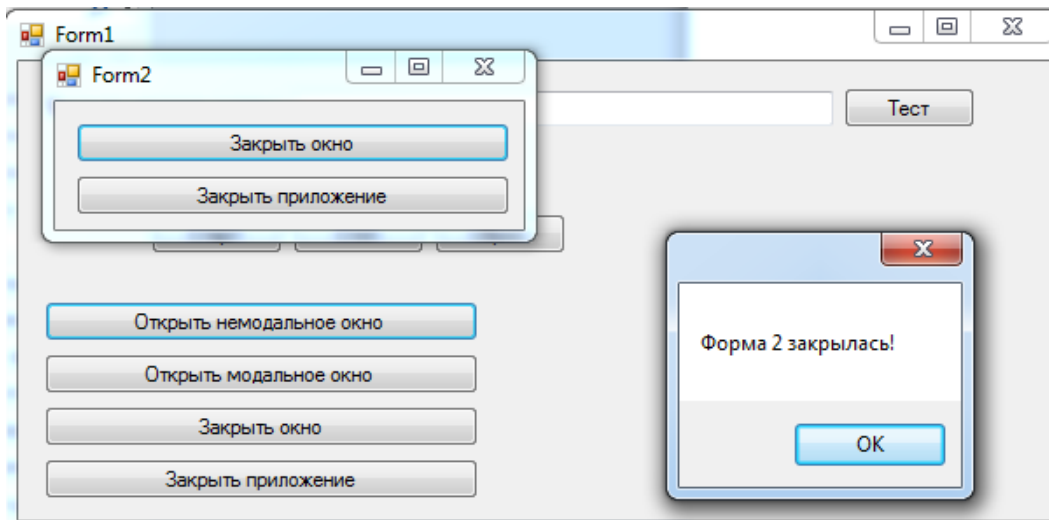


Рис. 54. Результат работы обработчика событий FormClosed.

13.2.5 Архитектурный аспект: «Super Form» и паттерн MVP применительно к Windows Forms

При разработке Windows Forms-приложений начинающие разработчики часто помещают всю логику приложения непосредственно в обработчики событий формы. Форма начинает: загружать данные из базы, валидировать ввод, вычислять результаты, форматировать вывод — то есть нарушает принцип SRP, превращаясь в антипаттерн «Super Object».

Правильный подход состоит в том, чтобы вынести логику в отдельные классы. Форма отвечает только за ввод/вывод. Это облегчает тестирование (сервис можно протестировать без запуска формы) и позволяет переиспользовать логику в других технологиях — например, в веб-приложении ASP.NET Core, использующем те же классы сервисов.

13.3 WPF и паттерн MVVM

WPF (Windows Presentation Foundation) появился в .NET 3.0 (2006) и принципиально отличается от Windows Forms двумя вещами:

1. Интерфейс описывается декларативно на языке XAML (Extensible Application Markup Language), а не программно.
2. Данные связываются с

элементами управления через механизм привязки данных (data binding) — элемент управления автоматически отображает изменения в объекте данных.

Именно эти два механизма делают паттерн MVVM естественным для WPF. XAML — это XML-язык для описания объектного графа. Каждый тег в XAML соответствует созданию объекта .NET, а атрибуты — установке свойств:

```
<!-- XAML: декларативно -->
<Button Content="Нажми меня" Width="100" Height="30"
        Background="LightBlue"/>
```

Эквивалентный код C#:

```
// C#: программно
var button = new Button();
button.Content = "Нажми меня";
button.Width = 100;
button.Height = 30;
button.Background = new SolidColorBrush(Colors.LightBlue);
```

Преимущество XAML: разметка чётко отделена от логики. Дизайнер и разработчик могут работать независимо — дизайнер редактирует XAML в Blend, разработчик пишет ViewModel на C#.

MVVM (Model-View-ViewModel) — это архитектурный шаблон проектирования, применяемый при разработке программного обеспечения для реализации принципа разделения ответственности (Separation of Concerns). Его фундаментальная задача заключается в строгой изоляции доменной (бизнес) логики и слоя доступа к данным от слоя пользовательского интерфейса.

Исторически MVVM является эволюцией паттерна Presentation Model (предложенного Мартином Фаулером) и был адаптирован Microsoft для декларативных интерфейсов.

Рассмотрим структурные компоненты паттерна с академической точки зрения:

1. Model (Модель)

Определение: Слой, инкапсулирующий предметную область (Domain Model), фундаментальные бизнес-правила и логику доступа к данным (Data Access Layer).

Функции: Управление состоянием данных, валидация бизнес-правил, взаимодействие с внешними источниками (базы данных, REST API, файловая система).

Свойства: Модель абсолютно независима и абстрагирована от других слоев. Она не содержит никаких ссылок на элементы интерфейса или логику представления.

2. View (Представление)

Определение: Слой пользовательского интерфейса (Presentation Layer), ответственный за визуализацию данных и перехват событий ввода.

Функции: Декларативное (как правило) описание структуры и компоновки графических элементов.

Свойства: View в парадигме MVVM должна быть максимально «пассивной» (Passive View). Она не должна содержать бизнес-логики (code-behind должен быть сведен к минимуму). View знает о существовании ViewModel и подписывается на ее публичные свойства.

3. ViewModel (Модель представления)

Определение: Абстракция слоя представления, выступающая в роли связующего звена (адаптера) между Model и View.

Функции:

Управление состоянием (State Management): Хранит текущее состояние интерфейса.

Трансформация данных: Извлекает данные из Model и форматирует их в структуры, оптимальные для немедленного биндинга (связывания) во View.

Обработка команд: Предоставляет методы или объекты команд (паттерн Command) для обработки действий пользователя, инициированных во View.

Свойства: Ключевое архитектурное правило — ViewModel ничего не знает о конкретной реализации View. Она не оперирует UI-компонентами (например, кнопками или текстовыми полями), что обеспечивает слабую связанность (loose coupling).

Архитектурное взаимодействие и механизм связи

Взаимодействие между компонентами MVVM строится на строгом графе зависимостей: View → ViewModel → Model (зависимость направлена только в одну сторону).

Обратная связь (от ViewModel к View) реализуется не через прямые вызовы методов, а через паттерн «Наблюдатель» (Observer) и механизм связывания данных (Data Binding).

ViewModel декларирует реактивные (наблюдаемые) свойства (например, через интерфейс INotifyPropertyChanged в .NET, LiveData/StateFlow в Android, Combine в iOS).

View подписывается на эти свойства.

При изменении данных в Model, ViewModel обновляет свои свойства, генерируя событие (уведомление).

View, перехватив событие, автоматически синхронизирует свой графический интерфейс с новым состоянием ViewModel.

Обоснование применения (Benefits)

Имплементация MVVM решает три ключевые инженерные задачи:

Тестируемость (Testability): Отсутствие зависимостей от UI-фреймворка позволяет покрывать логику представления (ViewModel) модульными тестами (Unit Tests) без инстанцирования графических компонентов.

Поддерживаемость (Maintainability): Слабая связанность модулей снижает риск возникновения регрессионных ошибок при модификации системы.

Параллельная разработка: UI-дизайнеры и верстальщики могут работать над View, пока инженеры-программисты реализуют ViewModel и Model, используя заглушки (mock-объекты) для связывания.

13.3.1 Привязка данных (Data Binding)

Привязка данных — механизм, автоматически синхронизирующий значение свойства UI-элемента со значением свойства объекта данных.

```
<!-- Элемент TextBlock автоматически отображает значение свойства Name -->
<TextBlock Text = "{Binding Name}" />
```

Когда свойство Name объекта изменится — TextBlock обновится автоматически. Именно поэтому свойства играют ключевую роль в .NET: привязка данных работает только со свойствами, но не с полями.

Для того чтобы привязка «знала» об изменении свойства, ViewModel должен реализовывать интерфейс INotifyPropertyChanged:

```
public interface INotifyPropertyChanged
{
    event PropertyChangedEventHandler? PropertyChanged;
}
```

Когда свойство меняется, ViewModel вызывает событие PropertyChanged (это стандартный механизм событий), только теперь его слушает не обработчик в форме, а движок привязки данных WPF.

13.3.2 Ключевые выводы по WPF + MVVM

1. ViewModel не содержит ни одной ссылки на элементы UI (TextBox, DataGrid и т.д.) — он работает только с данными и командами. Это означает, что ViewModel можно полностью покрыть модульными тестами без запуска оконного приложения.

2. Кнопка «Удалить» автоматически становится неактивной, когда ничего не выбрано — это обеспечивает CanExecute команды. В Windows

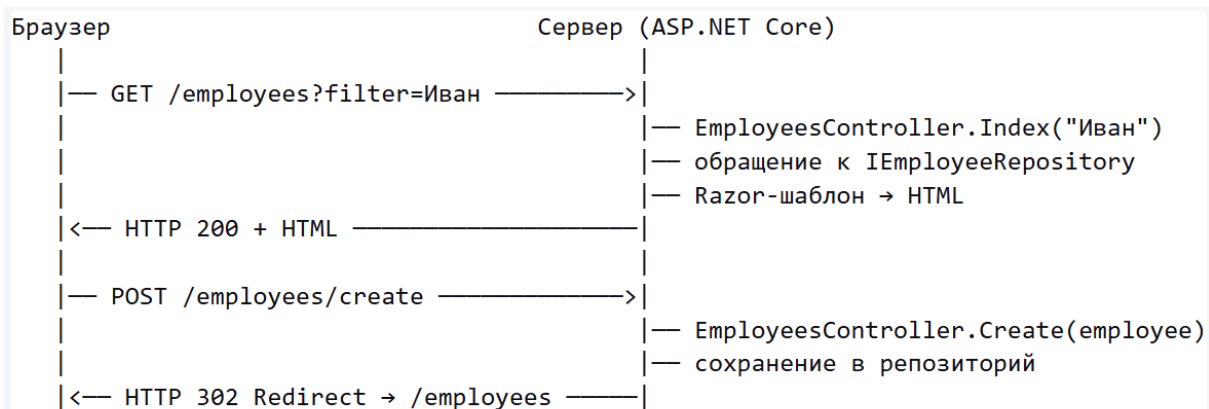
Forms то же самое пришлось бы реализовывать вручную через `button.Enabled = ....`

3. MVVM — не технология WPF, а паттерн проектирования (раздел 15.1). WPF лишь предоставляет инфраструктуру (data binding, ICommand), которая делает MVVM удобным. Тот же паттерн используется в MAUI, Blazor, Avalonia с теми же принципами.

13.4 Раздел 15.5. ASP.NET Core MVC

Архитектура веб-приложения:

В отличие от настольных приложений, веб-приложение разделено на две части: клиент (браузер) и сервер. Взаимодействие происходит через протокол HTTP:



Паттерн MVC «в действии»:

- **Controller** — принимает HTTP-запрос, вызывает Model, возвращает результат
- **Model** — классы данных и репозиторий (знакомые Employee, IEmployeeRepository)
- **View** — Razor-шаблон: HTML + выражения C# вида `@emp.Name`

14 Пример многопоточного поиска в текстовом файле с использованием технологии Windows Forms

В данном разделе пособия мы реализуем комплексный пример, который обобщает предыдущие примеры.

Пример реализован с использованием технологии Windows Forms. Он выполняет следующие действия:

- Запрашивает имя текстового файла для поиска с использованием стандартного диалога открытия файла.
- Вводит слово для поиска.
- Реализует четкий (без учета опечаток) поиск слова в файле. Найденное слово (которое может встретиться в файле несколько раз) выводится в результирующий список.
- Реализует нечеткий (на основе расстояния Дамерау-Левенштейна) многопоточный поиск в файле. При этом вводится максимальное расстояние для нечеткого поиска, на которое искомое слово может отличаться от слова в файле. Также вводится количество потоков, на которое разделяется массив слов исходного файла. Найденное слово (которое может встретиться в файле несколько раз) выводится в результирующий список с указанием номера потока и вычисленного расстояния Дамерау-Левенштейна.
- По результатам поиска формируется отчет в виде текстового файла в формате HTML. Выбор имени файла при сохранении проводится с использованием стандартного диалога сохранения файла.

Приложение реализовано с использованием одной формы, которая показана на рис. 43.

Рис. 55. Форма многопоточного поиска в текстовом файле.

14.1 Чтение информации из текстового файла

При нажатии на кнопку «Чтение из файла» выполняется чтение информации из текстового файла.

Для хранения списка слов, прочитанных из файла, в классе Form1 используется поле list:

```

/// <summary>
/// Список слов
/// </summary>
List<string> list = new List<string>();

```

Рассмотрим код обработчика кнопки:

```

private void buttonLoadFile_Click(object sender, EventArgs e)
{
    OpenFileDialog fd = new OpenFileDialog();
    fd.Filter = "Текстовые файлы|*.txt";

    if (fd.ShowDialog() == DialogResult.OK)
    {

```

```

Stopwatch t = new Stopwatch();
t.Start();

//Чтение файла в виде строки
string text = File.ReadAllText(fd.FileName);

//Разделительные символы для чтения из файла
char[] separators =
    new char[] { ' ', '.', ',', '!', '?', '/', '\t', '\n' };

string[] textArray = text.Split(separators);

foreach (string strTemp in textArray)
{
    //Удаление пробелов в начале и конце строки
    string str = strTemp.Trim();
    //Добавление строки в список, если строка не содержится
в списке
    if (!list.Contains(str)) list.Add(str);
}

t.Stop();
this.textBoxFileReadTime.Text = t.Elapsed.ToString();
this.textBoxFileReadCount.Text = list.Count.ToString();
}
else
{
    MessageBox.Show(«Необходимо выбрать файл»);
}
}

```

Для выбора текстового файла используется класс OpenFileDialog, который реализует стандартный диалог открытия файла. Свойство Filter содержит список расширений файлов, которые позволяет выбирать диалоговое окно. Открытие диалогового окна производится с помощью метода ShowDialog.

Если пользователь не выбрал текстовый файл (условие «fd.ShowDialog() == DialogResult.OK» не выполняется), то с использованием класса MessageBox выводится сообщение «Необходимо выбрать файл» и обработчик события завершает работу.

Если же условие истинно (файл успешно выбран), то выполняются основные действия обработчика события.

С использованием класса Stopwatch (который был рассмотрен в разделе пособия по параллельной обработке) объявляется и запускается таймер.

С использованием метода File.ReadAllText (который был рассмотрен в разделе пособия по обработке текстовых файлов) содержимое файла считывается в виде переменной text типа string. Свойство fd.FileName класса OpenFileDialog возвращает путь и имя файла.

С использованием метода Split класса string производится разделение содержимого переменной text на массив строк textArray. Метод Split принимает в качестве параметра массив символов, которые могут разделять слова в файле.

Далее в цикле foreach производится перебор всех слов в массиве textArray. Для текущего слова производится удаление пробелов в начале и конце. Если текущее слово еще не содержится в списке слов list, то оно добавляется в этот список. Таким образом, каждое слово в файле включается в список list только один раз.

После этого производится остановка таймера. В текстовое поле textBoxFileReadTime выводится время построения списка слов файла, а в текстовое поле textBoxFileReadCount выводится количество слов в списке list, то есть количество уникальных слов в файле. На рис. 43. у текстовых полей textBoxFileReadTime и textBoxFileReadCount серый фон, так как для них установлено свойство «ReadOnly=true», то есть они доступны только для чтения. Если поле доступно только для чтения, то пользователь не может вводить данные в такое поле, но его можно изменять в программе.

14.2 Четкий поиск в текстовом файле

В процедуре поиска используется не текстовый файл напрямую, а список list слов, прочитанных из файла.

Рассмотрим код обработчика кнопки «Четкий поиск»:

```

private void buttonExact_Click(object sender, EventArgs e)
{
    //Слово для поиска
    string word = this.textBoxFind.Text.Trim();

    //Если слово для поиска не пусто
    if (!string.IsNullOrEmpty(word) && list.Count > 0)
    {
        //Слово для поиска в верхнем регистре
        string wordUpper = word.ToUpper();

        //Временные результаты поиска
        List<string> tempList = new List<string>();

        Stopwatch t = new Stopwatch();
        t.Start();

        foreach (string str in list)
        {
            if (str.ToUpper().Contains(wordUpper))
            {
                tempList.Add(str);
            }
        }

        t.Stop();
        this.textBoxExactTime.Text = t.Elapsed.ToString();

        this.listBoxResult.BeginUpdate();

        //Очистка списка
        this.listBoxResult.Items.Clear();

        //Вывод результатов поиска
        foreach (string str in tempList)
        {
            this.listBoxResult.Items.Add(str);
        }
        this.listBoxResult.EndUpdate();
    }
    else
    {
        MessageBox.Show("Необходимо выбрать файл и ввести слово для
поиска");
    }
}

```

В начале работы обработчика проверяется содержимое поля «Слово для поиска» (элемент textBoxFind).

Если данное поле не пусто (условие `string.IsNullOrEmpty(word)` не выполняется) и прочитаны данные из файла (условие `list.Count>0` выполняется) то выполняются основные действия обработчика события. Если условие не выполняется, то выводится сообщение «Необходимо выбрать файл и ввести слово для поиска» и обработчик события завершает работу.

Далее перебираются все слова в списке `list`. Если текущее проверяемое слово включает слово для поиска как подстроку (что проверяется с использованием метода `Contains` класса `string`), то текущее проверяемое слово добавляется во временный список найденных слов `tempList`. Перед проверкой строки переводятся в верхний регистр с использованием метода `ToUpper` класса `string`.

После завершения поиска в текстовое поле `textBoxExactTime` выводится время поиска, а в список `listBoxResult` (элемент `ListBox`) список найденных слов.

Для заполнения данными элемента `ListBox` необходимо выполнить следующие действия:

- для начала обновления данных списка необходимо вызвать метод `BeginUpdate`;
- для работы со списком необходимо использовать коллекцию `Items`. Для очистки результатов предыдущего поиска следует вызвать метод `Items.Clear`;
- для добавления элементов в список необходимо использовать метод `Items.Add`;
- для завершения обновления данных списка нужно вызвать метод `EndUpdate`.

В результате четкого поиска в список будут выведены слова файла, которые включают искомое слово как подстроку.

Пример результатов четкого поиска представлен на рис. 44. В качестве файла используется текстовое содержимое статьи из «Википедии» о языке программирования C#. Для корректного поиска в русскоязычном тексте, текстовый файл должен быть сохранен в кодировке UTF-8, так как с ней по умолчанию работают методы класса File.

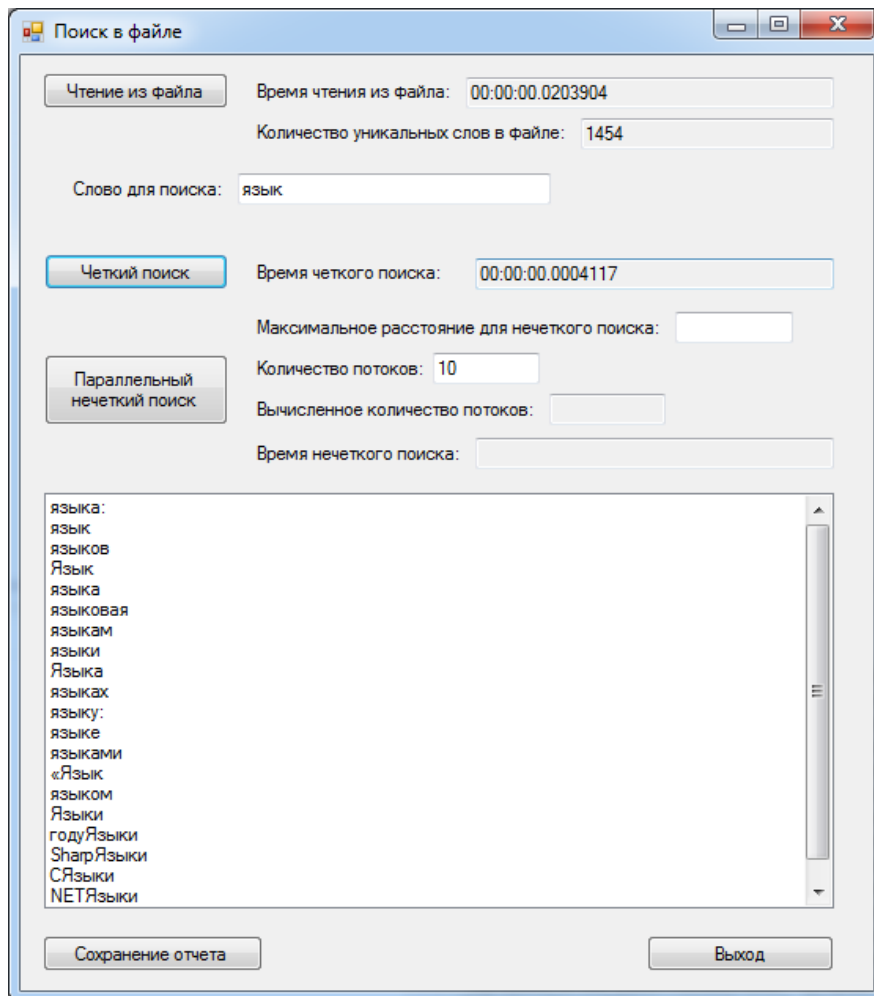


Рис. 56. Результаты четкого поиска.

14.3 Нечеткий поиск в текстовом файле

Нечеткий поиск в текстовом файле практически не отличается от четкого, только вместо проверки вхождения подстроки необходимо вызывать функцию вычисления расстояния Дамерау-Левенштейна, рассмотренную ранее. Однако дополнительную сложность придает требование параллельного поиска в массиве слов (пример параллельного поиска в массиве также был рассмотрен ранее).

Рассмотрим код обработчика кнопки «Параллельный нечеткий

ПОИСК»:

```
private void buttonApprox_Click(object sender, EventArgs e)
{
    //Слово для поиска
    string word = this.textBoxFind.Text.Trim();

    //Если слово для поиска не пусто
    if (!string.IsNullOrEmpty(word) && list.Count > 0)
    {
        int maxDist;
        if (!int.TryParse(this.textBoxMaxDist.Text.Trim(), out maxDist))
        {
            MessageBox.Show("Необходимо указать максимальное расстояние");
            return;
        }

        if (maxDist < 1 || maxDist > 5)
        {
            MessageBox.Show("Максимальное расстояние должно быть в диапазоне от 1
до 5");
            return;
        }

        int ThreadCount;
        if (!int.TryParse(this.textBoxThreadCount.Text.Trim(), out ThreadCount))
        {
            MessageBox.Show("Необходимо указать количество потоков");
            return;
        }

        Stopwatch timer = new Stopwatch();
        timer.Start();

        //-----
        // Начало параллельного поиска
        //-----

        //Результирующий список
        List<ParallelSearchResult> Result = new List<ParallelSearchResult>();

        //Деление списка на фрагменты для параллельного запуска в потоках
        List<MinMax> arrayDivList = SubArrays.DivideSubArrays(0, list.Count,
ThreadCount);
        int count = arrayDivList.Count;

        //Количество потоков соответствует количеству фрагментов массива
        Task<List<ParallelSearchResult>>[] tasks = new
Task<List<ParallelSearchResult>>[count];

        //Запуск потоков
        for (int i = 0; i < count; i++)
        {
            //Создание временного списка, чтобы потоки не работали параллельно с
одной коллекцией
            List<string> tempTaskList = list.GetRange(arrayDivList[i].Min,
arrayDivList[i].Max - arrayDivList[i].Min);
```

```

tasks[i] = new Task<List<ParallelSearchResult>>(
    //Метод, который будет выполняться в потоке
    ArrayThreadTask,
    //Параметры потока
    new ParallelSearchThreadParam()
    {
        tempList = tempTaskList,
        maxDist = maxDist,
        ThreadNum = i,
        wordPattern = word
    });

    //Запуск потока
    tasks[i].Start();
}

Task.WaitAll(tasks);

timer.Stop();

//Объединение результатов
for (int i = 0; i < count; i++)
{
    Result.AddRange(tasks[i].Result);
}

//-----
// Завершение параллельного поиска
//-----

timer.Stop();

//Вывод результатов

//Время поиска
this.textBoxApproxTime.Text = timer.Elapsed.ToString();

//Вычисленное количество потоков
this.textBoxThreadCountAll.Text = count.ToString();

//Начало обновления списка результатов
this.listBoxResult.BeginUpdate();

//Очистка списка
this.listBoxResult.Items.Clear();

//Вывод результатов поиска
foreach (var x in Result)
{
    string temp = x.word + "(расстояние=" + x.dist.ToString() + " поток="
+ x.ThreadNum.ToString() + ")";
    this.listBoxResult.Items.Add(temp);
}

//Окончание обновления списка результатов
this.listBoxResult.EndUpdate();
}
else
{
    MessageBox.Show("Необходимо выбрать файл и ввести слово для поиска");
}

```

```

    }
}

```

В начале обработчика, как и в случае четкого поиска, проверяется что список `list` не пуст и что указано слово для поиска. Если условие выполняется, то производится проверка и приведение к типу `int` полей «Максимальное расстояние для нечеткого поиска» и «Количество потоков». Для приведения к типу `int` используется метод `int.TryParse`. Для максимального расстояния также проверяется, что оно находится в диапазоне от 1 до 5.

Параллельный поиск практически полностью повторяет пример, рассмотренный в разделе 9. Разница состоит в том, что в данном случае обрабатывается не массив целых чисел, а массив строк.

Для хранения информации о найденных словах используется класс `ParallelSearchResult`:

```

/// <summary>
/// Результаты параллельного поиска
/// </summary>
public class ParallelSearchResult
{
    /// <summary>
    /// Найденное слово
    /// </summary>
    public string word { get; set; }

    /// <summary>
    /// Расстояние
    /// </summary>
    public int dist { get; set; }

    /// <summary>
    /// Номер потока
    /// </summary>
    public int ThreadNum { get; set; }
}

```

Класс содержит найденное слово, расстояние Дameraу-Левенштейна между найденным и искомым словами, и номер потока, в котором было найдено данное слово.

Для разделения массива на подмассивы применяется класс `SubArrays`.

Для параллельного поиска используется класс Task. Как и в рассмотренном ранее примере параллельного поиска создается массив объектов класса Task, потоки запускаются, ожидание завершения потоков осуществляется с помощью метода Task.WaitAll. Внутри потока выполняется метод ArrayThreadTask, который получает в качестве параметра объект класса ParallelSearchThreadParam.

После завершения работы всех потоков проводится объединение результатов с помощью свойства Result класса Task.

Далее осуществляется вывод результатов. Заполняются поля «Вычисленное количество потоков» (так как класс SubArrays может возвращать на один поток больше чем указано) и «Время нечеткого поиска».

Аналогично четкому поиску найденные слова выводятся в список listBoxResult, но в данном случае наряду с найденным словом выводится номер потока, в котором было найдено слово, и реальное расстояние Дамерау-Левенштейна, на которое данное слово отличается от искомого.

Рассмотрим метод ArrayThreadTask, который выполняется внутри потока. В качестве параметра метод ArrayThreadTask получает объект класса ParallelSearchThreadParam:

```

/// <summary>
/// Параметры которые передаются в поток
/// для параллельного поиска
/// </summary>
class ParallelSearchThreadParam
{
    /// <summary>
    /// Массив для поиска
    /// </summary>
    public List<string> tempList { get; set; }

    /// <summary>
    /// Слово для поиска
    /// </summary>
    public string wordPattern { get; set; }

    /// <summary>

```

```

    /// Максимальное расстояние для нечеткого поиска
    /// </summary>
    public int maxDist { get; set; }

    /// <summary>
    /// Номер потока
    /// </summary>
    public int ThreadNum { get; set; }
}

```

Класс содержит входной массив слов и слово для поиска, максимальное расстояние для нечеткого поиска и номер потока. Текст метода ArrayThreadTask:

```

/// <summary>
/// Выполняется в параллельном потоке для поиска строк
/// </summary>
public static List<ParallelSearchResult> ArrayThreadTask(object
paramObj)
{
    ParallelSearchThreadParam param =
    (ParallelSearchThreadParam)paramObj;

    ///Слово для поиска в верхнем регистре
    string wordUpper = param.wordPattern.Trim().ToUpper();

    ///Результаты поиска в одном потоке
    List<ParallelSearchResult> Result = new
List<ParallelSearchResult>();

    ///Перебор всех слов во временном списке данного потока
    foreach (string str in param.tempList)
    {
        ///Вычисление расстояния Дамерау-Левенштейна
        int dist = EditDistance.Distance(str.ToUpper(), wordUpper);

        ///Если расстояние меньше порогового, то слово добавляется в
результат
        if (dist <= param.maxDist)
        {
            ParallelSearchResult temp = new ParallelSearchResult()
            {
                word = str,
                dist = dist,
                ThreadNum = param.ThreadNum
            };

            Result.Add(temp);
        }
    }
}

```

```

    }
    return Result;
}

```

В методе перебираются все слова в массиве для поиска, переданном данному потоку. Если расстояние Дamerau-Левенштейна между текущим словом и искомым словом меньше максимально допустимого, то текущее слово считается найденным и помещается в список результата.

Список результата, являющийся возвращаемым значением метода, содержит объекты рассмотренного ранее класса `ParallelSearchResult`. Пример результатов нечеткого поиска представлен на рис. 45.

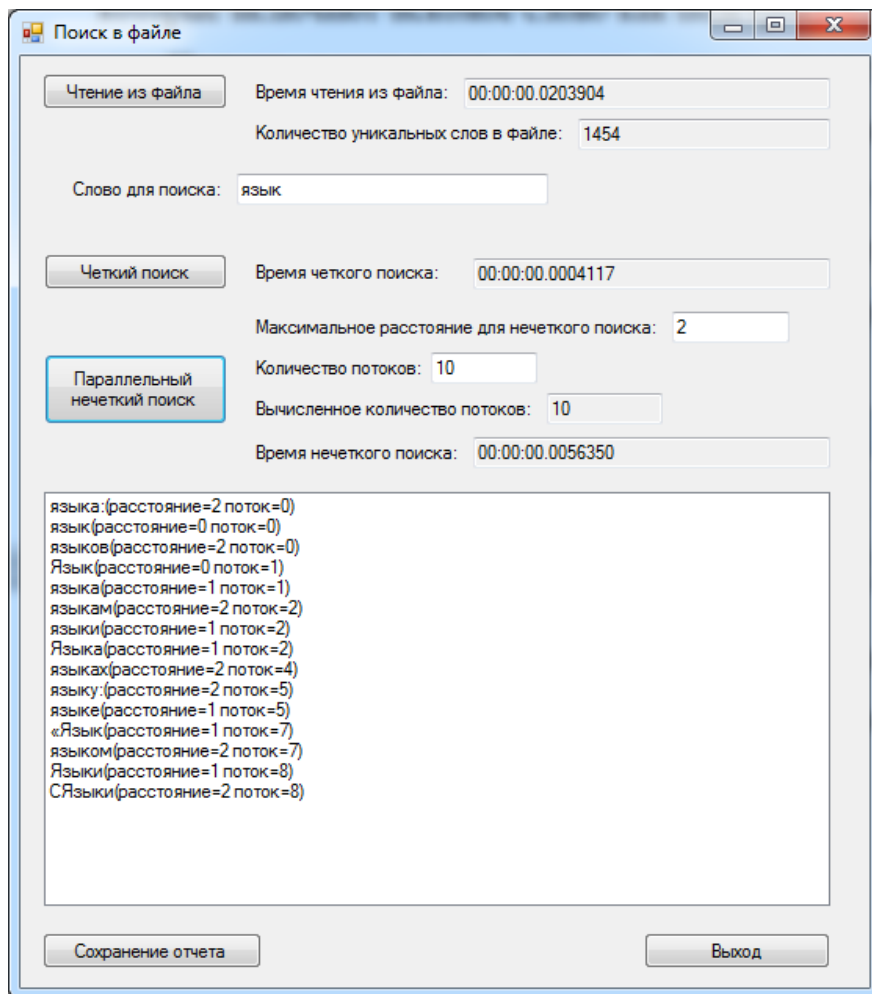


Рис. 57. Результаты нечеткого поиска.

14.4 Формирование отчета

При нажатии на кнопку «Сохранение отчета» проводятся формирование отчета и сохранение в текстовый файл.

Рассмотрим код обработчика кнопки:

```
private void buttonSaveReport_Click(object sender, EventArgs e)
{
    //Имя файла отчета
    string TempReportFileName = "Report_" +
    DateTime.Now.ToString("dd_MM_yyyy_hhmmss");

    //Диалог сохранения файла отчета
    SaveFileDialog fd = new SaveFileDialog();
    fd.FileName = TempReportFileName;
    fd.DefaultExt = ".html";
    fd.Filter = "HTML Reports|*.html";

    if (fd.ShowDialog() == DialogResult.OK)
    {
        string ReportFileName = fd.FileName;

        //Формирование отчета
        StringBuilder b = new StringBuilder();
        b.AppendLine("<html>");

        b.AppendLine("<head>");
        b.AppendLine("<meta http-equiv='Content-Type' content='text/html; charset=UTF-8' />");
        b.AppendLine("<title>" + "Отчет: " + ReportFileName + "</title>");
        b.AppendLine("</head>");

        b.AppendLine("<body>");

        b.AppendLine("<h1>" + "Отчет: " + ReportFileName + "</h1>");
        b.AppendLine("<table border='1'>");

        b.AppendLine("<tr>");
        b.AppendLine("<td>Время чтения из файла</td>");
        b.AppendLine("<td>" + this.textBoxFileReadTime.Text + "</td>");
        b.AppendLine("</tr>");

        b.AppendLine("<tr>");
        b.AppendLine("<td>Количество уникальных слов в файле</td>");
        b.AppendLine("<td>" + this.textBoxFileReadCount.Text + "</td>");
        b.AppendLine("</tr>");

        b.AppendLine("<tr>");
        b.AppendLine("<td>Слово для поиска</td>");
        b.AppendLine("<td>" + this.textBoxFind.Text + "</td>");
        b.AppendLine("</tr>");

        b.AppendLine("<tr>");
        b.AppendLine("<td>Максимальное расстояние для нечеткого поиска</td>");
        b.AppendLine("<td>" + this.textBoxMaxDist.Text + "</td>");
        b.AppendLine("</tr>");

        b.AppendLine("<tr>");
        b.AppendLine("<td>Время четкого поиска</td>");
        b.AppendLine("<td>" + this.textBoxExactTime.Text + "</td>");
        b.AppendLine("</tr>");

        b.AppendLine("<tr>");
        b.AppendLine("<td>Время нечеткого поиска</td>");
```

```

b.AppendLine("<td>" + this.textBoxApproxTime.Text + "</td>");
b.AppendLine("</tr>");

b.AppendLine("<tr valign='top'>");
b.AppendLine("<td>Результаты поиска</td>");
b.AppendLine("<td>");
b.AppendLine("<ul>");

foreach (var x in this.listBoxResult.Items)
{
    b.AppendLine("<li>" + x.ToString() + "</li>");
}

b.AppendLine("</ul>");
b.AppendLine("</td>");
b.AppendLine("</tr>");

b.AppendLine("</table>");

b.AppendLine("</body>");
b.AppendLine("</html>");

//Сохранение файла
File.AppendAllText(ReportFileName, b.ToString());

MessageBox.Show("Отчет сформирован. Файл: " + ReportFileName);
}
}

```

В переменной TempReportFileName формируется имя файла на основе текущих даты и времени.

Далее создается объект класса SaveFileDialog. При вызове метода ShowDialog пользователю предлагается сохранить файл.

Если пользователь не выбрал корректное имя файла для сохранения (условие «fd.ShowDialog() == DialogResult.OK» не выполняется), то обработчик события завершает работу.

Если пользователь выбрал корректное имя файла, то формируется отчет в формате HTML с использованием класса StringBuilder.

Отчет формируется в виде HTML-таблицы. После формирования отчета с помощью класса StringBuilder, отчет сохраняется в текстовый файл с помощью метода File.AppendAllText:

```
File.AppendAllText(ReportFileName, b.ToString());
```

Первым параметром метода является путь и имя файла, указанные в классе SaveFileDialog. Вторым параметром метода является отчет,

сформированный с помощью класса `StringBuilder` и преобразуемый к типу `String`.

Далее с применением класса `MessageBox` выводится сообщение пользователю об имени файла, содержащего отчет. Отображение файла отчета в браузере представлено на рис. 46.

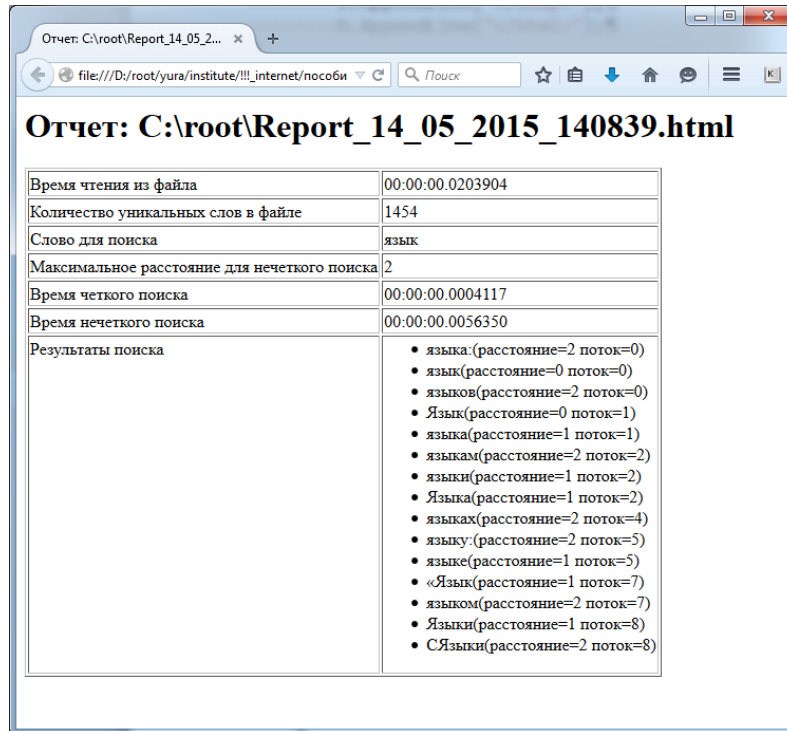


Рис. 58. Отображение файла отчета в браузере.

Текст сформированного файла «Report_14_05_2015_140839.html»:

```
<html>
<head>
<meta http-equiv='Content-Type' content='text/html; charset=UTF-8'/>
<title>Отчет: C:\root\Report_14_05_2015_140839.html</title>
</head>
<body>
<h1>Отчет: C:\root\Report_14_05_2015_140839.html</h1>
<table border='1'>
<tr>
<td>Время чтения из файла</td>
<td>00:00:00.0203904</td>
</tr>
<tr>
<td>Количество уникальных слов в файле</td>
<td>1454</td>
</tr>
<tr>
<td>Слово для поиска</td>
```

```

<td>язык</td>
</tr>
<tr>
<td>Максимальное расстояние для нечеткого поиска</td>
<td>2</td>
</tr>
<tr>
<td>Время четкого поиска</td>
<td>00:00:00.0004117</td>
</tr>
<tr>
<td>Время нечеткого поиска</td>
<td>00:00:00.0056350</td>
</tr>
<tr valign='top'>
<td>Результаты поиска</td>
<td>
<ul>
<li>языка:(расстояние=2 поток=0)</li>
<li>язык(расстояние=0 поток=0)</li>
<li>языков(расстояние=2 поток=0)</li>
<li>Язык(расстояние=0 поток=1)</li>
<li>языка(расстояние=1 поток=1)</li>
<li>языкам(расстояние=2 поток=2)</li>
<li>языки(расстояние=1 поток=2)</li>
<li>Языка(расстояние=1 поток=2)</li>
<li>языках(расстояние=2 поток=4)</li>
<li>языку:(расстояние=2 поток=5)</li>
<li>языке(расстояние=1 поток=5)</li>
<li>«Язык(расстояние=1 поток=7)</li>
<li>языком(расстояние=2 поток=7)</li>
<li>Языки(расстояние=1 поток=8)</li>
<li>Сязыки(расстояние=2 поток=8)</li>
</ul>
</td>
</tr>
</table>
</body>
</html>

```